

ADVANCED ALGORITHMS

Computer Science, ETH Zürich

BASED ON A SCRIPT BY MOHSEN GHAFFARI

Adapted by Bernhard Haeupler, Johannes Lengler, Maximilian Probst

These notes will be updated regularly. Please read critically; there are typos throughout, but there might also be mistakes. Feedback and comments would be greatly appreciated and should be emailed to raghu.ravi@inf.ethz.ch.

Last update: September 17, 2026

Contents

Notation and useful inequalities	ix
I Approximation Algorithms	1
1 Greedy algorithms	5
1.1 Minimum set cover	5
2 Approximation schemes	11
2.1 Knapsack	12
2.2 Bin packing	15
3 Randomized approximation schemes	23
3.1 Sampling	25
3.2 DNF counting	26
3.3 Network reliability	31
4 Rounding Linear Program Solutions	35
4.1 Minimum set cover	36
4.2 Minimizing congestion in multi-commodity routing	42
5 Online Algorithms and Competitive Analysis	51
5.1 Warm up: Ski rental	51
5.2 Paging	52
II Algorithms for Big Data	61
6 Advanced Hashing	63
6.1 Idealized Hash Functions vs. Practical Hash Functions	63
6.2 Signatures via Hashing	66
6.3 Hashing with Chaining	67

6.4	Linear Probing	68
7	Streaming Algorithms	75
7.1	The Streaming Model	75
7.2	Frequency Moments of a Stream	75
7.3	Estimating the first moment of a stream	76
7.4	Estimating the zeroth moment of a stream	81
7.5	Estimating the second moment of a stream	87
8	Dimensionality Reduction and Nearest-Neighbor Search	93
8.1	Dimensionality Reduction for Similarity Measures and Distances	95
8.2	Nearest Neighbor Search via Locality Sensitive Hashing	103
9	Data Clustering	109
9.1	Lloyd's Algorithm	111
9.2	k-means++: Initializing with guarantees	114
9.3	k-means : Parallelizing the initialization	121
III	Modern Graph Algorithms	125
10	Distance-Preserving Trees	127
10.1	The Random-Shift Algorithm	129
10.2	Probabilistic Tree Embeddings	133
10.3	Tightening the Random-Shift Analysis	136
11	Cut-Preserving Trees via MWU	141
11.1	The Multiplicative Weights Update method	142
11.2	Solving Linear Programs via the MWU method	146
11.3	Application: Cut-Preserving Trees	148
12	Graph Sparsification: Preserving Distances and Cuts	153
12.1	Distance-Preserving Sparsifiers	154
12.2	Cut-Preserving Sparsifiers	157
12.3	Warm up: Computing a cut sparsifier for $G = K_n$	158
12.4	Cut Sparsifiers for Expander Graphs	160
12.5	Decomposing a graph into expanders	161
12.6	Decomposing the graph into almost vertex-disjoint expanders with average-degree close to minimum-degree	165
12.7	Putting it all together	166

13 Maximum & Transshipment Flows	169
13.1 A Brief Intro to Flows	170
13.2 Maximum Flows	171
13.3 Transshipment Flows	176

Introduction to the Course

This course serves as an introduction to central ideas and techniques in modern algorithms. The emphasis in this course is on high-level concepts, ideas, and intuitions underlying these algorithms, and on learning to analyze them rigorously. The course is designed around algorithmic topics that are broadly applicable to modern computational problems encountered in practice, for example, in the areas of machine learning and computational biology. It also introduces foundational theoretical concepts that underlie state-of-the-art research in various important areas.

The course is structured into three parts, each consisting of four interconnected topics that are each studied over the course of a week:

1. **Approximation Algorithms:** Many fundamental algorithmic problems, such as Set Cover, Knapsack, or Bin Packing are NP-hard and do not admit *exact* polynomial time algorithms (unless $P = NP$). However, such problems are ubiquitous in practice and are often encountered in scheduling, computational biology, and many other areas.

In this chapter, we study introductory techniques to obtain efficient (i.e. polynomial-time) algorithms to compute solutions to NP-hard problems that are provably 'close' to an optimal solution. The closeness is quantified by an approximation ratio, providing a formal guarantee on the quality of the returned solution.

Techniques: Greedy Algorithm, Approximation Schemes, LP formulation, LP rounding, Online Algorithms.

2. **Algorithms for Big Data:** A fundamental challenge in modern computer science is the abundance of data. In this chapter, we study efficient algorithms and computational models that allow us to process such massive amounts of data.

We study highly practical hashing algorithms, discuss how to compress data sets while preserving important information, compute statistics in computationally challenging settings, and give a formal introduction to clustering techniques. This chapter places special emphasis on practical algorithms for key computational primitives and formal techniques to analyze problems in big data, data science, and machine learning.

Techniques: Hashing, Dimensionality Reduction, Streaming, Massive Parallel Computations, Clustering.

3. **Modern Graph Algorithms:** For many problems in computer science, data can be modelled as a graph network. It is then often desir-

able to extract information relating to distances or the connectivity/cut structure of the underlying graph.

In this chapter, we discuss recent algorithmic techniques that yield extremely fast algorithms to compute distance and cut-related information. These techniques mix methods from combinatorial graph algorithms with techniques from optimization. We first discuss tree embeddings that encode distance information of the graph in a compact tree. Then, we learn about trees that encode the cut structure of the graph. We then learn how to use such objects to compute cuts and commodity-flows that minimize underlying cost, referred to as transshipment (minimizing ℓ_1 -cost) and maxflow (minimizing ℓ_∞ -cost). Finally, we learn about graph compression beyond trees, e.g. we study sparse subgraphs of the original network that preserve the underlying properties.

Throughout this part, we make extensive use of the Multiplicative Weights Update Framework, which is a popular first-order optimization method very similar to gradient descent.

Techniques: Tree Embeddings, Multiplicative Weights Update Framework, Oblivious Routings, Cuts, Single-Commodity Flows, Multi-Commodity Flows, Expander Decomposition.

Lectures are accompanied by exercises and two graded homeworks to develop and advance algorithmic problem solving skills.

Additional Materials. Lectures are often accompanied by additional material on the same topic given on the course webpage. Our course is similar to various other recent courses, and lectures are partially inspired by the Stanford course [The Modern Algorithmic Toolbox](#) and the CMU course [Advanced Algorithms](#). These courses also cover additional topics for curious students. The part 'Algorithms for Big Data' is loosely designed as a formal and rigorous study of many of the algorithms encountered in the ETH engineering course [Big Data](#).

Our course is also a good starting point for exploring various topics more deeply over the course of your studies at ETH Zurich. The ETH courses [Linear & Combinatorial Optimization](#) and [Network & Integer Optimization](#) are a natural choice to learn more about the topics covered in 'Approximation Algorithms' and classic graph algorithms. The lecture [Approximations- und Online-Algorithmen](#) had substantial overlap with the first part of this course, but will be redesigned to be complementary and may possibly be merged with [Algorithmik für schwere Probleme](#). Many of the topics are also studied in the

theory of distributed systems explored in the course [Principles of Distributed Computing](#).

The part 'Algorithms for Big Data' touches upon various topics that are studied extensively in other theory courses:

- The topic of data science is studied extensively in the ETH course [Algorithmic Foundations of Data Science](#) .
- The course [Optimization For Data Science](#) focuses on convex problems and gradient based optimization methods.
- A rigorous introduction to machine learning can be found in the ETH course [Introduction to Machine Learning](#).

The interplay between combinatorial graph algorithms and optimization introduced in 'Modern Graph Algorithms' is covered in much more detail in the ETH course [Advanced Algorithms and Optimization](#).

Notation and useful inequalities

Commonly used notation

- $\mathcal{P}$: class of decision problems that can be solved on a deterministic sequential machine in polynomial time with respect to the input size
- $\mathcal{NP}$: class of decision problems that can be solved non-deterministically in polynomial time with respect to input size. That is, decision problems for which the “yes” instances have a proof that can be verified in polynomial time.
- $\mathcal{A}$: usually denotes the algorithm we are discussing about
- $\mathcal{I}$: usually denotes a problem instance
- ind.: independent / independently
- w.p.: with probability
- w.h.p: with high probability
We say that event X holds *with high probability* (w.h.p.) if

$$\Pr[X] \geq 1 - \frac{1}{poly(n)}$$

say, $\Pr[X] \geq 1 - \frac{1}{n^c}$ for some constant $c \geq 2$.

- L.o.E.: linearity of expectation
- u.a.r.: uniformly at random
- Integer range $[n] = \{1, \dots, n\}$
- $e \approx 2.718281828459$: the base of the natural logarithm

Useful distributions

Bernoulli Coin flip w.p. p . Useful for indicators

$$\Pr[X = 1] = p$$

$$\mathbb{E}[X] = p$$

$$\text{Var}(X) = p(1 - p)$$

Binomial Number of successes from n trials, each succeeding w.p. p ; Sample *with replacement* of n items, k of which are successes

$$\Pr[X = k] = \binom{n}{k} p^k (1 - p)^{n-k}$$

$$\mathbb{E}[X] = np$$

$$\text{Var}(X) = np(1 - p) \leq np$$

Geometric Number of Bernoulli trials until one success

$$\Pr[X = k] = (1 - p)^{k-1} p$$

$$\mathbb{E}[X] = \frac{1}{p}$$

$$\text{Var}(X) = \frac{1 - p}{p^2}$$

Hypergeometric Number of successes in n draws *without replacement*, from a population of N items in which K are successful:

$$\Pr[X = k] = \frac{\binom{K}{k} \binom{N-K}{n-k}}{\binom{N}{n}}$$

$$\mathbb{E}[X] = n \cdot \frac{K}{N}$$

$$\text{Var}(X) = n \cdot \frac{K}{N} \cdot \frac{N-K}{N} \cdot \frac{N-n}{N-1}$$

Exponential Parameter: λ ; Written as $X \sim \text{Exp}(\lambda)$

$$f_X(x) = \begin{cases} \lambda e^{-\lambda x} & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$$

$$\mathbb{E}[X] = \frac{1}{\lambda}$$

$$\text{Var}(X) = \frac{1}{\lambda^2}$$

Remark If $x_1 \sim \text{Exp}(\lambda_1), \dots, x_n \sim \text{Exp}(\lambda_n)$ are independent, then

- $\min\{x_1, \dots, x_n\} \sim \text{Exp}(\lambda_1 + \dots + \lambda_n)$
- $\Pr[\min\{x_1, \dots, x_n\} = x_k] = \frac{\lambda_k}{\lambda_1 + \dots + \lambda_n}$

Useful inequalities

- $\left(\frac{n}{k}\right)^k \leq \binom{n}{k} \leq \left(\frac{en}{k}\right)^k$
- $\binom{n}{k} \leq n^k$
- $\lim_{n \rightarrow \infty} \left(1 - \frac{1}{n}\right)^n = e^{-1}$
- $\sum_{i=1}^{\infty} \frac{1}{i^2} = \frac{\pi^2}{6}$
- $(1 - x) \leq e^{-x}$, for any x
- $(1 + 2x) \geq e^x$, for $x \in [0, 1]$
- $(1 + \frac{x}{2}) \geq e^x$, for $x \in [-1, 0]$
- $(1 - x) \geq e^{-x-x^2}$, for $x \in (0, \frac{1}{2})$
- $\frac{1}{1-x} \leq 1 + 2x$ for $0 \leq x \leq \frac{1}{2}$

Theorem (Linearity of Expectation).

$$\mathbb{E}\left(\sum_{i=1}^n a_i X_i\right) = \sum_{i=1}^n a_i \mathbb{E}(X_i)$$

Theorem (Variance).

$$\mathbb{V}(X) = \mathbb{E}(X^2) - \mathbb{E}(X)^2$$

Theorem (Variance of a Sum of Random Variables).

$$\mathbb{V}(aX + bY) = a^2\mathbb{V}(X) + b^2\mathbb{V}(Y) + 2ab\text{Cov}(X, Y)$$

Theorem (AM-GM inequality). *Given n non-negative numbers $x_1, \dots, x_n$,*

$$\frac{x_1 + \dots + x_n}{n} \geq (x_1 * \dots * x_n)^{1/n}$$

The equality holds if and only if $x_1 = \dots = x_n$.

Theorem (Markov's inequality). *If X is a nonnegative random variable and $a > 0$, then*

$$\Pr[X \geq a] \leq \frac{\mathbb{E}(X)}{a}$$

Theorem (Chebyshev's inequality). *If X is a random variable (with finite expected value μ and non-zero variance σ^2), then for any $k > 0$,*

$$\Pr[|X - \mu| \geq k\sigma] \leq \frac{1}{k^2}$$

Theorem (Bernoulli's inequality). *For every integer $r \geq 0$ and every real number $x \geq -1$,*

$$(1 + x)^r \geq 1 + rx$$

Theorem (Chernoff bound). *For independent Bernoulli variables $X_1, \dots, X_n$, let $X = \sum_{i=1}^n X_i$. Then*

$$\begin{aligned} \Pr[X \geq (1 + \epsilon) \cdot \mathbb{E}(X)] &\leq \exp\left(\frac{-\min\{\epsilon, \epsilon^2\} \mathbb{E}(X)}{3}\right) && \text{for } 0 < \epsilon \\ \Pr[X \leq (1 - \epsilon) \cdot \mathbb{E}(X)] &\leq \exp\left(\frac{-\epsilon^2 \mathbb{E}(X)}{2}\right) && \text{for } 0 < \epsilon < 1 \end{aligned}$$

By union bound, for $0 < \epsilon < 1$, we have

$$\Pr[|X - \mathbb{E}(X)| \geq \epsilon \cdot \mathbb{E}(X)] \leq 2 \exp\left(\frac{-\epsilon^2 \mathbb{E}(X)}{3}\right)$$

Remark 1 There is actually a tighter form of Chernoff bounds:

$$\forall \epsilon > 0, \Pr[X \geq (1 + \epsilon) \mathbb{E}(X)] \leq \left(\frac{e^\epsilon}{(1 + \epsilon)^{1+\epsilon}}\right)^{\mathbb{E}(X)}$$

This form is more complicated, but is especially better in the case of large $\epsilon > 1$. Both the basic and the tighter form also hold more generally for independent random variables X_i with arbitrary distributions (not necessarily identical) in $[0, 1]$.

Remark 2 We usually apply Chernoff bound to show that the probability of bad approximation is low by picking parameters such that $2 \exp(\frac{-\epsilon^2 \mathbb{E}(X)}{3}) \leq \delta$, then negating to get $\Pr[|X - \mathbb{E}(X)| \leq \epsilon \cdot \mathbb{E}(X)] \geq 1 - \delta$.

Theorem (Hoeffding's Inequality). *For independent random variables $X_1, \dots, X_n \in [0, 1]$, let $X = \sum_{i=1}^n X_i$. Then*

$$\Pr[|X - \mathbb{E}(X)| > t] < 2e^{-2t^2/n}$$

Theorem (Probabilistic Method). *Let $(\Omega, \mathcal{A}, P)$ be a probability space. Then, for $E \in \mathcal{A}$*

$$\Pr[E] > 0 \implies E \neq \emptyset$$

Combinatorics taking k elements out of n :

- no repetition, no ordering: $\binom{n}{k}$
- no repetition, ordering: $\frac{n!}{(n-k)!}$
- repetition, no ordering: $\binom{n+k-1}{k}$
- repetition, ordering: n^k

Part I

Approximation Algorithms

Many fundamental optimization problems we face in computer science, such as Set Cover, Knapsack, or Bin Packing are NP-hard. This implies that unless $P=NP$, we cannot find efficient polynomial-time algorithms that solve these problems exactly for all possible inputs. However, such problems are very relevant in practice, so we do not want to just give up and abandon such problems completely. An alternative is to relax our goal. Instead of searching for the perfect solution, we can design an approximation algorithm. This is an efficient algorithm that, while not guaranteed to find the optimal solution, is proven to return a solution that is provably "close" to the optimum. This closeness is quantified by an *approximation ratio*, which provides a formal guarantee on the quality of the returned solution.

In the first part of the course, we will discuss various methods for obtaining approximation algorithms. In particular, we will see greedy algorithms, discuss various ways of rounding a problem, and learn powerful techniques based on relaxing an integer linear program (ILP) into a linear program (LP).

Chapter 1

Greedy algorithms

Unless $\mathcal{P} = \mathcal{NP}$, we do not expect efficient algorithms for $\mathcal{NP}$ -hard problems. However, we are often able to design efficient algorithms that give solutions that are provably close/approximate to the optimum.

Definition 1.1 (α -approximation). *An algorithm $\mathcal{A}$ is an α -approximation algorithm for a minimization problem with respect to cost metric c if for any problem instance I and for some optimum solution OPT ,*

$$c(\mathcal{A}(I)) \leq \alpha \cdot c(OPT(I))$$

Maximization problems are defined similarly with $c(OPT(I)) \leq \alpha \cdot c(\mathcal{A}(I))$.

1.1 Minimum set cover

Consider a universe $\mathcal{U} = \{e_1, \dots, e_n\}$ of n elements, a collection of subsets $\mathcal{S} = \{S_1, \dots, S_m\}$ of m subsets of $\mathcal{U}$ such that $\mathcal{U} = \bigcup_{i=1}^m S_i$, and a non-negative¹ cost function $c : \mathcal{S} \rightarrow \mathbb{R}^+$. If $S_i = \{e_1, e_2, e_5\}$, then we say S_i *covers* elements e_1 , e_2 , and e_5 . For any subset $T \subseteq \mathcal{S}$, define the cost of T as the cost of all subsets in T . That is,

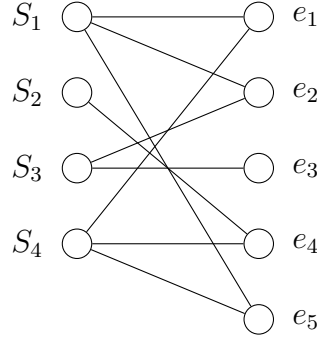
$$c(T) = \sum_{S_i \in T} c(S_i)$$

Definition 1.2 (Minimum set cover problem). *Given a universe of elements $\mathcal{U}$, a collection of subsets $\mathcal{S}$, and a non-negative cost function $c : \mathcal{S} \rightarrow \mathbb{R}^+$, find a subset $S^* \subseteq \mathcal{S}$ such that:*

- (i) S^* is a set cover: $\bigcup_{S_i \in S^*} S_i = \mathcal{U}$
- (ii) $c(S^*)$, the cost of S^* , is minimized

⁰ Chapter by Mohsen Ghaffari, adapted by Johannes Lengler

¹ If a set costs 0, then we can just remove all the elements covered by it for free.

Example

Suppose there are 5 elements e_1, e_2, e_3, e_4, e_5 , 4 subsets S_1, S_2, S_3, S_4 , and the cost function is defined as $c(S_i) = i^2$. Even though $S_3 \cup S_4$ covers all vertices, this costs $c(\{S_3, S_4\}) = c(S_3) + c(S_4) = 9 + 16 = 25$. One can verify that the minimum set cover is $S^* = \{S_1, S_2, S_3\}$ with a cost of $c(S^*) = 14$. Notice that we want a minimum cover with respect to c and not the number of subsets chosen from $\mathcal{S}$ (unless c is uniform cost).

1.1.1 A greedy minimum set cover algorithm

Since finding the minimum set cover is $\mathcal{NP}$ -complete, we are interested in algorithms that give a good approximation for the optimum. [Joh74] described a greedy algorithm *GreedySetCover* and proved that it gives an H_n -approximation². The intuition is as follows: Spread the cost $c(S_i)$ evenly among the vertices that are covered by S_i . We denote the resulting price-per-item by $\text{ppi}(S_i) = \frac{c(S_i)}{|S_i|}$, and we greedily select the set that has the lowest ppi. Then we remove all elements of this set from the universe and cover the rest recursively.

Algorithm 1 *GreedySetCover*($\mathcal{U}, \mathcal{S}, c$)

```

if  $\mathcal{U} = \emptyset$  then then return  $\emptyset$  ▷ Trivial case
else
     $S_i \leftarrow \arg \min_{S_i \in \mathcal{S}} \frac{c(S_i)}{|S_i|}$  ▷ Pick set with lowest price-per-item
     $\mathcal{U}' \leftarrow \mathcal{U} \setminus S_i$  ▷ Remove items in  $S_i$  from universe
     $\mathcal{S}' \leftarrow \{S \setminus S_i : S \in \mathcal{S}\}$  ▷ Remove items in  $S_i$  from sets
    return  $\{S_i\} \cup \text{GreedySetCover}(\mathcal{U}', \mathcal{S}', c)$  ▷ Solve recursively
end if

```

² $H_n = \sum_{i=1}^n \frac{1}{i} \approx \ln(n) + \gamma \leq \ln(n) + 0.6 \in \mathcal{O}(\log(n))$, where γ is the Euler-Mascheroni constant. See https://en.wikipedia.org/wiki/Euler-Mascheroni_constant.

Note that when we remove elements, the sets S_i might change, but we assume that the costs $c(S_i)$ remain the same. Additionally, it may happen that some of the S_j become empty, including the minimizing set S_i . In this case, we interpret the price-per-item $\text{ppi}(S_j)$ as infinite, so the set S_j will never be chosen (again). We may also just remove such sets explicitly. Similarly, it may happen that two sets S_j, S_k become equal after removing some elements. We could just remove the more expensive one in this case, but this is not necessary for the correctness of the algorithm.

Consider a run of *GreedySetCover* on the earlier example. In the first iteration, $\text{ppi}(S_1) = 1/3$, $\text{ppi}(S_2) = 4$, $\text{ppi}(S_3) = 9/2$, $\text{ppi}(S_4) = 16/3$. So, S_1 is chosen. In the second iteration, $\mathcal{U}' = \{e_3, e_4\}$, $\text{ppi}(S_1) = \infty$, $\text{ppi}(S_2) = 4$, $\text{ppi}(S_3) = 9$, $\text{ppi}(S_4) = 16$. So, S_2 is chosen. In the third iteration, $\mathcal{U}' = \{e_3\}$, $\text{ppi}(S_1) = \infty$, $\text{ppi}(S_2) = \infty$, $\text{ppi}(S_3) = 9$, $\text{ppi}(S_4) = \infty$. So, S_3 is chosen. Since all vertices are now covered, the algorithm terminates (coincidentally to the minimum set cover). Notice that ppi changes in each iteration according to which vertices remain uncovered.

Theorem 1.3. *GreedySetCover is an H_n -approximation algorithm.*

Proof. By construction, if $\mathcal{U} = \bigcup_{S_i \in \mathcal{S}} S_i$ then *GreedySetCover* terminates with a valid set cover T . It remains to show that $c(T) \leq H_n \cdot c(\text{OPT})$ for any minimum set cover OPT . We will prove this by induction over $n = |\mathcal{U}|$, where the base case $n = 0$ is trivial.

For the inductive step, let S_i be the set that *GreedySetCover* chooses in the first iteration, and let $\text{OPT} = \{O_1, \dots, O_p\}$ be an optimal solution. We claim that there is a $j \in [p]$ such that $\text{ppi}(O_j) \leq \frac{c(\text{OPT})}{n}$. Note that this implies $\text{ppi}(S_i) \leq \frac{c(\text{OPT})}{n}$, since S_i is the set that minimizes the ppi . To see the claim, assume for contradiction that $\forall j \in [p] : \text{ppi}(O_j) > \frac{c(\text{OPT})}{n}$. Then

$$\begin{aligned}
c(\text{OPT}) &= \sum_{j=1}^p c(O_j) = \sum_{j=1}^p \text{ppi}(O_j) \cdot |O_j| && \text{definition of ppi} \\
&> \sum_{j=1}^p \frac{c(\text{OPT})}{n} \cdot |O_j| && \text{by assumption} \\
&= \frac{c(\text{OPT})}{n} \sum_{j=1}^p |O_j| \\
&\geq \frac{c(\text{OPT})}{n} \cdot n && \text{OPT covers } \mathcal{U} \\
&= c(\text{OPT}).
\end{aligned}$$

Since this is a contradiction, we have indeed shown the claim and hence $\text{ppi}(S_i) \leq \frac{c(OPT)}{n}$.

To finish the induction, denote $k := |S_i|$. The remaining problem has universe $\mathcal{U}'$ of size $|\mathcal{U}'| = n - k$, and an optimal solution OPT' satisfies $c(OPT') \leq c(OPT)$ because OPT is still a valid cover for $\mathcal{U}'$ (though probably not minimum). Hence, by induction hypothesis, *GreedySetCover* covers $\mathcal{U}'$ with cost at most $H_{n-k} \cdot c(OPT') \leq H_{n-k} \cdot c(OPT)$. Together, the solution T of *GreedySetCover* on the full problem $\mathcal{U}$ costs at most

$$\begin{aligned} c(T) &\leq c(S_i) + H_{n-k} \cdot c(OPT) \\ &= \text{ppi}(S_i)|S_i| + H_{n-k} \cdot c(OPT) && \text{definition of ppi} \\ &\leq \frac{c(OPT)}{n} \cdot |S_i| + H_{n-k} \cdot c(OPT) && \text{since } \text{ppi}(S_i) \leq \frac{c(OPT)}{n} \\ &= c(OPT) \cdot \left(\frac{k}{n} + H_{n-k} \right) \leq c(OPT) \cdot H_n, \end{aligned}$$

where the last step follows because $\frac{k}{n} = \sum_{i=n-k+1}^n \frac{1}{n} \leq \sum_{i=n-k+1}^n \frac{1}{i}$. $\square$

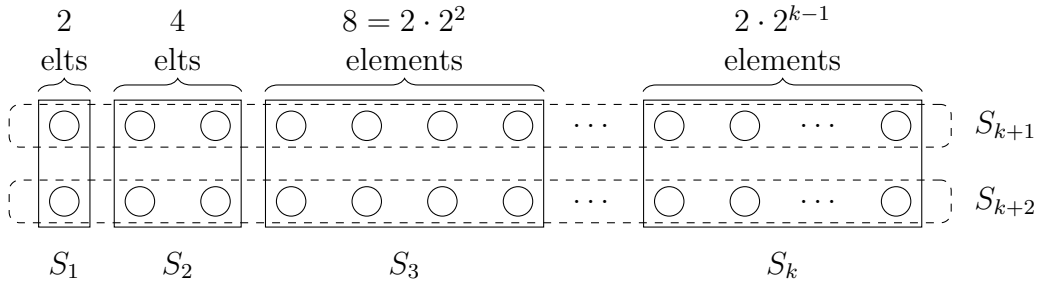
Remark Here is an alternative way to think about the proof. Whenever *GreedySetCover* picks a set $S_i = \{e_{i,1}, \dots, e_{i,k}\}$ then we say that each of the items $e_{i,j}$ for $j \in [k]$ pays $\text{price}(e_{i,j}) := \text{ppi}(S_i)$. Note that those payments cover exactly the cost of the set S_i because $c(S_i) = \text{ppi}(S_i) \cdot |S_i| = \sum_{j=1}^k \text{price}(e_{i,j})$. Hence, the total payment by all elements equals the cost of the solution. In the proof we have shown that when universe has size n , the next element(s) pay a price of at most $\frac{c(OPT)}{n}$.

Next we provide an example to show this bound is indeed tight.

Tight bound example for *GreedySetCover* Consider $n = 2 \cdot (2^k - 1)$ elements, for some $k \in \mathbb{N} \setminus \{0\}$. Partition the elements into groups of size $2 \cdot 2^0, 2 \cdot 2^1, 2 \cdot 2^2, \dots, 2 \cdot 2^{k-1}$. Let $\mathcal{S} = \{S_1, \dots, S_k, S_{k+1}, S_{k+2}\}$. For $1 \leq i \leq k$, let S_i cover the group of size $2 \cdot 2^{i-1} = 2^i$. Let S_{k+1} and S_{k+2} cover half of each group (i.e. $2^k - 1$ elements each) such that $S_{k+1} \cap S_{k+2} = \emptyset$.

Suppose $c(S_i) = 1, \forall i \in \{1, \dots, k+2\}$. The greedy algorithm will pick S_k , then $S_{k-1}, \dots$, and finally S_1 . This is because $2 \cdot 2^{k-1} > n/2$ and $2 \cdot 2^{i-1} > (n - \sum_{j=i+1}^k 2 \cdot 2^{j-1})/2$, for $1 \leq i \leq k-1$. This greedy set cover costs $k = \mathcal{O}(\log(n))$. Meanwhile, the minimum set cover is $S^* = \{S_{k+1}, S_{k+2}\}$ with a cost of 2.

A series of works by Lund and Yannakakis [LY94], Feige [Fei98], and Dinur and Steurer [DS14, Corollary 1.5] showed that it is $\mathcal{NP}$ -hard to always approximate set cover to within $(1 - \epsilon) \cdot \ln |\mathcal{U}|$, for any constant $\epsilon > 0$.



Theorem 1.4 ([DS14, Corollary 1.5]). *It is $\mathcal{NP}$ -hard to always approximate set cover to within $(1 - \epsilon) \cdot \ln |\mathcal{U}|$, for any constant $\epsilon > 0$.*

Proof. See [DS14, Corollary 1.5]

□

Chapter 2

Approximation schemes

In the last chapter, we described simple greedy algorithms that approximate the optimum for minimum set cover, maximal matching and minimum vertex cover within a constant factor of the optimum solution. We now want to devise algorithms which come arbitrarily close to the optimum solution. For that purpose we formalize the notion of efficient $(1 + \epsilon)$ -approximation algorithms for minimization problems, a la [Vaz13].

Let I be an instance from the problem of interest (e.g. minimum set cover). Denote $|I|$ as the size of the problem instance in bits, and $|I_u|$ as the size of the problem instance in unary. For example, if the input is a number x of at most n bits, then $|I| = \lfloor \log_2(x) \rfloor + 1 = \mathcal{O}(n)$ while $|I_u| = \mathcal{O}(2^n)$. This distinction of “size of input” will be important when we discuss the knapsack problem later.

Definition 2.1 (Polynomial time approximation scheme (PTAS)). *For a given cost metric c , an optimal algorithm OPT and a parameter ϵ , an algorithm $\mathcal{A}_\epsilon$ is a PTAS for a minimization problem if*

- $c(\mathcal{A}_\epsilon(I)) \leq (1 + \epsilon) \cdot c(OPT(I))$
- $\mathcal{A}_\epsilon$ runs in $\text{poly}(|I|)$ time

Note that ϵ is a parameter of the algorithm, and is not considered as input. Thus the runtime for PTAS may depend arbitrarily on ϵ . If we define ϵ as an input parameter for the algorithm, we can obtain a stricter definition, namely that of fully polynomial time approximation schemes (FPTAS). Assuming $\mathcal{P} \neq \mathcal{NP}$, FPTAS is the best one can hope for on $\mathcal{NP}$ -hard problems.

Definition 2.2 (Fully polynomial time approximation scheme (FPTAS)). *For a given cost metric c , an optimal algorithm OPT and input parameter ϵ , an algorithm $\mathcal{A}$ is an FPTAS for a minimization problem if*

⁰ Chapter by Mohsen Ghaffari, adapted by Johannes Lengler

- For any $\epsilon > 0$, $c(\mathcal{A}(I)) \leq (1 + \epsilon) \cdot c(OPT(I))$,
- $\mathcal{A}$ runs in $\text{poly}(|I|, \frac{1}{\epsilon})$ time.

For example, if an approximation scheme has runtime $O(n^{1/\epsilon})$, then it is a PTAS, but not a FPTAS.

One can define $(1 - \epsilon)$ -approximations, PTAS, and FPTAS for maximization problems similarly.

2.1 Knapsack

Definition 2.3 (Knapsack problem). *Consider a set $\mathcal{S}$ with n items. Each item i has size $s_i \in \mathbb{Z}^+$ and profit $p_i \in \mathbb{Z}^+$. Given a budget B , find a subset $S^* \subseteq \mathcal{S}$ such that:*

- (i) Selection S^* fits budget: $\sum_{i \in S^*} s_i \leq B$
- (ii) Selection S^* has maximum value: $\sum_{i \in S^*} p_i$ is maximized

Notice that any item which has $s_i > B$ cannot be chosen, due to the size constraint, and therefore we can ignore it. In $\mathcal{O}(n)$ time, we can remove any such item and relabel the remaining ones as items 1, 2, 3, ... Thus, without loss of generality, we can assume that $s_i \leq B, \forall i \in \{1, \dots, n\}$.

Let $p_{\max} = \max_{i \in \{1, \dots, n\}} p_i$ denote the highest profit for an item. Also, let $OPT = OPT(I) \subseteq \{1, \dots, n\}$ denote the optimal set of items for instance I , and denote its profit by $p(OPT)$. Then by the previous paragraph we have $p_{\max} \leq p(OPT)$ because we can always pick the highest valued item.

Example Denote the i -th item by $i : \langle s_i, p_i \rangle$. Consider an instance with $\mathcal{S} = \{1 : \langle 10, 130 \rangle, 2 : \langle 7, 103 \rangle, 3 : \langle 6, 91 \rangle, 4 : \langle 4, 40 \rangle, 5 : \langle 3, 38 \rangle\}$ and budget $B = 10$. Then, the best subset $S^* = \{2 : \langle 7, 103 \rangle, 5 : \langle 3, 38 \rangle\} \subseteq \mathcal{S}$ yields a total profit of $103 + 38 = 141$.

2.1.1 An exact algorithm via dynamic programming

The maximum achievable profit is at most $np_{\max}$, as we can have at most n items, each having profit at most $p_{\max}$. Define the size of a subset as the sum of the size of the sets involved. Using dynamic programming (DP), we can form an n -by- $(np_{\max})$ matrix M where $M[i, p]$ is the smallest size of a subset chosen from $\{1, \dots, i\}$ such that the total profit is at least p . To handle boundaries, define $M[0, p] = \infty$ for $p > 0$ and set $M[i, p] = 0$ for all $p \leq 0$

and $i \geq 0$. Then, in order to compute $M[i, p]$ we may either use item i or not, which yields the following recursion:

$$M[i, p] = \min\{M[i - 1, p], s_i + M[i - 1, p - p_i]\}.$$

Since each cell can be computed in $\mathcal{O}(1)$ using DP via the above recurrence, matrix M can be filled in time $\mathcal{O}(n^2 p_{\max})$ and S^* may be extracted from the table $M[., .]$ as follows: we find the maximum value $p \in [p_{\max}, np_{\max}]$ for which $M[n, p] \leq B$ and we back-track from there to extract the optimal set S^* .

Remark This dynamic programming algorithm is *not* a PTAS because $\mathcal{O}(n^2 p_{\max})$ can be exponential in input problem size $|I|$. Namely the number $p_{\max}$ which appears in the runtime is encoded by $\log_2(p_{\max})$ bits in the input, so the actual value $p_{\max}$ can be exponential in the input size. In such a case, we say that this Dynamic Programming provides a *pseudo-polynomial time algorithm*.

2.1.2 FPTAS via profit rounding

Algorithm 2 *FPTAS-Knapsack*($\mathcal{S}, B, \epsilon$)

$k \leftarrow \max\{1, \lfloor \frac{\epsilon}{n} p_{\max} \rfloor\}$ $\triangleright$ Choice of k to be justified later

for $i \in \{1, \dots, n\}$ **do**

$p'_i = k \cdot \lfloor \frac{p_i}{k} \rfloor$ $\triangleright$ Round down the profits

end for

Run DP in Section 2.1.1 with budget B , sizes s_i , and rounded profits p'_i .
Fill out the table only for profits that are multiples of k .

return Items selected by DP

Theorem 2.4. *FPTAS-Knapsack is a FPTAS for the knapsack problem.*

Proof. Since *FPTAS-Knapsack* only needs to fill out every k -th column, it runs in time $\mathcal{O}(\frac{n^2 p_{\max}}{k}) = \mathcal{O}(\frac{n^3}{\epsilon}) \in \text{poly}(n, \frac{1}{\epsilon})$. It remains to show that it gives a $(1 - \epsilon)$ -approximation. For $k = 1$ we solve the original problem, so let us assume that $k = \lfloor \frac{\epsilon}{n} p_{\max} \rfloor$.

Suppose we are given a knapsack instance $I = (\mathcal{S}, B)$. We denote by OPT the (unknown) optimal solution (set of items) for I , and by OPT' the optimal solution of the rounded problem that *FPTAS-Knapsack* returns. For any solution S , we denote by $p(S)$ and $'(S)$ the profits of S in the original and the rounded problem, respectively.

Let $\text{loss}(i) \geq 0$ be the decrease in value by using rounded p'_i for item i . By the profit rounding definition, for each item i ,

$$\text{loss}(i) = p_i - k \cdot \lfloor \frac{p_i}{k} \rfloor \leq k.$$

For any solution S we write $\text{loss}(S) := p(S) - p'(S)$. Then

$$\begin{aligned} \text{loss}(S) &= \sum_{i \in S} \text{loss}(i) \leq \sum_{i=1}^n \text{loss}(i) \\ &\leq nk && \text{loss}(i) \leq k \text{ for any item } i \\ &\leq \epsilon \cdot p_{\max} && \text{since } k = \lfloor \frac{\epsilon}{n} p_{\max} \rfloor \\ &\leq \epsilon \cdot p(\text{OPT}) && \text{since } p_{\max} \leq p(\text{OPT}) \end{aligned}$$

Crucially, we only changed the profits, not the sizes. Hence, every solution to instance I remains a solution after rounding, i.e., the budget requirement is still satisfied. In particular, OPT is still a solution to the rounded problem, and it achieves profit at least

$$p'(\text{OPT}) = p(\text{OPT}) - \text{loss}(\text{OPT}) \geq (1 - \epsilon) \cdot p(\text{OPT}).$$

Since OPT' is the optimal solution for the rounded problem, it has at least as much rounded profit as any other solution, in particular at least as much as OPT . Hence,

$$p'(\text{OPT}') \geq p'(\text{OPT}) \geq (1 - \epsilon) \cdot p(\text{OPT}).$$

Finally, since we rounded down,

$$p(\text{OPT}') \geq p'(\text{OPT}') \geq (1 - \epsilon) \cdot p(\text{OPT}).$$

Since OPT' is the output of *FPTAS-Knapsack* for instance I and $p(\text{OPT}')$ is the profit for that instance, we have shown that *FPTAS-Knapsack* achieves a $(1 - \epsilon)$ -approximation for the knapsack problem. $\square$

Example Recall the earlier example where budget $B = 10$ and $\mathcal{S} = \{1 : \langle 10, 130 \rangle, 2 : \langle 7, 103 \rangle, 3 : \langle 6, 91 \rangle, 4 : \langle 4, 40 \rangle, 5 : \langle 3, 38 \rangle\}$. For $\epsilon = \frac{1}{2}$, one would set $k = \max\{1, \lfloor \frac{\epsilon p_{\max}}{n} \rfloor\} = \max\{1, \lfloor \frac{130/2}{5} \rfloor\} = 13$. After rounding, we have $\mathcal{S}' = \{1 : \langle 10, 130 \rangle, 2 : \langle 7, 91 \rangle, 3 : \langle 6, 91 \rangle, 4 : \langle 4, 39 \rangle, 5 : \langle 3, 26 \rangle\}$. The optimum subset from $\mathcal{S}'$ is $\text{OPT}' = \{3 : \langle 6, 91 \rangle, 4 : \langle 4, 39 \rangle\}$ which translates to a total profit of $p(\text{OPT}') = 91 + 40 = 131$ in the original problem. As expected, $131 = p(\text{OPT}') \geq (1 - \frac{1}{2}) \cdot p(\text{OPT}) = 70.5$. Note that the solution OPT' has a small loss of 1, while the optimal solution $\text{OPT} = \{2 : \langle 7, 103 \rangle, 5 : \langle 3, 38 \rangle\}$ has a much larger loss of 24. This explains why OPT is no longer optimal after rounding.

remark It is also easy to write an exact DP for the knapsack problem by rounding the sizes instead of the profits. This runs in time $O(n^2B)$, so it is pseudo-polynomial in the budget instead of the profits. Why did we round the profits instead of the sizes?

One answer is that rounding profits keeps feasible sets feasible, so in particular our output OPT' is feasible. If we rounded the sizes, it would not be clear what to return if OPT' is not feasible.

A second, more systematic answer looks at the profit of an optimal solution as a function of the inputs p_i . I.e., we fix B and the sizes s_i , and consider the function $p_{OPT}(p_1, \dots, p_n)$, which gives the profit of the optimal solution $p(OPT(I))$ for the instance I with budget B , sizes s_i and profits p_i . It is easy to see that this function is *continuous* in the profits: changing a single profit by ϵ changes the profit of the optimal solution by at most ϵ .¹ Hence rounding the profits by a small amount will change the optimal profit in a very controlled way, as we quantified with the notion of **loss** in the proof.

On the other hand, we can also regard the profit of the optimal solution as a function of the *sizes*, i.e., consider a function $p_{OPT}(s_1, \dots, s_n)$ for fixed budget B and profits p_i . This function is not continuous: changing a single size s_i by ϵ can change the profit of an optimal solution by an arbitrary amount, for example because item i no longer fits into the budget, and the profit from the other items is much smaller. Thus even if we round the sizes by a little amount, we have no guarantee about the solution quality.

2.2 Bin packing

Definition 2.5 (Bin packing problem). *Given a set $\mathcal{S}$ with n items where each item i has $\text{size}(i) \in (0, 1]$, find the minimum number of unit-sized bins (i.e. bins of size 1) that can hold all n items.*

For any problem instance I , let $OPT(I)$ be an optimal bin assignment and $|OPT(I)|$ be the corresponding minimum number of bins required. One can see that $\text{size}(I) := \sum_{i=1}^n \text{size}(i) \leq |OPT(I)|$.

Example Consider $\mathcal{S} = \{0.5, 0.1, 0.1, 0.1, 0.5, 0.4, 0.5, 0.4, 0.4\}$, where $|\mathcal{S}| = n = 9$. Since $\text{size}(\mathcal{S}) = \sum_{i=1}^n \text{size}(i) = 3$, at least 3 bins are needed. One can verify that 3 bins suffice: $b_1 = b_2 = b_3 = \{0.5, 0.4, 0.1\}$. Hence, $|OPT(\mathcal{S})| = 3$.

¹This is actually stronger than just being continuous. It is called Lipschitz-continuous with Lipschitz constant one.

0.1	0.1	0.1
0.4	0.4	0.4
0.5	0.5	0.5
b_1	b_2	b_3

2.2.1 First-fit: A 2-approximation algorithm

FirstFit processes items one-by-one, creating new bins if an item cannot fit into one of the existing bins. For a unit-sized bin b , we use $\text{size}(b)$ to denote the sum of the **size** of items that are put into b , and define $\text{free}(b) = 1 - \text{size}(b)$.

Algorithm 3 *FirstFit*($\mathcal{S}$)

```

 $B \rightarrow \emptyset$  ▷ Collection of bins
for  $i \in \{1, \dots, n\}$  do
  if  $\text{size}(i) \leq \text{free}(b)$  for some bin  $b \in B$ . then
    Pick the smallest such  $b$ .
     $\text{free}(b) \leftarrow \text{free}(b) - \text{size}(i)$  ▷ Put item  $i$  into existing bin  $b$ 
  else
     $B \leftarrow B \cup \{b'\}$  ▷ Put item  $i$  into a fresh bin  $b'$ 
     $\text{free}(b') = 1 - \text{size}(i)$ 
  end if
end for
return  $B$ 

```

Lemma 2.6. *Using FirstFit, at most one bin is less than half-full. That is, $|\{b \in B : \text{size}(b) \leq \frac{1}{2}\}| \leq 1$, where B is the output of FirstFit.*

Proof. Suppose, for contradiction, that there are two bins b_i and b_j such that $i < j$, $\text{size}(b_i) \leq \frac{1}{2}$ and $\text{size}(b_j) \leq \frac{1}{2}$. Then, *FirstFit* could have put all items in b_j into b_i , and would not have created b_j . This is a contradiction. $\square$

Theorem 2.7. *FirstFit is a 2-approximation algorithm for bin packing.*

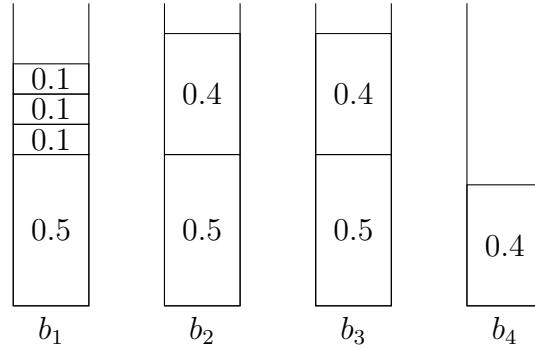
Proof. Suppose *FirstFit* terminates with $|B| = m$ bins. By Lemma 2.6, $\sum_{i=1}^n \text{size}(i) > \frac{m-1}{2}$, as $m-1$ bins are at least half-full. Since $\sum_{i=1}^n \text{size}(i) \leq$

$|OPT(I)|$, we have

$$m - 1 < 2 \cdot \sum_{i=1}^n \text{size}(i) \leq 2 \cdot |OPT(I)|$$

That is, $m \leq 2 \cdot |OPT(I)|$ since both m and $|OPT(I)|$ are integers. $\square$

Recall example with $\mathcal{S} = \{0.5, 0.1, 0.1, 0.1, 0.5, 0.4, 0.5, 0.4, 0.4\}$. *FirstFit* will use 4 bins: $b_1 = \{0.5, 0.1, 0.1, 0.1\}$, $b_2 = b_3 = \{0.5, 0.4\}$, $b_4 = \{0.4\}$. As expected, $4 = |FirstFit(\mathcal{S})| \leq 2 \cdot |OPT(\mathcal{S})| = 6$.



We can improve the approximation ratio by first sorting the items before applying *FirstFit*, which yields the following algorithm.

Algorithm 4 *SortedFirstFit*($\mathcal{S}$)

Sort items in $\mathcal{S}$ decreasingly

return *FirstFit*($\mathcal{S}$)

Theorem 2.8. *SortedFirstFit is a $3/2$ -approximation algorithm for bin packing.*

Proof. Let the sizes be $s_1 \geq s_2 \geq \dots \geq s_n$, and assume that *SortedFirstFit* needs k bins $B_1, \dots, B_k$. Let r be the number of items that are larger than $1/2$, so $s_r > 1/2$ and $s_{r+1} \leq 1/2$. Note that all those r items need to be in different bins. Hence, if $r \geq \frac{2k}{3}$ then $|OPT(S)| \geq r \geq \frac{2k}{3}$, or equivalently $k \leq \frac{3}{2}|OPT(S)|$. In this case we are immediately done.

Hence, assume that $r < \frac{2k}{3}$. Let us call the first $k_1 := \lceil \frac{2k}{3} \rceil - 1$ bins *early*, and the other $k_2 = k - k_1$ bins *late*. Then all items of size larger than $1/2$ fit into early bins, so all items in late bins have size at most $1/2$. In particular, every late bin must contain at least two items, except possibly for the very last bin. Hence, there are at least $2(k_2 - 1) + 1 = 2k - 2\lceil \frac{2k}{3} \rceil + 1 > k_1 - 1$

items in the late bins. Since both sides are integers, the number of items in the late bins is at least k_1 .

Now take k_1 items from the late bins and pair them up with the k_1 early bins. (With the bins themselves, not with items in the early bins.) For any such pair (i, b) of item i and bin b , the item i was not put into bin b by the algorithm, so item i did not fit into b , or in formula $s_i + \text{size}(b) > 1$. Since we have k_1 such pairs, the total size of the problem is at least $\text{size}(S) > k_1$, and hence $|OPT(S)| \geq \text{size}(S) > k_1 = \lceil \frac{2k}{3} \rceil - 1$. Since $|OPT(S)|$ is an integer it follows that $|OPT(S)| \geq \lceil \frac{2k}{3} \rceil \geq \frac{2k}{3}$. $\square$

It is natural to wonder whether we can do better than a $\frac{3}{2}$ -approximation. Unfortunately, unless $\mathcal{P} = \mathcal{NP}$, we cannot do so efficiently. To do that, we would need to be able to discriminate between bin packing instances that fit into two bins and instances that do not fit into two bins. To do that, we would need to solve the partition problem (which is $\mathcal{NP}$ -hard) efficiently.

Definition 2.9 (Partition problem). *Given a multiset $\mathcal{S}$ of (possibly repeated) positive integers $x_1, \dots, x_n$, is there a way to partition $\mathcal{S}$ into $\mathcal{S}_1$ and $\mathcal{S}_2$ such that $\sum_{x \in \mathcal{S}_1} x = \sum_{x \in \mathcal{S}_2} x$?*

Theorem 2.10. *It is $\mathcal{NP}$ -hard to solve bin packing with an approximation factor better than $\frac{3}{2}$.*

Proof. Suppose some polytime algorithm $\mathcal{A}$ solves bin packing with a $(\frac{3}{2} - \epsilon)$ -approximation for $\epsilon > 0$. Given an instance of the partition problem with $\mathcal{S} = \{x_1, \dots, x_n\}$, let $X = \sum_{i=1}^n x_i$. Assume that $x_i \leq X/2$ for all i , as otherwise a bipartition is trivially impossible. Define a bin packing instance $\mathcal{S}' = \{\frac{2x_1}{X}, \dots, \frac{2x_n}{X}\}$. Since $\sum_{x \in \mathcal{S}'} x = 2$, at least two bins are required. By construction, one can bipartition $\mathcal{S}$ if and only if only two bins are required to pack $\mathcal{S}'$. Since $\mathcal{A}$ gives a $(\frac{3}{2} - \epsilon)$ -approximation, if OPT on $\mathcal{S}'$ returns 2 bins, then $\mathcal{A}$ on $\mathcal{S}'$ will return also $\lfloor 2 \cdot (\frac{3}{2} - \epsilon) \rfloor = 2$ bins. Therefore, as $\mathcal{A}$ solves bin-packing with a $(\frac{3}{2} - \epsilon)$ -approximation in polytime, we would get an algorithm for solving the partition problem in polytime. Contradiction. $\square$

The above rules out the possibility of a proper PTAS for bin packing since a $(1 + \epsilon)$ -approximation of 2, for $\epsilon < 0.5$, would be strictly less than 3. Another way to view this negative result is to say that we need to allow the approximation algorithm to have at least an additive +1 loss, in comparison to the optimum. But we can still aim for an approximation that is within a $(1 + \epsilon)$ factor of optimum modulo an additive +1 error, i.e., achieving a number of bins that is at most $(1 + \epsilon)OPT + 1$.

In the following sections, we will show how to achieve this goal, with a runtime that is exponential in $\frac{1}{\epsilon}$. To do this, we first consider two simplifying

assumptions and design algorithms for this simplified case. Then, we see how to adapt the algorithm and remove these two assumptions.

2.2.2 Special case 1: Exact solving with $\mathcal{A}_\epsilon$

In this section, we make the following two assumptions:

Assumption (1) All items have at least size ϵ , for some $\epsilon > 0$.

Assumption (2) There are only k different possible sizes (k is a constant).

Define $M = \lceil \frac{1}{\epsilon} \rceil$. By assumption 1, there are at most M items in a bin. In addition, define $R = \binom{M+k}{M}$. By assumption 2, there are at most R item arrangements in one bin. Since at most n bins are needed, the total number of bin configurations is at most $\binom{n+R}{R} \leq (n+R)^R = \mathcal{O}(n^R)$. Since k and ϵ are constant, R is also constant and one can enumerate over all possible bin configurations (denote this algorithm as $\mathcal{A}_\epsilon$) to *exactly* solve bin packing, in this special case, in $\mathcal{O}(n^R) \in \text{poly}(n)$ time.

Remark 1 The number of configurations are computed by solving combinatorics problems of the following form: If x_i defines the number of items of the i^{th} possible size, how many non-negative integer solutions are there to $x_1 + \dots + x_k \leq M$? This type of problems can be solved by counting how many ways there are to put n indistinguishable balls into k distinguishable bins and is generally known under *stars and bars*.²

Remark 2 The number of bin configurations is computed out of n bins, which is a very crude upper bound for the number of bins. The bounds get better if one uses better bounds for the number of bins, but this upper bound suffices for our purposes.

2.2.3 Special case 2: PTAS

In this section, we remove the second assumption and require only:

Assumption (1) All items have at least size ϵ , for some $\epsilon > 0$.

Our goal is to reuse the exact algorithm $\mathcal{A}_\epsilon$ on a slightly modified problem instance J that satisfies both assumptions. For this, we sort the items by size and partition them (in sorted order) into k non-overlapping groups of

²See slides 22 and 23 of <http://www.cs.ucr.edu/~neal/2006/cs260/piyush.pdf> for illustration of $\binom{M+k}{M}$ and $\binom{n+R}{R}$.

$Q \leq \lceil n\epsilon^2 \rceil$ elements each. To obtain a constant number of different sizes, we round the sizes of all items in a group to the largest size of that group, resulting in at most k different item sizes. We can now call $\mathcal{A}_\epsilon$ on J to solve the modified instance exactly in polynomial time. Since J only *rounds up* sizes, $\mathcal{A}_\epsilon(J)$ will yield a satisfying bin assignment for instance I , with possibly “spare slack”. The entire procedure is described in *PTAS-BinPacking*.

Algorithm 5 *PTAS-BinPacking*($I = \mathcal{S}, \epsilon$)

```

 $k \leftarrow \lceil \frac{1}{\epsilon^2} \rceil$ 
 $Q \leftarrow \lceil n\epsilon^2 \rceil$ 
Sort  $n$  items by size
Partition items into  $k$  non-overlapping groups of consecutive items, each
with  $\leq Q$  items
for  $i \in \{1, \dots, k\}$  do
     $i_{max} \leftarrow \max_{\text{item } j \text{ in group } i} \text{size}(j)$ 
    for item  $j$  in group  $i$  do
         $\text{size}(j) \leftarrow i_{max}$ 
    end for
end for
Denote the modified instance as  $J$ 
return  $\mathcal{A}_\epsilon(J)$ 

```

It remains to show that the solution to the modified instance $OPT(J)$ yields a $(1 + \epsilon)$ -approximation of $OPT(I)$.

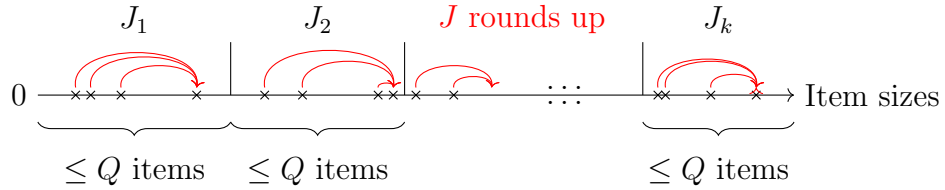


Figure 2.1: Sort and partition items into k groups, each with $\leq Q$ items; Label groups in ascending sizes; J rounds up item sizes.

Lemma 2.11. $|OPT(I)| \leq |OPT(J)| \leq |OPT(I)| + Q - 1$

Proof. The first inequality is trivial, since rounding up can only make it harder to pack items into bins. For the second inequality, denote the sizes of the items in I in decreasing order by $s_1 \geq \dots \geq s_n$.

We will pack J by taking the $Q - 1$ largest items in J and pack each of them into its own bin. Denote the instance of the remaining $n - Q + 1$

items by J' . We will argue that the instance J' is at most as hard as I . Assume that the items in J and J' are also sorted in descending order, and consider the j -th item in J' , for $j \in \{1, \dots, n - Q + 1\}$. The j -th item in J' is the $(j + Q - 1)$ -th item in J . It was obtained from the $(j + Q - 1)$ -th item in I by rounding up. The rounding procedure replaced the size s_{j+Q-1} by another size in the same group, so it could increase the item by at most $Q - 1$ ‘positions’ to size s_j . Hence, we have shown that the j -th item in J' is at most as large as the j -th item in I . Therefore, instance J' is at most as hard as I , and $|OPT(J')| \leq |OPT(I)|$. The lemma now follows from $|OPT(J)| \leq |OPT(J')| + Q - 1 \leq |OPT(I)| + Q - 1$. $\square$

Theorem 2.12. *PTAS-BinPacking is an $(1 + \epsilon)$ -approximation algorithm for bin packing with Assumption (1).*

Proof. By Assumption (1), all item sizes are at least ϵ , so $|OPT(I)| \geq n\epsilon$. Then, $Q = \lceil n\epsilon^2 \rceil \leq \epsilon \cdot |OPT(I)| + 1$ and the statement follows from Lemma 2.11. $\square$

2.2.4 General case

We now consider the general case where we do not make any assumptions on the problem instance I . First, we lower bound the minimum item size by putting aside all items with size smaller than $\min\{\frac{1}{2}, \frac{\epsilon}{2}\}$, thus allowing us to use *PTAS-BinPacking*. Then, we add back the small items in a greedy manner with *FirstFit* to complete the packing.

Algorithm 6 *General-PTAS-BinPacking*($I = \mathcal{S}, \epsilon$)

$\epsilon' \leftarrow \min\{\frac{1}{2}, \frac{\epsilon}{2}\}$	$\triangleright$ See analysis why we chose such an ϵ'
$X \leftarrow$ Items with size $< \epsilon'$	$\triangleright$ Ignore small items
$P \leftarrow \text{PTAS-BinPacking}(\mathcal{S} \setminus X, \epsilon')$	$\triangleright$ By Theorem 2.12,
	$\triangleright P = (1 + \epsilon') \cdot OPT(\mathcal{S} \setminus X) $
$P' \leftarrow$ Using <i>FirstFit</i> , add items in X to P	$\triangleright$ Handle small items
return Resultant packing P'	

Theorem 2.13. *General-PTAS-BinPacking uses $\leq (1 + \epsilon)|OPT(I)| + 1$ bins*

Proof. If *FirstFit* does not open a new bin, the theorem trivially holds. Suppose *FirstFit* opens a new bin (using m bins in total), then we know that at least $(m - 1)$ bins are strictly more than $(1 - \epsilon')$ -full.

$$\begin{aligned}
|OPT(I)| &\geq \sum_{i=1}^n \text{size}(i) && \text{Lower bound on } |OPT(I)| \\
&> (m-1)(1-\epsilon') && \text{From above observation}
\end{aligned}$$

Hence,

$$\begin{aligned}
m &< \frac{|OPT(I)|}{1-\epsilon'} + 1 && \text{Rearranging} \\
&\leq |OPT(I)| \cdot (1+2\epsilon') + 1 && \text{Since } \frac{1}{1-\epsilon'} \leq 1+2\epsilon', \text{ for } \epsilon' \leq \frac{1}{2} \\
&\leq (1+\epsilon) \cdot |OPT(I)| + 1 && \text{By choice of } \epsilon' = \min\{\frac{1}{2}, \frac{\epsilon}{2}\}
\end{aligned}$$

□

Remark Our PTAS has a very bad dependency on $1/\epsilon$, the runtime is doubly exponential in $1/\epsilon$. This is due to our brute-force solution when the number k of different item sizes is constant, in our case $k \approx 1/\epsilon^2$. This part of the algorithm can be improved so that the runtime is “only” exponential in $1/\epsilon^2$, up to logarithmic terms in the exponent. On the other hand, the runtime is proven to be exponential in $1/\epsilon$ (also ignoring logarithmic terms) unless the Exponential Time Hypothesis (ETH) fails [CJZ14].³ It is an open question whether the best exponent is closer to $1/\epsilon$ or $1/\epsilon^2$.

³The Exponential Time Hypothesis says that the SAT problem can not be solved in sub-exponential time. It is a stronger hypothesis than $\mathcal{P} \neq \mathcal{NP}$, which is equivalent to saying that SAT can not be solved in *polynomial* time. The ETH is widely believed in computer science and has a remarkable acronym, only beaten by the “*strong ETH*”, also widely believed, which asserts that SAT can not be solved in time $O(2^{(1-\epsilon)n})$ for any constant $\epsilon > 0$.

Chapter 3

Randomized approximation schemes

In this chapter, we study the class of algorithms which extends FPTAS by allowing randomization.

Definition 3.1 (Fully polynomial randomized approximation scheme (FPRAS)).

Assume that each input I maps to a positive real number $SOL(I)$ that the randomized algorithm $\mathcal{A}$ is supposed to approximate. Then a Polynomial-Time Randomized Approximation Scheme or PRAS is a set of algorithms $\mathcal{A} = \mathcal{A}_\epsilon$ for every $\epsilon > 0$ such that

1. For any $\epsilon > 0$, $\Pr[|\mathcal{A}_\epsilon(I) - SOL(I)| \leq \epsilon \cdot SOL(I)] \geq \frac{3}{4}$
2. For any $\epsilon > 0$, the algorithm $\mathcal{A}_\epsilon$ runs in expected time $\text{poly}(|I|)$ on input I .

If the runtime is $\text{poly}(|I|, \frac{1}{\epsilon})$ then we speak of a Fully Polynomial-Time Randomized Approximation Scheme or FPRAS.

The probability $\frac{3}{4}$ in Condition 1 is arbitrary as long as it is strictly larger than $1/2$.¹ We can easily amplify the success probability by invoking several independent copies of the algorithm $\mathcal{A}$ and then return the median. This is known as *probability amplification* or *median trick*. We will also return to the median trick in [Section 7.3.3](#). The following lemma asserts that this trick indeed boosts the probability.

⁰ Chapter by Mohsen Ghaffari, adapted by Johannes Lengler

¹Probabilities less than $1/2$ are usually not enough. Consider the situation that the correct answer is 1, and that we have an algorithm that output either 1 or 2, both with probability $1/2$. Then this algorithm satisfies Condition 1 with probability $1/2$ instead of $3/4$, but we can't use it to discriminate between the two potential solutions 1 and 2.

Lemma 3.2. *There is a $k = \mathcal{O}(\log \frac{1}{\delta})$ such that the median answer $MED = MED(\mathcal{A}_\epsilon, k, I)$ of k independent runs of $\mathcal{A}_\epsilon$ on I satisfies*

$$\Pr[|MED - SOL(I)| \leq \epsilon \cdot SOL(I)] \geq 1 - \delta.$$

Proof. Let us call $\mathcal{E}$ the event that $MED - SOL(I) > \epsilon \cdot SOL(I)$, or equivalently $MED > (1 + \epsilon) \cdot SOL(I)$. This is one of the two ways in which the distance between MED and $SOL(I)$ can be too large. We will show $\Pr[\mathcal{E}] \leq \delta/2$, and one can show analogously that the other way is also unlikely: $\Pr[SOL(I) - MED > \epsilon \cdot SOL(I)] \leq \delta/2$. Then a union bound over both ways will finish the proof.

We need to estimate how likely it is that the median is larger than $(1 + \epsilon)SOL(I)$. This happens if and only if² at least $k/2$ of the k runs of $\mathcal{A}_\epsilon$ yield a value larger than $(1 + \epsilon)SOL(I)$. Each run has probability at most $1/4$ to give this result, and the runs are independent. Hence, $\Pr[\mathcal{E}]$ is upper bounded by the probability that a binomial distribution with k trials and success probability $1/4$ yields at least $k/2$ successes. Let X be a $\text{Bin}(k, 1/4)$ random variable. Then by a Chernoff bound, this probability is at most

$$\Pr[\mathcal{E}] \leq \Pr[X \geq k/2] = \Pr[X \geq (1 + 1) \cdot \mathbb{E}(X)] \leq e^{-\mathbb{E}(X)/3} \leq e^{-k/12} \leq \delta/2,$$

where the last step holds if we choose $k \geq 12 \ln(\frac{2}{\delta}) = \mathcal{O}(\log(\frac{1}{\delta}))$. $\square$

Remark We assumed in Condition 2 that the *expected* runtime is $\text{poly}(|I|)$. We can strengthen this condition into a *deterministic* runtime bound without cost. Let the runtime of $\mathcal{A}_\epsilon$ on input I be T , and let $E := \mathbb{E}[T] = \text{poly}(|I|)$. Then by Markov's inequality we have $\Pr[T \geq 8E] \leq 1/8$. Now consider the truncated algorithm $\mathcal{A}_{\epsilon, \text{trunc}}$ that runs $\mathcal{A}_\epsilon$ for time $8E$. If $\mathcal{A}_\epsilon$ terminates in this time, then $\mathcal{A}_{\epsilon, \text{trunc}}$ returns the output of $\mathcal{A}_\epsilon$. Otherwise, it returns an arbitrary output like 0. The output of $\mathcal{A}_{\epsilon, \text{trunc}}$ is bad if either the output of $\mathcal{A}_\epsilon$ is bad or if $\mathcal{A}_\epsilon$ takes too long. Thus, by a union bound,

$$\begin{aligned} \Pr[|\mathcal{A}_{\epsilon, \text{trunc}}(I) - SOL(I)| > \epsilon \cdot SOL(I)] \\ &\leq \Pr[|\mathcal{A}_\epsilon(I) - SOL(I)| > \epsilon \cdot SOL(I)] + \Pr[T \geq 8E] \\ &\leq 1/4 + 1/8 \leq 3/8. \end{aligned}$$

Hence, the algorithm $\mathcal{A}_{\epsilon, \text{trunc}}$ is still a PRAS with slightly worse success probability $5/8$ (which we can boost via the median trick), but with a deterministic runtime bound of $8E$.

²If k is odd. For even k we would need to specify a tie-breaking rule.

Remark Sometimes we only have a *one-sided error*, so we have a guarantee that all outputs of the algorithm land on the same side of the optimum. For example, if we solve a minimization problem and each run of the algorithm gives us a different solution, then we know that the output of the algorithm is never smaller than the optimum. In such cases, we do not need to resort to the median, but we can take the minimum instead. Surprisingly, this does not improve the asymptotic number of repetitions that we need to take in order to boost the success probability from $3/4$ to $1 - \delta$ under Condition 1.

Remark It would be a bad idea to take the *mean* instead of the median. Consider an algorithm which outputs the exactly correct solution 1 with probability $3/4$, and outputs 100 otherwise. Then the mean is 25.75, which is not close to the correct answer 1. In fact, it is not close to *any* output that the algorithm has ever given, while the median (for odd k) occurs among the outputs of the algorithm. Moreover, even a single outlier can distort the mean if it is very large, while the median is robust against a small number of nonsense outputs. Despite these drawbacks, there are situations where the mean is useful, especially if we have an *unbiased estimator* with *bounded variance* for the result. We will return to this notion in [Section 7.3.2](#).

3.1 Sampling

We briefly repeat the most basic version of an approximation algorithm: *sampling*. If we want to know what fraction of a set S satisfies some property P , then we may sample k elements from S uniformly at random, and count the number X of them that satisfy property P . Then we return X/k . The most classical application is polls, where we want to know which fraction of people want to vote for some political party or have a certain opinion. Another example is *target shooting*, where we want to estimate the volume of some geometric region R . Then we enclose R in some bounding box B , sample k points from B uniformly at random and report the fraction of points that are in R .

We leave it as an exercise to show that this satisfies Condition 1 for some $k = \mathcal{O}(\frac{1}{\epsilon^2 f})$, where f is the fraction that we aim for. Note that this can be improved to failure probability δ instead of $3/4$ by repeating the process $O(\log(1/\delta))$ times, for a total runtime of $k = \mathcal{O}(\frac{\log(1/\delta)}{\epsilon^2 f})$. Whether this is a FPRAS depends on f : if f is a constant or polynomially bounded in the input size, then we obtain an FPRAS (assuming that we can efficiently sample uniformly at random, and that we can evaluate property P efficiently). On the other hand, it is clear that $\Omega(1/f)$ are needed: if we make fewer than

$1/f$ samples, then in expectation we sample no elements with property f at all, so we have no information about f other than it is probably small.

3.2 DNF counting

Definition 3.3 (Disjunctive Normal Form (DNF)). *A formula F on n Boolean variables $x_1, \dots, x_n$ is said to be in a Disjunctive Normal Form (DNF) if*

- $F = C_1 \vee \dots \vee C_m$ is a disjunction (that is, logical OR) of clauses
- $\forall i \in [m]$, a clause $C_i = l_{i,1} \wedge \dots \wedge l_{i,|C_i|}$ is a conjunction (that is, logical AND) of distinct literals
- $\forall j \in [n]$, a literal $l_j \in \{x_j, \neg x_j\}$ is either the variable x_j or its negation.

Let $\alpha : [n] \rightarrow \{0, 1\}$ be a truth assignment to the n variables. Formula F is said to be satisfiable if there exists a satisfying assignment α such that F evaluates to true under α (i.e. $F[\alpha] = 1$).

Any clause with both x_j and $\neg x_j$ is trivially false. As they can be removed in a single scan of F , we assume that F does not contain such trivial clauses.

Example Let $F = (x_1 \wedge \neg x_2 \wedge \neg x_4) \vee (x_2 \wedge x_3) \vee (\neg x_3 \wedge \neg x_4)$ be a Boolean formula on 4 variables, where $C_1 = x_1 \wedge \neg x_2 \wedge \neg x_4$, $C_2 = x_2 \wedge x_3$ and $C_3 = \neg x_3 \wedge \neg x_4$. Drawing the truth table, one sees that there are 9 satisfying assignments to F , one of which is $\alpha(1) = 1, \alpha(2) = \alpha(3) = \alpha(4) = 0$.

Remark Another common normal form for representing Boolean formulas is the *Conjunctive Normal Form* (CNF). Formulas in CNF are conjunctions of disjunctions (as compared to disjunctions of conjunctions in DNF). In particular, a DNF formula is always satisfiable unless all clauses are trivial, but it is $\mathcal{NP}$ -complete to determine if a CNF formula is satisfiable.

In this section, we are interested in the number of satisfying assignment for each given DNF. Suppose F is a Boolean formula in DNF. Let $N = N(F) = |\{\alpha : F[\alpha] = 1\}|$ be the number of satisfying assignments to F . If we let $S_i = \{\alpha : C_i[\alpha] = 1\}$ be the set of satisfying assignments to clause C_i , then we see that $N(F) = |\bigcup_{i=1}^m S_i|$. We are interested in polynomial-time algorithms for computing or approximating N . Equivalently, we are interested in the probability $\frac{N}{2^n}$ that a uniformly random assignment is satisfying. In the above example, $|S_1| = 2$, $|S_2| = 4$, $|S_3| = 4$, and $N = 9$.

In the following, we present two failed attempts to compute N and then present *DNF-Count*, a FPRAS for DNF counting via sampling.

3.2.1 Failed attempt 1: Principle of Inclusion-Exclusion

By definition of $N = |\bigcup_{i=1}^m S_i|$, one may be tempted to apply the Principle of Inclusion-Exclusion to expand:

$$|\bigcup_{i=1}^m S_i| = \sum_{i=1}^m |S_i| - \sum_{i < j} |S_i \cap S_j| + \dots$$

However, there are exponentially many terms and there exist instances where truncating the sum yields arbitrarily bad approximation.

3.2.2 Failed attempt 2: Sampling (wrongly)

We could try to solve the problem by sampling assignments u.a.r. and count the number of satisfying ones. This works if the fraction $N/2^n$ of satisfying assignment is at most polynomially small. Unfortunately, this does not need to be the case. Thus, this approach will *not* yield a FPRAS for DNF counting.

Still, we can make the idea work. However, we need to make sure that we estimate a fraction on which we have a polynomial lower bound.

3.2.3 An FPRAS for DNF counting via sampling

Consider an m -by- 2^n boolean matrix A where

$$A[i, j] = \begin{cases} 1 & \text{if assignment } \alpha_j \text{ satisfies clause } C_i \\ 0 & \text{otherwise} \end{cases}$$

Remark Due to its large size we will never be able to build the matrix A . It is used here as an explanation of why this attempt works.

Let M denote the total number of 1's in A . The crucial insights about M are the following.

- We can compute the number M . In fact, as we will see in a moment, we can even sample uniformly at random among the M 1's in A .
- The desired quantity N differs from M only by polynomial factors:

$$N \leq M \leq N \cdot m, \tag{3.1}$$

since N is the number of columns with at least one 1-entry, and each such column has between 1 and m such entries.

	α_1	α_2	α_3	$\dots$	α_{2^n}
C_1	0	1	0	$\dots$	0
C_2	1	1	0	$\dots$	1
C_3	0	0	0	$\dots$	0
$\dots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	
C_m	0	1	0	$\dots$	1

Table 3.1: Visual representation of the matrix A . Red 1's indicate the topmost clause C_i satisfied for each assignment α_j

We can compute M row-wise. Recall that S_i is the set of assignments that satisfy C_i . Since $|S_i| = 2^{n-|C_i|}$, $M = \sum_{i=1}^m |S_i| = \sum_{i=1}^m 2^{n-|C_i|}$.

We now interpret N as the number of “topmost” 1's in A , where “topmost” means that there is no other 1 above in the same column. Obviously there is exactly one topmost 1 for each satisfying assignment, so there are exactly N “topmost” 1's in A .

DNF-Count estimates the fraction $f = \frac{N}{M}$ of “topmost” 1's among the M 1's, then returns $f \cdot M$ as an estimate of N . To estimate the fraction of “topmost” 1's, it performs three steps.

1. Pick a 1 in A uniformly at random.
2. Check if the assignment satisfies any clauses with a smaller index.
3. Repeat those steps $\Theta(\frac{9m}{\epsilon^2})$ times.

We now verify that the algorithm does what we claim. To understand why the algorithm picks a 1 uniformly at random, we consider such a uniformly random 1. Then the probability that this one is in row i is $\frac{\#\{1\text{'s in row } i\}}{\#\{1\text{'s in } A\}} = \frac{2^{n-|C_i|}}{M}$. Hence, we can achieve uniform distribution of all 1's in A by picking row i with probability $\frac{2^{n-|C_i|}}{M}$. For completeness, we give a formal proof below.

Lemma 3.4. *DNF-Count samples a ‘1’ in A uniformly at random at each step.*

Proof. Fix any 1 in A . Assume it is in row i and column j . Then

$$\begin{aligned}
 \Pr[C_i \text{ and } \alpha_j \text{ are chosen}] &= \Pr[C_i \text{ is chosen}] \cdot \Pr[\alpha_j \text{ is chosen} \mid C_i \text{ is chosen}] \\
 &= \frac{2^{n-|C_i|}}{M} \cdot \frac{1}{2^{n-|C_i|}} \\
 &= \frac{1}{M},
 \end{aligned}$$

Algorithm 7 *DNF-Count*(F, ϵ)

```

1:  $X \leftarrow 0$  ▷ Empirical number of “topmost” 1’s sampled
2: for  $k = \frac{9m}{\epsilon^2}$  times do
3:    $C_i \leftarrow$  Sample one of  $m$  clauses, where  $\Pr[C_i \text{ chosen}] = \frac{2^{n-|C_i|}}{M}$ 
4:    $\alpha_j \leftarrow$  Sample one of  $2^{n-|C_i|}$  satisfying assignments of  $C_i$ 
5:    $IsTopmost \leftarrow \text{True}$ 
6:   for  $l \in \{1, \dots, i-1\}$  do ▷ Check if  $\alpha_j$  is “topmost”
7:     if  $C_l[\alpha_j] = 1$  then ▷ Checkable in  $\mathcal{O}(n)$  time
8:        $IsTopmost \leftarrow \text{False}$ 
9:     end if
10:  end for
11:  if  $IsTopmost$  then
12:     $X \leftarrow X + 1$ 
13:  end if
14: end for
15: return  $\frac{M \cdot X}{k}$ 

```

which means that every 1 is chosen with the same probability $1/M$. $\square$

Lemma 3.5. *In DNF-Count, $\Pr \left[\left| \frac{M \cdot X}{k} - N \right| \leq \epsilon \cdot N \right] \geq \frac{3}{4}$.*

Proof. Let X_i be the indicator variable whether the i -th sampled assignment is topmost, where $p = \Pr[X_i = 1]$. By Lemma 3.4, $p = \Pr[X_i = 1] = \frac{N}{M}$. Let $X = \sum_{i=1}^k X_i$ be the empirical number of topmost 1’s. Then, $\mathbb{E}(X) = kp$ by linearity of expectation. By picking $k = \frac{9m}{\epsilon^2}$,

$$\begin{aligned}
& \Pr \left[\left| \frac{M \cdot X}{k} - N \right| \geq \epsilon \cdot N \right] \\
&= \Pr \left[\left| X - \frac{k \cdot N}{M} \right| \geq \frac{\epsilon \cdot k \cdot N}{M} \right] && \text{Multiply by } \frac{k}{M} \\
&= \Pr [|X - kp| \geq \epsilon kp] && \text{Since } p = \frac{N}{M} \\
&\leq 2 \exp \left(-\frac{\epsilon^2 kp}{3} \right) && \text{By Chernoff bound} \\
&= 2 \exp \left(-\frac{3m \cdot N}{M} \right) && \text{Since } k = \frac{9m}{\epsilon^2} \text{ and } p = \frac{N}{M} \\
&\leq 2 \exp(-3) && \text{Since } M \leq m \cdot N \\
&\leq \frac{1}{4}
\end{aligned}$$

□

Lemma 3.6. *DNF-Count runs in $\text{poly}(F, \frac{1}{\epsilon}) = \text{poly}(n, m, \frac{1}{\epsilon})$ time.*

Proof. There are $k \in \mathcal{O}(\frac{m}{\epsilon^2})$ iterations. In each iteration, we spend $\mathcal{O}(m + n)$ sampling C_i and α_j , and $\mathcal{O}(nm)$ for checking if a sampled α_j is topmost. The latter dominates the former, so in total *DNF-Count* runs in $\mathcal{O}(\frac{m^2 n}{\epsilon^2})$ time. □

Theorem 3.7. *DNF-Count is a FPRAS for DNF counting.*

Proof. By Lemmas 3.5 and 3.6. □

Remark The method presented here works also for a more general situation. Computing the number N is equivalent to computing the probability that the formula is satisfied for a uniformly random assignment, i.e. if we set each variable to 0 or 1 with probability $1/2$ each, independently of each other. As a generalization, we can set the i -th variable to 1 with some probability p_i and to 0 with probability $1 - p_i$, still independently for different i . Then we still ask for the probability that such a random assignment satisfies F .

The same algorithm still works, with some small adaptation of the probabilities. Instead of picking row i with probability $\frac{2^{n-|C_i|}}{M}$ in line 3, we need to recompute the probability that a randomly chosen 1 in A is in row i , which is

$$\begin{aligned} \Pr[\text{random 1 is in row } i] &= \frac{\sum_{j: \alpha_j \text{ satisfies } C_i} \Pr[\alpha_j]}{\sum_{r=1}^m \sum_{j: \alpha_j \text{ satisfies } C_r} \Pr[\alpha_j]} \\ &= \frac{\Pr[C_i \text{ is satisfied}]}{\sum_{r=1}^m \Pr[C_r \text{ is satisfied}]}. \end{aligned}$$

We can compute the probabilities in the latter expression efficiently since $\Pr[C_i \text{ is satisfied}] = \prod_{x_s \in C_i} p_s \cdot \prod_{\neg x_s \in C_i} (1 - p_s)$. Moreover, in line 4 of the algorithm we don't pick a satisfying assignment of C_i *uniformly* at random, but instead we set all variables that do not occur in C_i randomly with their corresponding probabilities p_s . This ensures that we sample a 1 in column j with probability proportional to $\Pr[\alpha_j]$ (and all 1's in the same column have the same probability). Also, we still have a bound comparable to (3.1), except that now we need to use probabilities instead of integers. Consider a random entry (i, j) from the matrix A , that means an entry where the index j (the assignment a_j) is picked with probability $\Pr[\alpha_j]$ given by the distribution on assignments, and the index i (the clause C_i) is picked uniformly at random, i.e., each clause has probability $1/m$. We claim that

$$\Pr[A(i, j) = 1] \leq \Pr[\alpha_j \text{ is satisfying}] \leq m \cdot \Pr[A(i, j) = 1], \quad (3.2)$$

which is why we can still choose the same value of k .

Indeed, to see (3.2), consider a *fixed and satisfying* α_j . There are between 1 and m satisfied clauses $C_{i'}$ for α_j , where each pair (i', j) has the same probability $\frac{1}{m} \Pr[\alpha_j]$. Those i' are described by the condition $A(i', j) = 1$. Hence,

$$\frac{1}{m} \Pr[\alpha_j] \leq \sum_{i': A(i', j)=1} \Pr[A(i', j)] \leq \Pr[\alpha_j].$$

Summing over all satisfying assignments α_j yields $\Pr[\alpha_j \text{ is satisfying}]$ on the left (with a factor $1/m$) and on the right, while the middle term gives $\Pr[A(i, j) = 1]$ for a random entry (i, j) of A . This implies (3.2).

3.3 Network reliability

Let us consider an undirected graph $G = (V, E)$, where each edge $e \in E$ has a certain probability of failing, namely p_e , independent of other edges. In the *network reliability* problem, we are interested in calculating or estimating the probability that the network becomes disconnected. For simplicity we study the symmetrical case, where $p_e = p$ for every edge $e \in E$.

As a side remark, we note that we are aiming to obtain a $(1 \pm \varepsilon)$ -approximation of the quantity $P := \Pr[G \text{ is disconnected}]$. Such an approximation is not necessarily a $(1 \pm \varepsilon)$ -approximation of the probability of the converse, $1 - P = \Pr[G \text{ is connected}]$ and vice versa, since P may be very close to 1.

Observation 3.8. *The graph becomes disconnected if and only if there is an edge cut in which all edges fail. For an edge cut of k edges, the probability of all its edges failing is p^k .*

Reduction to DNF counting: If the graph had only a few cuts, there would be a natural and easy way to formulate the problem as a variant of DNF counting: each edge would be represented by a variable, and every clause would correspond to a cut postulating that all of its edges have failed. The probability of disconnecting is then the probability that a random assignment satisfies the DNF formula, when each variable is true with probability p and false otherwise. In formula, if X_e is the indicator variable with $X_e = 0$ iff the edge e fails, then

$$G \text{ disconnected} \iff \bigvee_{\text{cuts } C} \left(\bigwedge_{e \in C} \neg X_e \right).$$

Unfortunately, there are exponentially many cuts in a general graph. However, due to a result by Karger [Kar01], there are only few *small* cuts. We will now distinguish two cases based on the minimum cut size s . If s is small, then the probability of disconnection is so high that a naive approach works. If s is large, then we will show that large cuts are negligible.

- When $p^s \geq \frac{1}{n^4}$, this means that the probability of the network disconnection is rather high. As mentioned previously, this is not the case of a large interest, since the motivation behind studying this problem is understanding how to build reliable networks. Nevertheless, since the probability of disconnection is rather high, the naive Monte-Carlo method of sampling the graph and checking whether it stays connected from section 3.2.2 is sufficient, since we only need $\tilde{\mathcal{O}}(n^4)$ samples to achieve concentration.
- When $p^s \leq \frac{1}{n^4}$, we show that the large cuts do not contribute to the probability of disconnection too much, and therefore they can be safely ignored. By Karger's result, for any $\alpha \geq 1$ there are at most $n^{2\alpha}$ cuts with at most αs edges.³ For concreteness, let us select a threshold $i_0 := 4s$ and show that cuts with more than $4s$ edges are negligible. Let P be the probability that G becomes disconnected, and note that $P \geq p^s$. Before we dive into the calculation, note that our assumption $p^s \leq n^{-4}$ implies $p^{-1/2} \geq n^{2/s}$ by taking the $-1/(2s)$ -th root, and hence $pn^{2/s} \leq p^{1/2}$. Moreover, $s \leq n - 1$ since there are always edge cuts of size at most $n - 1$ (by isolating a vertex). This implies $p^{1/2} \leq 1 - \frac{1}{2n}$ since otherwise we would have $p^s = (p^{1/2})^{2s} > (1 - \frac{1}{2n})^{2n-2} > 1/e$, contradicting that we are in case $p^s \leq \frac{1}{n^4}$.

Now let us come to the actual calculation. For $i \geq s$ the number of cuts with i edges is at most $n^{2i/s}$ (from Karger's result with $\alpha = i/s$), and each of them has a probability of p^i to get disconnected. Hence, the probability that a cut with at least $i_0 = 4s$ edges disconnects is at

³You may have seen this fact before, e.g., in the context of Karger's randomized contraction algorithm in your undergraduate algorithmic classes. In any case, we will ask you to prove it in the exercises.

most

$$\begin{aligned}
\sum_{i=4s}^{\infty} n^{2i/s} p^i &= \sum_{i=4s}^{\infty} (n^{2/s} p)^i \\
&= (n^{2/s} p)^{4s} \cdot \frac{1}{1 - n^{2/s} p} && \text{formula for geometric series} \\
&\leq p^{2s} \cdot 2n && \text{since } n^{2/s} p \leq p^{1/2} \leq 1 - \frac{1}{2n} \\
&\leq P \cdot \frac{2}{n^3} && \text{since } p^s \leq P \text{ and } p^s \leq n^{-4}.
\end{aligned}$$

Hence, the error introduced by ignoring large cuts is negligible compared to P .⁴ Thus, we can ignore those large cuts.

The number of cuts smaller than i_0 is at most n^8 and therefore the reduction to DNF-counting is polynomial.⁵ Note that we also need to *find* all cuts of small size. Fortunately, Karger's contraction algorithm has the nice property that for every cut of size at most i , it returns this cut with probability at least $n^{-2i/s}$. Hence, it provides us with a way of sampling cuts of size less than i_0 , and by the coupon collector argument we only need to wait for time $\tilde{O}(n^8)$ until we have seen all such cuts.

⁴At least for $\varepsilon > n^{-3}$. For general ε we may choose $i_0 = \max\{\mathcal{O}(\log_n \varepsilon^{-1}), \Theta(1)\}$.

⁵The exponent is not tight and can easily be improved.

Chapter 4

Rounding Linear Program Solutions

Linear programming (LP) and integer linear programming (ILP) are versatile models but with different solving complexities — LPs are solvable in polynomial time while ILPs are $\mathcal{NP}$ -hard.

Definition 4.1 (Linear program (LP)). *The canonical form of an LP is*

$$\begin{aligned} & \text{minimize} && \mathbf{c}^T \mathbf{x} \\ & \text{subject to} && A\mathbf{x} \geq \mathbf{b} \\ & && \mathbf{x} \geq 0 \end{aligned}$$

where $\mathbf{x}$ is the vector of n variables (to be determined), $\mathbf{b}$ and $\mathbf{c}$ are vectors of (known) coefficients, and A is a (known) matrix of coefficients. $\mathbf{c}^T \mathbf{x}$ and $\text{obj}(\mathbf{x})$ are the objective function and objective value of the LP respectively. For an optimal variable assignment $\mathbf{x}^*$, $\text{obj}(\mathbf{x}^*)$ is the optimal value.

ILPs are defined similarly with the additional constraint that variables take on integer values. As we will be relaxing ILPs into LPs, to avoid confusion, we use $\mathbf{y}$ for ILP variables to contrast against the $\mathbf{x}$ variables in LPs.

Definition 4.2 (Integer linear program (ILP)). *The canonical form of an ILP is*

$$\begin{aligned} & \text{minimize} && \mathbf{c}^T \mathbf{y} \\ & \text{subject to} && A\mathbf{y} \geq \mathbf{b} \\ & && \mathbf{y} \geq 0 \\ & && \mathbf{y} \in \mathbb{Z}^n \end{aligned}$$

where $\mathbf{y}$ is the vector of n variables (to be determined), $\mathbf{b}$ and $\mathbf{c}$ are vectors of (known) coefficients, and A is a (known) matrix of coefficients. $\mathbf{c}^T \mathbf{y}$ and $\text{obj}(\mathbf{y})$ are the objective function and objective value of the LP respectively. For an optimal variable assignment $\mathbf{y}^*$, $\text{obj}(\mathbf{y}^*)$ is the optimal value.

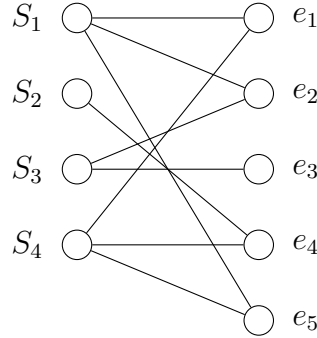
Remark We can define LPs and ILPs for maximization problems similarly. One can also solve maximization problems with a minimization LPs using the same constraints but negated objective function. The optimal value from the solved LP will then be the negation of the maximized optimal value.

In this chapter, we illustrate how one can model set cover and multi-commodity routing as ILPs, and how to perform rounding to yield approximations for these problems. As before, Chernoff bounds will be a useful inequality in our analysis toolbox.

4.1 Minimum set cover

Recall the minimum set cover problem and the example from Section 1.1.

Example



Suppose there are $n = 5$ vertices and $m = 4$ subsets $S = \{S_1, S_2, S_3, S_4\}$, where the cost function is defined as $c(S_i) = i^2$. Then, the minimum set cover is $S^* = \{S_1, S_2, S_3\}$ with a cost of $c(S^*) = 14$.

In Section 1.1, we saw that a greedy selection of sets that minimizes the price-per-item of remaining sets gave an H_n -approximation for set cover. We now show how to formulate set cover as an ILP, reduce it into a LP, and how to round the solutions to yield an approximation to the original set cover instance. Consider the following ILP:

ILP_{Set cover}

$$\begin{aligned}
& \text{minimize} && \sum_{i=1}^m y_i \cdot c(S_i) && \triangleleft \text{Cost of chosen set cover} \\
& \text{subject to} && \sum_{i: e_j \in S_i} y_i \geq 1 \quad \forall j \in [n] && \triangleleft \text{Every item } e_j \text{ is covered} \\
& && y_i \in \{0, 1\} \quad \forall i \in [m] && \triangleleft \text{Indicator whether set } S_i \text{ is chosen}
\end{aligned}$$

Upon solving $\text{ILP}_{\text{Set cover}}$, the set $\{S_i : i \in [m] \wedge y_i^* = 1\}$ is the optimal solution for a given set cover instance. However, as solving ILPs is $\mathcal{NP}$ -hard, we consider relaxing the integral constraint by replacing binary y_i variables by real-valued/fractional $x_i \in [0, 1]$. Such a relaxation will yield the corresponding LP:

LP_{Set cover}

$$\begin{aligned}
& \text{minimize} && \sum_{i=1}^m x_i \cdot c(S_i) && \triangleleft \text{Cost of chosen fractional set cover} \\
& \text{subject to} && \sum_{i: e_j \in S_i} x_i \geq 1 \quad \forall j \in [n] && \triangleleft \text{Every item } e_j \text{ is fractionally covered} \\
& && 0 \leq x_i \leq 1 \quad \forall i \in [m] && \triangleleft \text{Relaxed indicator variables}
\end{aligned}$$

Since LPs can be solved in polynomial time, we can find the optimal fractional solution to $\text{LP}_{\text{Set cover}}$ in polynomial time.

Observation As the set of solutions of $\text{ILP}_{\text{Set cover}}$ is a subset of $\text{LP}_{\text{Set cover}}$, $\text{obj}(\mathbf{x}^*) \leq \text{obj}(\mathbf{y}^*)$.

Example The corresponding ILP for the example set cover instance is:

$$\begin{aligned}
& \text{minimize} && y_1 + 4y_2 + 9y_3 + 16y_4 \\
& \text{subject to} && y_1 + y_4 \geq 1 && \triangleleft \text{Sets covering } e_1 \\
& && y_1 + y_3 \geq 1 && \triangleleft \text{Sets covering } e_2 \\
& && y_3 \geq 1 && \triangleleft \text{Sets covering } e_3 \\
& && y_2 + y_4 \geq 1 && \triangleleft \text{Sets covering } e_4 \\
& && y_1 + y_4 \geq 1 && \triangleleft \text{Sets covering } e_5 \\
& && \forall i \in \{1, \dots, 4\}, y_i \in \{0, 1\}
\end{aligned}$$

After relaxing:

$$\begin{array}{ll}
\text{minimize} & x_1 + 4x_2 + 9x_3 + 16x_4 \\
\text{subject to} & x_1 + x_4 \geq 1 \\
& x_1 + x_3 \geq 1 \\
& x_3 \geq 1 \\
& x_2 + x_4 \geq 1 \\
& x_1 + x_4 \geq 1 \\
& \forall i \in \{1, \dots, 4\}, 0 \leq x_i \leq 1 \quad \triangleleft \text{Relaxed indicator variables}
\end{array}$$

Solving it using a LP solver¹ yields: $x_1 = 1, x_2 = 1, x_3 = 1, x_4 = 0$. Since the solved $\mathbf{x}^*$ are integral, $\mathbf{x}^*$ is also the optimal solution for the original ILP. In general, the solved $\mathbf{x}^*$ may be fractional, which does not immediately yield a set selection.

We now describe two ways to round the fractional assignments $\mathbf{x}^*$ into binary variables $\mathbf{y}$ so that we can interpret them as proper set selections.

4.1.1 (Deterministic) Rounding for Vertex Cover

There is an interesting special case of the general Set Cover problem, namely the Vertex Cover problem. In the problem, the input is a graph with costs on the vertices, and we want to select a set of vertices of minimal cost such that the set contains at least one endpoint for every edge. This is a special case of Set Cover, where the items to be covered are the edges of the graph, and for each vertex we get the corresponding set of all incident edges.

This case is special because every item (edge) is contained in exactly two sets (is incident to exactly two vertices). Therefore, each constraint $\sum_{i:e_j \in S_i} x_i \geq 1$ in the resulting $\text{ILP}_{\text{Vertex cover}}$ and $\text{LP}_{\text{Vertex cover}}$ contains exactly two summands.

This allows us to round $\mathbf{x}^*$ as follows:

$$\forall i \in [m], \text{ set } y_i = \begin{cases} 1 & \text{if } x_i^* \geq \frac{1}{2} \\ 0 & \text{else} \end{cases}$$

Theorem 4.3. *The rounded $\mathbf{y}$ is a feasible solution to $\text{ILP}_{\text{Vertex cover}}$.*

¹Using Microsoft Excel. See tutorial: http://faculty.sfasu.edu/fisherwarre/lp_solver.html

Or, use an online LP solver such as: <http://online-optimizer.appspot.com/?model=builtin:default.mod>

Proof. Since $\mathbf{x}^*$ is a feasible (not to mention, optimal) solution for $\text{LP}_{\text{Vertex cover}}$, in each constraint, there is at least one x_i^* that is greater or equal to $\frac{1}{2}$. Hence, every element is covered by some set y_i in the rounding. $\square$

Theorem 4.4. *The rounded $\mathbf{y}$ is a 2-approximation to $\text{ILP}_{\text{Vertex cover}}$.*

Proof. By the rounding, $y_i \leq 2 \cdot x_i^*, \forall i \in [m]$. Therefore,

$$\text{obj}(\mathbf{y}) \leq 2 \cdot \text{obj}(\mathbf{x}^*) \leq 2 \cdot \text{obj}(\mathbf{y}^*)$$

$\square$

We thus obtain a 2-approximation for the Vertex Cover problem.

Integrality gap A helpful way of estimating whether the approximation factor is good is to consider the *integrality gap*. The integrality gap of vertex cover is defined as

$$\text{integrality gap of VC} = \sup_{\text{all input instances } I \text{ of VC}} \frac{\text{opt. cost of ILP for } I}{\text{opt. cost of LP for } I}.$$

In the analysis above we have implicitly shown that the integrality gap of vertex cover is at most 2, because we can round any feasible solution of the LP to a feasible solution of the ILP and lose at most a factor of 2. Conversely, it is easy to see that the integrality gap of vertex cover is exactly 2. Consider as input I the complete graph K_n where all vertices have cost 1. Then the best solution of the ILP has cost $n - 1$, since any set of size at most $n - 2$ misses two vertices u, v , and hence does not cover the edge $\{u, v\}$. On the other hand, the LP has a solution of cost $n/2$, by setting all x_i to $1/2$. Hence, the gap for this input instance is at least $\frac{n-1}{n/2} \rightarrow 2$ for $n \rightarrow \infty$.

Why is the integrality gap related to the approximation ratio? Our approach is to find an optimal solution $\mathbf{x}^*$ of the LP and round it to a feasible solution $\mathbf{y}$ of the ILP. But we have just seen that there are instances in which *every* feasible solution of the ILP is by a factor (almost) 2 worse than $\mathbf{x}^*$. Hence, every way of transforming the solution $\mathbf{x}^*$ into a feasible integral solution must lose a factor of 2 on these instances. If we use the generic argument (with $f = 2$)

$$\text{obj}(\mathbf{y}) \leq 2 \cdot \text{obj}(\mathbf{x}^*) \leq 2 \cdot \text{obj}(\mathbf{y}^*),$$

then there is no way of avoiding the factor 2 in the first step. To improve the analysis, we would need to improve the bound $\text{obj}(\mathbf{x}^*) \leq \text{obj}(\mathbf{y}^*)$, i.e., we would need to quantify how much easier the problem gets by relaxing from ILP to LP. This is usually hard to do. The integrality gap of 2 implies that there exist *some* problem instances on which this inequality is not tight, but we would need to quantify this for *all* instances.

4.1.2 (Randomized) Rounding for general Set Cover

For the general Set Cover problem, if every item is contained in at most f sets, then we need to choose the threshold for deterministic rounding as $1/f$ in order to maintain feasibility after rounding. The resulting algorithm gives an f -approximation, which may be unsatisfactory. By introducing randomness in the rounding process, we show that one can obtain a $\ln(n)$ -approximation (in expectation) with arbitrarily high probability through probability amplification.

Consider the following rounding procedure:

1. Interpret each x_i^* as probability for picking S_i . That is, $\Pr[y_i = 1] = x_i^*$.
2. For each i , independently set y_i to 1 with probability x_i^* .

Theorem 4.5. $\mathbb{E}(\text{obj}(\mathbf{y})) = \text{obj}(\mathbf{x}^*)$.

Proof.

$$\begin{aligned}
 \mathbb{E}(\text{obj}(\mathbf{y})) &= \mathbb{E}\left(\sum_{i=1}^m y_i \cdot c(S_i)\right) \\
 &= \sum_{i=1}^m \mathbb{E}(y_i) \cdot c(S_i) && \text{By linearity of expectation} \\
 &= \sum_{i=1}^m \Pr(y_i = 1) \cdot c(S_i) && \text{Since each } y_i \text{ is an indicator variable} \\
 &= \sum_{i=1}^m x_i^* \cdot c(S_i) && \text{Since } \Pr(y_i = 1) = x_i^* \\
 &= \text{obj}(\mathbf{x}^*).
 \end{aligned}$$

□

Although the rounded selection to yield an objective cost that is close to the optimum (in expectation) of the LP, we need to consider whether all constraints are satisfied.

Theorem 4.6. For any $j \in [n]$, item e_j is not covered w.p. $\leq e^{-1}$.

Proof. For any $j \in [n]$,

$$\begin{aligned}
\Pr[\text{Item } e_j \text{ not covered}] &= \Pr\left[\sum_{i:e_j \in S_i} y_i = 0\right] \\
&= \prod_{i:e_j \in S_i} (1 - x_i^*) && \text{Since the } y_i \text{ are chosen independently} \\
&\leq \prod_{i:e_j \in S_i} e^{-x_i^*} && \text{Since } (1 - x) \leq e^{-x} \\
&= e^{-\sum_{i:e_j \in S_i} x_i^*} \\
&\leq e^{-1}
\end{aligned}$$

The last inequality holds because the optimal solution $\mathbf{x}^*$ satisfies the j^{th} constraint in the LP that $\sum_{i:e_j \in S_i} x_i^* \geq 1$. $\square$

Since $e^{-1} \approx 0.37$, we would expect the rounded $\mathbf{y}$ not to cover several items. However, one can amplify the success probability by considering independent roundings and taking the union (See *ApxSetCoverILP*).

Algorithm 8 *ApxSetCoverILP*($\mathcal{U}, \mathcal{S}, c$)

```

ILPSet cover  $\leftarrow$  Construct ILP of problem instance
LPSet cover  $\leftarrow$  Relax integral constraints on indicator variables  $\mathbf{y}$  to  $\mathbf{x}$ 
 $\mathbf{x}^* \leftarrow$  Solve LPSet cover
 $T \leftarrow \emptyset$   $\triangleright$  Selected subset of  $\mathcal{S}$ 
for  $k \cdot \ln(n)$  times (for any constant  $k > 1$ ) do
  for  $i \in [m]$  do
     $y_i \leftarrow$  Set to 1 with probability  $x_i^*$ 
    if  $y_i = 1$  then
       $T \leftarrow T \cup \{S_i\}$   $\triangleright$  Add to selected sets  $T$ 
    end if
  end for
end for
return  $T$ 

```

Similar to Theorem 4.5, we can see that $\mathbb{E}(\text{obj}(T)) \leq (k \cdot \ln(n)) \cdot \text{obj}(\mathbf{y}^*)$. Furthermore, Markov's inequality tells us that the probability of $\text{obj}(T)$ being z times larger than its expectation is at most $\frac{1}{z}$.

Theorem 4.7. *ApxSetCoverILP gives a valid set cover w.p. $\geq 1 - n^{1-k}$, and it gives a valid $2k \cdot \ln(n)$ -approximation for set cover w.p. $\geq 1/2 - n^{1-k}$.*

Proof. For all $j \in [n]$,

$$\begin{aligned}
\Pr[\text{Item } e_j \text{ not covered by } T] &= \Pr[e_j \text{ not covered by all } k \ln(n) \text{ roundings}] \\
&\leq (e^{-1})^{k \ln(n)} \\
&= n^{-k}
\end{aligned}$$

Taking union bound over all n items,

$$\Pr[T \text{ is not a valid set cover}] \leq \sum_{i=1}^n n^{-k} = n^{1-k}$$

So, T is a valid set cover with probability $\geq 1 - n^{1-k}$.

For the second statement, recall $\mathbb{E}(\text{obj}(T)) \leq k \cdot \ln(n) \cdot \text{obj}(\mathbf{y}^*)$. By Markov's inequality $\Pr[\text{obj}(T) \geq 2k \cdot \ln(n) \cdot \text{obj}(\mathbf{y}^*)] \leq 1/2$. Hence,

$$\begin{aligned}
&\Pr[T \text{ is a valid set cover and } \text{obj}(T) < 2k \cdot \ln(n) \cdot \text{obj}(\mathbf{y}^*)] \\
&= 1 - \Pr[T \text{ is not a valid set cover or } \text{obj}(T) \geq 2k \cdot \ln(n) \cdot \text{obj}(\mathbf{y}^*)] \\
&\geq 1 - (\Pr[T \text{ is not a valid set cover}] + \Pr[\text{obj}(T) \geq 2k \cdot \ln(n) \cdot \text{obj}(\mathbf{y}^*)]) \\
&\geq 1 - (n^{1-k} + 1/2) = 1/2 - n^{1-k}.
\end{aligned}$$

□

Note that the success probability of $1/2 - n^{1-k}$ can be further amplified by taking several *independent* samples of *ApxSetCoverILP*, then returning the lowest cost valid set cover sampled. With z samples, the probability that *all* repetitions fail is less than $(1/2 - n^{1-k})^z$, so we succeed w.p. $\geq 1 - (1/2 - n^{1-k})^z$, which gets exponentially close to 1 with growing z .

4.2 Minimizing congestion in multi-commodity routing

A multi-commodity routing (MCR) problem involves routing multiple (s_i, t_i) flows across a network with the goal of minimizing congestion, where congestion is defined as the largest ratio of flow over capacity of any edge in the network. In this section, we discuss two variants of the multi-commodity routing problem. In the first variant (special case), we are given the set of possible paths $\mathcal{P}_i$ for each (s_i, t_i) source-target pairs. In the second variant (general case), we are given only the network. In both cases, [RT87] showed that one can obtain an approximation of $\mathcal{O}(\frac{\log(m)}{\log \log(m)})$ with high probability.

Definition 4.8 (Multi-commodity routing problem). *Consider a directed graph $G = (V, E)$ where $|E| = m$ and each edge $e = (u, v) \in E$ has a capacity $c(u, v)$. The in-set/out-set of a vertex v is denoted as $in(v) = \{(u, v) \in E : u \in V\}$ and $out(v) = \{(v, u) \in E : u \in V\}$ respectively. Given k triplets (s_i, t_i, d_i) , where $s_i \in V$ is the source, $t_i \in V$ is the target, and $d_i \geq 0$ is the demand for the i^{th} commodity respectively, denote $f(e, i) \in [0, 1]$ as the fraction of d_i that is flowing through edge e . The task is to minimize the congestion parameter λ by finding paths p_i for each $i \in [k]$, such that:*

$$(i) \text{ (Valid sources): } \sum_{e \in out(s_i)} f(e, i) - \sum_{e \in in(s_i)} f(e, i) = 1, \forall i \in [k]$$

$$(ii) \text{ (Valid sinks): } \sum_{e \in in(t_i)} f(e, i) - \sum_{e \in out(t_i)} f(e, i) = 1, \forall i \in [k]$$

(iii) (Flow conservation): For each commodity $i \in [k]$,

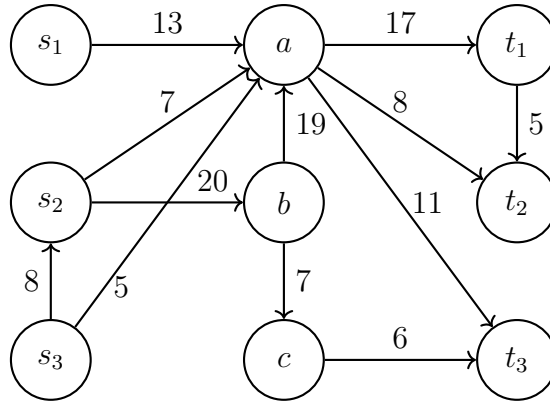
$$\sum_{e \in out(v)} f(e, i) - \sum_{e \in in(v)} f(e, i) = 0, \quad \forall v \in V \setminus \{s_i, t_i\}$$

(iv) (Single path): All demand for commodity i passes through a single path p_i (no repeated vertices).

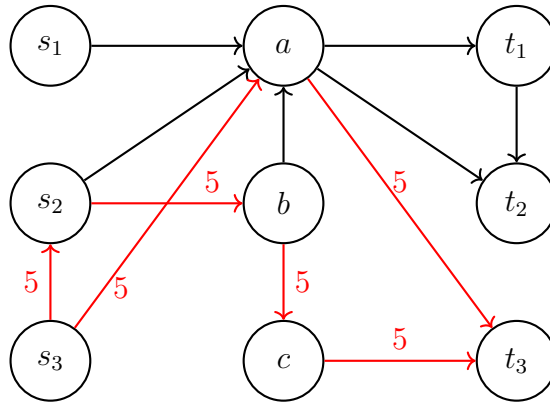
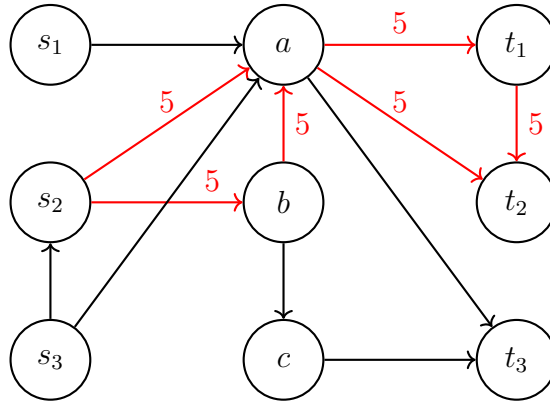
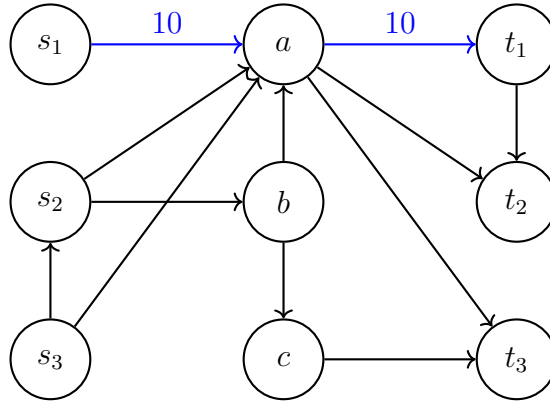
(v) (Congestion factor): $\forall e \in E, \sum_{i=1}^k d_i \mathbb{1}_{e \in p_i} \leq \lambda \cdot c(e)$, where indicator $\mathbb{1}_{e \in p_i} = 1 \iff e \in p_i$.

(vi) (Minimum congestion): λ is minimized.

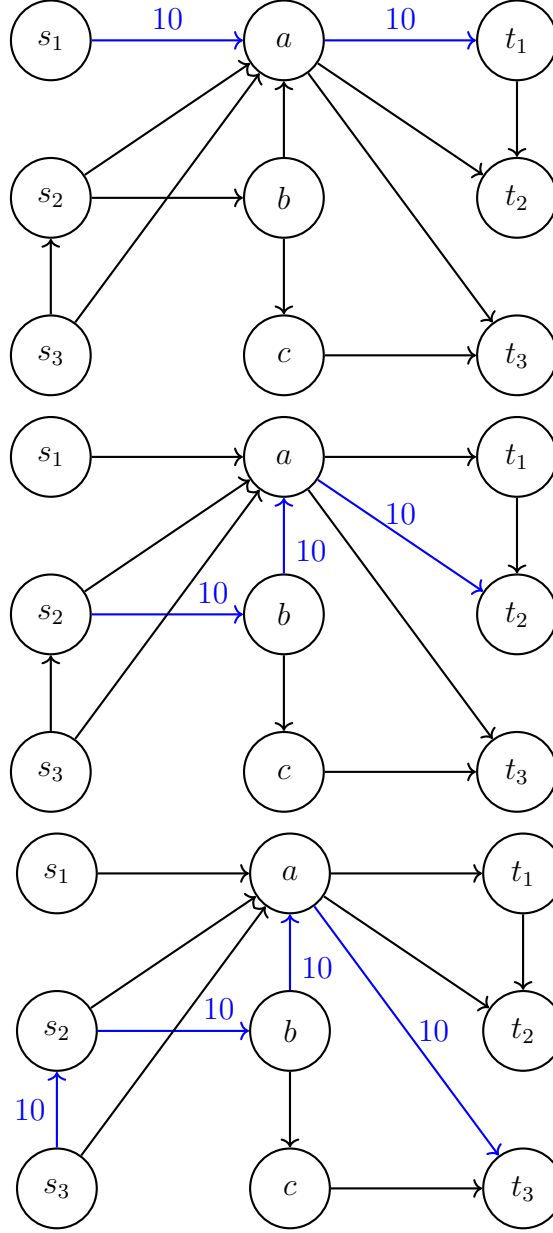
Example Consider the following flow network with $k = 3$ commodities with edge capacities as labelled:



For demands $d_1 = d_2 = d_3 = 10$, there exists a flow assignment such that the total demands flowing on each edge is below its capacity:



Although the assignment attains congestion $\lambda = 1$ (due to edge (s_3, a)), the path assignments for commodities 2 and 3 violate the property of “single path”. Forcing all demand of each commodity to flow through a single path, we have a minimum congestion of $\lambda = 1.25$ (due to edges (s_3, s_2) and (a, t_2)):



4.2.1 Special case: Given sets of $s_i - t_i$ paths $\mathcal{P}_i$

For each commodity $i \in [k]$, we are to select a path p_i from a given set of valid s_i - t_i -paths $\mathcal{P}_i$, where each edge in all paths in $\mathcal{P}_i$ has capacities $\geq d_i$. Because we intend to pick a single path for each commodity to send *all* demands through, constraints (i)-(iii) of MCR are fulfilled trivially. Using $y_{i,p}$ as indicator variables whether path $p \in \mathcal{P}_i$ is chosen, we can model the following ILP:

ILP_{MCR-Given-Paths}

$$\text{minimize} \quad \lambda \quad \triangleleft (1)$$

$$\text{subject to} \quad \sum_{i=1}^k \left(d_i \cdot \sum_{p \in \mathcal{P}_i, e \in p} y_{i,p} \right) \leq \lambda \cdot c(e) \quad \forall e \in E \quad \triangleleft (2)$$

$$\sum_{p \in \mathcal{P}_i} y_{i,p} = 1 \quad \forall i \in [k] \quad \triangleleft (3)$$

$$y_{i,p} \in \{0, 1\} \quad \forall i \in [k], p \in \mathcal{P}_i \quad \triangleleft (4)$$

$\triangleleft (1)$ Congestion parameter λ

$\triangleleft (2)$ Congestion factor relative to selected paths

$\triangleleft (3)$ Exactly one path chosen from each $\mathcal{P}_i$

$\triangleleft (4)$ Indicator variable for path $p \in \mathcal{P}_i$

Relax the integral constraint on $y_{i,p}$ to $x_{i,p} \in [0, 1]$ and solve the corresponding LP. Define $\lambda^* = \text{obj}(\text{LP}_{\text{MCR-Given-Paths}})$ and denote $\mathbf{x}^*$ as a fractional path selection that achieves λ^* . To obtain a valid path selection, for each commodity $i \in [k]$, pick path $p \in \mathcal{P}_i$ with weighted probability $\frac{x_{i,p}^*}{\sum_{p \in \mathcal{P}_i} x_{i,p}^*} = x_{i,p}^*$. Note that by constraint (3), $\sum_{p \in \mathcal{P}_i} x_{i,p}^* = 1$.

Remark 1 For a fixed i , a path is selected *exclusively* (only one!) (cf. set cover's roundings where we may pick multiple sets for an item).

Remark 2 The weighted sampling is independent across different commodities. That is, the choice of path amongst $\mathcal{P}_i$ does not influence the choice of path amongst $\mathcal{P}_j$ for $i \neq j$.

Theorem 4.9. $\Pr[\text{obj}(\mathbf{y}) \geq \frac{2c \log m}{\log \log m} \max\{1, \lambda^*\}] \leq \frac{1}{m^{c-1}}$

Proof. Fix an arbitrary edge $e \in E$. For each commodity i , define an indicator variable $Y_{e,i}$ whether edge e is part of the chosen path for commodity i . By randomized rounding, $\Pr[Y_{e,i} = 1] = \sum_{p \in \mathcal{P}_i, e \in p} x_{i,p}^*$. Denoting

4.2. MINIMIZING CONGESTION IN MULTI-COMMODITY ROUTING⁴⁷

$Y_e = \sum_{i=1}^k d_i \cdot Y_{e,i}$ as the total demand on edge e in *all* k chosen paths,

$$\begin{aligned}
 \mathbb{E}(Y_e) &= \mathbb{E}\left(\sum_{i=1}^k d_i \cdot Y_{e,i}\right) \\
 &= \sum_{i=1}^k d_i \cdot \mathbb{E}(Y_{e,i}) && \text{By linearity of expectation} \\
 &= \sum_{i=1}^k d_i \sum_{p \in \mathcal{P}_i, e \in p} x_{i,p} && \text{Since } \Pr[Y_{e,i} = 1] = \sum_{p \in \mathcal{P}_i, e \in p} x_{i,p} \\
 &\leq \lambda^* \cdot c(e) && \text{By MCR constraint and optimality of the solved LP}
 \end{aligned}$$

For every edge $e \in E$, applying² the tight form of Chernoff bounds with $(1 + \epsilon) = \frac{2 \log m}{\log \log m}$ on variable $\frac{Y_e}{c(e)}$ gives

$$\Pr \left[\frac{Y_e}{c(e)} \geq \frac{2c \log m}{\log \log m} \max\{1, \lambda^*\} \right] \leq \frac{1}{m^c}$$

Finally, we use a union bound over all m edges. □

4.2.2 General: Given only a network

In the general case, we may not be given path sets $\mathcal{P}_i$ and there may be exponentially many s_i - t_i paths in the network. However, we show that one can still formulate an LP and round it (slightly differently) to yield the same approximation factor. With the following LP, we encode the task of finding a set of $s_i - t_i$ -flows (one for each i) that minimizes congestion. The variable $y_{i,e}$ encodes how much flow is transported along edge e for task i . Note that the resulting solution will be a *flow* for each i , not a single path.

²See Corollary 2 of https://courses.engr.illinois.edu/cs598csc/sp2011/Lectures/lecture_9.pdf for details.

LP_{Flow}

$$\begin{aligned}
& \text{minimize} && \lambda && \triangleleft (1) \\
& \text{subject to} && \sum_{e \in \text{out}(s_i)} x_{e,i} - \sum_{e \in \text{in}(s_i)} x_{e,i} = d_i && \forall i \in [k] \triangleleft (2) \\
& && \sum_{e \in \text{in}(t_i)} x_{e,i} - \sum_{e \in \text{out}(t_i)} x_{e,i} = d_i && \forall i \in [k] \triangleleft (3) \\
& && \sum_{e \in \text{out}(v)} x_{e,1} - \sum_{e \in \text{in}(v)} x_{e,1} = 0 && \forall v \in E, \triangleleft (4) \\
& && && \forall v \in V \setminus \{s_1 \cup t_1\} \\
& && && \vdots \\
& && \sum_{e \in \text{out}(v)} x_{e,k} - \sum_{e \in \text{in}(v)} x_{e,k} = 0 && \forall v \in E, \triangleleft (4) \\
& && && \forall v \in V \setminus \{s_k \cup t_k\} \\
& && \sum_{i=1}^k x_{e,i} \leq \lambda \cdot c(e) && \forall e \in E \text{ Congestion} \\
& && x_{i,p} \geq 0 && \forall i \in [k], e \in E
\end{aligned}$$

◁ (1) Congestion parameter λ

◁ (2) Valid sources

◁ (3) Valid sinks

◁ (4) Flow conservation

Note that LP_{net} is already an LP formulation, so we may solve it in polynomial time. This yields a flow $f(e, i) = x_{e,i}$ for each commodity i . To extract the path candidates $\mathcal{P}_i$ for commodity i , we decompose the flow $x_{e,i}$ into at most m paths $p_{i,j}$, where $j \in \{1 \dots m\}$ and we identify the path $p_{i,j}$ with a unit flow along $p_{i,j}$. I.e., we find weights $w_{i,j} \geq 0$, $j \in \{1, \dots, m\}$ such that $x_{e,i} = \sum_{j: e \in p_{i,j}} w_{i,j} p_{i,j}$.³ This yields a set $\mathcal{P}_i$ of paths for each commodity.

³See <https://www.youtube.com/watch?v=zgutyA9JM4&t=1020s> (17:00 to 29:50) for details on how to find such a flow decomposition. The main idea is to consider the edge e of minimal flow in x_i , find an s_i - t_i path p that uses e , and remove the flow $x_{i,e} \cdot p$ and the edge e from the network. Then repeat the process until all edges are removed.

4.2. MINIMIZING CONGESTION IN MULTI-COMMODITY ROUTING 49

Now consider the corresponding $\text{ILP}_{\text{MCR-Given-Paths}}$ as in the previous section, where we only allow the paths in $\mathcal{P}_i$ for the i -th commodity. Denote the corresponding LP relaxation by LP_{path} . By construction, the optimal congestion λ^* of LP_{net} and LP_{path} is identical.⁴ Hence, by the same method as before we can find a solution $\mathbf{y}$ to $\text{ILP}_{\text{MCR-Given-Paths}}$ which satisfies $\text{obj}(\mathbf{y}) \leq \frac{2c \log m}{\log \log m} \max\{1, \lambda^*\}$. This is also an $O(\frac{\log m}{\log \log m})$ -approximation of the original problem $\text{ILP}_{\text{MCR-Given-Network}}$.

⁴And they have the same optimal flow. In particular, we do not need to find the optimal solution of LP_{path} from scratch, we can deduce it from the already computed optimal solution of LP_{net} .

Chapter 5

Online Algorithms and Competitive Analysis

5.1 Warm up: Ski rental

We now study the class of online problems where one has to commit to provably good decisions as data arrive in an online fashion. To measure the effectiveness of online algorithms, we compare the quality of the produced solution against the solution from an optimal offline algorithm that knows the whole sequence of information a priori. Such a comparison is called *competitive analysis*.

Remark We do not assume that the optimal offline algorithm has to be computationally efficient. Under the competitive analysis framework, only the *quality of the best possible solution* matters.

Definition 5.1 (α -competitive online algorithm). *Let σ be an input sequence, c be a cost function, $\mathcal{A}$ be the online algorithm and OPT be the optimal offline algorithm. Then, denote $c_{\mathcal{A}}(\sigma)$ as the cost incurred by $\mathcal{A}$ on σ and $c_{OPT}(\sigma)$ as the cost incurred by OPT on the same sequence. We say that an online algorithm is α -competitive if for any input sequence σ , $c_{\mathcal{A}}(\sigma) \leq \alpha \cdot c_{OPT}(\sigma)$.*

Definition 5.2 (Ski rental problem). *Suppose we wish to ski every day but we do not have any skiing equipment initially. On each day, we can choose between:*

- *Rent the equipment for a day at CHF 1*
- *Buying the equipment (once and for all) for CHF B*

In the toy setting where we may break our leg on each day (and cannot ski thereafter), let d be the (unknown) total number of days we ski. What is the best online strategy for renting/buying?

Claim 5.3. $\mathcal{A}$ = “Rent for B days, then buy on day $B+1$ ” is a 2-competitive algorithm.

Proof. If $d \leq B$, the optimal offline strategy is to rent everyday, incurring a cost of $c_{OPT}(d) = d$. $\mathcal{A}$ will rent for d days and also incur a loss of $c_A(d) = d = c_{OPT}(d)$. If $d > B$, the optimal offline strategy is to buy the equipment immediately, incurring a loss of $c_{OPT}(d) = B$. $\mathcal{A}$ will rent for B days and then buy the equipment for CHF B , incurring a cost of $c_A(d) = 2B \leq 2c_{OPT}(d)$. Thus, for any d , $c_A(d) \leq 2 \cdot c_{OPT}(d)$. $\square$

5.2 Paging

Definition 5.4 (Paging problem [ST85]). *Suppose we have a fast memory (cache) that can fit k pages and an unbounded sized slow memory. Accessing items in the cache costs 0 units of time while accessing items in the slow memory costs 1 unit of time. When accessing an item in the slow memory, we must bring it into the cache by evicting an incumbent item if the cache was full. Starting with an empty cache, what is the best online strategy for maintaining items in the cache to minimize the total access cost on a sequence of queries?*

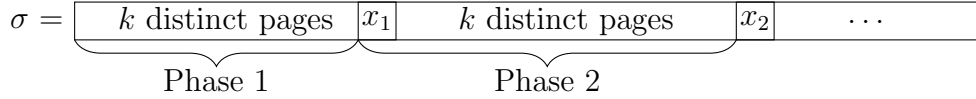
Denote *cache miss* as accessing an item that is not in the cache. Any sensible strategy should aim to reduce the number of cache misses. For example, if $k = 3$ and $\sigma = \{1, 2, 3, 4, 2, 3, 4, \dots, 2, 3, 4\}$, keeping item 1 in the cache will incur several cache misses. Instead, the strategy should aim to keep items $\{2, 3, 4\}$ in the cache. We formalize this notion in the following definition of *conservative strategy*.

Definition 5.5 (Conservative strategy). *A strategy is conservative if on any consecutive subsequence that includes only k distinct pages, there are at most k cache misses.*

Remark Some natural paging strategies such as “Least Recently Used (LRU)” and “First In First Out (FIFO)” are conservative.

Claim 5.6. *Any conservative online algorithm $\mathcal{A}$ is k -competitive.*

Proof. For any given input sequence σ , partition σ into m *maximal phases* — $P_1, P_2, \dots, P_m$ — where each phase has k distinct pages, and a new phase is created only if the next element is different from the ones in the current phase. Let x_i be the first item that does not belong in Phase i .



After x_{i-1} , the element x_{i-1} is in the cache. Hence, of the k other elements in $P_i \setminus \{x_{i-1}\} \cup \{x_i\}$, at least one is not in the cache. Thus, OPT has to pay ≥ 1 to handle the elements in $P_i \setminus \{x_{i-1}\} \cup \{x_i\}$, for any $i > 1$, and it also needs to pay 1 for the first element. Therefore, $c_{\text{OPT}} \geq m$. On the other hand, since $\mathcal{A}$ is conservative, $\mathcal{A}$ has $\leq k$ cache misses per phase. Hence, $c_{\mathcal{A}}(\sigma) \leq k \cdot m \leq k \cdot c_{\text{OPT}}(\sigma)$. $\square$

We show next that the competitive ratio k is tight for all deterministic algorithms.

Claim 5.7. *If $\mathcal{A}$ is a deterministic online algorithm that is α -competitive, then $\alpha \geq k$.*

Proof. Consider the following iteratively constructed adversarial input sequence σ on $k + 1$ pages. Assume the adversary has already decided about the first $\sigma(1), \dots, \sigma(t)$. Since $\mathcal{A}$ is deterministic, the adversary can simulate $\mathcal{A}$ for time t and knows which elements are in the cache at time t . Since the cache has size k , at least one item is not in the cache. Then the adversary picks $\sigma(t + 1)$ as the item not in the cache. By construction, $c_{\mathcal{A}}(\sigma) = |\sigma|$.

On the other hand, since OPT can see the entire sequence σ , OPT can choose to evict the page i that is requested furthest in the future. The next request for page i has to be at least k requests ahead in the future, since by definition of i all other pages $j \neq i \in \{1, \dots, k + 1\}$ have to be requested before i . Thus, in every k steps, OPT has ≤ 1 cache miss. Therefore, $c_{\text{OPT}} \leq \frac{|\sigma|}{k}$ which implies: $k \cdot c_{\text{OPT}} \leq |\sigma| = c_{\mathcal{A}}(\sigma)$. $\square$

Remark A randomized algorithm can achieve $\mathcal{O}(\log k)$ -competitiveness. This will be covered in the next lecture.

5.2.1 Types of adversaries

Since online algorithms are analyzed on all possible input sequences, it helps to consider adversarial inputs that may induce the worst case performance

for a given online algorithm $\mathcal{A}$. To this end, one may wish to classify the classes of adversaries designing the input sequences (in increasing power):

Oblivious The adversary designs the input sequence σ at the beginning. It knows the algorithm $\mathcal{A}$, but does not know any randomness used by $\mathcal{A}$.

Adaptive At each time step t , the adversary knows all randomness used by algorithm $\mathcal{A}$ thus far. In particular, it knows the exact state of the algorithm. With these in mind, it then picks the $(t + 1)$ -th element in the input sequence.

Fully adaptive The adversary knows all possible randomness that will be used by the algorithm $\mathcal{A}$ when running on the full input sequence σ . For instance, assume the adversary has access to the same pseudorandom number generator used by $\mathcal{A}$ and can invoke it arbitrarily many times while designing the adversarial input sequence σ .

Remark If $\mathcal{A}$ is deterministic, then all three classes of adversaries have the same power.

An oblivious adversary models the most common setting in theoretical computer science: to be competitive for a worst-case input sequence, since the input is typically controlled by an unknown user. However, the user has no access to (or interest in) the random number generator of our algorithm. The other two adversaries appear in interactive or in cryptographic settings where an attacker tries to break the algorithm.

Remark For an oblivious adversary, an α -competitive online algorithm $\mathcal{A}$ refers to the expected cost over the random choices of $\mathcal{A}$, so for any input sequence σ we require $\mathbb{E}[c_{\mathcal{A}}(\sigma)] \leq \alpha \cdot c_{OPT}(\sigma)$.

5.2.2 Random Marking Algorithm (RMA)

Consider the Random Marking Algorithm (RMA), a $\mathcal{O}(\log k)$ -competitive algorithm for paging against oblivious adversaries:

- Initialize all pages as marked
- Upon request of a page p
 - If p is not in cache,
 - * If all pages in cache are marked, unmark all
 - * Evict a random unmarked page

- Mark page p

In the initial phase of the algorithm in which the cache is not yet full, we simply insert new pages and mark them. We will ignore this initial phase in the analysis.

Example Suppose $k = 3$, $\sigma = (1, 3, 4, 2, 5, 2, 1, 3)$.

After the initial three steps, the cache is:

Cache	1	3	4
Marked?	✓	✓	✓

When $\sigma(4) = 2$ arrives, all pages are unmarked. Suppose the random eviction chose page ‘3’. The newly added page ‘2’ is then marked.

Cache	1	2	4
Marked?	✗	✓	✗

When $\sigma(5) = 5$ arrives, suppose random eviction chose page ‘4’ (between pages ‘1’ and ‘4’). The newly added page ‘5’ is then marked.

Cache	1	2	5
Marked?	✗	✓	✓

When $\sigma(6) = 2$ arrives, page ‘2’ in the cache is marked (no change).

Cache	1	2	5
Marked?	✗	✓	✓

When $\sigma(7) = 1$ arrives, page ‘1’ in the cache is marked. At this point, any page request that is not from $\{1, 2, 5\}$ will cause a full unmarking of all pages in the cache.

Cache	1	2	5
Marked?	✓	✓	✓

When $\sigma(8) = 3$ arrives, all pages are unmarked. Suppose the random eviction chooses page ‘5’. The newly added page ‘3’ is then marked.

Cache	1	2	3
Marked?	✗	✗	✓

We denote a phase as the time period between 2 consecutive full unmarking steps. That is, each phase is a maximal run where we access k distinct pages. In the above example, $\{2, 5, 2, 1\}$ is such a phase for $k = 3$.

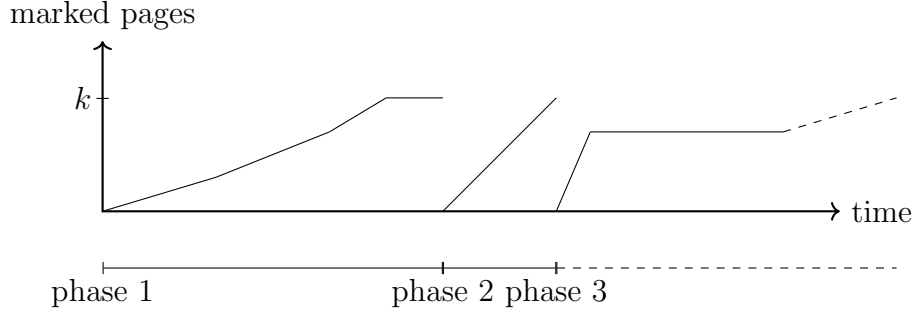


Figure 5.1: The number of marked pages within a phase is monotonically increasing.

Observation As pages are only unmarked at the beginning of a new phase, the number of unmarked pages is monotonically decreasing within a phase.

Theorem 5.8. *RMA is $\mathcal{O}(\log k)$ -competitive against any oblivious adversary.*

Proof. Let P_i be the set of pages at the *start* of phase i . Since requesting a marked page does not incur any cost, it suffices to analyze the *first time* any request occurs within the phase.

Let m_i be the number of *unique new requests* (pages that are not in P_i) and o_i as the number of *unique old requests* (pages that are in P_i). By definition, $o_i \leq k$ and $m_i + o_i = k$.

We have $c_{RMA}(\text{Phase } i) = (\text{Cost due to new requests}) + (\text{Cost due to old requests})$. Since every new request incurs a cost of 1, the cost due to these requests is m_i . We now focus on the extra cost incurred from the old requests, that is when an old page is requested that has already been kicked out upon the arrival of a new request.

Order the old requests in the order which they appear in the phase and let x_j be the j^{th} old request, for $j \in \{1, \dots, o_i\}$. Define l_j as the number of *distinct new requests* before x_j . and consider the first time the j^{th} old request x_j occurs. At this point there are $l_j + j - 1$ marked pages in the cache, namely l_j new pages and $j - 1$ old pages. The other $k - l_j - (j - 1)$ pages in the cache are a subset of the $k - (j - 1)$ pages in P_i that have not been queried yet in this phase. Since the adversary is oblivious, all these pages have the same probability still to be in the cache. Since x_j is one of them, the probability that x_j is still in the cache is $\frac{k - l_j - (j - 1)}{k - (j - 1)}$. Then,

Cost due to old requests

$$\begin{aligned}
&= \sum_{j=1}^{o_i} \Pr[x_j \text{ is not in cache when requested}] && \text{Sum over old requests} \\
&= \sum_{j=1}^{o_i} \frac{l_j}{k - (j - 1)} && \text{From above} \\
&\leq \sum_{j=1}^{o_i} \frac{m_i}{k - (j - 1)} && \text{Since } l_j \leq m_i = |\mathcal{N}| \\
&\leq m_i \cdot \sum_{j=1}^k \frac{1}{k - (j - 1)} && \text{Since } o_i \leq k \\
&= m_i \cdot \sum_{j=1}^k \frac{1}{j} && \text{Rewriting} \\
&= m_i \cdot H_k && \text{Since } \sum_{i=1}^n \frac{1}{i} = H_n
\end{aligned}$$

Together for new and old requests, we get $c_{RMA}(\text{Phase } i) \leq m_i + m_i \cdot H_k$.

We now analyze OPT's performance. By definition of phases, among all requests between two consecutive phases (say, $i - 1$ and i), a total of $k + m_i$ distinct pages are requested. So, OPT has to incur at least $\geq m_i$ to bring in these new pages. To avoid double counting, we lower bound $c_{OPT}(\sigma)$ for both odd and even i : $c_{OPT}(\sigma) \geq \sum_{\text{odd } i} m_i$ and $c_{OPT}(\sigma) \geq \sum_{\text{even } i} m_i$. Together,

$$2 \cdot c_{OPT}(\sigma) \geq \sum_{\text{odd } i} m_i + \sum_{\text{even } i} m_i \geq \sum_i m_i$$

Therefore, we have:

$$c_{RMA}(\sigma) \leq \sum_i (m_i + m_i \cdot H_k) = \mathcal{O}(\log k) \sum_i m_i \leq \mathcal{O}(\log k) \cdot c_{OPT}(\sigma)$$

□

Remark In the above example, $k = 3$, phase 1 = (2, 5, 2, 1), $P_1 = \{1, 3, 4\}$, new requests = {2, 5}, old requests = {1}. Although '2' appeared twice, we only care about analyzing the first time it appeared.

5.2.3 Lower Bound for Paging via Yao's Principle

Yao's Principle Often, it is considerably easier to obtain (distributional) lower bounds against deterministic algorithms, than to (directly) obtain deterministic lower bound instances against randomized algorithms. We use Yao's principle (also called Yao's Minimax Principle) to bridge this gap. Informally, this principle tells us that if *no* deterministic algorithm can do well on a given distribution of random inputs (D), then for any randomized algorithm, there is a deterministic bad input so that the cost of the randomized algorithm on this particular input will be high (C). We next state and prove Yao's principle.

Before getting to the principle, let us observe that given the sequence of random bits used, a randomized algorithm behaves deterministically. Hence, one may view a randomized algorithm as a random choice from a distribution of deterministic algorithms.

Let $\mathcal{X}$ be the space of problem inputs and $\mathcal{A}$ be the space of all possible deterministic algorithms. Denote probability distributions over $\mathcal{A}$ and $\mathcal{X}$ by $p_a = \Pr[A = a]$ and $q_x = \Pr[X = x]$, where X and A are random variables for input and deterministic algorithm, respectively. Define $c(a, x)$ as the cost of algorithm $a \in \mathcal{A}$ on input $x \in \mathcal{X}$.

Theorem 5.9 ([Yao77]).

$$C = \max_{x \in \mathcal{X}} \mathbb{E}_p[c(a, x)] \geq \min_{a \in \mathcal{A}} \mathbb{E}_q[c(a, x)] = D$$

Proof.

$$\max_{x \in \mathcal{X}} \mathbb{E}_p[c(a, x)] \geq \mathbb{E}_q[\mathbb{E}_p[c(a, x)]] = \mathbb{E}_p[\mathbb{E}_q[c(a, x)]] \geq \min_{a \in \mathcal{A}} \mathbb{E}_q[c(a, x)].$$

□

Remark While it is common to use $c(a, x)$ as the cost (e.g., the runtime) of algorithm a on input x , Yao's principle is also true for any other quantity. For example, we could also define $c(a, x)$ as the competitive ratio $\frac{\text{cost of } a \text{ on } x}{\text{cost of best offline algorithm for } x}$, which sometimes makes handling the expectations easier.

Application to the paging problem

Theorem 5.10. *Any (randomized) algorithm has competitive ratio $\Omega(\log k)$ against an oblivious adversary.*

Proof. Consider an arbitrary deterministic algorithm a . We will indeed study $c(a, \sigma)$ as the competitiveness ratio of algorithm a on input σ , so

$$c(a, \sigma) = \frac{\text{cost}(a, \sigma)}{\text{OPT}(\sigma)},$$

where $\text{cost}(a, \sigma)$ is the cost of a on σ and $\text{OPT}(\sigma)$ is the cost of the best offline algorithm for input σ . Fix some (large) $m = |\sigma|$ and let q_σ be the uniform distribution over $\{1, \dots, k+1\}^m$, i.e., the i -th page is drawn from $\{1, \dots, k+1\}$ uniformly at random.

By construction of σ , the probability of having a cache miss is $\frac{1}{k+1}$ for any algorithm a , regardless of what A does. Hence, $\mathbb{E}_q[\text{cost}(a, \sigma)] = \frac{m}{k+1}$.

On the other hand, consider again the greedy offline algorithm a' that choose to evict the page that is requested furthest in the future. As before, we denote a phase as a maximal run where there are k distinct page requests. This means that

$$\text{OPT}(\sigma) \leq \text{cost}(a', \sigma) = \text{number of phases} = \frac{m}{\text{average phase length}}.$$

Therefore, the competitiveness ratio is

$$c(a, \sigma) \geq \frac{\text{cost}(a, \sigma)}{m} \cdot \text{average phase length}.$$

Both the term $\frac{\text{cost}(a, \sigma)}{m}$ and the average phase length are averages over independent random variables, and are therefore strongly concentrated around their expectation. This allows us to compute the expectations separately¹:

$$\mathbb{E}_q[c(a, \sigma)] \geq (1 - o(1)) \cdot \mathbb{E}_q \left[\frac{\text{cost}(a, \sigma)}{m} \right] \cdot \mathbb{E}_q[\text{average phase length}].$$

We already know $\mathbb{E}_q[\frac{\text{cost}(a, \sigma)}{m}] = \frac{1}{k+1}$. To analyze the average length of a phase, also known as *Coupon Collector problem*, suppose there are i distinct pages so far, for $0 \leq i \leq k$. The probability of the next request being new is $\frac{k+1-i}{k+1}$, and one expects to get $\frac{k+1}{k+1-i}$ requests before having $i+1$ distinct pages. Thus, the expected length of a phase is $\sum_{i=0}^k \frac{k+1}{k+1-i} = (k+1) \cdot H_{k+1}$.

¹This can be formally proven by the Chernoff bound. Call the two factors f_1 and f_2 and consider for every $0 \leq x \leq 1$ the event $\mathcal{E}_x$ that “ $f_1 \leq \sqrt{(1-x) \cdot \mathbb{E}[f_1]}$ or $f_2 \leq \sqrt{(1-x) \cdot \mathbb{E}[f_2]}$ ”. Note that this event is unlikely due to concentration, but that it is a necessary condition for the event $f_1 \cdot f_2 \leq (1-x) \cdot \mathbb{E}[f_1] \cdot \mathbb{E}[f_2]$, or equivalently for the event $f_1 \cdot f_2 - \mathbb{E}[f_1] \cdot \mathbb{E}[f_2] \leq -x \cdot \mathbb{E}[f_1] \cdot \mathbb{E}[f_2]$. Then we may bound the lower tail of the expectation $\mathbb{E}[f_1 \cdot f_2] - \mathbb{E}[f_1] \cdot \mathbb{E}[f_2]$ by integrating over all x , and obtain that it is $-o(\mathbb{E}[f_1] \cdot \mathbb{E}[f_2])$.

If m is large enough, then by the law of large numbers the average phase length converges in expectation to $(k+1) \cdot H_{k+1}$.

Putting both bounds together we have shown that for $\mathbb{E}_q[c(a, \sigma)] \geq (1 - o(1)) \cdot \frac{1}{k+1} \cdot (k+1)H_{k+1} = (1 - o(1))H_{k+1}$. Note that the bound was independent of a , so we also have $D = \min_{a \in \mathcal{A}} \mathbb{E}_q[c(a, x)] \geq (1 - o(1))H_k$. From Yao's Principle we know that $C \geq D$, hence for every *randomized* algorithm a there exists an input σ such that $\mathbb{E}[c(a, \sigma)] \geq D$, or in other words $\mathbb{E}[\text{cost}(a, \sigma)] \geq D \cdot \text{OPT}(\sigma) \geq (1 - o(1))H_k \cdot \text{OPT}(\sigma)$. Note that the expectation is now taken over the random choices of a . Hence, no randomized algorithm can achieve a competitiveness ratio better than $\Omega(\log k)$. $\square$

Part II

Algorithms for Big Data

Chapter 6

Advanced Hashing

Hashing is one of the backbones of efficient computing, and in this chapter we will explore several of its powerful applications. Specifically, we will study:

1. **Signatures:** How to assign large items compact signatures while still distinguishing distinct items reliably. This is useful, for example, in string matching and fingerprinting algorithms.
2. **Hash Tables via Chaining:** We review the classic hashing with chaining where we store items hashing to the same value in a linked list/bucket.
3. **Linear Probing:** We then introduce linear probing, an even simpler hash table implementation that is surprisingly efficient but requires more advanced analysis techniques to understand its performance.

A key theme of this chapter is the interplay between theory and practice. Often, it is convenient to reason about hashing under idealized assumptions, which makes the analysis clean and intuitive. The second step, however, is to ask whether the same algorithms continue to perform well with realistic, practical hash functions, and if so, under what conditions. This perspective will guide our study of independence requirements, collision probabilities, and load balancing throughout the chapter.

6.1 Idealized Hash Functions vs. Practical Hash Functions

For hashing applications, it is often convenient to assume that hash functions have full independence. This is also often called the Simple Uniform Hashing Assumption.

Definition 6.1. Given a family of functions $\mathcal{H}_{U,m}$ mapping universe $[U] = \{0, 1, \dots, U-1\}$ to a domain $[m]$. We say $\mathcal{H}_{U,m}$ is

1. uniform if $\forall x \in [U]$ and $i \in [m]$, $\Pr_{h \in \mathcal{H}}[h(x) = i] = 1/m$, and
2. fully-independent if the variables $h(0), h(1), \dots, h(U-1)$ are independent.

Remark We will leave the underlying distribution implicit, in our lectures we will only use the uniform distribution, i.e. each hash function in the family $\mathcal{H}_{U,m}$ receives the same probability mass.

Being uniform and fully-independent is the most that you could expect from any (family of) hash function(s). And, assuming that your underlying hash function is drawn from such a family, analyzing hashing schemes is often quite straightforward. Thus, this assumption is particularly useful when designing algorithms, as it enables us to focus on concise statements of the problems.

While all hash functions that we work with henceforth are uniform, unfortunately, the ask for full independence is unrealistic for most scenarios. Again, this idealization helps when first designing algorithms. But it can be verified that any fully-independent function requires $\Omega(U \log_2 m)$ random bits, which makes it vastly impractical since in most applications U is much larger than m . Let us therefore explore the concept of k -wise independent hash functions. We start by defining k -wise independent random variables.

Definition 6.2. We say that a set of random variables $X_0, X_1, \dots, X_{n-1} \in [m]$ are k -wise independent if for any subset $I \subseteq [n]$ with $|I| \leq k$ and values $x_0, x_1, \dots, x_{n-1} \in [m]$, we have

$$\Pr[\wedge_{i \in I} X_i = x_i] = \prod_{i \in I} \Pr[X_i = x_i].$$

We next define k -wise independent hash functions.

Definition 6.3. We say $\mathcal{H}_{k,U,m}$ is a k -wise independent family if $h(0), h(1), \dots, h(U-1)$ are k -wise independent random variables.

Note that this gives a clean generalization of full independence in [Theorem 6.1](#), which translates to U -wise independence in the above definition. While having k -wise independence for large k is naturally desirable, it comes at a cost; the larger k , the more space and evaluation time, a hash function takes. Thus, in our later applications, we will try to show that k -wise independence for a small k suffices.

Let us start with 2-wise independence (1-wise independence is trivial). The most commonly known hash function that is uniform and 2-wise independent is the function

$$h_{a,b}(x) = (ax + b) \mod m$$

where m is assumed to be prime and a, b are uniformly, independently taken from $[m]$. We also assume that the domain $[U]$ coincides with the co-domain, i.e. $U = m$. In the exercises, you are asked to remove this assumption. Let us prove that these properties indeed hold.

Lemma 6.4. *For m prime, $\mathcal{G}_{2,m,m} = \cup_{a,b \in [m]} \{h_{a,b}(x) = (ax + b) \mod m\}$ is a uniform, 2-wise independent family of hash functions.*

Proof. Note that for any $x, y \in [m], x \neq y$, and any $i, j \in [m]$, we have that the events $h_{a,b}(x) = i$ and $h_{a,b}(y) = j$ can be written as a set of linear equations (over the finite field $[m]$ with modulo m multiplication and addition)

$$\begin{pmatrix} 1 & x \\ 1 & y \end{pmatrix} \begin{pmatrix} b \\ a \end{pmatrix} \equiv \begin{pmatrix} i \\ j \end{pmatrix} \mod m$$

and since $\det \begin{pmatrix} 1 & x \\ 1 & y \end{pmatrix} \mod m \equiv y - x \mod m \not\equiv 0$, we have that $\begin{pmatrix} 1 & x \\ 1 & y \end{pmatrix}$ is invertible, and thus there is a unique solution a^*, b^* to this system of linear equations (in fact, one can prove a one-to-one correspondence between pairs $(a, b) \in [m]^2$ and $(i, j) \in [m]^2$). Since each pair a, b is sampled with probability $1/m^2$, we thus have that

$$\Pr_{a,b}[h_{a,b}(x) = i \wedge h_{a,b}(y) = j] = \Pr_{a,b}[a = a^* \wedge b = b^*] = 1/m^2.$$

From this, we immediately obtain uniformity since for each $i \in [m]$ (and any $y \neq x$), we have

$$\Pr_{a,b}[h_{a,b}(x) = i] = \sum_{j \in [m]} \Pr_{a,b}[h_{a,b}(x) = i \wedge h_{a,b}(y) = j] = 1/m.$$

Thus,

$$\Pr_{a,b}[h_{a,b}(x) = i \wedge h_{a,b}(y) = j] = \Pr_{a,b}[h_{a,b}(x) = i] \cdot \Pr_{a,b}[h_{a,b}(y) = j]$$

and therefore $h(x)$ and $h(y)$ are 2-wise independent random variables. $\square$

For the rest of this chapter, we need even stronger hash functions. Luckily, we can generalize our example of a family 2-wise independent functions to get k -wise independence for any $k \geq 2$. More precisely, given k , we take $a_1, \dots, a_k \in [m]$ drawn independently and uniformly at random and define the hash function

$$h_k(x) = \sum_{i=1}^k a_i x^{i-1} \mod m$$

Our new proof is a generalization of the proof before.

Lemma 6.5. *For m being prime, $\mathcal{G}_{k,m,m} = \cup_{a_1, a_2, \dots, a_k \in [m]} \{h_k(x) = \sum_{i=1}^k a_i x^{i-1} \mod m\}$ is a uniform, k -independent family of hash functions.*

Proof. For any distinct $x_1, x_2, \dots, x_k \in [m]$, i.e. $x_i \neq x_j$ for all $i \neq j$, and $i_1, i_2, \dots, i_k \in [m]$, we can write $h_k(x_\ell) = i_\ell$ as linear equations

$$\begin{pmatrix} x_1^0 & x_1^1 & \dots & x_1^{k-1} \\ x_2^0 & x_2^1 & \dots & x_2^{k-1} \\ \dots & \dots & \dots & \dots \\ x_k^0 & x_k^1 & \dots & x_k^{k-1} \end{pmatrix} \begin{pmatrix} a_1 \\ a_2 \\ \dots \\ a_k \end{pmatrix} = \begin{pmatrix} i_1 \\ i_2 \\ \dots \\ i_k \end{pmatrix}$$

The matrix on the left side is also known as [Vandermonde Matrix](#) and it is well-known that this matrix is non-singular if x_i 's are distinct. Thus, we have a unique solution $(a_1^*, a_2^*, \dots, a_k^*)$ for every tuple of values $(i_1, i_2, \dots, i_k)$ and since the former are chosen independently and uniformly at random, each such event occurs with probability $1/m^k$.

The rest of the proof follows by the same line of argumentation as before. $\square$

The most important takeaway from this section is that higher independence requires evaluating a higher-degree polynomial, which translates into higher space and time bounds for hash functions. Therefore, it is highly desirable to use as little independence as possible.

Finally, we point out that the hash functions presented here have rather large hidden constants as they require taking a modulo, which is a rather slow operation. See [here](#) for a very good resource on practical hash functions that are often much faster (see in particular the multiply-shift hash function).

6.2 Signatures via Hashing

Assume you are given a collection of n strings $s_1, s_2, \dots, s_n \in [U]$ where $[U]$ is a very large universe. Upon the arrival of a new string s_{n+1} , you want to check if this string is a duplicate of some string we have seen before.

To solve this problem efficiently, we can use a hash function $h : [U] \mapsto [n^2]$ and then check if $h(s_{n+1})$ hashes to any of the other hash values. If $s_{n+1} = s_i$ for some i then we have a collision and detect the duplicate. Otherwise, the probability of a collision with string s_i is

$$\Pr[h(s_{n+1}) = h(s_i) \mid s_{n+1} \neq s_i] = 1/n^2$$

Thus, by a simple union bound, we can conclude that a collision occurs with probability at most $n/n^2 = 1/n$ and thus essentially negligible. Note that here, we only used the independence between $h(s_{n+1})$ and $h(s_i)$ in each of the analyzed events! Thus, it suffices to use a 2-independent hash function!

6.3 Hashing with Chaining

Given a collection of items $S \subseteq [U]$ where U is the universe size. Our goal is to have a fast data structure for insertions/deletions to S and membership queries, i.e. to check if some $x \in U$ is in S or not. We let n be a pre-fixed upper bound on the size of the collection S at any point in time.

In hashing with chaining, the hash table H is taken to be an m -sized array (typically m is $\Theta(n)$) and each entry $H[j]$ stores a linked list (or bucket) containing all items $x \in S$ with $h(x) = j$. Clearly, the time for insertions/deletions or membership queries is linearly related to the size of the list.

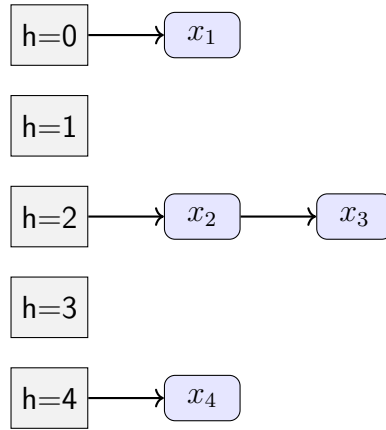


Figure 6.1: Hashing with chaining: collisions are stored in linked lists.

Expected Runtime. Let us first bound the expected runtime, that is, we bound the expected length of the list containing key x .

$$\mathbb{E}[|H[h(x)]|] = 1 + \sum_{x \neq y \in S} \Pr[h(y) = h(x)] = 1 + \sum_{x \neq y \in S} 1/m \leq 1 + n/m.$$

This is pretty much what we expect: the expected time of operations is guided by the *load* n/m of the hash table. Again, we only used independence of $h(x)$ and $h(y)$ in each of the events and thus, a 2-independent hash function suffices!

Worst-Case Runtime. For simplicity, assume $m = n$. For any k , we have that the probability that list $H[i]$ has length k is

$$\Pr[|H[i]| = k] \leq \sum_{X \subseteq S, |X|=k} \Pr[\forall x \in X, h(x) = i] = \binom{n}{k} \frac{1}{n^k} \leq \left(\frac{e}{k}\right)^k.$$

using $\binom{n}{k} \leq \left(\frac{ne}{k}\right)^k$. But for $k = O(\log n / \log \log n)$, this yields a probability of less than n^{-c} for some arbitrarily large constant c . Using a union bound over the n buckets and over all the bucket sizes $\geq k$ yields that w.p. less than n^{-c+2} no bucket has more than $O(\log n / \log \log n)$ items. Thus, we have w.h.p. that the worst-case operation time is $O(\log n / \log \log n)$. Furthermore, we only used $\Theta(\log n / \log \log n)$ -wise independence in our proof!

6.4 Linear Probing

In linear probing¹, we again store a dynamic collection $S \subseteq [U]$ in a hash table H where the size of S is upper bounded by n at all times. We use a hash function $h : [U] \mapsto [m]$ and assume throughout that $m \geq \frac{3}{2}n$ at all times. However, this time H consists of a single m -sized array and a key $x \in S$ is stored at the first free position in the array at or after index $h(x)$ (here we work with a wrap-around of the array but we will ignore it in our description below to avoid cluttering the discussion).

More precisely, let S be initially empty. We implement the operations:

- **MEMBER?**(x) : let $i = h(x)$, and then check all positions $i \leq j \leq k$ where k is the smallest integer at least i such that $H[k]$ is empty. If x is stored in some entry $H[j]$ return YES otherwise return NO.
- **INSERT**(x): Let k be the smallest integer larger-equal to $i = h(x)$ such that $H[k]$ is empty. Store x in $H[k]$.
- **DELETE**(x): Find the position i where x is stored in H , delete x from $H[i]$, then recursively find an element y that is hashed to $h(y) \leq i$ and stored after position i and move it to $H[i]$.

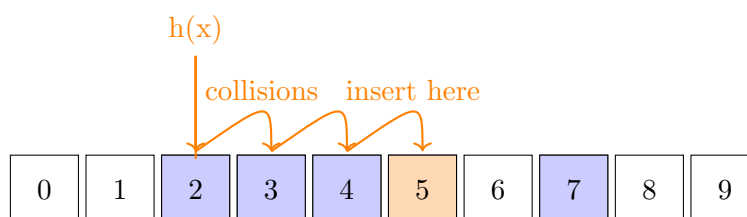


Figure 6.2: Linear probing: when a collision occurs, the next free cell is used. This strategy is what you would probably adopt in street parking: if the first space is blocked, you drive a little further down the road and check the next spot, if it is free, you park, if it is taken, you keep on driving.

It is not hard to verify that the algorithm maintains the invariant that each element $x \in S$ is stored in $H[i]$ for some $i \geq h(x)$ such that all locations $H[h(x)], H[h(x) + 1], \dots, H[i - 1]$ store an item from S . This immediately implies the correctness of the query operation. We next show how to obtain the following runtime bound. Notably, the result is tight: using a hash function with lower k -wise independence could not yield the same bounds.

Theorem 6.6. *Using a 5-wise independent hash function, each operation in Linear Probing takes $O(1)$ time in expectation.*

Linear Probing is very popular in practice due to its simplicity and efficiency. The latter can be explained by how modern computer architecture works: access to memory is done in cache lines and thus even if the item x is not stored in $H[h(x)]$, accessing $H[h(x) + 1], H[h(x) + 2], \dots$ is fairly cheap as it is already pre-loaded in the same cache line. And even in the case of a cache miss, further access is still relatively fast due to modern prefetching techniques. We point the reader to this [resource](#) which inspired these lecture notes and has references to empirical study and further theoretical work (like what kind of practical hash functions work well with linear probing).

Expected Runtime Analysis To analyze the runtime of each of these operations pertaining to an element x , the crucial object to consider is that of a run $R(x)$ which is either empty if $H[h(x)]$ is empty (this could happen in a membership query), or otherwise, it is the maximal interval such that $h(x) \in R(x)$ and $H[r]$ stores an item for every $r \in R(x)$. Clearly, each operation pertaining to x can be implemented in worst-case time $O(|R(x)| + 1)$.

¹Linear probing is also often referred to as Open Addressing. Other names are also used but are less common.

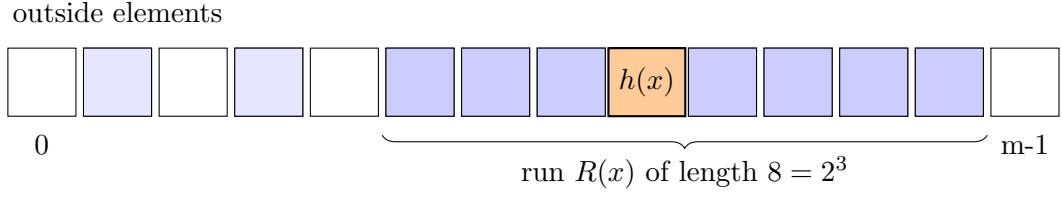


Figure 6.3: The central object in our analysis is a run $R(x)$ of an item $x \in S$: a contiguous block of occupied cells (light blue), containing x (orange). Some other cells outside the run are also occupied (light blue).

For our analysis, we make use of dyadic intervals. That is, we assume that m is a power of 2. We then partition $[m]$ for every $0 \leq \ell \leq \log_2 m$ into ℓ -intervals, that are all the intervals of the form $[i \cdot 2^\ell, (i+1) \cdot 2^\ell)$ for any i .

Given a particular ℓ -interval I , note that when we sample $S \setminus \{x\}$ via h , we expect that there at most $2^\ell \cdot n/m \leq 2^{\ell+2}$ elements from $S \setminus \{x\}$ that have their hash value in I . We will say that a particular ℓ -interval I is *almost-full* if at least $2^\ell \cdot \frac{3}{4}$ items from $S \setminus \{x\}$ hash into I . We make the following observation.

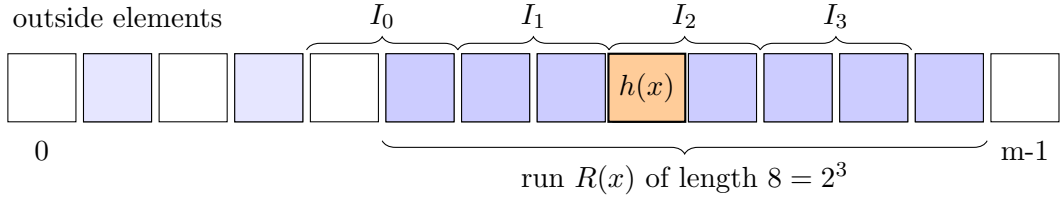


Figure 6.4: Illustration of the proof of [Theorem 6.7](#): The run from [Figure 6.3](#). For $\ell = 1$, we have that the four dyadic intervals I_0, I_1, I_2, I_3 of length 2 that first intersect with the run $R(x)$. It is not hard to see that I_1, I_2, I_3 are full by definition. Since no element that ends up at position y has hash value larger y , we have that all elements from the first 4 intervals have also hashed into these intervals.

Observation 6.7. *If $|R(x)| \in [2^{\ell+2}, 2^{\ell+3})$, then there is at least one ℓ -interval that intersects $R(x)$ and is almost-full.*

Proof. Let $I_0, I_1, \dots, I_3$ be the first four ℓ -intervals that intersect $R(x)$. Clearly, since the run is of length at least $2^{\ell+2}$, we have that I_1, I_2, I_3 are all full and I_0 contains at least one item (as otherwise it would not intersect $R(x)$). But

at the same time, we know that the run started in I_0 and so all items in the first four intervals hashed into I_0, I_1, I_2 or I_3 . But since at least $3 \cdot 2^\ell + 1$ items from S hashed into these four intervals, we have that at least $3 \cdot 2^\ell$ elements from $S \setminus \{x\}$ hashed into these intervals. Thus, on average at least $\frac{3}{4} \cdot 2^\ell$ items from $S \setminus \{x\}$ hash into these intervals and thus at least one of them is almost-full. $\square$

Given this result, let us define P_ℓ to denote the probability that a particular ℓ -interval is almost-full. We have that if $R(x)$ is of size $[2^{\ell+2}, 2^{\ell+3})$, then there are at most $(2^4 + 1)$ ℓ -intervals that could possibly intersect with $R(x)$ (the run of x is contained in the interval $(h(x) - 2^{\ell+3}, h(x) + 2^{\ell+3})$). We can therefore upper bound the probability that $R(x)$ is of size $[2^{\ell+2}, 2^{\ell+3})$, by the probability that any of these ℓ -intervals is almost-full to obtain an upper bound of $(2^4 + 1) \cdot P_\ell$.

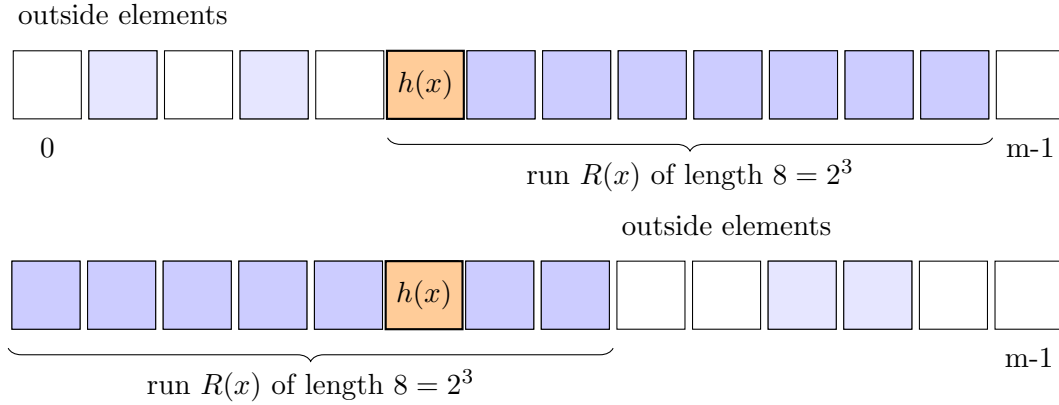


Figure 6.5: A run $R(x)$ of length $8 = 2^3$ could intersect various ℓ -intervals after $h(x)$, but also before $h(x)$ depending on where the run starts. But there are at most $(2^4 + 1)$ 2-intervals that could possibly intersect.

Let us now fix any such ℓ -interval I . For any $y \in S \setminus \{x\}$, we let Y_y be the random indicator variable that takes value 1 if $h(y) \in I$ and 0 otherwise. We let $X = \sum_{y \in S \setminus \{x\}} Y_y$ and have that $\mu = \mathbb{E}[X] = \sum_{y \in S \setminus \{x\}} \mathbb{E}[Y_y] \leq n \cdot 2^\ell / m \leq \frac{2}{3} \cdot 2^\ell$. Note that

$$X \geq \frac{3}{4} 2^\ell \implies |X - \mu| \geq \frac{1}{12} 2^\ell > \frac{1}{10} \mu. \quad (6.1)$$

Assuming h is a fully-independent hash function, which implies variables Y_y are independent, we can apply a Chernoff bound to obtain

$$P_\ell \leq \Pr[|X - \mu| \geq \frac{1}{10} \mu] < e^{-\Omega(\mu)} = e^{-\Omega(2^\ell)} \quad (6.2)$$

Since there are at most $(2^4 + 1)$ relevant ℓ -intervals, by a union bound, we derive that $R(x)$ is of size $[2^{\ell+2}, 2^{\ell+3})$ with probability $P_\ell = O(1/2^{\Omega(2^\ell)})$. Note that if $R(x)$ is of this size, the cost of the operation pertaining to x is $O(2^\ell)$ since we might need to inspect every element of $R(x)$. Summing over all possible values of ℓ (and considering that we only start with intervals of size 2^2), we obtain the total expected runtime

$$O(1) + \sum_{\ell=0}^{\log_2 m} P_\ell \cdot O(2^\ell) = O(1) + \sum_{\ell=0}^{\log_2 m} 1/2^\ell = O(1) \quad (6.3)$$

where we use the crude upper bound $O(2^\ell/2^{2^\ell}) = O(1/2^\ell)$ and that the last sum is geometric. This yields [Theorem 6.6](#).

Removing Full Independence Assumption via Higher-Moment Chebyshev As for previous applications, we show that full independence is not needed. Instead, we argue that a 5-wise independent hash function h suffices.

To this end, we use a tool that is very commonly used in such applications. A higher-order moment version of Chebyshev's inequality. For concreteness, let us state the inequality for 4-wise independence that we will apply.

Lemma 6.8. *Let $Y_0, Y_1, \dots, Y_{n-1} \in \{0, 1\}$ be 4-wise independent random variables each taking value 1 with probability p . Let $X = \sum_i Y_i$, and $\mu = \mathbb{E}[X] = np$, and $p \geq 1/n$, then for any $d > 0$,*

$$\Pr[|X - \mu| > d\sqrt{\mu}] < 4/d^4.$$

We use this tool to bound P_ℓ using only 5-independence. Using $\frac{1}{12}2^\ell > \frac{1}{10}\sqrt{2^\ell\mu}$, we get from [Theorem 6.8](#)

$$P_\ell \leq \Pr[|X - \mu| \geq \frac{1}{10}\sqrt{2^\ell\mu}] < \frac{4}{(\frac{\sqrt{2^\ell}}{10})^4} = 40000/2^{2\ell} = O(1/2^{2\ell}).$$

Here we used that the variables Y_y are 4-wise independent. This is given since h is 5-wise independent and we have only fixed x and therefore $h(x)$ so far, which leaves the other hash values of items in $S \setminus \{x\}$ to be 4-wise independent.

While this compares meagerly to [\(6.2\)](#), it suffices to make the calculation [\(6.3\)](#) hold again since $O(1/2^{2\ell}) \cdot O(2^\ell) = O(1/2^\ell)$. We once again stress that this result is tight: less independence does not yield constant expected update time!

Proving the 4-th Moment Chebyshev Bound. It remains to prove a bound on the 4-th moment. The proof below generalizes the classic proof of Chebyshev's inequality.

Lemma 6.9. *Let $Y_0, Y_1, \dots, Y_{n-1} \in \{0, 1\}$ be 4-wise independent random variables each taking value 1 with probability p . Let $X = \sum_i Y_i$ and $\mu = \mathbb{E}[X] = np$ with $p \geq 1/n$, then for any $d > 0$,*

$$\Pr[|X - \mu| > d\sqrt{\mu}] < 4/d^4.$$

Proof. We first use Markov's inequality, which yields

$$\Pr[|X - \mu| > d\sqrt{\mu}] = \Pr[(X - \mu)^4 > d^4\mu^2] \leq \frac{\mathbb{E}[(X - \mu)^4]}{d^4\mu^2}.$$

The key part of the analysis is to analyze the fourth moment $\mathbb{E}[(X - \mu)^4]$. We can write

$$\begin{aligned} \mathbb{E}[(X - \mu)^4] &= \sum_{i,j,k,l \in [n]} \mathbb{E}[(Y_i - p)(Y_j - p)(Y_k - p)(Y_l - p)] \\ &= \sum_i \mathbb{E}[(Y_i - p)^4] + \binom{4}{2} \sum_{i < j \in [n]} \mathbb{E}[(Y_i - p)^2(Y_j - p)^2] \end{aligned}$$

which follows because terms with $i \neq j \neq k \neq l$ contribute zero since by 4-wise independence yields

$$\begin{aligned} \mathbb{E}[(Y_i - p)(Y_j - p)(Y_k - p)(Y_l - p)] &= \mathbb{E}[(Y_i - p)] \cdot \mathbb{E}[(Y_j - p)(Y_k - p)(Y_l - p)] \\ &= 0 \cdot \mathbb{E}[(Y_j - p)(Y_k - p)(Y_l - p)] \end{aligned}$$

where we also used that $\mathbb{E}[Y_i - p] = 0$. And similarly, for terms with $i \neq j$, $\mathbb{E}[(Y_i - p)(Y_j - p)^3]$, we can again pull out the first term and obtain $\mathbb{E}[(Y_i - p)(Y_j - p)^3] = \mathbb{E}[(Y_i - p)] \cdot \mathbb{E}[(Y_j - p)^3] = 0$.

Bounding $\mathbb{E}[(Y_i - p)^4]$: We observe that $-1 \leq Y_i - p \leq 1$ and therefore for every $\ell \geq 0$, we have $(Y_i - p)^\ell \leq 1$. Thus, $\mathbb{E}[(Y_i - p)^4] \leq \mathbb{E}[(Y_i - p)^2] = p \cdot (1 - p)^2 + (1 - p) \cdot p^2 = p - p^2 \leq p$.

Bounding $\mathbb{E}[(Y_i - p)^2(Y_j - p)^2]$: We have independence of Y_i and Y_j and thus

$$\mathbb{E}[(Y_i - p)^2(Y_j - p)^2] = \mathbb{E}[(Y_i - p)^2] \cdot \mathbb{E}[(Y_j - p)^2] = \mathbb{E}[(Y_i - p)^2]^2.$$

Then, we can just use again that $\mathbb{E}[(Y_i - p)^2] \leq p$ and thus $\mathbb{E}[(Y_i - p)^2(Y_j - p)^2] \leq p^2$.

Completing the Proof: It remains to put everything together to obtain

$$\begin{aligned}\mathbb{E}[(X - \mu)^4] &= \sum_i \mathbb{E}[(Y_i - p)^4] + \binom{4}{2} \sum_{i < j \in [n]} \mathbb{E}[(Y_i - p)^2(Y_j - \mu_j)^2] \\ &\leq \sum_i p + \binom{4}{2} \sum_{i < j} p^2 \\ &< pn + 6 \cdot p^2 n^2 / 2 \leq 4(pn)^2\end{aligned}$$

where we used the assumption $p \geq 1/n$ in the final inequality. This finally yields

$$\Pr[|X - \mu| > d\sqrt{\mu}] \leq \frac{\mathbb{E}[(X - \mu)^4]}{d^4 \mu^2} \leq \frac{4(pn)^2}{d^4 (pn)^2} \leq \frac{4}{d^4}.$$

□

In the exercise, we will ask you to prove the following generalization of the above statement for any k . The tool below is extremely useful in bounding the amount of independence necessary. Note that for $k = \Theta(\log n)$, probabilities typically become polynomially small, which suffices for many applications, and thus, quite often, $O(\log n)$ -wise independence is good enough.

Theorem 6.10. *For any even $k \geq 2$, let $Y_0, Y_1, \dots, Y_{n-1} \in \{0, 1\}$ be k -wise independent random variables each taking value 1 with probability p . Let $X = \sum_i Y_i$ and $\mu = \mathbb{E}[X] = np$, and $p \geq 1/n$, then for any $d > 0$,*

$$\Pr[|X - \mu| > d\sqrt{\mu}] = O(k^k / (2d)^k),$$

and in particular, for k constant the probability is upper bounded by $O(d^{-k})$.

References

This chapter was written by Maximilian Probst. The content of the chapter is similar to the content in lectures 3 and 4 of [Advanced Algorithms](#) by Jelani Nelson. The content for linear probing is heavily inspired by Mikkel Thorup's excellent lecture notes [\[Tho15\]](#).

Chapter 7

Streaming Algorithms

7.1 The Streaming Model

Thus far, we have been ensuring that our algorithms run fast. What if our system does not have sufficient memory to store all data to post-process it? For example, a router has a relatively small amount of memory while a tremendous amount of routing data flows through it. In a memory-constrained setting, can one compute something meaningful, possibly approximately, with a limited amount of memory?

More formally, we now look at a slightly different class of algorithms where data elements from $[n] = \{0, \dots, n-1\}$ arrive one at a time, in a stream $S = a_1, \dots, a_m$, where $a_i \in [n]$ arrives in the i^{th} time step. At each step, our algorithm performs some computation¹ and discards the item a_i . When the end of the stream is announced², i.e. after the m -th item was discarded, the algorithm should give us a value that approximates some value of interest.

7.2 Frequency Moments of a Stream

The streaming model was originally introduced in [AMS96] by Alon, Matias and Szegedy while they were working at AT&T. In this paper, they give a systematic study of how to maintain the moments of a stream. The paper has since been extremely influential and has won a [Gödel prize in 2005](#).

Let us now define what we mean by moments of a stream. For a given stream S , and items $j \in [n]$, we define f_j as the number of times item j appears in a stream S . Then, the k^{th} moment of a stream S is defined

¹Usually this requires only little time so we ignore the runtime.

²In general, the length of the stream, m , may not be known.

as $F_k \stackrel{\text{def}}{=} \sum_{j=1}^n (f_j)^k$. We will see algorithms for estimating the following moments:

- The 0-th moment: defining $0^0 = 0$, the zeroth moment $F_0 = \sum_{j=1}^n f_j^0$ is the number of distinct elements in the stream S .
- The 1-st moment: the first moment $F_1 = \sum_{j=1}^n f_j = m$ is simply the number of elements in the stream S .
- The 2-nd moment: $F_2 = \sum_{j=1}^n f_j^2$ measures the spikiness of the stream. This is very well-motivated by applications in databases.

The goal of this chapter is to study various streaming algorithms to compute frequency moments. We note that the algorithm for estimating the 2-nd moment that we present extends to estimate the k -th moment for arbitrary $k \geq 2$, however, as k increases, space goes to n , and has an $\Omega(n)$ space lower bound for the ∞ -moment, i.e. how often the item that appears the most often appears. Since ∞ moments are of high interest, people have studied various interesting variants that bypass these lower bounds. These algorithms are known under the name *heavy hitters* (see for example the [Misra-Gries heavy hitters algorithm](#)).

7.3 Estimating the first moment of a stream

Let us first consider the problem of estimating the first moment of a stream a.k.a. estimating the number of elements of the stream S .

A trivial exact solution would be to use $\mathcal{O}(\log m)$ bits to maintain a counter, incrementing for each element observed. To maintain an exact solution, it can be shown to be necessary to have $\Omega(\log m)$ bits of space.

Thus, if the amount of space available is $o(\log m)$, we have to resort to an approximation algorithm. For the rest of this section, we prove the following theorem which obtains a $(1 + \varepsilon)$ -approximation using only $O(\log \log m)$ bits for $\varepsilon \in \Theta(1)$ with constant probability. Note that this is an exponential improvement over maintaining an exact counter.

Theorem 7.1. *There is an algorithm that for any stream $S = a_1, \dots, a_m$ with $a_i \in [n]$ and $\epsilon, \delta > 0$, uses space $O(\epsilon^{-2} \log \log(m) \cdot \log(1/\delta))$, and outputs with probability at least $1 - \delta$, an estimate $\tilde{D}$ such that $(1 - \epsilon)F_1 \leq \tilde{D} \leq (1 + \epsilon)F_1$.*

7.3.1 Morris Counter

A very famous algorithm for approximate counting is [Morris' Algorithm](#) given below. This first version of the algorithm serves as a warm-up and achieves the guarantees stated in [Theorem 7.1](#) for $\varepsilon > 1/\sqrt{2}$ and some $\delta = \Omega(1)$.

There are two ideas that are crucial in Morris' algorithm:

1. Instead of maintaining the current exact count which requires $\mathcal{O}(\log m)$ bits, the algorithm tries to maintain the exponent i such that for current length of the stream m , we have $2^i \leq m < 2^{i+1}$. Since the largest i that needs to be recorded is in $O(\log \log m)$, this improves the space usage exponentially.
2. Unfortunately, maintaining i precisely is difficult as one would exactly have to know when m is a power of two.

Instead, the algorithm uses randomization: given current estimate X of exponent i , it tries to increase X after having observed 2^X new elements since the last increase of X . To do so, whenever a new item arrives, it increases X with probability 2^{-X} . As we will see, 2^X yields an unbiased estimator of 2^i (shifted by 1).

We give the algorithm MORRIS below.

Algorithm 9 MORRIS($S = \{a_1, \dots, a_m\}$)

```

 $X \leftarrow 0$ 
for  $a_i \in S$  do                                ▷ Items arrive in streaming fashion
     $r \leftarrow$  random integer drawn from  $[2^X]$ 
    if  $r = 0$  then                                ▷ If not,  $x$  is unchanged.
         $X \leftarrow X + 1$ 
    end if
end for
return  $2^X - 1$                                 ▷ Estimate  $m$  by  $2^X - 1$ 

```

For analysis, denote X_m as the value of counter X after exactly m items arrive.

Theorem 7.2. $\mathbb{E}[2^{X_m} - 1] = m$. That is, MORRIS is an unbiased estimator for the length of the stream.

Proof. Equivalently, let us prove $\mathbb{E}[2^{X_m}] = m + 1$, by induction on $m \in \mathbb{N}^+$. On the first element ($m = 1$), X increments with probability 1, so $\mathbb{E}[2^{X_1}] = 2^1 = m + 1$. Suppose it holds for some $m \in \mathbb{N}$, then

$$\begin{aligned}
\mathbb{E}[2^{X_{m+1}}] &= \sum_{j=1}^m \mathbb{E}[2^{X_{m+1}} | X_m = j] \Pr[X_m = j] && \text{Condition on } X_m \\
&= \sum_{j=1}^m (2^{j+1} \cdot 2^{-j} + 2^j \cdot (1 - 2^{-j})) \cdot \Pr[X_m = j] && \text{Increment } x \text{ w.p. } 2^{-j} \\
&= \sum_{j=1}^m (2^j + 1) \cdot \Pr[X_m = j] && \text{Simplifying} \\
&= \sum_{j=1}^m 2^j \cdot \Pr[X_m = j] + \sum_{j=1}^m \Pr[X_m = j] && \text{Splitting the sum} \\
&= \mathbb{E}[2^{X_m}] + \sum_{j=1}^m \Pr[X_m = j] && \text{Definition of } \mathbb{E}[2^{X_m}] \\
&= \mathbb{E}[2^{X_m}] + 1 && \sum_{i=1}^m \Pr[X_m = j] = 1 \\
&= (m + 1) + 1 && \text{Induction hypothesis} \\
&= m + 2
\end{aligned}$$

□

Claim 7.3. $\mathbb{E}[2^{2X_m}] = \frac{3}{2}m^2 + \frac{3}{2}m + 1$

Proof. Prove the above claim again by induction on m as an exercise. □

Claim 7.4. $\text{Var}(2^{X_m} - 1) = \mathbb{E}[(2^{X_m} - 1 - m)^2] \leq \frac{m^2}{2}$

Proof. Use the Claim 7.3 and establish this claim as an exercise. □

Theorem 7.5. For $\ell > 0$, $\Pr[|(2^{X_m} - 1) - m| > \ell m] \leq \frac{1}{2\ell^2}$.

Proof. We can now use Chebyshev's inequality to derive

$$\begin{aligned}
\Pr[|(2^{X_m} - 1) - m| > \ell m] &\leq \frac{\text{Var}(2^{X_m} - 1)}{(\ell m)^2} && \text{Chebyshev's inequality} \\
&\leq \frac{m^2/2}{\ell^2 m^2} && \text{By Claim 7.4} \\
&= \frac{1}{2\ell^2}
\end{aligned}$$

□

Note that for Morris Algorithm for constant $\ell > 1/\sqrt{2}$, we obtain an algorithm that requires with constant probability at most $O(\log \log m)$ bits and obtains a meaningful guarantee from the above bound, yielding a constant approximation. Here, we point out that drawing a random r from $[2^x]$ and testing $r = 0$ requires $O(x)$ bits. Since we have with constant probability that $x \in O(\log m)$, storing the sampled number takes too much memory. Instead, we actually flip x coins, and check that all of them are zero. For this, we need a counter for how many bits we have already sampled, which takes $O(\log x) = O(\log \log m)$ bits to store.

7.3.2 Morris Counter+ (The mean trick)

Naturally, we would like for a much more fine-grained estimator, i.e. an estimator that allows for a better approximation factor than $(1 + 1/\sqrt{2}) \approx 1.7$. We therefore use a common trick in streaming: we compute multiple estimators and take the mean to reduce variance.

Algorithm 10 MORRIS+($S = \{a_1, \dots, a_m\}, k$)

```

for  $i = 1, 2, \dots, k$  do
     $Y_i \leftarrow \text{MORRIS}(S = \{a_1, \dots, a_m\})$ 
end for
return  $\bar{Y} = \frac{1}{k} \sum_{i=1}^k Y_i$ 

```

To analyze this algorithm, we use the facts that for $Y_1, Y_2, \dots, Y_k$ being independent random variables, we have $\text{Var}(\sum_i Y_i) = \sum_i \text{Var}(Y_i)$ and that for every $c > 0$, we have $\text{Var}[c \cdot X] = c^2 \cdot \text{Var}[X]$.

Applying Chebyshev's inequality again but this time to variable $\bar{Y} = \frac{1}{k} \sum_{i=1}^k Y_i$ which is an unbiased estimator of $m + 1$ by [Theorem 7.2](#) and linearity of expectation, we obtain

$$\begin{aligned}
 \Pr[|\bar{Y} - (m + 1)| > \ell m] &\leq \frac{\text{Var}(\bar{Y})}{(\ell m)^2} && \text{Chebyshev's inequality} \\
 &= \frac{\frac{1}{k^2} \cdot \sum_{i=1}^k \text{Var}(Y_i)}{(\ell m)^2} && \text{By } \text{Var}(\sum_i Y_i) = \sum_i \text{Var}(Y_i) \text{ and Scaling} \\
 &= \frac{\frac{1}{k^2} \cdot \sum_{i=1}^k m^2/2}{(\ell m)^2} && \text{By } \text{Theorem 7.4} \\
 &\leq \frac{\frac{1}{k} m^2/2}{(\ell m)^2} = \frac{1}{2\ell^2 k} && \text{Simplifying}
 \end{aligned}$$

Thus, we know that by using k copies, and therefore $O(k \log \log m)$ space, we can choose $\ell = 1/\sqrt{k}$ and obtain an estimator that is $(1 \pm \frac{1}{\sqrt{k}})$ -approximate with probability at least $1/2$ by the above calculation. Setting $k = \varepsilon^{-2}$, we thus obtain [Theorem 7.1](#) for $\delta = 1/2$.

7.3.3 Morris Counter++ (The median trick)

Finally, we want to boost the probability that our estimator is approximately correct. For this, let us choose ℓ in the analysis above to be $2/\sqrt{k}$, so the accuracy decreases slightly to $(1 + \frac{2}{\sqrt{k}})$ -approximate, but the failure probability decreases to at most $1/4$.

Now we can use another common trick to reduce the failure probability further from $1/4$ to any $0 < \delta < 1/2$: we run $t = 12 \log(1/\delta)$ copies of the above algorithm, and then only output the median estimate (we assume for simplicity that t is an odd number).

Algorithm 11 MORRIS++($S = \{a_1, \dots, a_m\}, k, \delta$)

for $i = 1, 2, \dots, t = 12 \log(1/\delta)$ **do**
 $Z_i \leftarrow \text{MORRIS+}(S = \{a_1, \dots, a_m\})$
end for
return median among all Z_i .

Clearly, the median deviates from an $(1 + 2/\sqrt{k})$ -approximation only if at least half the calls to MORRIS+(S) fail. But at most a quarter of them do so in expectation. Let us denote by I_i the indicator variable that the i -th call to MORRIS+(S) fails, i.e Z_i is not a $(1 \pm \frac{2}{\sqrt{k}})$ -approximation of m .

Since all calls are independent and therefore I_i are independent, we can use Hoeffding's inequality (or by re-arranging also Chernoff bounds work) to bound the failure probability by

$$\Pr \left[\sum_{i=1}^t I_i \geq t/2 \right] \leq \Pr \left[\sum_{i=1}^t I_i - \mathbb{E} \left[\sum_{i=1}^t I_i \right] \geq t/4 \right] \leq e^{-t/12} \leq \delta.$$

Thus, using $O(k \log \log(m) \cdot \log(1/\delta))$ space, we obtain a $(1 + \frac{2}{\sqrt{k}})$ -approximation with probability at least $1 - \delta$. Setting $k = (\varepsilon/2)^{-2}$, this yields [Theorem 7.1](#).³

³Note that technically, we only have proven that with constant probability, MORRIS requires $O(\log \log m)$ bits but we have used here that this is also a bound on the expected number of bits required. We leave it to the reader to think about this detail.

7.4 Estimating the zeroth moment of a stream

Next, we discuss how to compute the (approximate) zeroth moment of a stream a.k.a. the number of distinct elements.

A trivial exact solutions could either use $\mathcal{O}(n)$ bits to track if elements exists, or use $\mathcal{O}(m \log n)$ bits to remember the whole stream. Again, this is optimal (but hardly satisfying) for exact solutions.

In this section, we present an algorithm to obtain an approximation of the number of distinct elements. We summarize the result we obtain in the following theorem.

Theorem 7.6. *There is an algorithm that for any stream $S = a_1, \dots, a_m$ with $a_i \in [n]$ and $\epsilon, \delta > 0$, uses space $O((\log n + \epsilon^{-2} \cdot \log \log(n)) \log(1/\delta))$, and outputs with probability at least $1 - \delta$, an estimate $\tilde{D}$ such that $(1 - \epsilon)F_0 \leq \tilde{D} \leq (1 + \epsilon)F_0$.*

7.4.1 The FM Algorithm

Before proving [Theorem 7.6](#), we present a slightly modified version of the famous algorithm by Flajolet and Martin [[FM85](#)] for this problem. This will serve as a warm-up to get acquainted with some of the techniques that we will then use. We summarize the result of the FM algorithm in the following theorem.

Theorem 7.7. *There is an algorithm that for any stream $S = a_1, \dots, a_m$ with $a_i \in [n]$ and $\delta > 0$, uses space $O(\log n \cdot \log(1/\delta))$, and outputs with probability at least $1 - \delta$, an estimate $\tilde{D}$ such that $F_0/3 \leq \tilde{D} \leq 3F_0$.*

Henceforth, let J be the set of all distinct elements in S . We have $F_0 = |J|$ by definition. We assume that n is a power of 2. The main idea of the algorithm is to pick a uniform, 2-wise independent hash function $h : [n] \mapsto [n]$ (see [Theorem 6.3](#) and family $\mathcal{H}_{2,n,n}$ from [Theorem 6.5](#)) and to record the smallest hash value $Y = \min_{j \in J} h(j)$. Note that this can be achieved by storing the hash function, and upon arrival of element a_i , if $h(a_i) < Y$, then we update x to $h(a_i)$. Note that since $a_i = a_j$ implies $h(a_i) = h(a_j)$, the final value Y is the minimum hash value attained over F_0 elements (instead of m). Using the uniformity of the hash function, by a relatively straightforward analysis, this yields that $\mathbb{E}[Y] \approx n/F_0$ and thus Y/n is close to being an unbiased estimator for $1/F_0$. The space of this algorithm is clearly $O(\log n)$ for storing hash function and min hash Y .

We will use a slightly different implementation: we couple the algorithmic idea above with the idea behind Morris' algorithm and do not maintain

Y directly but instead aim to maintain an exponent r such that $2^r \approx Y$. Concretely, we do not record the smallest hash value, but instead the largest exponent r such that some $h(a_i)$ is divisible by 2^r . We use variable X to store the above r , and use the function $\text{ZEROS}(y)$ to extract the largest exponent for which we have divisibility which coincides with the number of trailing zeros in the binary representation of y . For example, if $y = 20 = (\dots 10100)_2$, then $\text{ZEROS}(y) = 2$ and clearly y is divisible by 2^2 but not by 2^3 .

The full algorithm is given below in procedure FM.

Algorithm 12 FM($S = a_1, \dots, a_m$)

$h : [n] \mapsto [n] \leftarrow$ uniform, 2-wise independent random hash function
 $X \leftarrow 0$
for $a_i \in S$ **do** ▷ Items arrive in streaming fashion
 $X \leftarrow \max\{X, \text{ZEROS}(h(a_i))\}$
end for
return $2^{X+\frac{1}{2}}$ ▷ Estimate of F_0

The space analysis is straightforward: we need $O(\log n)$ bits to store the hash function h and then require only $O(\log \log n)$ bits to store the variable X . Together, this yields $O(\log n)$ space. While this does not improve on the version of this algorithm that directly stores the minimum hash value, we will later develop an algorithm that uses the same analytic tools and indeed benefits from this space-saving trick.

7.4.2 Analysis of FM

Let us now give the analysis of the algorithm. We denote by J the set of items from S that appear in the stream S . For each $j \in S$ and non-negative integer r , we define

$$Y_{j,r} = \begin{cases} 1 & \text{if } \text{ZEROS}(h(j)) \geq r \\ 0 & \text{otherwise} \end{cases}$$

We let $Y_r = \sum_{j \in J} Y_{j,r}$.

Let us compute expectation and variance:

$$\begin{aligned} \mathbb{E}[Y_r] &= \sum_{j \in J} \mathbb{E}[Y_{j,r}] = \sum_{j \in J} \frac{1}{2^r} = \frac{F_0}{2^r} \\ \text{Var}[Y_r] &= \sum_{j \in J} \mathbb{E}[Y_{j,r}^2] - \mathbb{E}[Y_{j,r}]^2 \leq \sum_{j \in J} \mathbb{E}[Y_{j,r}^2] = \sum_{j \in J} \mathbb{E}[Y_{j,r}] = \frac{F_0}{2^r}. \end{aligned}$$

Note that while we only used linearity of expectation in the first bound, in the second bound we used that $\text{Var}(\sum_{j \in J} Y_{j,r}) = \sum_{j \in J} \text{Var}(Y_{j,r})$ since $Y_{j,r}$ variables are 2-wise independent.

The algorithm outputs estimate $\hat{D} = 2^{X+\frac{1}{2}}$ for F_0 . We now bound the probability that $\hat{D}$ is far from the number of distinct elements F_0 . More concretely, we choose τ_{high} to be the smallest integer such that $2^{\tau_{high}+\frac{1}{2}} > 3F_0$ and τ_{low} be the largest integer such that $2^{\tau_{low}+\frac{1}{2}} < F_0/3$. The remainder of this section proves that with good probability, we have

$$\tau_{low} < X < \tau_{high} \implies F_0/3 \leq \hat{D} \leq 3F_0.$$

We prove the two inequalities separately. We start with $X \geq \tau_{high}$ where the crucial step is to use Markov's inequality

$$\begin{aligned} \Pr[X \geq \tau_{high}] &\leq \Pr[Y_{\tau_{high}} \geq 1] && \text{Since } X \geq \tau_{high} \Rightarrow Y_{\tau_{high}} \geq 1 \\ &\leq \frac{\mathbb{E}[Y_{\tau_{high}}]}{1} && \text{By Markov's inequality} \\ &= \frac{F_0}{2^{\tau_{high}}} && \text{Since } \mathbb{E}[Y_r] = \frac{F_0}{2^r} \\ &\leq \frac{\sqrt{2}}{3} && \text{Since } 2^{\tau_{high}+\frac{1}{2}} > 3F_0 \end{aligned}$$

and for the other direction, i.e. $X \leq \tau_{low}$, we use Chebyshev's inequality

$$\begin{aligned} \Pr[X \leq \tau_{low}] &\leq \Pr[Y_{\tau_{low}+1} = 0] && \text{Since } X \leq \tau_{low} \Rightarrow Y_{\tau_{low}+1} = 0 \\ &\leq \Pr[\mathbb{E}[Y_{\tau_{low}+1}] - Y_{\tau_{low}+1} \geq \mathbb{E}[Y_{\tau_{low}+1}]] && \text{Implied} \\ &\leq \Pr[|Y_{\tau_{low}+1} - \mathbb{E}[Y_{\tau_{low}+1}]| \geq \mathbb{E}[Y_{\tau_{low}+1}]] && \text{Adding absolute sign} \\ &\leq \frac{\text{Var}[Y_{\tau_{low}+1}]}{(\mathbb{E}[Y_{\tau_{low}+1}])^2} && \text{By Chebyshev's inequality} \\ &\leq \frac{2^{\tau_{low}+1}}{F_0} && \text{Since } \text{Var}[Y_r] \leq \mathbb{E}[Y_r] = \frac{F_0}{2^r} \\ &\leq \frac{\sqrt{2}}{3} && \text{Since } 2^{\tau_{low}} \cdot \sqrt{2} < \frac{F_0}{3} \end{aligned}$$

But by a simple union bound, we have that the probability of failure, i.e. the estimate $\hat{D}$ not satisfying $F_0/3 \leq \hat{D} \leq 3F_0$, is at most the probability of violating upper or lower bound and thus at most $2 \cdot \frac{\sqrt{2}}{3}$. Thus, with probability at least $1 - \frac{2\sqrt{2}}{3} \approx 0.0572$, the estimate is within the claimed bounds.

Finally, we can use the mean trick from Section 7.3.2 to boost the success probability to be larger 0.5 and then the median trick from Section 7.3.3 to boost the success probability to $1 - \delta$ at the cost of slightly increasing the

space by $O(\log(1/\delta))$. This yields [Theorem 7.7](#). However, here this trick and also the mean trick cannot help us to boost the accuracy of the estimate. We therefore turn to a more clever refinement of the algorithm.

7.4.3 BJKST a.k.a. FM+

In this section, we present our main result summarized in [Theorem 7.6](#). The algorithm was given in BJKST'02 but we will refer to the algorithm simply as FM+ (which is easier to pronounce). The refined algorithm is given below.

Algorithm 13 FM+($S = a_1, \dots, a_m$)

```

 $h : [n] \mapsto [n] \leftarrow$  uniform, 2-wise independent random hash function.
 $g : [n] \mapsto [b] \leftarrow$  uniform, 2-wise independent random hash function where
 $b = C \cdot \epsilon^{-4} \cdot \log^2(n)$ .
 $\mathcal{B} \leftarrow \emptyset$ 
 $X \leftarrow 0$ 
for  $a_i \in S$  do                                 $\triangleright$  Items arrive in streaming fashion
    if  $\text{ZEROS}(h(a_i)) \geq X$  then
         $\mathcal{B} \leftarrow \mathcal{B} \cup \{(g(a_i), \text{ZEROS}(h(a_i)))\}$ 
        while  $|\mathcal{B}| \geq C' \cdot \epsilon^{-2}$  do                 $\triangleright$  For  $C' = 576$ .
             $X \leftarrow X + 1$ 
            Remove from  $\mathcal{B}$  all elements  $(\alpha, \beta)$  with  $\beta < X$ .
        end while
    end if
end for
return  $|\mathcal{B}| \cdot 2^X$                                  $\triangleright$  Estimate of  $F_0$ 

```

The main idea is still to use hash function h to uniformly hash values from the set J which is the set of distinct elements appearing in stream S . But this time, we maintain 2^X to be roughly of order F_0/ϵ^2 which implies that we expect roughly $1/\epsilon^2$ elements in J to hash to a value with at least X trailing zeros. We record these elements and store them in a bag $\mathcal{B}$. Now, $|\mathcal{B}| \cdot 2^X$ serves as an estimator for $F_0 = |J|$. Note that we now expect $F_0/2^X$ elements to hash into $\mathcal{B}$ and the size of this sample which is roughly $1/\epsilon^2$ yields much more robustness than before where we only recorded a single element hashing to the number with the most trailing zeros.

Note that the algorithm slowly increases X as a result of the ever-growing number of distinct elements that have been observed so far. This necessitates that whenever we increment X , we adjust $\mathcal{B}$ by removing all elements that hashed to a value with exactly $X - 1$ trailing zeros.

The most straightforward way to maintain $\mathcal{B}$ is to let it be the set of all items a_i that were seen so far and hash to a value with at least X trailing zeros. This is achieved by placing each item a_i upon arrival into $\mathcal{B}$ if its hash has at least X trailing zeros. Then, whenever X increases, one could simply check the number of trailing zeros for each element in $\mathcal{B}$ under hash function h explicitly to adjust $\mathcal{B}$ accordingly. But note that this solution requires $O(\log n)$ bits of space for each of the elements in $\mathcal{B}$.

Instead, the algorithm stores in $\mathcal{B}$ tuples $(g(a_i), \text{ZEROS}(h(a_i)))$ for each item a_i that arrived up until now where g is a hash function that maps each item a_i to a number using only $O(\log b) = O(\log \log n + \log \epsilon^{-1})$ bits and $\text{ZEROS}(h(a_i))$ only requires $O(\log \log n)$ bits. Assuming that there are no collisions among any items hashed via g , we have that under this implementation, we place as many items initially into $\mathcal{B}$ as before. Using the explicit number of trailing zeros stored with each element, we can further still adjust $\mathcal{B}$ whenever X increases.

Overall, the algorithm now requires only $O(\log n + \epsilon^{-2} \cdot (\log \log n + \log \epsilon^{-1}))$ bits, a bound that comes extremely close to a lower bounds of $\Omega(\log n)$ and $\Omega(\epsilon^{-2})$.

7.4.4 Analysis of FM+

We use almost the same proof set-up as in the previous section. We assume here that throughout the process, there is no collision between any two distinct elements hashed via g . In the exercises, you are asked to prove that this occurs with probability at least $5/6$.

We will henceforth condition on the event that there is no collision between elements hashed via g , i.e. at any point during the algorithm, there is a one-to-one correspondence between elements with at least X trailing zeros and elements in the bag $\mathcal{B}$.

We let $Y_{j,r}$ denote the indicator variable taking value 1 if $j \in [n]$ has at least r trailing zeros, and 0 otherwise. We let $Y_r = \sum_{j \in J} Y_{j,r}$.

The analysis of expectation and variance of Y_r is the same previously.

$$\begin{aligned} \mathbb{E}[Y_r] &= \sum_{j \in J} \mathbb{E}[Y_{j,r}] = \sum_{j \in J} \frac{1}{2^r} = \frac{F_0}{2^r} \\ \text{Var}[Y_r] &= \sum_{j \in J} \mathbb{E}[Y_{j,r}^2] - \mathbb{E}[Y_{j,r}]^2 \leq \sum_{j \in J} \mathbb{E}[Y_{j,r}^2] = \sum_{j \in J} \mathbb{E}[Y_{j,r}] = \frac{F_0}{2^r}. \end{aligned}$$

Next, note that our algorithm returns an inaccurate estimate in the event

$$\begin{aligned} |Y_X \cdot 2^X - F_0| \geq \epsilon \cdot F_0 &\iff |Y_X - F_0/2^X| \geq \epsilon \cdot F_0/2^X \\ &\iff \bigcup_{r=0}^{\log_2(n)} \{|Y_r - F_0/2^r| \geq \epsilon \cdot F_0/2^r \wedge X = r\} \end{aligned}$$

where in the second equivalence, we partition the event into a separate event for each value r that X can take.

We next let τ be the unique integer such that $12 \cdot \epsilon^{-2} \leq F_0/2^\tau < 24 \cdot \epsilon^{-2}$. We next split the events above into events where $r < \tau$ and events when $r \geq \tau$ and argue separately about the probability of failure in either event. More concretely, we have

$$\begin{aligned} \Pr[|Y_X \cdot 2^X - F_0| \geq \epsilon \cdot F_0] &= \Pr[\bigcup_{r=0}^{\log_2(n)} \{|Y_r - F_0/2^r| \geq \epsilon \cdot F_0/2^r \wedge X = r\}] \\ &\leq \sum_{r=0}^{\log_2(n)} \Pr[|Y_r - F_0/2^r| \geq \epsilon \cdot F_0/2^r \wedge X = r] \\ &\leq \sum_{r=0}^{\tau-1} \Pr[|Y_r - F_0/2^r| \geq \epsilon \cdot F_0/2^r] + \sum_{r=\tau}^{\log_2(n)} \Pr[X = r] \end{aligned}$$

We now give an upper bound on the failure probability from the events where $X \geq \tau$

$$\begin{aligned} \sum_{r=\tau}^{\log_2(n)} \Pr[X = r] &\leq \Pr[X \geq \tau] \\ &= \Pr[Y_{\tau-1} > C' \cdot \epsilon^{-2}] \end{aligned}$$

where the last equality follows since the event that there are more than C'/ϵ^2 elements in S that hash to a value with at least $\tau-1$ trailing zeros implies that if $X \leq \tau-1$, then the bag $\mathcal{B}$ would contain all of these elements contradicting the while-loop condition in our algorithm that ensures that $\mathcal{B}$ is of small size at all times.

We can then again use Markov's Inequality to derive

$$\begin{aligned} \Pr[Y_{\tau-1} > C' \cdot \epsilon^{-2}] &\leq \mathbb{E}[Y_{\tau-1}] \cdot \frac{1}{C' \cdot \epsilon^{-2}} \\ &\leq \frac{F_0}{2^{\tau-1}} \cdot \frac{1}{C' \cdot \epsilon^{-2}} \\ &\leq 48 \cdot \epsilon^{-2} \cdot \frac{1}{C' \cdot \epsilon^{-2}} \\ &= \frac{48}{C'} \end{aligned}$$

where we use how we chose τ . Choosing $C' = 576$, we obtain a probability of failure of at most $1/12$.

For the probability of the lower bound failing, we again use Chebyshev's Inequality

$$\begin{aligned}
\sum_{r=0}^{\tau-1} \Pr[|Y_r - F_0/2^r| \geq \epsilon \cdot F_0/2^r] &\leq \sum_{r=0}^{\tau-1} \frac{\text{Var}[Y_r]}{(\epsilon \cdot F_0/2^r)^2} \\
&\leq \sum_{r=0}^{\tau-1} \frac{F_0/2^r}{(\epsilon \cdot F_0/2^r)^2} \\
&= \sum_{r=0}^{\tau-1} \frac{2^r}{\epsilon^2 \cdot F_0} \\
&\leq \frac{2^\tau}{\epsilon^2 \cdot F_0} \\
&\leq \frac{\epsilon^2 F_0/12}{\epsilon^2 F_0} \\
&= 1/12.
\end{aligned}$$

Here we used our previous calculation of the variance of Y_r , a calculation of a geometric partial sum, and then our choice of τ .

Together, this bounds the probability of failure, i.e. of the estimate being inaccurate, by at most $1/12 + 1/12 = 1/6$. Using that the probability of failure via a collision in g is at most $1/6$, we thus have failure probability at most $1/3$. Using the median trick from [Section 7.3.3](#), we can boost the success probability to $1 - \delta$, using $O(\log \delta^{-1})$ copies of the above algorithm. This yields [Theorem 7.6](#).

7.5 Estimating the second moment of a stream

In this section, we describe algorithms from [\[AMS96\]](#) that estimates the second moment of a stream $F_2 = \sum_{j \in [n]} f_j^2$. The algorithm also has a generalization that estimates the k^{th} moment of a stream for general $k \geq 2$, however, we will not cover this algorithm in these notes. We prove the following result.

Theorem 7.8. *There is an algorithm that for any stream $S = a_1, \dots, a_m$ with $a_i \in [n]$ and $\delta, \epsilon > 0$, uses space $O(\frac{\log(n) \log(1/\delta)}{\epsilon^2})$, and outputs with probability at least $1 - \delta$, an estimate $\tilde{D}$ such that $(1 - \epsilon)F_2 \leq \tilde{D} \leq (1 + \epsilon)F_2$ where F_2 is the second moment of the stream S .*

7.5.1 The AMS Algorithm

The AMS algorithm first draws a uniform, 4-wise independent hash function h that maps $[n]$ to $\{-1, +1\}$, i.e. every element $j \in [n]$ has value $h(j)$ being -1 with probability $1/2$ and $+1$ with probability $1/2$. To obtain h , we can use a hash function $g_4(x) : [n] \mapsto [2]$ and then map 0 to -1 . Recall that we can store such a hash function with only $O(\log n)$ bits (and that while we technically have that $+1$ and -1 are drawn with probability at most $1/2 + n^{-100}$, we henceforth ignore this issue and assume perfect uniformity in our algorithm and analysis).

The algorithm then maintains a sum Z that is initialized to 0 and whenever a new element a_i is scanned, we add $h(a_i)$ to Z . Note that this yields $Z = \sum_{j \in [n]} h(j)f_j$ at the end of the algorithm, i.e. each element j is added or subtracted the number of times it occurred in the stream. Since addition and subtraction occur with the same probability $\mathbb{E}[Z] = 0$. However, in the analysis, we will see that Z^2 is a very good estimator of F_2 .

Algorithm 14 AMS-2($S = \{a_1, \dots, a_m\}$)

$h : [n] \mapsto \{-1, +1\} \leftarrow$ uniform, 4-wise independent random hash function.	
$Z \leftarrow 0$	
for $a_i \in S$ do	▷ Items arrive in streaming fashion
$Z \leftarrow Z + h(a_i)$	▷ At the end, $Z = \sum_{j \in [n]} h(j)f_j$
end for	
return Z^2	▷ Estimate of $F_2 = \sum_{j \in [n]} f_j^2$

7.5.2 Analyzing the AMS Algorithm

For convenience, we henceforth use h_j in lieu of $h(j)$. We start by verifying that Z^2 is an unbiased estimator.

Lemma 7.9. *In AMS-2, Z^2 is an unbiased estimator, i.e. $\mathbb{E}[Z^2] = F_2$.*

Proof.

$$\begin{aligned}
\mathbb{E}[Z^2] &= \mathbb{E}\left[\left(\sum_{j \in [n]} h_j \cdot f_j\right)^2\right] \\
&= \mathbb{E}\left[\sum_{j \in [n]} h_j^2 f_j^2 + 2 \sum_{1 \leq i < j \leq n} h_i h_j f_i f_j\right] && \text{Expanding } \left(\sum_{j \in [n]} h_j \cdot f_j\right)^2 \\
&= \sum_{j \in [n]} \mathbb{E}[h_j^2 f_j^2] + \sum_{i \neq j \in [n]} \mathbb{E}[h_i h_j f_i f_j] && \text{Linearity of expectation} \\
&= \sum_{j \in [n]} \mathbb{E}[h_j^2] f_j^2 + \sum_{i \neq j \in [n]} \mathbb{E}[h_i h_j] f_i f_j && f_i \text{'s are (unknown) constants} \\
&= \sum_{j \in [n]} \mathbb{E}[h_j^2] f_j^2 + \sum_{i \neq j \in [n]} \mathbb{E}[h_i] \mathbb{E}[h_j] f_i f_j && \text{Since } \{h_i\}_{i \in [n]} \text{ are pairwise independent} \\
&= \sum_{j \in [n]} f_j^2 && \text{Since } h_j^2 = 1, \forall j \in [n] \text{ and } \mathbb{E}[h_j] = 0 \\
&= F_2
\end{aligned}$$

□

As usual, we want to also analyze the variance of Z^2 so that we can use Chebyshev's inequality later to bound the probability of large deviations.

Lemma 7.10. $\text{Var}[Z^2] \leq 2F_2^2$.

Proof. As before, $\mathbb{E}[h_i] = 0$ and $h_i^2 = 1$ for all $i \in [n]$. By 4-wise independence, the expectation of any product of at most 4 different h_i 's is the product of their expectations. Thus we get $\mathbb{E}[h_i h_j h_k h_l] = \mathbb{E}[h_i] \mathbb{E}[h_j] \mathbb{E}[h_k] \mathbb{E}[h_l] = 0$, as well as $\mathbb{E}(h_i^3 h_j) = \mathbb{E}(h_i h_j) = 0$ and $\mathbb{E}(h_i^2 h_j h_k) = \mathbb{E}(h_j h_k) = 0$, where the indices i, j, k, l are distinct. This allows us to compute $\mathbb{E}[Z^4]$:

$$\begin{aligned}
\mathbb{E}[Z^4] &= \mathbb{E}\left[\left(\sum_{j \in [n]} h_j f_j\right)^4\right] && \text{Since } Z = \sum_{j \in [n]} h_j f_j \text{ at the end} \\
&= \sum_{j \in [n]} \mathbb{E}[h_j^4] f_j^4 + 6 \sum_{i < j \in [n]} \mathbb{E}[h_i^2 h_j^2] f_i^2 f_j^2 && \text{L.o.E. and 4-wise independence} \\
&= \sum_{i=1}^n f_i^4 + 6 \sum_{i < j \in [n]} f_i^2 f_j^2 && \text{Since } h_i^4 = h_i^2 = 1, \forall i \in [n]
\end{aligned}$$

Note that the coefficient of $\sum_{i < j \in [n]} \mathbb{E}[h_i^2 h_j^2] f_i^2 f_j^2$ is $\binom{4}{2} = 6$ and that all other terms vanish by the computation above.

$$\begin{aligned}
\text{Var}[Z^2] &= \mathbb{E}[(Z^2)^2] - (\mathbb{E}[Z^2])^2 && \text{Definition of variance} \\
&= \sum_{j \in [n]} f_j^4 + 6 \sum_{i < j \in [n]} f_i^2 f_j^2 - (\mathbb{E}[Z^2])^2 && \text{From above} \\
&= \sum_{j \in [n]} f_j^4 + 6 \sum_{i < j \in [n]} f_i^2 f_j^2 - \left(\sum_{j \in [n]} f_j^2 \right)^2 && \text{By Lemma 7.9} \\
&= 4 \sum_{i < j \in [n]} f_i^2 f_j^2 && \text{Expand and simplify} \\
&\leq 2 \left(\sum_{j \in [n]} f_j^2 \right)^2 && \text{Introducing } f_j^4 \text{ terms} \\
&= 2(F_2)^2 && \text{By Lemma 7.9}
\end{aligned}$$

□

We can now use Chebyshev's Inequality to bound the probability of deviation.

Theorem 7.11. *We have $\Pr[|Z^2 - F_2| > \ell F_2] \leq \frac{2}{\ell^2}$ for any $\ell > 0$.*

Proof.

$$\begin{aligned}
\Pr[|Z^2 - F_2| > \ell F_2] &\leq \frac{\text{Var}(Z^2)}{(\ell F_2)^2} && \text{By Chebyshev's inequality} \\
&\leq \frac{2F_2^2}{(\ell^2 F_2^2)} && \text{By Lemma 7.10} \\
&= \frac{2}{\ell^2}
\end{aligned}$$

□

We can again apply the mean trick (Section 7.3.2) to decrease the variance by a factor of k and have a smaller upper bound on the probability of error. In particular, if we pick $k = \frac{10}{\epsilon^2}$ repetitions of ASM-2 and output the mean value of the output Z^2 we have :

$$\Pr[\text{error}] \leq \frac{\text{Var}[Z^2]^{\frac{1}{k}}}{(\epsilon F_2)^2} \leq \frac{1}{k} \cdot \frac{2}{\epsilon^2} = \frac{1}{5}$$

When the failure probability is less than $\frac{1}{2}$, one can then call the routine k times independently, and return the median (Section 7.3.3). On the whole, for any given $\epsilon > 0$ and $\delta > 0$, $\mathcal{O}(\frac{\log(n) \log(1/\delta)}{\epsilon^2})$ space suffices to yield a $(1 \pm \epsilon)$ -approximation algorithm that succeeds with probability $> 1 - \delta$, yielding Theorem 7.8.

References

This chapter was originally written by Mohsen Ghaffari. It was rewritten by Maximilian Probst Gutenberg with the presentation of BJKST being newly added (replacing another version of FM+).

Chapter 8

Dimensionality Reduction and Nearest-Neighbor Search

In modern systems, data is typically stored as high-dimensional vectors. A streaming service could store a vector $\mathbf{x} \in \mathbb{R}^d$ for each user, where d is the number of songs or movies available and $\mathbf{x}(i)$ is a score of how much the user liked the i -th song/movie. Other examples are to store pictures as pixel vectors, or to store webpages as bags-of-words, i.e., representing each page by a vector $\mathbf{x} \in \mathbb{R}^d$ where d is the number of words in the dictionary and $\mathbf{x}(i)$ records the number of occurrences of the i -th word in the dictionary. Typical tasks that these modern systems like to perform on these vectors are:

- given a user $\mathbf{x}$ in the database of a streaming service, find a user $\mathbf{y}$ that is similar in the database and use this users' assessments to make a recommendation to $\mathbf{x}$.
- find webpages that are near-duplicates, i.e. whose content mostly agrees. When training large natural language processing models (NLPs) or large language models (LLMs) one then wants to exclude near duplicates to speed up learning and avoid biases.
- clustering/ labeling/ categorizing all similar data points the same for classification and machine learning tasks.
- use these vectors for training for NLP, ML, and LLM tasks.

Today, we study how to relate such vectors to each other and how to deal with the high dimensionality of the data. It is important to keep in mind that similarity is often subjective, and for different tasks, different similarity measures might work best. It is therefore crucial to make an informed decision

when settling for an appropriate measure. In this lecture, for concreteness, we study three very popular similarity measures/distances:

1. **Jaccard Similarity:**¹ for vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}_{\geq 0}^d$, defined as

$$\text{Jac}(\mathbf{x}, \mathbf{y}) = \frac{\sum_{1 \leq i \leq d} \min\{\mathbf{x}(i), \mathbf{y}(i)\}}{\sum_{1 \leq i \leq d} \max\{\mathbf{x}(i), \mathbf{y}(i)\}}.$$

2. **Cosine Similarity:** for non-zero vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$, defined as

$$\text{Cos}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\|_2 \|\mathbf{y}\|_2}.$$

Recall that $\mathbf{x} \cdot \mathbf{y} = \|\mathbf{x}\|_2 \|\mathbf{y}\|_2 \cdot \cos(\phi)$ and so the cosine similarity is $\text{Cos}(\mathbf{x}, \mathbf{y}) = \cos(\phi)$ where ϕ is the angle between the two vectors.

3. **Euclidean Distance:** for any vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$, defined as $\|\mathbf{x} - \mathbf{y}\|_2$.

These three measures are chosen for their popularity, distinctness and for their interesting geometric interpretations:

1. **Jaccard Similarity** compares coordinate-wise overlap.
2. **Cosine Similarity** compares directional alignment.
3. **Euclidean Distance** compares distances.

In the first part of the lecture, we show that we can hash high-dimensional vectors $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n \in \mathbb{R}^d$ to low-dimensional *sketches*, e.g. vectors $h(\mathbf{x}_1), h(\mathbf{x}_2), \dots, h(\mathbf{x}_n) \in \mathbb{R}^k$ for some small/reasonable k such that $h(\mathbf{x}_i)$ and $h(\mathbf{x}_j)$ are (approximately) close if and only if $\mathbf{x}_i$ and $\mathbf{x}_j$ are close with respect to the underlying similarity/distance measure. Consider trying to cluster all vectors that are similar/close, which requires doing pairwise comparisons: instead of time $O(n^2 \cdot d)$, this brings the time to compare all pairs down to $O(n^2 \cdot k)$ where k is much smaller than d .

In the second part of the lecture, we discuss how to further speed up such searches. We study the (approximate) nearest neighbor problem that knows vectors $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and given a new vector $\mathbf{q} \in \mathbb{R}^d$ returns the closest (or at least almost closest) vector from $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$. Note that for small dimensions (say $d \leq 20$), this problem can be solved in reasonable time via quadtrees. But for larger dimensions, quadtrees do not scale and

¹A more popular way of defining Jaccard similarity for two sets $S, T \subseteq \{d\}$ is by defining $\text{Jac}(S, T) = |S \cap T| / |S \cup T|$. This is equivalent to our definition by setting $\mathbf{x}(i)$ to 1 if the i -th item of the universe is in S , and $\mathbf{y}(i)$ to 1 if it is in T ; and otherwise to 0.

the problem becomes very difficult – this is also often referred to as the curse of dimensionality. Building on techniques from sketching and hashing, we give an approach to solve this very hard problem approximately with time depending only on the closeness of the query point $\mathbf{q}$ to the closest point in the data set X .

8.1 Dimensionality Reduction for Similarity Measures and Distances

8.1.1 Dimensionality Reduction for Jaccard Similarity via MinHash

Let us start by constructing a signature for the Jaccard similarity measure. In this section, we assume that vectors are 0-1, i.e. the data points $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n \in \{0, 1\}^d$.²

We use the idea of taking the min-hash, which is easiest to learn by just looking at it. Let us take a fully-independent uniform hash function $h : [d] \mapsto [m]$ for some m large enough to make collision probabilities very small (say $m = \Theta(n^2 \log n)$). We condition henceforth on this event.

Then, for each vector, we take as its signature the smallest hash value assigned to any of its 'set' coordinates

$$s(\mathbf{x}) = \min\{h(i) \mid \mathbf{x}(i) = 1\}.$$

It turns out that h now gives us an unbiased estimator for the Jaccard similarity for two vectors.

Lemma 8.1. *For vectors $\mathbf{x}, \mathbf{y} \in \{0, 1\}^d$, $\Pr[s(\mathbf{x}) = s(\mathbf{y})] = \text{Jac}(\mathbf{x}, \mathbf{y})$.*

Proof. Let $I = \{i \mid \max\{\mathbf{x}(i), \mathbf{y}(i)\} = 1\}$. Let $J = \{i \mid \min\{\mathbf{x}(i), \mathbf{y}(i)\} = 1\}$. The Jaccard similarity is exactly the ratio of these two sets, i.e. $\text{Jac}(\mathbf{x}, \mathbf{y}) = |J|/|I|$.

But note that by choice of our hash function, the values $h(i)$ for $i \in I$ are ordered randomly, and so it is equally likely for any item $i \in I$ to be hashed to the smallest number under h . That is for any $i' \in I$, $\Pr[h(i') = \min_{i \in I} h(i)] = 1/|I|$. But this implies that an index $j \in J$ is hashed to the smallest value among all indices in I with probability $|J|/|I|$.

It remains to observe that $s(\mathbf{x}) = s(\mathbf{y})$ iff and only if an index in J is hashed to the smallest value among all indices in I . $\square$

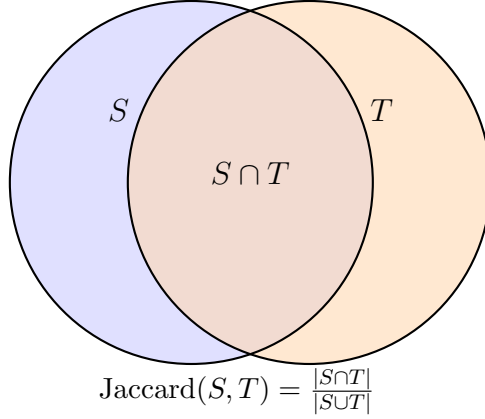


Figure 8.1: Geometric intuition for Jaccard similarity: ratio of intersection to union.

We next transform this insight into an estimator of the Jaccard similarity that works with high probability.

Lemma 8.2. *For some $0 < \epsilon, \delta < 1$, let $k = \lceil \epsilon^{-2} \log(1/\delta) \rceil$, let us take independent signature functions $s_1, s_2, \dots, s_k$ as described above. For vectors $\mathbf{x}, \mathbf{y} \in \{0, 1\}^d$, we have for estimator $\hat{J} = \frac{1}{k} \sum_i \mathbf{1}[s_i(\mathbf{x}) = s_i(\mathbf{y})]$ that*

$$\Pr[|\text{Jac}(\mathbf{x}, \mathbf{y}) - \hat{J}| > \epsilon] < 2\delta.$$

Proof. We have that variables $X_i = \mathbf{1}[s_i(\mathbf{x}) = s_i(\mathbf{y})]$ are independent r.v.'s with mean $\text{Jac}(\mathbf{x}, \mathbf{y})$ (by [Theorem 8.1](#)), which yields $X = \sum_i X_i$ with $\text{Exp}[X] = k \cdot \text{Jac}(\mathbf{x}, \mathbf{y})$. Thus, Hoeffding's inequality applied to X yields

$$\begin{aligned} \Pr[|\hat{J} - \text{Jac}(\mathbf{x}, \mathbf{y})| > \epsilon] &= \Pr[|X - k \cdot \text{Jac}(\mathbf{x}, \mathbf{y})| > \epsilon \cdot k] \\ &\leq 2e^{-2\epsilon^2 \cdot k} < 2e^{-\log(1/\delta)} = 2e^{\log(\delta)} = 2\delta. \end{aligned}$$

□

Note that we have not chosen a multiplicative guarantee in the bound above since the application of Chernoff bounds would have the number of independent functions scale inversely in the Jaccard similarity of $\mathbf{x}$ and $\mathbf{y}$. Instead, we opted for making a small additive error. Since we typically search for fairly related items, it suffices to preserve up to some large $\epsilon > 0$ (i.e. some ϵ not too far from 1) and thus the number of dimensions necessary is very small.

²While techniques do not seamlessly translate to general vectors, simple algorithms exist that further extend on the core ideas presented in this section.

Note that from [Theorem 8.2](#), we can store for each data point $\mathbf{x}_i$ only the bit vector $s(\mathbf{x}) = s_1(\mathbf{x}_i) \circ s_2(\mathbf{x}_i) \dots s_k(\mathbf{x}_i)$ and to get an estimator for $\text{Jac}(\mathbf{x}, \mathbf{y})$, we can take the number of positions such that $s(\mathbf{x}) = s(\mathbf{y})$ and plug this number into the formula above.

8.1.2 Dimensionality Reduction for Cosine Similarity via Hyperplane Projections

We next study the Cosine similarity, defined for non-zero vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ to be

$$\text{Cos}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\|_2 \|\mathbf{y}\|_2}.$$

Since $\text{Cos}(\mathbf{x}, \mathbf{y})$ only depends on the angle of vectors $\mathbf{x}$ and $\mathbf{y}$, we will assume w.l.o.g. that all vectors in our data set $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$ are unit vectors, i.e. $\|\mathbf{x}_i\|_2 = 1$ and thus are on the unit sphere $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n \in \mathbb{S}^{d-1}$.

Again, we would like to compute a signature $s(\mathbf{x})$ for each vector $\mathbf{x}$ such that for similar vectors $\mathbf{x}, \mathbf{y}$, the assigned signatures agree with good probability and otherwise, disagree with good probability.

Intuition for the Random Hyperplane Method. The main idea here is to use random hyperplanes to partition $\mathbb{R}^d$ into two equal parts, so-called halfspaces. For $d = 2$, a hyperplane is simply a line through the origin associated with a direction. Finally, we assign the two halfspaces values -1 and $+1$, respectively, and define the signature $s(\mathbf{x})$ for the vector be $+1$ if it is in the $+1$ halfspace, and -1 otherwise. Here the sign depends on the direction of the line.

Each directed line in $\mathbb{R}^2$ can be described by an angle $\rho \in [0, 2\pi]$. Sampling hyperplanes uniformly, each angle $[0, 2\pi]$ has equal probability of being chosen. Given two vectors $\mathbf{x}$ and $\mathbf{y}$ and let $\theta = \arccos(\mathbf{x} \cdot \mathbf{y})$ be the angle between them. Then, $\mathbf{x}$ and $\mathbf{y}$ are both on the $+1$ side of the hyperplane with probability $(\pi - \theta)/2\pi$ and on the -1 side with the same probability. We thus get $\Pr[s(\mathbf{x}) = s(\mathbf{y})] = 1 - \frac{\theta}{\pi}$.

Implementing the Random Hyperplane Method. Let us now discuss how to implement this method efficiently. Recall that a hyperplane of an d -dimensional space can be described by the vector $\mathbf{r} \in \mathbb{R}^d$ that is orthogonal to the hyperplane. $\mathbf{r}$ is also called the *normal vector*. We have that $\langle \mathbf{r}, \mathbf{x} \rangle > 0$ indicates that $\mathbf{x}$ is on the $+1$ side of the hyperplane associated with $\mathbf{r}$, and $\langle \mathbf{r}, \mathbf{x} \rangle < 0$ indicates that it is on the -1 side. That is, we define the signature function by $s(\mathbf{x}) = \text{sign}(\langle \mathbf{r}, \mathbf{x} \rangle)$.

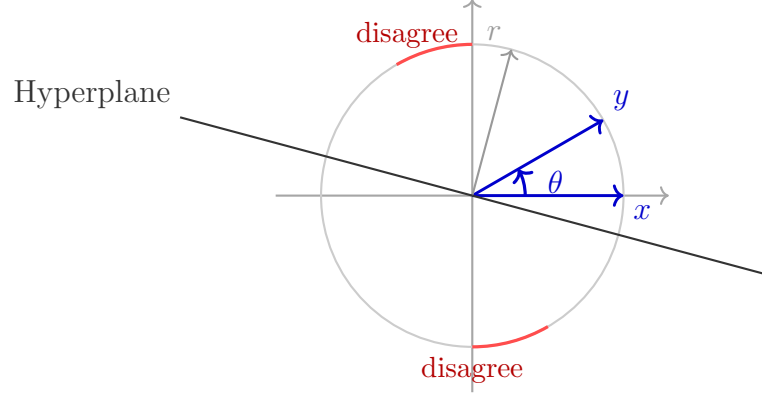


Figure 8.2: Random hyperplane sketch for cosine similarity. A random Gaussian vector $\mathbf{r}$ defines a hyperplane through the origin (recall that it is the line orthogonal to $\mathbf{r}$ through the origin). The probability that $\mathbf{x}$ and $\mathbf{y}$ lie on different sides equals θ/π for $\theta = \arccos(\langle \mathbf{x}, \mathbf{y} \rangle)$.

It remains to choose the random vector $\mathbf{r} \in \mathbb{R}^d$. To this end, we sample each coordinate $\mathbf{r}(i)$ from the normal distribution with mean 0 and variance 1, i.e $\mathbf{r}(i) \sim \mathcal{N}(0, 1)$. Sampling $\mathbf{r}$ from the multivariate normal distribution is a standard way of sampling a vector that chooses each direction uniformly. That is because $\mathbf{r}$ is sampled from a distribution that is invariant under rotation, that is, for every rotation matrix $\mathbf{Q} \in \mathbb{R}^{d \times d}$, we have that a vector $\mathbf{z}$ is sampled with sample probability as vector $\mathbf{Q}\mathbf{z}$.

To see this property, recall that the probability density function (pdf) of the standard normal distribution $\mathcal{N}(0, 1)$ at point z is

$$f(z) = \frac{1}{\sqrt{2\pi}} e^{-z^2/2}$$

and thus the joint probability distribution for our sampling for any point $\mathbf{z} \in \mathbb{R}^d$ is

$$\begin{aligned} f(\mathbf{z} = (z_1, z_2, \dots, z_d)) &= \prod_{i=1}^d f(z_i) \\ &= \prod_{i=1}^d \frac{1}{\sqrt{2\pi}} e^{-z_i^2/2} \\ &= \left(\frac{1}{\sqrt{2\pi}} \right)^d \cdot e^{-\sum_i z_i^2/2} \\ &= \left(\frac{1}{\sqrt{2\pi}} \right)^d \cdot e^{-\|\mathbf{z}\|_2^2/2} \end{aligned}$$

And therefore each point at distance $\|\mathbf{z}\|_2^2$ from the origin has the same probability of being sampled, and thus we sample uniformly over all directions. Note that this proof does not only hold for rotations but for all matrices $\mathbf{Q} \in \mathbb{R}^d$ such that $\|\mathbf{Q}\mathbf{x}\|_2 = \|\mathbf{x}\|_2$ for all $\mathbf{x} \in \mathbb{R}^d$. These matrices are orthogonal matrices, i.e. matrices where $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$. Every rotation matrix is an orthogonal matrix, but there are also orthogonal matrices that not only rotate the underlying space **but also** reflect.

Finally, let us generalize the above proof sketch that vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ take the same signature value with probability scaling linearly in their angle.

Lemma 8.3. *For any $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$, and signature function s as defined above, we have for $\theta = \arccos(\langle \mathbf{x}, \mathbf{y} \rangle)$ that*

$$\Pr[s(\mathbf{x}) = s(\mathbf{y})] = 1 - \frac{\theta}{\pi}$$

Proof. We first show that we can effectively reduce to two dimensions and assume $\mathbf{x} = (1, 0)$ and $\mathbf{y} = (\cos \theta, \sin \theta)$.

This follows since there is an orthogonal matrix $\mathbf{Q} \in \mathbb{R}^{d \times d}$ with $\mathbf{x}' = \mathbf{Q}\mathbf{x} = (1, 0, \dots, 0)$ and $\mathbf{y}' = \mathbf{Q}\mathbf{y} = (\cos \theta, \sin \theta, 0, \dots, 0)$ (in the exercises we ask you to construct such a matrix). Using $\mathbf{r}' = \mathbf{Q}\mathbf{r}$ and since $\mathbf{Q}$ is an orthogonal matrix, we have that $\langle \mathbf{r}', \mathbf{x}' \rangle = \langle \mathbf{r}, \mathbf{x} \rangle$ and $\langle \mathbf{r}', \mathbf{y}' \rangle = \langle \mathbf{r}, \mathbf{y} \rangle$. Since only the first two coordinates of $\mathbf{r}', \mathbf{x}'$ and $\mathbf{y}'$ determine the inner products of interest, we can restrict all vectors to the first two coordinates. Since the underlying distribution is invariant to orthogonal transformations, we further have $\mathbf{r}' \sim \mathbf{Q}\mathbf{r}$ and thus $\mathbf{r}'(1) \sim \mathcal{N}(0, 1)$ and $\mathbf{r}'(2) \sim \mathcal{N}(0, 1)$.

We may thus restrict attention to the first two coordinates of each vector. Writing $(\mathbf{r}'(1), \mathbf{r}'(2)) = \rho(\cos \varphi, \sin \varphi)$ in polar coordinates, φ is uniform on $[0, 2\pi)$ and independent of the length of the vector ρ .

$$s(\mathbf{x}) = \text{sign}(\langle \mathbf{r}', \mathbf{x}' \rangle) = \text{sign}(\cos \varphi)$$

$$s(\mathbf{y}) = \text{sign}(\langle \mathbf{r}', \mathbf{y}' \rangle) = \text{sign}(\cos(\theta) \cos(\varphi) + \sin(\theta) \sin(\varphi)) = \text{sign}(\cos(\varphi - \theta)).$$

They disagree exactly when $\cos \varphi$ and $\cos(\varphi - \theta)$ have opposite signs, which occurs for two arcs of total length 2θ on the circle. Since φ is uniform,

$$\Pr[s(\mathbf{x}) \neq s(\mathbf{y})] = \frac{2\theta}{2\pi} = \frac{\theta}{\pi}, \quad \Pr[s(\mathbf{x}) = s(\mathbf{y})] = 1 - \frac{\theta}{\pi}.$$

□

Again, we can take the mean of independent trials to obtain a good random estimator.

8.1.3 Dimensionality Reduction for Euclidean Distances via Johnson-Lindenstrauss

In this section, we see the most famous result. Pairwise distances in Euclidean space can be preserved extremely well even after hashing to only few dimensions.

Lemma 8.4 (Johnson Lindenstrauss Lemma). *For vectors $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and an accuracy $0 < \epsilon < 1$, there is a distribution over matrices $\mathbf{A} \in \mathbb{R}^{k \times d}$ where $k = O(\epsilon^{-2} \log(n))$ such that w.h.p. for all $i, j \in [n]$,*

$$(1 - \epsilon)\|\mathbf{x}_i - \mathbf{x}_j\|_2 \leq \|\mathbf{A}\mathbf{x}_i - \mathbf{A}\mathbf{x}_j\|_2 \leq (1 + \epsilon)\|\mathbf{x}_i - \mathbf{x}_j\|_2.$$

We will prove a slightly stronger statement.

Lemma 8.5. *For any $0 < \epsilon, \delta < 1$, there is a distribution over matrices $\mathbf{A} \in \mathbb{R}^{k \times d}$ with $k = O(\epsilon^{-2} \log(1/\delta))$ such that for any vector $\mathbf{x} \in \mathbb{R}^d$, with probability at least $1 - \delta$,*

$$(1 - \epsilon)\|\mathbf{x}\|_2^2 \leq \|\mathbf{A}\mathbf{x}\|_2^2 \leq (1 + \epsilon)\|\mathbf{x}\|_2^2.$$

Note that for matrix $\mathbf{A} \in \mathbb{R}^{k \times d}$ sampled as above for $\delta < n^{-3}$, we have for all vectors $\mathbf{x}_i - \mathbf{x}_j$ that

$$\begin{aligned} (1 - \epsilon)\|\mathbf{x}_i - \mathbf{x}_j\|_2^2 &\leq \|\mathbf{A}(\mathbf{x}_i - \mathbf{x}_j)\|_2^2 \leq (1 + \epsilon)\|\mathbf{x}_i - \mathbf{x}_j\|_2^2 \\ \implies (1 - \epsilon)\|\mathbf{x}_i - \mathbf{x}_j\|_2 &\leq \|\mathbf{A}(\mathbf{x}_i - \mathbf{x}_j)\|_2 \leq (1 + \epsilon)\|\mathbf{x}_i - \mathbf{x}_j\|_2. \end{aligned}$$

and since there are only $\binom{n}{2}$ such vectors to preserve, $\mathbf{A}$ succeeds with probability at least $1 - \binom{n}{2} \cdot n^{-3} \geq 1 - 1/n$, i.e. w.h.p. as desired. Thus, [Theorem 8.5](#) yields [Theorem 8.4](#) as a corollary.

Analyzing a Random Projection. We start by considering a single random projection of $\mathbf{x}$. Note that we can assume w.l.o.g. that $\|\mathbf{x}\|_2^2 = 1$ since $\|\mathbf{A}(c \cdot \mathbf{x})\|_2^2 = c^2 \|\mathbf{A}\mathbf{x}\|_2^2$.

That is, we sample a vector $\mathbf{r} \in \mathbb{R}^d$ where each coordinate $\mathbf{r}(i) \sim \mathcal{N}(0, 1)$, i.e., from the normal distribution with mean 0 and variance 1, and analyze the inner product $\mathbf{y} = \langle \mathbf{x}, \mathbf{r} \rangle$.

Here, we exploit a simple but powerful fact about the normal distribution.³

³This fact can be derived from invariance of multivariate normal distributions under orthogonal transformations.

Fact 8.6 (Normal Distribution is closed under Additions). *Given independent Gaussians $X_1, X_2, \dots, X_n$ where each $X_i \sim \mathcal{N}(\alpha_i, \beta_i)$, i.e. with mean α_i and variance β_i , then we have $X = \sum_i X_i \sim \mathcal{N}(\sum_i \alpha_i, \sum_i \beta_i)$.*

Let us now learn how to exploit this fact: we use that for a random variable X , the variance of αX is $\alpha^2 \text{Var}(X)$ and thus $\mathbf{x}(i)\mathbf{r}(i) \sim \mathcal{N}(0, \mathbf{x}(i)^2)$ and so we get

$$y = \langle \mathbf{x}, \mathbf{r} \rangle = \sum_i \mathbf{x}(i)\mathbf{r}(i) \sim \mathcal{N}(0, \sum_i \mathbf{x}(i)^2).$$

That is y has mean 0 and variance $\sum_i \mathbf{x}(i)^2 = \|\mathbf{x}\|_2^2 = 1$. This yields

$$\text{Var}[y] = \mathbb{E}[y^2] - \mathbb{E}[y]^2 = \mathbb{E}[y^2] - 0 = \mathbb{E}[y^2]$$

and thus $\mathbb{E}[y^2] = \|\mathbf{x}\|_2^2 = 1$. This is quite remarkable: we have just proven that the square of the inner product with a random vector is an unbiased estimator for the length of the vector $\mathbf{x}$!

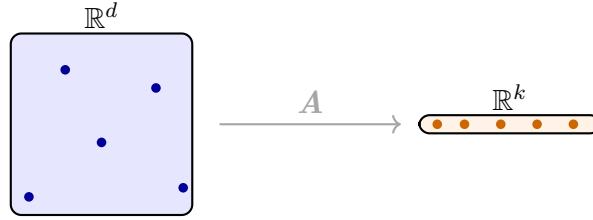


Figure 8.3: Johnson–Lindenstrauss projection: points in $\mathbb{R}^d$ are linearly mapped to $\mathbb{R}^k$ while approximately preserving pairwise distances.

Concentration from Independent Trials. As you have seen many times before, we can now just use multiple samples and hope for concentration: we let the matrix $\mathbf{R}$ have each row be a random vector $\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_k \in \mathbb{R}^d$ with each entry sampled from $\mathcal{N}(0, 1)$. And use the matrix $\mathbf{A} = \frac{1}{\sqrt{k}} \mathbf{R}$ to create our sketches. This implies that we have $\|\mathbf{A}\mathbf{x}\|_2^2 = \|\frac{1}{\sqrt{k}} \mathbf{R}\mathbf{x}\|_2^2 = \frac{1}{k} \|\mathbf{R}\mathbf{x}\|_2^2$.

Thus, taking $y_i = \langle \mathbf{r}_i, \mathbf{x} \rangle$, we get

$$\mathbb{E}[\|\mathbf{A}\mathbf{x}\|_2^2] = \frac{1}{k} \mathbb{E}[\|\mathbf{R}\mathbf{x}\|_2^2] = \frac{1}{k} \mathbb{E} \left[\sum_{i=1}^k y_i^2 \right] = \frac{1}{k} \sum_{i=1}^k \mathbb{E}[y_i^2] = \|\mathbf{x}\|^2 = 1$$

by linearity of expectation.

Our proof of concentration follows a Chernoff-bound style argument. That is, we first consider the moment generating functions for some $t > 0$ and then use Markov's inequality

$$\begin{aligned}\mathbb{P}[\|\mathbf{Ax}\|_2^2 > (1 + \epsilon)] &= \mathbb{P}[\|\mathbf{Rx}\|_2^2 > (1 + \epsilon)k] \\ &= \mathbb{P}[e^{t \cdot \|\mathbf{Rx}\|^2} > e^{t \cdot (1 + \epsilon)k}] \\ &\leq \frac{\mathbb{E}[e^{t \cdot \|\mathbf{Rx}\|^2}]}{e^{t \cdot (1 + \epsilon)k}}.\end{aligned}$$

We next exploit the independence of random vectors $\mathbf{r}_i$.

$$\mathbb{E}[e^{t \cdot \|\mathbf{Rx}\|^2}] = \mathbb{E}[e^{t \cdot \sum_{i=1}^k y_i^2}] = \mathbb{E}\left[\prod_{i=1}^k e^{t \cdot y_i^2}\right] = \mathbb{E}[e^{t \cdot y^2}]^k.$$

To compute the moment-generating function for y^2 , we recall that the pdf of the normal distribution with mean 0 and variance 1 is $f(z) = \frac{1}{\sqrt{2\pi}} e^{-z^2/2}$ and calculate

$$\begin{aligned}\mathbb{E}[e^{t \cdot y^2}] &= \int_{z=-\infty}^{\infty} f(z) \cdot e^{t \cdot z^2} dz \\ &= \int_{z=-\infty}^{\infty} \frac{1}{\sqrt{2\pi}} e^{-z^2/(2\|\mathbf{x}\|_2^2)} e^{t \cdot z^2} dz \\ &= \frac{1}{\sqrt{2\pi}} \int_{z=-\infty}^{\infty} e^{-\left(\frac{1/\|\mathbf{x}\|_2^2 - 2t}{2}\right) z^2} dz\end{aligned}$$

Here, we use the fact that $\int_{z=-\infty}^{\infty} e^{-\alpha z^2} = \sqrt{\pi/\alpha}$ which yields

$$\mathbb{E}[e^{t \cdot y^2}] = \frac{1}{\sqrt{2\pi}} \cdot \sqrt{\frac{2\pi}{(1 - 2t)}} = \frac{1}{(1 - 2t)^{1/2}}.$$

Recall that we have k terms and therefore, we consider the k -th power of this expected value. We conclude

$$\begin{aligned}\frac{\mathbb{E}[e^{t \cdot \|\mathbf{Rx}\|^2}]}{e^{t \cdot (1 + \epsilon)k}} &= ((1 - 2t)^{k/2} \cdot e^{t \cdot (1 + \epsilon)k})^{-1} \\ &\leq ((1 - 2t)^{k/2} \cdot (1 + 2t \cdot (1 + \epsilon))^{k/2})^{-1} \\ &= ((1 - 2t) \cdot (1 + 2t \cdot (1 + \epsilon)))^{-k/2} \\ &= (1 + 2t \cdot (1 + \epsilon) - 2t - 4t^2 \cdot (1 + \epsilon))^{-k/2} \\ &= (1 + 2t \cdot \epsilon - 4t^2 \cdot (1 + \epsilon))^{-k/2}\end{aligned}$$

where we use $1 + x \leq e^x$ for all $x \in \mathbb{R}$ in the inequality. We maximize w.r.t. t the expression

$$1 + 2t \cdot \epsilon - 4t^2 \cdot (1 + \epsilon).$$

Taking $t = \frac{\epsilon}{4(1+\epsilon)}$, the expression evaluates to $1 + (1 + \epsilon)^{-1}\epsilon^2/4 > 1 + \epsilon^2/8 \geq e^{\epsilon^2/16}$ using $1 + x \geq e^{x/2}$ for $|x| < 1$. We thus have that $\|\mathbf{A}\mathbf{x}\|_2^2 > (1 + \epsilon)$ with probability at most $e^{-\epsilon^2 \cdot k/32}$. A similar analysis yields the corresponding lower bound, and for some carefully chosen $k = \Theta(\epsilon^{-2} \log(1/\delta))$, this yields [Theorem 8.5](#).

8.2 Nearest Neighbor Search via Locality Sensitive Hashing

In the previous section, we learned how to construct sketches for Jaccard similarity, cosine similarity, and Euclidean distances. While these sketches allow us to estimate similarities efficiently, they still require comparing many pairs of data points if we want to find nearest neighbors. *Locality Sensitive Hashing (LSH)* provides a probabilistic method to further reduce this cost.

Before we delve into the framework, let us formalize the problem.

Definition 8.7 (Approximate Nearest Neighbor (ANN)). *In the c -approximate nearest neighbor (c -ANN) problem, for $c \geq 1$, one is given a distance/dissimilarity measure d that relates points in $\mathbb{R}^d$ and a collection of n points $X = \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n \in \mathbb{R}^d$. A c -ANN data structure preprocesses X efficiently and then answers queries for points $\mathbf{q} \in \mathbb{R}^d$ by returning a point $\mathbf{x}_i \in X$ such that $d(\mathbf{q}, \mathbf{x}_i) \leq c \cdot \min_j d(\mathbf{q}, \mathbf{x}_j)$.*

Using bucketing over the available distances, we can reduce this problem to the point location problem on equal balls (PLEB) problem.

Definition 8.8. *In the (r_1, r_2) -PLEB problem, we are given radii $r_1 < r_2$ and the collection of points X . The (r_1, r_2) -PLEB problem then returns upon query with point $\mathbf{q} \in \mathbb{R}^d$ either:*

- a point $\mathbf{x}_i \in X$ such that $d(\mathbf{q}, \mathbf{x}_i) \leq r_2$, or
- concludes that there is no point at distance at most r_1 to $\mathbf{q}$ in X .

Now, given a fast data structure to solve the (r_1, r_2) -PLEB problem for any choice of r_1 and $r_2 = r_1 \cdot c$, we can simply create a (c^i, c^{i+1}) -PLEB data structure for every $i = 0, 1, \dots, D$ and for any query $\mathbf{q} \in \mathbb{R}^d$, if $\min_i d(\mathbf{q}, \mathbf{x}_i) \in [1, c^D]$, we can recover a point $\mathbf{x}_i$ that is at distance at most c^2 times the

distance to the closest vertex. Assuming that distances are polynomially bounded in n and using $c = (1 + \epsilon/2)$, we blow up space and search time by a factor of at most $O(\log_c(n)) = O(\log n/\epsilon)$.

Finally, let us discuss how to implement a data structure for the (r_1, r_2) -PLEB problem. The key idea is that of a locally sensitive hash function.

Definition 8.9 (Locality Sensitive Hashing). *A family of hash functions $\mathcal{H}$ is called (r_1, r_2, p_1, p_2) -LSH for a distance (or similarity) measure d if, for any vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$:*

- *If $d(\mathbf{x}, \mathbf{y}) \leq r_1$, then $\Pr_{h \sim \mathcal{H}}[h(\mathbf{x}) = h(\mathbf{y})] \geq p_1$.*
- *If $d(\mathbf{x}, \mathbf{y}) \geq r_2$, then $\Pr_{h \sim \mathcal{H}}[h(\mathbf{x}) = h(\mathbf{y})] \leq p_2$.*

The idea is now to hash points $X \cup \{\mathbf{q}\}$ to buckets using such a hash function, i.e. points that receive the same hash value end up in the same bucket. While a single signature bit does not achieve anything meaningful (it addresses only 2 buckets), taking the concatenation of k signatures $s_1(\mathbf{x}) \circ s_2(\mathbf{x}) \circ \dots \circ s_k(\mathbf{x})$ as the address of the bucket reduces the number of items per bucket significantly. The key to efficiency is now to balance probabilities for collisions in the bucket that $\mathbf{q}$ hashes to:

- on the one hand side, we would like to collide with similar points with good probability. Note that each such point agrees on every bit with probability p_1 and thus agrees on all bits with probability p_1^k .
- on the other hand, one wants to avoid too many collisions with points that are far. Since each of them agrees on each bit with probability less than p_2 , the expected number of such collisions is bounded by $n \cdot p_2^k$.

Sketching the success probability of finding a close point versus the number of bad collisions then gives an S -curve for values of k . Optimizing probabilities, we get the following data structure, which can then be used to give an ANN data structure.

Theorem 8.10. *There is a data structure that solves the (r_1, r_2) -PLEB problem given a (r_1, r_2, p_1, p_2) -LSH family $\mathcal{H}$ for data points $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\} \in \mathbb{R}^d$ using $O(dn + n^{1+\gamma})$ space such that given any vector $\mathbf{p} \in \mathbb{R}^d$, the data structure returns in expected $O(n^\gamma)$ time, hash function and distance evaluations a correct solution, for $\gamma = \frac{\log(1/p_1)}{\log(1/p_2)}$. The query succeeds with constant probability.⁴*

⁴We assume henceforth that hash functions require only $O(1)$ time to evaluate and $O(n)$ time and space to sample. We further assume p_1 and p_2 to be constants.

Proof. Set $k = \log(n)/\log(1/p_2)$ and $\ell = 2n^\gamma$. Create L hash tables $H_1, H_2, \dots, H_\ell$ with 2^k buckets and for each hash table H_i sample k signature functions $s_{i,1}, s_{i,2}, \dots, s_{i,k}$ from $\mathcal{H}$ and store a pointer to each item $\mathbf{x} \in X$ in the bucket associated with the hash $s_{i,1}(\mathbf{x}) \circ s_{i,2}(\mathbf{x}) \circ \dots \circ s_{i,k}(\mathbf{x})$. Each hash table requires $O(n)$ words of space to store the $O(\log n)$ -sized pointers to the points in X . Additionally, we have to store the points in X explicitly using $O(nd)$ space (this is where the pointers point to).

Next, consider a query with point $\mathbf{q} \in \mathbb{R}^d$. We implement it by considering in each hash table the collisions with $\mathbf{q}$ and check for each point that is hashed to the same bucket whether it is at distances at most r_2 from $\mathbf{q}$.

If there is a point that is close by, we have a collision probability in hash table H_i of $p_1^{\log n / \log(1/p_2)} = n^{-\log(1/p_1)/\log(1/p_2)} = n^{-\gamma}$. Thus, we have that there is a collision in some hash table H_i with probability $1 - (1 - n^{-\gamma})^{2n^\gamma} \geq 1 - e^{-2}$ using $1 - x \leq e^{-x}$.

On the other hand, the number of collisions with far vertices in H_i is in expectation $n \cdot p_2^k = 1$ and thus, in expectation, we have at most $O(\ell) = O(n^\gamma)$ collisions with far vertices across all hash tables. $\square$

Finally, we show that for the measures/distances we considered earlier in the lecture, there are good locally-sensitive hash functions.

ANN for Jaccard Similarity. For Jaccard similarity (considering only points in $[0, 1]^d$), we can use the signature functions $s(\mathbf{x}) = (\min\{h(i) \mid \mathbf{x}(i) = 1\})$ which satisfies $\Pr[s(\mathbf{x}) = s(\mathbf{y})] = \text{Jac}(\mathbf{x}, \mathbf{y})$ where h is a hash function with no collisions on the input data points.

Lemma 8.11. *For any r_1 and $c > 1$, let the Jaccard distance between any two vectors $\mathbf{x}, \mathbf{y}$ be defined by $d(\mathbf{x}, \mathbf{y}) = 1 - \text{Jac}(\mathbf{x}, \mathbf{y})$. Then, the family of signature functions s is $(r_1, r_2 = c \cdot r_1, 1 - r_1, 1 - r_2)$ -LSH for Jaccard distances.*

Proof. Immediate from $\Pr[s(\mathbf{x}) = s(\mathbf{y})] = \text{Jac}(\mathbf{x}, \mathbf{y})$. $\square$

Note that using this LSH family in [Theorem 8.10](#) yields

$$\gamma = \frac{\log(1/p_1)}{\log(1/p_2)} = \frac{-\log(1 - r_1)}{-\log(1 - r_1 \cdot c)} \leq \frac{-\log(1 - r_1)}{-\log((1 - r_1)^c)} = \frac{1}{c}$$

For large Jaccard similarity, say above 50%, estimates for Jaccard similarity and distance have roughly the same approximation. Indeed, the application is to find near-duplicates and therefore we are mostly interested in this exact scenario. Now, even for relatively small $c > 1$, $\gamma \leq 1/c$ is substantially smaller 1 and we get a sublinear query time.

ANN for Cosine Similarity. For Cosine similarity, we had that for vectors in $\mathbb{R}_{\geq 0}^d$, we have a signature function s such that $\Pr[s(\mathbf{x}) = s(\mathbf{y})] = 1 - \frac{\theta}{\pi}$ where θ is the angle between $\mathbf{x}$ and $\mathbf{y}$. This implies the following.

Theorem 8.12. *For any r_1 and $c > 1$, let the Cosine distance be equal to the Cosine Similarity. Then, the family of signature functions s is $(r_1, r_2 = c \cdot r_1, 1 - r_1/\pi, 1 - r_2/\pi)$ -LSH for Cosine distances.*

Proof. Again, immediate from $\Pr[s(\mathbf{x}) = s(\mathbf{y})] = 1 - \frac{\theta}{\pi}$. $\square$

A similar calculation as above shows that again we have $\gamma \leq 1/c$ for Cosine similarity/distance.

ANN for Euclidean Distances. While LSH families for Jaccard Distance and Cosine Distance are immediate from our sketching techniques, our sketching technique for Euclidean distances rather preserves distances than creating collisions for similar items.

Luckily, not much more is needed beyond what we have already seen. Consider again a random vector $\mathbf{r} \sim \mathcal{N}(0, 1)^d$. We then consider the hash function

$$h(\mathbf{x}) = \left\lfloor \frac{\langle \mathbf{r}, \mathbf{x} \rangle + t}{w} \right\rfloor$$

where $t \sim \mathcal{U}[0, w]$, i.e. t is sampled uniformly from $[0, w]$ for $w \approx r_1$. That is, we again project onto a random vector $\mathbf{r}$, and then we assign different values based on the length of the projection by taking randomly shifted segments of length w along $\mathbf{r}$. Note that two vectors at distance roughly $w/4$ that are projected along $\mathbf{r}$ end up with the same value with probability at least $1/2$: their projections along $\mathbf{r}$ have smaller length than the original length, thus they have distance at most $w/2$ on $\mathbf{r}$ and thus the choice of t is such that the translation places them into the same segment with probability $1/2$.

Let us analyze this process formally: for $\mathbf{d} = \mathbf{x} - \mathbf{y}$, we have that $\langle \mathbf{r}, \mathbf{d} \rangle \sim \mathcal{N}(0, \|\mathbf{d}\|_2^2)$. Fixing $z = |\langle \mathbf{r}, \mathbf{d} \rangle|$, we have that $\mathbf{x}$ and $\mathbf{y}$ end up with the same hash with probability

$$\Pr_t[h(\mathbf{x}) = h(\mathbf{y})] = \max\left\{1 - \frac{z}{w}, 0\right\}.$$

We therefore get via the density function of the normal distribution that

$$\Pr_t[h(\mathbf{x}) = h(\mathbf{y})] = \frac{1}{\sqrt{2\pi\|\mathbf{d}\|_2^2}} \int_{z=-w}^w \left(1 - \frac{|z|}{w}\right) \cdot e^{-z^2/(2\|\mathbf{d}\|_2^2)} dz$$

This turns out to be a hard integral to solve by hand! Using numerical optimization (by a computer), one can find w that optimizes the collision probabilities and again obtain that h is an LSH such that $\gamma \leq 1/c$, as we have seen for the other two distances.

References

This lecture follows the JL analysis and definitions for the section on Approximate Nearest Neighbor/ PLEB/ LSH from the [Advanced Algorithm Design](#) course at Princeton by Matt Weinberg and Huacheng Yu, see in particular lectures 11 and 20.

Chapter 9

Data Clustering

In this chapter, we examine a clustering task widely used in data science. Given a large dataset, clustering allows us to group closely related elements so that each group can be represented by a prototypical example. Among the many clustering methods available, we will focus on ***k*-means clustering**.

k-means objective Let $X = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$ denote a set of n points. We aim to partition X into k clusters by choosing k centers $c_1, \dots, c_k \in \mathbb{R}^d$ and assigning each point to its nearest center. Define the cost (potential) of the centers $C = \{c_1, \dots, c_k\}$ by

$$\phi(X, C) = \sum_{x \in X} \min_{c \in C} \|x - c\|^2.$$

The goal of *k*-means is to find C with $|C| = k$ minimizing $\phi(X, C)$.

Why “means”? A key property of the *k*-means objective is that, once we fix which points belong to a given cluster, the best representative center of that cluster is its *mean*. The following lemma formalizes this observation.

Lemma 9.1. *Let $A \subset \mathbb{R}^d$ be a non-empty finite set of points, and let*

$$\mu_A = \frac{1}{|A|} \sum_{x \in A} x$$

denote its centroid. Then, for any $c \in \mathbb{R}^d$,

$$\sum_{x \in A} \|x - c\|^2 = \sum_{x \in A} \|x - \mu_A\|^2 + |A| \|\mu_A - c\|^2.$$

In particular, the minimum of the left-hand side is attained at $c = \mu_A$. In the notation of the k -means objective, we can equivalently write

$$\phi(A, c) = \phi(A, \mu_A) + |A| \|c - \mu_A\|^2.$$

where we adopt the convention that

$$\phi(a, C) := \min_{c \in C} \|a - c\|^2 \quad \text{and} \quad \phi(A, c) := \sum_{a \in A} \|a - c\|^2.$$

Hence, once the cluster membership A is fixed, the optimal center (minimizer of $\phi(A, c)$) is the mean μ_A .

Proof. Fix any $c \in \mathbb{R}^d$. For each $x \in A$, write

$$x - c = (x - \mu_A) + (\mu_A - c).$$

Taking squared norms and using $\|u + v\|^2 = \|u\|^2 + \|v\|^2 + 2\langle u, v \rangle$, we obtain

$$\|x - c\|^2 = \|x - \mu_A\|^2 + \|\mu_A - c\|^2 + 2\langle x - \mu_A, \mu_A - c \rangle.$$

Summing over all $x \in A$ gives

$$\sum_{x \in A} \|x - c\|^2 = \sum_{x \in A} \|x - \mu_A\|^2 + |A| \|\mu_A - c\|^2 + 2 \left\langle \sum_{x \in A} (x - \mu_A), \mu_A - c \right\rangle.$$

Since μ_A is the centroid, we have

$$\sum_{x \in A} (x - \mu_A) = 0,$$

and therefore the last term vanishes. This leaves

$$\sum_{x \in A} \|x - c\|^2 = \sum_{x \in A} \|x - \mu_A\|^2 + |A| \|\mu_A - c\|^2.$$

The first term does not depend on c , and the second term is minimized uniquely when $\mu_A - c = 0$, i.e., when $c = \mu_A$. Hence $c = \mu_A$ is the unique minimizer. $\square$

Example (1-dimensional case, $k = 2$). Consider the set $X = \{1, 2, 3, 10, 13\} \subset \mathbb{R}$. We wish to cluster these into two groups.

A natural choice is to split into $\{1, 2, 3\}$ and $\{10, 13\}$. Their centroids are:

$$\mu_{\{1,2,3\}} = \frac{1+2+3}{3} = 2, \quad \mu_{\{10,13\}} = \frac{10+13}{2} = 11.5.$$

Thus choosing centers $c_1 = 2$ and $c_2 = 11.5$ gives

$$\begin{aligned}\phi(X, \{c_1, c_2\}) &= \sum_{x \in \{1, 2, 3\}} (x - 2)^2 + \sum_{x \in \{10, 13\}} (x - 11.5)^2 \\ &= (1 + 0 + 1) + (2.25 + 2.25) = 2 + 4.5 = 6.5.\end{aligned}$$

This illustrates that once an assignment is fixed, the best centers (in the squared-distance sense) are the means of their clusters.

Computational hardness and overview of algorithms. Although the k -means objective is simple to state, it is computationally intractable in the worst case: even for $k = 2$, the decision version of the problem is NP-hard [Das08]. As a consequence, exact optimal solutions are rarely found in practice; instead one relies on heuristic or approximation algorithms. In the following three subsections we focus on three prevalent approaches: (i) the classical Lloyd's algorithm, (ii) the improved seeding strategy k -means++, and (iii) the large-scale/parallel seeding method k -means||.

9.1 Lloyd's Algorithm

Lloyd's algorithm is the classical heuristic method for the k -means objective [Llo82]. It accepts an initial set of k centers and then iterates the following two steps:

- **Assignment step:** Assign each point $x \in X$ to the nearest center.
- **Update step:** For each cluster, recompute its center as the mean of its assigned points.

The iteration continues until assignments no longer change.

Example (Lloyd's algorithm in 1D, $k = 2$). Let $X = \{1, 2, 3, 10, 13\} \subset \mathbb{R}$ and choose the initial centers

$$C_0 = \{c_1 = 10, c_2 = 13\}.$$

Iteration 1 — Assignment. Each point is assigned to its nearest center:

$$\{1, 2, 3, 10\} \rightarrow c_1 = 10, \quad \{13\} \rightarrow c_2 = 13.$$

Iteration 1 — Update. Recompute centers as means of assigned points:

$$c_1 \leftarrow \frac{1 + 2 + 3 + 10}{4} = 4, \quad c_2 \leftarrow 13.$$

Thus $C_1 = \{4, 13\}$.

Iteration 2 — Assignment. Nearest-center assignment with $C_1 = \{4, 13\}$:

$$\{1, 2, 3\} \rightarrow 4, \quad \{10, 13\} \rightarrow 13.$$

Iteration 2 — Update.

$$c_1 \leftarrow \frac{1 + 2 + 3}{3} = 2, \quad c_2 \leftarrow \frac{10 + 13}{2} = 11.5.$$

Thus $C_2 = \{2, 11.5\}$.

Iteration 3 — Assignment. With $C_2 = \{2, 11.5\}$ the assignments remain

$$\{1, 2, 3\} \rightarrow 2, \quad \{10, 13\} \rightarrow 11.5.$$

Iteration 3 — Update. Recomputing means yields the same centers:

$$c_1 = 2, \quad c_2 = 11.5.$$

Since assignments no longer change, the algorithm has converged. The final clustering is

$$\{1, 2, 3\} \quad \text{and} \quad \{10, 13\},$$

with centers 2 and 11.5.

Convergence. Each iteration of Lloyd's algorithm strictly decreases (or leaves unchanged) the cost $\phi(X, C)$: in the assignment step, points are assigned to the closest center, which cannot increase the cost; in the update step, each center is replaced by the mean of its assigned points, which by [Theorem 9.1](#) minimises the sum of squared distances within that cluster. Thus the cost sequence

$$\phi(X, C_0) \geq \phi(X, C_1) \geq \phi(X, C_2) \geq \dots$$

is monotonically non-increasing.

Moreover, there are only finitely many possible cluster assignments (at most k^n), and whenever the assignment changes, the cost strictly decreases. Therefore the algorithm must eventually reach a state where the assignments no longer change. Such a state is a *local minimum* of the k -means objective.

Further Properties.

- Each iteration takes $O(n k d)$ time for n points in $\mathbb{R}^d$, k clusters.
- In the worst case, the number of iterations can be exponential in the number n of input points. [AV06a, Vat09]

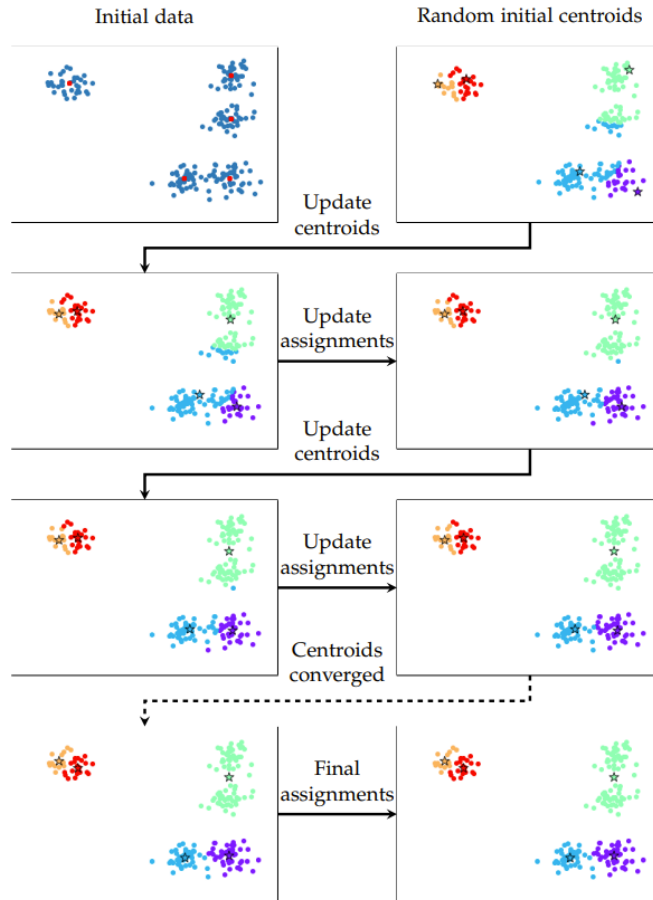


Figure 9.1: Lloyd's algorithm with random initial centers on data points drawn from 5 Gaussians.

Takeaway. Lloyd's algorithm is widely used in practice due to its simplicity and typically fast convergence. However, the quality of the final solution depends heavily on the initialization: a poor choice of initial centers may lead to significantly worse clusterings (higher cost) or require many more iterations to converge. For instance, as illustrated in Fig. 9.1, random initialization may cause the algorithm to converge to a poor local minimum. In

the next subsection, we introduce the improved seeding strategy k -means++, which is designed to reduce this sensitivity to initialization.

9.2 k -means++: Initializing with guarantees

Motivation. The performance of Lloyd’s algorithm depends heavily on the initial centers. The k -means++ method (D²-sampling) is a simple randomized seeding strategy that, in expectation, yields an $O(\log k)$ -approximation to the optimal k -means cost.

Algorithm 15 k -means++ seeding (D²-sampling) [AV06b]

```

1: Input: point set  $X \subseteq \mathbb{R}^d$ , number of centers  $k$ 
2: Uniformly sample  $c_1 \in X$  and set  $C_1 = \{c_1\}$ .
3: for  $t = 1, 2, \dots, k - 1$  do
4:   Sample  $c_{t+1} \in X$  with probability  $\Pr[c_{t+1} = x] = \frac{\phi(x, C_t)}{\phi(X, C_t)}$ .
5:   Set  $C_{t+1} = C_t \cup \{c_{t+1}\}$ .
6: end for
7: return  $C := C_k$ 

```

Example (1D, $k = 2$). Let $X = \{1, 2, 5\}$. Suppose the first center is $c_1 = 2$, so $C_1 = \{2\}$. Then $\phi(1, C_1) = 1$, $\phi(2, C_1) = 0$, $\phi(5, C_1) = 9$, and $\phi(X, C_1) = 10$. The next center is sampled with probabilities $\Pr[c_2 = 1] = \frac{1}{10}$, $\Pr[c_2 = 2] = 0$, $\Pr[c_2 = 5] = \frac{9}{10}$. If $c_2 = 5$, we return $C_2 = \{2, 5\}$.

9.2.1 Preliminary Lemmas

The analysis proceeds in two parts. First, we show that whenever k -means++ selects a center from an optimal cluster, the expected cost contributed by that cluster becomes bounded within a constant factor of its optimal cost. This is easiest to see for the very first center, which is chosen uniformly at random, yielding a 2-approximation in the case $k = 1$. We then extend this idea to the case where additional centers are selected using D²-sampling.

$k = 1$ gives a 2-approximation. When $k = 1$, k -means++ picks a random point. With μ_X the mean and $\phi^* = \phi(X, \mu_X)$, one has:

Lemma 9.2 (Lemma 3.1 in [AV06b]). *If c is drawn uniformly at random from X , then $\mathbb{E}[\phi(X, c)] \leq 2\phi^*$.*

Proof. By Lemma 9.1 with $A = X$, $\phi(X, c) = \phi^* + n\|\mu_X - c\|^2$. Averaging over uniform $c \in X$ gives $\mathbb{E}[\|\mu_X - c\|^2] = \phi^*/n$, hence $\mathbb{E}[\phi(X, c)] = 2\phi^*$. $\square$

Hitting a new optimal cluster is good. Let A be any finite set and C a current center set. If we add one new center $c \in A$ chosen with probability proportional to $\phi(c, C)$ (D²-sampling), then the expected cost of A drops to a constant factor of its best possible:

Lemma 9.3 (Lemma 3.2 in [AV06b]). *Consider two non-empty finite point sets A and C . Sample $c \in A$ with $\Pr[c = a] = \phi(a, C)/\phi(A, C)$. Then $\mathbb{E}[\phi(A, C \cup \{c\})] \leq 8\phi(A, \mu_A)$.*

Proof. We expand the expectation over the choice of c :

$$\mathbb{E}[\phi(A, C \cup \{c\})] = \sum_{c' \in A} \frac{\phi(c', C)}{\phi(A, C)} \sum_{a \in A} \min(\phi(a, C), \phi(a, c')). \quad (1)$$

For any $a, c' \in A$ and any $x \in C$, the squared triangle inequality yields

$$\|c' - x\|^2 \leq 2\|c' - a\|^2 + 2\|a - x\|^2.$$

Taking the minimum over $x \in C$ and summing over $a \in A$ gives

$$\phi(c', C) \leq \frac{2}{|A|} (\phi(A, c') + \phi(A, C)). \quad (2)$$

Using (2) inside (1), we obtain

$$\phi(c', C) \sum_{a \in A} \min(\phi(a, C), \phi(a, c')) \leq \frac{2}{|A|} (\phi(A, C) + \phi(A, c')) \min(\phi(A, C), \phi(A, c')).$$

For any nonnegative reals x, y ,

$$(x + y) \min(x, y) \leq 2xy.$$

Applying this with $x = \phi(A, C)$ and $y = \phi(A, c')$, we get

$$\phi(c', C) \sum_{a \in A} \min(\phi(a, C), \phi(a, c')) \leq \frac{4}{|A|} \phi(A, C) \phi(A, c'). \quad (3)$$

Plugging (3) into (1),

$$\mathbb{E}[\phi(A, C \cup \{c\})] \leq \sum_{c' \in A} \frac{4}{|A|} \phi(A, c') = \frac{4}{|A|} \sum_{c' \in A} \phi(A, c').$$

Finally, by Lemma 9.2,

$$\frac{1}{|A|} \sum_{c' \in A} \phi(A, c') = 2 \phi(A, \mu_A).$$

Multiplying both sides by 4 completes the proof:

$$\mathbb{E}[\phi(A, C \cup \{c\})] \leq 8 \phi(A, \mu_A).$$

□

9.2.2 Main analysis: $O(\log k)$ approximation

With Lemma 9.2 and Lemma 9.3 in hand, we can now prove that k -means++ achieves an $O(\log k)$ -approximation in expectation. The key idea is to compare the algorithm's progress against the optimal clustering C^* . At any time during the seeding process, we say that an optimal cluster is *hit/covered* if it already contains a center in C_t , and *uncovered* otherwise. When k -means++ selects a new center, it is more likely to come from an uncovered optimal cluster with large contribution to the current cost. Once such a cluster is “hit,” Lemma 9.3 ensures that its expected cost is reduced to a constant factor of optimal. However, the algorithm may also select additional centers from clusters that have already been hit, in which case some optimal clusters may remain uncovered by the end of the seeding phase. Bounding the expected cost contribution of those uncovered clusters is what yields the overall $O(\log k)$ factor. Our presentation closely follows the exposition of Dasgupta [Das13].

Theorem 9.4 (Theorem 3.1 in [AV06b]). *Let C be the set of k centers returned by the k -means++ algorithm. Then*

$$\mathbb{E}[\phi(X, C)] \leq O(\log k) \cdot \phi(X, C^*),$$

where C^* is an optimal set of k centers and expectation is over the random choices of the algorithm.

Setup. Let the optimal clustering be $A_1^*, \dots, A_k^*$, and let $\phi^* = \sum_{i=1}^k \phi(A_i^*, \mu_{A_i^*})$. After t sampled centers, $C_t = \{c_1, \dots, c_t\}$, define covered/hit and uncovered optimal cluster indices

$$H_t = \{i : A_i^* \cap C_t \neq \emptyset\}, \quad U_t = \{1, \dots, k\} \setminus H_t, \quad W_t = t - |H_t|.$$

Let $X_t^{\mathcal{H}} = \bigcup_{i \in H_t} A_i^*$ and $X_t^{\mathcal{U}} = \bigcup_{i \in U_t} A_i^*$.

We define the *potential*

$$\Psi_t := \frac{W_t \cdot \phi(X_t^{\mathcal{U}}, C_t)}{|U_t|}.$$

Note that if $|U_t| = 0$, we define Ψ_t to be 0. This can possibly only happen in the last step and it does not affect the arguments that we present.

The idea is that Ψ_t tracks how costly the uncovered clusters remain. If $W_t > 0$, then we have wasted W_t selections on already-covered clusters, so at the end there will still be at least W_t uncovered clusters. The ratio $\phi(X_t^{\mathcal{U}}, C_t)/|U_t|$ is the average cost per uncovered cluster, so Ψ_t scales this average by how many wasted steps we have made. Controlling Ψ_t therefore controls the remaining cost from uncovered clusters.

When $t = k$, one has $W_k = |U_k|$ and hence

$$\phi(X, C_k) = \phi(X_k^{\mathcal{H}}, C_k) + \phi(X_k^{\mathcal{U}}, C_k) = \phi(X_k^{\mathcal{H}}, C_k) + \Psi_k.$$

Key lemmas. We first bound the cost contributed by the optimal clusters that have already been hit. By Lemma 9.2 for the first cluster that is hit and Lemma 9.3 for subsequent hits, each covered cluster remains within a constant factor of its optimal cost, in expectation. A simple induction yields the following result.

Lemma 9.5. *For every $t \in \{1, \dots, k\}$, $\mathbb{E}[\phi(X_t^{\mathcal{H}}, C_t)] \leq 8 \sum_{i \in H_t} \phi(A_i^*, \mu_{A_i^*}) \leq 8\phi^*$.*

Next, we state the key lemma of the analysis. Intuitively, Lemma 9.6 shows that the potential Ψ_t cannot increase too quickly in expectation.

Lemma 9.6. *For every $t \in \{1, \dots, k-1\}$ and center set C_t ,*

$$\mathbb{E}[\Psi_{t+1} - \Psi_t \mid C_t] \leq \frac{\phi(X_t^{\mathcal{H}}, C_t)}{k-t}.$$

Let I denote the (random) index such that the $(t+1)$ st center lies in the optimal cluster A_I^* . We distinguish two cases depending on whether the newly hit cluster is covered/hit or uncovered.

Case 1: $I \in U_t$ (a new cluster is hit). This is the “good” case: we successfully hit a previously uncovered optimal cluster. Then $H_{t+1} = H_t \cup \{I\}$, $U_{t+1} = U_t \setminus \{I\}$, and $W_{t+1} = W_t$. Hence,

$$\Psi_{t+1} = \frac{W_{t+1} \phi(X_{t+1}^{\mathcal{U}}, C_{t+1})}{|U_{t+1}|} \leq \frac{W_t (\phi(X_t^{\mathcal{U}}, C_t) - \phi(A_I^*, C_t))}{|U_t| - 1}.$$

The expected contribution of A_I^* is

$$\mathbb{E}[\phi(A_I^*, C_t) \mid C_t, I \in U_t] = \sum_{i \in U_t} \frac{\phi(A_i^*, C_t)}{\phi(X_t^{\mathcal{U}}, C_t)} \cdot \phi(A_i^*, C_t) \geq \frac{\phi(X_t^{\mathcal{U}}, C_t)}{|U_t|}.$$

Plugging this in gives

$$\mathbb{E}[\Psi_{t+1} \mid C_t, I \in U_t] \leq \frac{W_t(\phi(X_t^{\mathcal{U}}, C_t) - \frac{\phi(X_t^{\mathcal{U}}, C_t)}{|U_t|})}{|U_t| - 1} = \frac{W_t \phi(X_t^{\mathcal{U}}, C_t)}{|U_t|} = \Psi_t.$$

Thus,

$$\mathbb{E}[\Psi_{t+1} - \Psi_t \mid C_t, I \in U_t] \leq 0.$$

Case 2: $I \in H_t$ (a wasted step). Here, we pick a point from a cluster already hit: $H_{t+1} = H_t$, $U_{t+1} = U_t$, and $W_{t+1} = W_t + 1$. Then

$$\Psi_{t+1} - \Psi_t \leq \frac{(W_t + 1)\phi(X_t^{\mathcal{U}}, C_t)}{|U_t|} - \frac{W_t \phi(X_t^{\mathcal{U}}, C_t)}{|U_t|} = \frac{\phi(X_t^{\mathcal{U}}, C_t)}{|U_t|}.$$

Hence,

$$\mathbb{E}[\Psi_{t+1} - \Psi_t \mid C_t, I \in H_t] \leq \frac{\phi(X_t^{\mathcal{U}}, C_t)}{|U_t|}.$$

Proof of Lemma 9.6. Combining the two cases:

$$\begin{aligned} \mathbb{E}[\Psi_{t+1} - \Psi_t \mid C_t] &= \mathbb{E}[\Psi_{t+1} - \Psi_t \mid C_t, I \in U_t] \cdot \Pr(I \in U_t \mid C_t) \\ &\quad + \mathbb{E}[\Psi_{t+1} - \Psi_t \mid C_t, I \in H_t] \cdot \Pr(I \in H_t \mid C_t) \\ &\leq 0 + \frac{\phi(X_t^{\mathcal{U}}, C_t)}{|U_t|} \cdot \frac{\phi(X_t^{\mathcal{H}}, C_t)}{\phi(X, C_t)}. \end{aligned}$$

Finally, using $|U_t| = k - t + W_t \geq k - t$ and $\phi(X_t^{\mathcal{U}}, C_t) \leq \phi(X, C_t)$ gives:

$$\mathbb{E}[\Psi_{t+1} - \Psi_t \mid C_t] \leq \frac{\phi(X_t^{\mathcal{H}}, C_t)}{k - t}.$$

□

Combining the bound on the expected increase of the potential from Lemma 9.6 with the fact that covered clusters contribute at most a constant factor of the optimal cost (Lemma 9.5) in expectation, we obtain the following bound on the drift of the potential.

Lemma 9.7. *For every $t \in \{1, 2, \dots, k-1\}$, it holds that*

$$\mathbb{E}[\Psi_{t+1} - \Psi_t] \leq \frac{8\phi^*}{k-t}.$$

Proof. From Lemma 9.6 we know that for every fixed choice of C_t ,

$$\mathbb{E}[\Psi_{t+1} - \Psi_t \mid C_t] \leq \frac{\phi(X_t^{\mathcal{H}}, C_t)}{k-t}.$$

Taking expectation over the randomness up to time t and applying the law of total expectation,

$$\mathbb{E}[\Psi_{t+1} - \Psi_t] = \mathbb{E}[\mathbb{E}[\Psi_{t+1} - \Psi_t \mid C_t]] \leq \frac{\mathbb{E}[\phi(X_t^{\mathcal{H}}, C_t)]}{k-t}.$$

By Lemma 9.5, $\mathbb{E}[\phi(X_t^{\mathcal{H}}, C_t)] \leq 8\phi^*$, which gives

$$\mathbb{E}[\Psi_{t+1} - \Psi_t] \leq \frac{8\phi^*}{k-t}.$$

□

Proof of Theorem 9.4. At the end of seeding, the expected cost splits into the contribution of covered optimal clusters and the (banked) contribution of uncovered ones:

$$\begin{aligned} \mathbb{E}[\phi(X, C_k)] &= \mathbb{E}[\phi(X_k^{\mathcal{H}}, C_k)] + \mathbb{E}[\Psi_k] \\ &\leq 8\phi^* + \sum_{t=1}^{k-1} \mathbb{E}[\Psi_{t+1} - \Psi_t] \quad (\text{by Lemma 9.5 and telescoping of } \Psi_t) \\ &\leq 8\phi^* + \sum_{t=1}^{k-1} \frac{8\phi^*}{k-t} \quad (\text{by Lemma 9.7}) \\ &= 8\phi^* + 8\phi^* \sum_{j=1}^{k-1} \frac{1}{j} = O(\log k) \phi^*. \end{aligned}$$

This completes the proof.

Discussion. In practice, one typically initializes the centers using k -means++ and then runs Lloyd's algorithm to refine them. Figure 9.2 shows how k -means++ tends to spread centers across the dataset by sampling points in

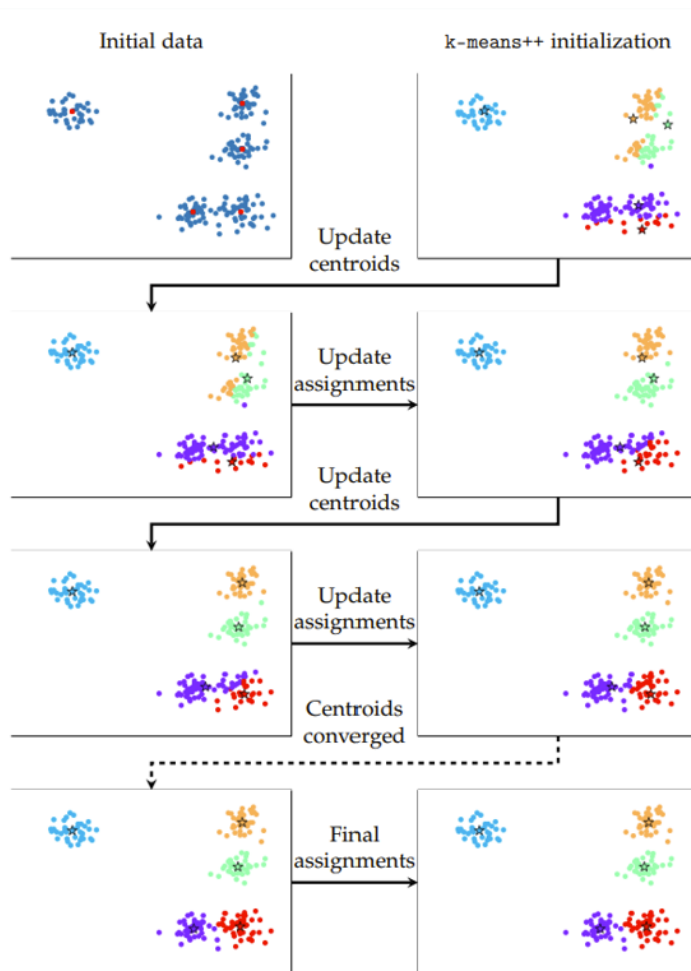


Figure 9.2: Lloyd's algorithm with k -means++ initialization on data points X that are generated from 5 Gaussians.

regions with large residual cost, often resulting in substantially better initializations than uniform random seeding. However, a drawback is that k -means++ requires k sequential passes over the data, since each new center is chosen based on distances to all points. When k is large or the dataset is distributed, this can be prohibitively slow. To address this limitation, Bahmani et al. introduced k -means|| [BMV⁺12], a parallelizable variant that samples multiple candidate centers in each round, reducing the number of passes to roughly $O(\log n)$ while achieving similar guarantees.

9.3 k-means||: Parallelizing the initialization

The key idea behind k -means|| is to preserve the “distance-proportional” sampling strategy of k -means++, but to perform it in parallel rather than adding one center at a time. Instead of selecting a single new center in each step, k -means|| samples many candidate centers per round, in proportion to their contribution to the current cost. After a logarithmic number of rounds, this produces an oversampled set of centers C with size much larger than k , but guaranteed to contain good representatives for all clusters.

A final downsampling (typically by running k -means++ on a weighted instance) reduces the set back to exactly k centers. This approach reduces the number of data passes from k to $O(\log n)$, making it suitable for large-scale and distributed environments (e.g., MapReduce, Spark).

Algorithm 16 k -means|| overseeding [BMV⁺12]

```

1: Input:  $X \subseteq \mathbb{R}^d$ , oversampling factor  $\ell \geq 1$ , number of rounds  $r$ 
2: Uniformly sample  $c_1 \in X$  and set  $C_1 = \{c_1\}$ .
3: for  $t = 1, 2, \dots, r$  do
4:   for each  $x \in X$  (independently) do
5:     Add  $x$  to  $C'_t$  with probability  $\min\left\{1, \ell \cdot \frac{\phi(x, C_t)}{\phi(X, C_t)}\right\}$ .
6:   end for
7:   Set  $C_{t+1} = C_t \cup C'_t$ .
8: end for
9: return  $C_{r+1}$ 

```

Theorem 9.8 (Overseeding guarantee). *Let C_{r+1} be the candidate set returned by Algorithm 16. Suppose the oversampling factor satisfies $\ell \geq k$ and the number of rounds is chosen as*

$$r \geq 1000 \cdot \log\left(\frac{\phi(X, \mu_X)}{\phi^*}\right),$$

where ϕ^* is the optimal k -means cost. Then,

$$\mathbb{E}[\phi(X, C_{r+1})] \leq 100 \cdot \phi^*.$$

Settled vs. unsettled clusters. Fix an optimal solution $C^* = \{c_1^*, \dots, c_k^*\}$ with clusters $A_1^*, \dots, A_k^*$ and cost $\phi^* = \sum_{i=1}^k \phi(A_i^*, \mu_{A_i^*})$. For a current center set C , say that A_i^* is *settled* if $\phi(A_i^*, C) \leq 10 \phi(A_i^*, \mu_{A_i^*})$ and *unsettled* otherwise.

The following lemma is a central part of the analysis. Proving it is the main goal of Exercise 3 on this week’s exercise sheet.

Lemma 9.9. *Let C be the current set of sampled centers and let A^* be an unsettled optimal cluster. In one round of Algorithm 16, the probability that A^* is not made settled is at most*

$$\exp\left(-\frac{\ell \phi(A^*, C)}{5 \phi(X, C)}\right).$$

Let U_t be the set of unsettled indices with respect to C_t , and write

$$\phi_U^t := \sum_{i \in U_t} \phi(A_i^*, C_t).$$

Lemma 9.10. *For every t , it holds that $\mathbb{E}[\phi_U^{t+1}] \leq \max(0.99\phi_U^t, 10\phi^*)$.*

Proof. Without loss of generality, we can assume that $\phi_U^t \geq 10\phi^*$. In particular, at most half of the current cost can come from clusters that are already settled. Hence

$$\phi_U^t \geq \frac{1}{2}(\phi(X, C_t)).$$

Partition the unsettled clusters into

$$H = \left\{i \in U_t : \phi(A_i^*, C_t) \geq \frac{\phi_U^t}{2k}\right\} \quad \text{and} \quad L = U_t \setminus H.$$

By construction, $\sum_{i \in L} \phi(A_i^*, C_t) \leq \frac{\phi_U^t}{2}$.

For any unsettled cluster A_i^* , the “hitting” lemma (proved earlier) gives that in one overseeding round with oversampling factor $\ell \geq k$,

$$\Pr[A_i^* \text{ remains unsettled} \mid C_t] \leq \exp\left(-\frac{\ell \phi(A_i^*, C_t)}{5 \phi(X, C_t)}\right).$$

In particular, for a *heavy* cluster $i \in H$,

$$\begin{aligned} \Pr[\text{not settled} \mid C_t] &\leq \exp\left(-\frac{\ell}{5 \phi(X, C_t)} \cdot \frac{\phi_U^t}{2k}\right) \\ &\leq \exp\left(-\frac{1}{10} \cdot \frac{\phi_U^t}{\phi(X, C_t)}\right) \\ &\leq e^{-1/20} \\ &\leq 1 - \frac{1}{50}, \end{aligned}$$

where we used $\ell \geq k$ and $\phi_U^t \geq \phi(X, C_t)/2$. Thus each heavy cluster is settled with probability at least $1/50$.

By linearity of expectation, the expected *drop* in unsettled cost from heavy clusters is at least

$$\frac{1}{50} \sum_{i \in H} \phi(A_i^*, C_t) = \frac{1}{50} \left(\phi_U^t - \sum_{i \in L} \phi(A_i^*, C_t) \right) \geq \frac{1}{50} \cdot \frac{\phi_U^t}{2} = \frac{\phi_U^t}{100}.$$

□

Proof of Theorem 9.8. Using Lemma 9.2 (the $k=1$ two-approximation), after the first uniform sample we have

$$\mathbb{E}[\phi_U^1] \leq 2 \phi(X, \mu_X).$$

By Lemma 9.10, for every t ,

$$\mathbb{E}[\phi_U^{t+1}] \leq \max(0.99 \phi_U^t, 10 \phi^*).$$

Unrolling this recurrence yields, for all $r \geq 1$,

$$\mathbb{E}[\phi_U^r] \leq 10 \phi^* + (0.99)^{r-1} \cdot \mathbb{E}[\phi_U^1] \leq 10 \phi^* + (0.99)^{r-1} \cdot 2 \phi(X, \mu_X).$$

With $r \geq 1000 \cdot \log\left(\frac{\phi(X, \mu_X)}{\phi^*}\right)$, we get

$$(0.99)^r \cdot 2 \phi(X, \mu_X) \leq 2 \phi(X, \mu_X) \cdot \left(\frac{\phi^*}{\phi(X, \mu_X)}\right)^{10} \leq \phi^*.$$

Therefore $\mathbb{E}[\phi_U^r] \leq 11 \phi^*$.

Finally, the total cost at the end of overseeding satisfies

$$\phi(X, C_{r+1}) = \underbrace{\phi_U^{r+1}}_{\leq \phi_U^r} + \underbrace{\sum_{i \notin U_{r+1}} \phi(A_i^*, C_{r+1})}_{\leq 10 \phi^*} \leq \phi_U^r + 10 \phi^*,$$

where we used monotonicity of the potential and that settled clusters contribute at most $10 \phi^*$ by definition. Taking expectations gives

$$\mathbb{E}[\phi(X, C_{r+1})] \leq \mathbb{E}[\phi_U^r] + 10 \phi^* \leq 21 \phi^* \leq 100 \phi^*,$$

which proves the theorem. □

Algorithm 17 Final selection via weighted k -means++

-
- 1: **Input:** $X \subseteq \mathbb{R}^d$, candidate set B (e.g., $B = C_{r+1}$), number of centers k
 - 2: **for** each $b \in B$ **do**
 - 3: $w_b \leftarrow$ number of points in X for which b is the closest center in B
 - 4: **end for**
 - 5: Run k -means++ (Algorithm 15) on B using weights $\{w_b\}_{b \in B}$.
 - 6: **return** the resulting k centers
-

9.3.1 Weighted k -means++

After the overseeding phase, we have a candidate center set C_{r+1} of size much larger than k , but which approximates the geometry of the optimal clustering up to a constant factor (Theorem 9.8). The final step is to reduce this candidate set back down to k centers. To do this, we treat each candidate $b \in C_{r+1}$ as representing the points in X for which b is the nearest candidate, and assign it a corresponding weight. We then run k -means++ on the weighted instance.

The weighted k -means++ procedure behaves exactly like the original k -means++ seeding, except that when selecting a new center, each $b \in B$ is sampled with probability proportional to $w_b \cdot \phi(b, C)$. This ensures that candidates representing large portions of the dataset are appropriately favored during center selection.

Theorem 9.11 (Weighted sampling preserves approximation). *Let C be the set of k centers returned by Algorithm 17 applied to the candidate set C_{r+1} . Then, in expectation,*

$$\mathbb{E}[\phi(X, C)] \leq O(\log k) \cdot \phi^*.$$

Thus k -means||, followed by weighted k -means++, achieves the same $O(\log k)$ approximation guarantee as k -means++ seeding, while requiring only $O(\log(\phi(X, \mu_X)/\phi^*))$ passes over the data—a dramatic improvement in large-scale or parallel environments.

Part III

Modern Graph Algorithms

Chapter 10

Distance-Preserving Trees

Many challenging graph problems become significantly easier when restricted to trees. In particular, certain $\mathcal{NP}$ -hard problems admit exact polynomial-time solutions on trees, and for others, we can achieve much better approximation guarantees when working with tree structures.

Motivated by this, one would like to design a general framework for an arbitrary graph $G = (V, E, w)$ consisting of the following steps:

1. Construct a tree T that approximately preserves the distances in G .
2. Solve the problem efficiently on T .
3. Map the solution obtained on T back to the original graph G .
4. Argue that this mapped solution approximates well the exact solution on G .

Ideally, the tree T would satisfy for all pairs $u, v \in V$ the inequalities

$$d_G(u, v) \leq d_T(u, v) \leq \alpha \cdot d_G(u, v),$$

where $\alpha \geq 1$ is the *stretch* factor of the tree embedding. Now the question is whether we can prove that such a tree exists for every graph for some small stretch c .

Unfortunately, such a construction is hopeless. As can be seen from [Figure 10.1](#), a stretch of $\Omega(n)$ is unavoidable on some graphs. Take, for example, the cycle graph G over n vertices. A tree T is obtained by removing any single one of the edges on the cycle (see [Figure 10.1](#)). While the endpoints of all the edges that are included in the tree have stretch 1, for the excluded edge, we have that the endpoints are distance $n - 1$ in the tree T (see [\[WS11\]](#)).

Once again, we call upon probability to come to the rescue. Consider that we sample among the n possible subtrees of a cycle graph. Let us analyze the

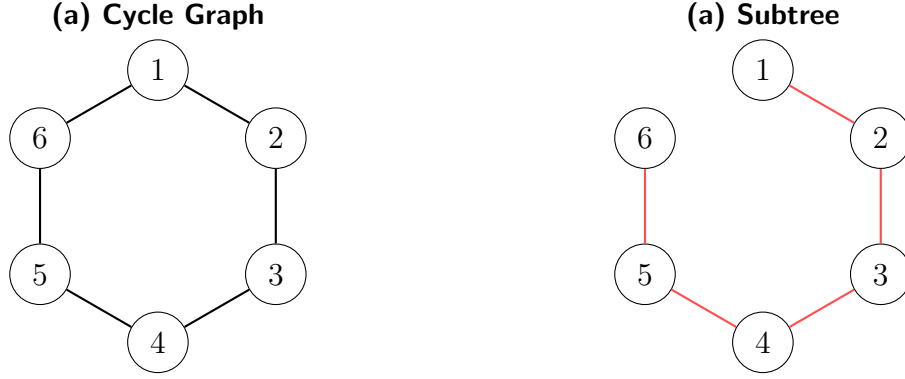


Figure 10.1: The cycle graph and a subtree obtained by removing a single edge.

expected stretch for a fixed edge e , i.e., the expected distance in the sampled tree between the endpoints of e .

In the cycle graph, there are n edges in total. To form a spanning tree, we need to remove only one. The edge e is kept in the tree with probability $\Pr[e \in T] = (n-1)/n$. In this case, its endpoints are distance 1 apart. The edge is removed with probability $\Pr[e \notin T] = 1/(n-1)$. In this case, one has to travel between the endpoints of e along the remainder of the cycle and thus its endpoints are distance $n-1$ apart.

The expected distance between endpoints of $e = (u, v)$ in T is

$$\begin{aligned} \Pr[e \in T] \cdot \text{dist}_T(u, v) + \Pr[e \notin T] \cdot \text{dist}_T(u, v) &= \frac{n-1}{n} \cdot 1 + \frac{1}{n} \cdot (n-1) \\ &< 2 \end{aligned}$$

This is surprisingly low and gives us hope that randomness is the right tool.

More generally, we relax our requirement and consider a *distribution* $\mathcal{T}$ over trees with the following properties:

- **Non-contraction:** For every tree $T \in \mathcal{T}$ and every $u, v \in V$,

$$d_G(u, v) \leq d_T(u, v),$$

- **Expected stretch bound:** For every $u, v \in V$,

$$\mathbb{E}_{T \sim \mathcal{T}} [d_T(u, v)] \leq \alpha \cdot d_G(u, v),$$

Fakcharoenphol, Rao, and Talwar [FRT03] proved in 2003 that a distribution with $\alpha = O(\log n)$ stretch exists. This is tight, meaning it matches a lower

bound up to constant factors! Notably, the distribution is over trees that are **not** required to be subgraphs of G . It is still a major open question to give such a distribution with tight $O(\log n)$ stretch (the best stretch obtained is $O(\log n \log \log n)$).

In previous iterations of this course, we taught the algorithm by Fakcharoenphol, Rao, and Talwar. However, with this course in mind, Kyng, Probst Gutenberg and Rieder [KGR25] have recently (in 2025) developed an alternative algorithm based on the modern random-shift framework. We present this algorithm in this lecture.

10.1 The Random-Shift Algorithm

Without further ado, let us present the key algorithmic primitive towards sampling a low-stretch tree embedding: the random-shift procedure in [Algorithm 18](#). The procedure was developed by Miller, Peng and Xu in 2013 and is one of the cornerstones of modern graph algorithms.

The input to the algorithm (besides the input graph G) is a parameter D and the goal is to compute a clustering of V into components of diameter $O(D \log n)$ such that few edges cross between clusters.

To achieve this goal, the algorithm picks a head start δ^u for every vertex u that is chosen from the exponential distribution with mean D . Each vertex v is assigned to the vertex u that minimizes $\text{dist}_G(u, v) - \delta^u$. For convenience, we assume throughout the rest of the chapter that there is always a unique minimizer (this holds with probability 1).

Gary Miller has suggested a very nice analogy of this process. We can think of this as a horse race towards a fixed vertex v . We have a horse for each vertex $u \in V$ starting from u with a random head start δ^u . The horse u must then travel the distance $\text{dist}_G(u, v)$ to v . We say that horse u is at the finish line at time $\text{dist}_G(u, v) - \delta^u$. The horse u that is first at the finish line wins, and u is exactly the vertex that becomes the cluster center of v . We will sometimes say, u wins v and say that $\text{dist}_G(u, v) - \delta^u$ is the winning time.

While the head starts play a big role in which vertices become cluster centers, the fact that the horse still needs to ride the distance heavily favors horses that start closer to v . Horses at very large distances, even with a solid head start, have almost no chance of winning the race. Intuitively, it should thus almost never occur that the center of the cluster containing v is a very distant vertex. We will use this analogy frequently throughout the analysis.

Note that we can implement the pseudo-code given in [Algorithm 18](#) in $O(m + n \log n)$ time by reducing to a call to Dijkstra's single-source shortest

Algorithm 18 RANDOMSHIFTDECOMP($G = (V, E, w : E \mapsto [1, W]), D$)

```

for each vertex  $u \in V$  do
    Pick  $\delta^u$  independently from the exponential distribution with mean  $D$ .
end for
for each vertex  $u \in V$  do
     $C^u \leftarrow \{v \in V \mid \text{dist}_G(u, v) - \delta_u \leq \min_{x \in V} \text{dist}_G(x, v) - \delta_x\}$ .
     $\text{CENTER}(C^u) \leftarrow u$ .
end for
return  $(\mathcal{C} = \{C^u\}_{u \in V} \setminus \{\emptyset\}, \text{CENTER})$ 

```

paths algorithm. In the exercises, we ask you to prove this formally.

It remains to prove that the clustering indeed has small diameter clusters and only a small fraction of the edges are cut. We prove the following theorem.

Theorem 10.1. *Given a graph $G = (V, E, w : E \mapsto [1, W])$ and a parameter $D > 0$, consider the clustering $\mathcal{C}$ and center function CENTER returned by [Algorithm 18](#). For every $v \in V$, let $\mathcal{C}(v)$ be the cluster containing v in $\mathcal{C}$, and define $\text{CENTER}(v) = \text{CENTER}(\mathcal{C}(v))$. Then, with high probability,*

1. *for any $v \in V$, $\text{dist}_G(v, \text{CENTER}(v)) \leq \text{diam}_G(\mathcal{C}(v)) = O(D \log n)$, and*
2. *for any $u, v \in V$, $\mathbb{P}[\mathcal{C}(u) \neq \mathcal{C}(v)] = O(\text{dist}_G(u, v)/D)$. In particular, any edge $e = (x, y) \in E$, we have $\text{dist}_G(x, y) \leq w(e)$ and thus e crosses the clustering with probability $O(w(e)/D)$.*

The types of clusterings $\mathcal{C}$ with these guarantees are often referred to as **low-diameter decompositions** (LDDs) of G . The guarantees obtained are tight up to constant factors, i.e., one essentially cannot hope for a better trade-off between vertex-to-center distance and separation factor!

Correctness of Property 1 in Theorem 10.1. For any vertex $v \in V$, consider the cluster center $\text{CENTER}(v)$ that is taken to be the minimizer x of $\min_x \text{dist}_G(x, v) - \delta^x$, i.e. the winner of the horse race. Since $\text{dist}_G(v, v) = 0$ and $\delta_v \geq 0$, we have $\min_x \text{dist}_G(x, v) - \delta^x \leq 0$. The finish line is reached by the winner before or at time 0. But this means that $\text{dist}_G(v, x = \text{CENTER}(v))$ is at most equal to the head start δ^x .

Let us now study how large the head start δ^u can be. Recall that δ^u is sampled from the exponential distribution with mean D which has cumulative distribution function (cdf) $F_D(y) = \Pr[\delta^u < y] = 1 - e^{-y/D}$. We thus

get that δ^u exceeds $10D \log n$ with probability $e^{-10 \log n} = n^{-10}$. By a union bound, we thus have that **all** vertices $u \in V$ have head start $\delta^u \leq 10D \log n$ with probability at least $1 - n^{-9}$. In particular, this includes the minimizer $x = \arg \min_x \text{dist}_G(x, v) - \delta^x$.

We have thus proven that $\text{dist}_G(v, x = \text{CENTER}(v)) \leq 10D \log n = O(D \log n)$, as desired. Since this bound holds for all vertices v w.h.p., the diameter of every cluster is at most $20D \log n$ w.h.p..

Correctness of Property 2 in Theorem 10.1. Let us now focus on bounding the probability that vertices $u, v \in V$ are separated, i.e. that $\text{CENTER}(u) \neq \text{CENTER}(v)$.

- **Defining the Winning Margin:** Let $c_u = \text{CENTER}(u)$, and recall that $t_u = \text{dist}_G(u, c_u) - \delta^{c_u} = \min_x \text{dist}_G(x, u) - \delta^x$ is the finish time for u .

Let us now define the *winning margin* $\text{GAP}(u)$ at u , that is, the gap between the time of the winner and the second-best competitor

$$\text{GAP}(u) = (\min_{x \neq c_u} \text{dist}_G(x, v) - \delta^x) - t_u.$$

Winning times t_u and t_v are strongly *correlated*. In particular, for $\text{GAP}(u) > 2 \text{dist}_G(u, v)$, for every $x \neq c_u$, we have

$$\begin{aligned} \text{dist}_G(x, u) - \delta^x &> t_u + 2 \text{dist}_G(u, v) \\ \implies \text{dist}_G(x, v) + \text{dist}_G(u, v) - \delta^x &> t_u + 2 \text{dist}_G(u, v) \\ \iff \text{dist}_G(x, v) - \delta^x &> t_u + \text{dist}_G(u, v) \\ \implies \text{dist}_G(x, v) - \delta^x &> (t_v - \text{dist}_G(u, v)) + \text{dist}_G(u, v). \\ \iff \text{dist}_G(x, v) - \delta^x &> t_v. \end{aligned}$$

where we used in the second last line $t_v = \min_x \text{dist}_G(x, v) - \delta^x \leq (\text{dist}_G(u, c_u) + \text{dist}_G(u, v)) - \delta^{c_u} = t_u + \text{dist}_G(u, v)$ by the triangle inequality. But if every competitor $x \neq c_u$ arrives *after* the winner, then only c_u is left to arrive at the winning time t_v and therefore c_u is also the winner of v .

- **Bounding the Winning Margin:** Let us next argue about the probability of the event $\text{GAP}(u) \leq 2 \cdot \text{dist}_G(u, v)$. As seen above, this upper bounds the probability that u and v are separated, i.e. $\text{CENTER}(u) \neq \text{CENTER}(v)$.

We start with a short detour into order statistics for the exponential distributions. A staple of order statistics is to analyze for i.i.d. random variables $X_1, X_2, \dots, X_k$ from an exponential distribution with mean D , the gap $X^{(1)} - X^{(2)}$ between the largest value $X^{(1)} = \max_i X_i$ and the second largest value $X^{(2)} = \max_{j, X_j \neq X^{(1)}} X_j$.

The key trick is to use the **memoryless** property of the exponential distribution: for a random variable X drawn from the exponential distribution and non-negative values d and s , we have $\Pr[X \leq d + s \mid X > s] = \Pr[X \leq d]$. That is, even after conditioning, the 'remaining' value is again exponentially distributed. Thus, for any $d \geq 0$, then

$$\begin{aligned} \Pr[X^{(1)} - X^{(2)} \leq d \mid X^{(1)} > X^{(2)}] &= \Pr[X^{(1)} \leq d + X^{(2)} \mid X^{(1)} > X^{(2)}] \\ &= \Pr[X \leq d] = 1 - e^{-d/D}. \end{aligned}$$

where X is again a r.v. drawn from the exponential distribution with mean D ! This means the gap $X^{(1)} - X^{(2)}$ is exponentially distributed and thus below our threshold with probability $\Pr[X^{(1)} - X^{(2)} \leq 2 \cdot \text{dist}_G(u, v)] = 1 - e^{-2\text{dist}_G(u, v)/D} = O(\text{dist}_G(u, v)/D)$ using $1 + x \leq e^x$.

Let us now come back to our problem. Let us define independent r.v. $Y_1 = \delta^{v_1} - \text{dist}_G(v_1, x), Y_2 = \delta^{v_2} - \text{dist}_G(v_2, x), \dots, Y_n = \delta^{v_n} - \text{dist}_G(v_n, x)$. We take largest value $Y^{(1)} = \max_i Y_i$ and second largest value $Y^{(2)} = \max_{j, Y_j \neq Y^{(1)}} Y_j$. Rewrite minimization problems as maximization problems yields

$$\begin{aligned} \text{GAP}(u) &= (\min_{x \neq c_u} \text{dist}_G(x, u) - \delta^x) - (\min_x \text{dist}_G(x, u) - \delta^x) \\ &= -(\max_{x \neq c_u} \delta^x - \text{dist}_G(x, u)) + (\max_x \delta^x - \text{dist}_G(x, u)) \\ &= (\max_x \delta^x - \text{dist}_G(x, u)) - (\max_{x \neq c_u} \delta^x - \text{dist}_G(x, u)) \\ &= Y^{(1)} - Y^{(2)}. \end{aligned}$$

But is the gap $\text{GAP}(u) = Y^{(1)} - Y^{(2)}$ exponentially distributed, like we saw for $X^{(1)} - X^{(2)}$? The answer is no: consider the case where vertex u is isolated, thus $\text{dist}(x, u) = \infty$ for $u \neq x$, resulting in u winning for u and everyone else's finish time being ∞ . Thus, in this case, the gap is ∞ with probability 1. However, the gap remains **stochastically lower bounded**, which suffices for our purpose.

Formally, for any v_i , let us condition on v_i winning u , i.e. $Y^{(1)} = \delta^{v_i} - \text{dist}_G(v_i, u)$. Knowing the winner of the horse race does not reveal by how much she won! Thus, for $s = Y^{(2)}$, we can define

$\Delta = \max\{0, \text{dist}_G(v_i, u) - s\}$ and we have that $Y^{(1)} > s + \Delta$ with probability 1. We obtain

$$\begin{aligned} \Pr[Y^{(1)} > d + s \mid Y^{(1)} > s + \Delta, v_i = \text{CENTER}(u)] \\ &= \Pr[Y^{(1)} > d + s + \Delta - \Delta \mid Y^{(1)} > s + \Delta, v_i = \text{CENTER}(u)] \\ &\geq \Pr[Y^{(1)} > d + s + \Delta \mid Y^{(1)} > s + \Delta, v_i = \text{CENTER}(u)] \\ &= \Pr[Y > d] = e^{-d/D}. \end{aligned}$$

where Y is again a r.v. drawn from the exponential distribution with mean D .

As seen above, being stochastically dominated by the exponential distribution yields $\text{GAP}(u) \leq 2 \cdot \text{dist}_G(u, v)$ with probability $O(\text{dist}_G(u, v)/D)$, as desired.

10.2 Probabilistic Tree Embeddings

We now present the algorithm to sample a probabilistic tree embedding T from a distribution over trees.

The Algorithm. Let $L = \lceil \log_2(nW) \rceil$. We first take clusterings $\mathcal{C}_0, \dots, \mathcal{C}_L$ of increasing diameters. For every level $0 < l < L$, let us sample a clustering $(\mathcal{C}_l, \text{CENTER}_l)$ obtained from invoking [Algorithm 18](#) on G with parameter 2^l . For the first and last levels, we define $\mathcal{C}_0 = \{\{v\} \mid v \in V\}$ and $\mathcal{C}_L = \{V\}$, let CENTER_0 be the identity function on the vertices, and CENTER_L be a constant map to an arbitrary fixed vertex $r \in V$.

For the rest of the section, we assume that each clustering $\mathcal{C}_l$ is a refinement of the clustering $\mathcal{C}_{l+1}$. That is, for any $0 \leq l < L$ and $C \in \mathcal{C}_l$, there is a larger cluster $C' \supseteq C$ contained in clustering $\mathcal{C}_{l+1}$. In the exercises, we ask you to show that this property can be obtained with only a constant loss in the guarantees achieved by [Theorem 10.1](#).

Finally, consider the tree T that has a node associated with each cluster $C \in \mathcal{C}_l$ and for each $0 \leq l < L$, an edge from any node $C \in \mathcal{C}_l$ to the node $C \subseteq C' \in \mathcal{C}_{l+1}$ of weight $\text{dist}_G(\text{CENTER}_l(C), \text{CENTER}_{l+1}(C'))$. We identify each vertex $v \in V$ with the node $\{v\}$ from $\mathcal{C}_0$. This completes the algorithm.

Analysis. Recall that our goal is to show for any two vertices $u, v \in V$, that $\text{dist}_G(u, v) \leq \text{dist}_T(u, v)$ and $\mathbb{E}[\text{dist}_T(u, v)] \leq \alpha \cdot \text{dist}_G(u, v)$ for some c .

Let l denote the smallest integer such that for some cluster $C \in \mathcal{C}_l$, we have $u, v \in C$. Consider the unique uv -tree path in T

$$\pi = \langle \mathcal{C}_0(u) = \{u\}, \mathcal{C}_1(u), \dots, \mathcal{C}_l(u) = \mathcal{C}_l(v), \mathcal{C}_{l-1}(v), \dots, \mathcal{C}_0(v) = \{v\} \rangle.$$

Note that we have

$$\text{dist}_T(u, v) = w(\pi) = \sum_{z \in \{u, v\}} \sum_{0 \leq i < l} \text{dist}_G(\text{CENTER}_i(z), \text{CENTER}_{i+1}(z))$$

and thus by the triangle inequality, $w(\pi) \geq \text{dist}_G(\text{CENTER}_0(u), \text{CENTER}_0(v)) = \text{dist}_G(u, v)$ which yields the lower bound.

For the upper bound, let us first observe that by the triangle inequality, we have $\text{dist}_G(\text{CENTER}_i(z), \text{CENTER}_{i+1}(z)) \leq \text{dist}_G(z, \text{CENTER}_i(z)) + \text{dist}_G(z, \text{CENTER}_{i+1}(z))$ and thus

$$\text{dist}_T(u, v) = w(\pi) \leq 2 \cdot \sum_{z \in \{u, v\}} \sum_{0 \leq i < l} \text{dist}_G(z, \text{CENTER}_{i+1}(z)).$$

Since l is a random variable, it will be more convenient to sum over all levels i up to L and use an indicator variable $\mathbf{1}[i < l]$. Taking expectations, and using linearity of expectations, and the guarantees from [Algorithm 18](#), we obtain

$$\begin{aligned} \mathbb{E}[\text{dist}_T(u, v)] &= 2 \sum_{z \in \{u, v\}} \sum_{0 \leq i < L} \mathbb{E}[\mathbf{1}[i < l] \cdot \text{dist}_G(z, \text{CENTER}_{i+1}(z))] \\ &= 2 \sum_{z \in \{u, v\}} \sum_{0 \leq i < L} \Pr[i < l] \cdot \mathbb{E}[\text{dist}_G(z, \text{CENTER}_{i+1}(z))] \\ &= 2 \sum_{z \in \{u, v\}} \sum_{0 \leq i < L} \text{dist}_G(u, v) / 2^i \cdot \mathbb{E}[\text{dist}_G(z, \text{CENTER}_{i+1}(z))] \\ &= O(\text{dist}_G(u, v)) \cdot \sum_{z \in \{u, v\}} \sum_{0 \leq i < L} \mathbb{E}[\text{dist}_G(z, \text{CENTER}_{i+1}(z))] / 2^i \\ &= O(\text{dist}_G(u, v)) \cdot \sum_{z \in \{u, v\}} \sum_{0 \leq i < L} O(2^i \cdot \log n) / 2^i \\ &= O(\text{dist}_G(u, v)) \cdot \sum_{z \in \{u, v\}} \sum_{0 \leq i < L} O(\log n) \\ &= O(\text{dist}_G(u, v)) \cdot 2 \cdot L \cdot O(\log n) \\ &= O(\text{dist}_G(u, v) \cdot \log^2 n). \end{aligned}$$

using $L \leq \log Wn = O(\log n)$.

But, lo and behold, that is not what we promised! We get an expected stretch of $O(\log^2 n)$ instead of the claimed $O(\log n)$. But we claim that one can tighten the analysis as follows: a more careful analysis of the random-shift algorithm in [Algorithm 18](#) allows us to tighten the bound on the vertex-to-center distance *on average* over all levels. That is, we will later show that the following property holds for every vertex $x \in V$:

$$\sum_{0 \leq i \leq L} \mathbb{E}[\text{dist}_G(x, \text{CENTER}_i(x))]/2^i = O(\log n) \quad (10.1)$$

Plugging this into the calculations above then immediately yields

$$\begin{aligned} \mathbb{E}[\text{dist}_G(u, v)] &= O(\text{dist}_G(u, v)) \cdot \sum_{z \in \{u, v\}} \sum_{0 \leq i < L} \mathbb{E}[\text{dist}_G(z, \text{CENTER}_{i+1}(z))/2^i] \\ &= \text{dist}_G(u, v) \cdot \sum_{z \in \{u, v\}} O(\log n) \\ &= O(\text{dist}_G(u, v) \cdot \log n). \end{aligned}$$

as promised. It remains to prove [\(10.1\)](#), which is done in the next section.

Recap. Let us briefly recap what we have proven so far:

- we first showed that in [Theorem 10.1](#) that the random-shift algorithm yields tight low-diameter decompositions $\mathcal{C}$ for any parameter D .
- we then showed that any tight low-diameter decomposition algorithm can be invoked for scales $2^0, 2^1, \dots, 2^L$ and used to construct a tree embedding with stretch $O(\log^2 n)$.
- we finally observed that to shave off another logarithmic factor, one has to go beyond the guarantees achieved by low-diameter decompositions. We claimed that this is possible when averaging over scales **and** that the random-shift algorithm already exhibits this property. This yields an $O(\log n)$ -approximation probabilistic tree embedding T .

Tree 'Embedding'. Until now, we have not really addressed why T is referred to as a tree embedding. However, it is not hard to see that each node in T is identified with a cluster center and thus with a vertex in G . We denote the map described above by $\Pi_{V(T) \rightarrow V}$.

Further, for any edge (u, v) in T which is of length $\text{dist}_G(u, v)$ where u and v are center vertices at different levels, we can find a path in G between the vertices u and v of the same length. We denote the map from edges $e = (u, v) \in E(T)$ to uv -paths in G of the same length by $\Pi_{E(T) \rightarrow 2^E}$.

10.3 Tightening the Random-Shift Analysis

In this section, we give a proof of (10.1) for the case when $\mathcal{C}_1, \mathcal{C}_2, \dots, \mathcal{C}_{L-1}$ are computed via the standard random-shift algorithm from Algorithm 18 where $\mathcal{C}_i$ is returned by a call with parameter 2^l .

Studying the Bad Case. Before starting the analysis, let us build some intuition for the bad case. Let us fix some vertex v . We would like to understand what causes the expected v -to-center distance at a certain scale 2^l to be large, i.e., of order $\Omega(2^l \cdot \log n)$.

Consider first the following example: let $G = (X \cup \{v\}, E)$ be a graph over $n+1$ vertices, being the star centered at v , with edge (v, x) for $x \in X$ of weight $2^l(\log n - 1)$. What is the expected distance of v to its cluster center?

We have n vertices at distance exactly $2^l(\log n - 1)$. Thus, if one of the vertices from X becomes the center, the distance to the center is $2^l(\log n - 1) = \Omega(2^l \log n)$, which is pretty bad (this should occur at most at constantly many scales).

Let us get an idea for the order of the largest head start among vertices in X . We have n such head start random variables $\delta_1, \delta_2, \dots, \delta_n$ that are i.i.d. from the exponential distribution with mean 2^l . Thus, we have $\delta^{(1)} = \max_i \delta_i$ for $d \geq 0$,

$$\Pr[\delta^{(1)} > d] = 1 - \prod_i \Pr[\delta_i \leq d] = 1 - (1 - e^{-d/2^l})^n \approx 1 - e^{-n \cdot e^{-d/2^l}}.$$

using $1 + x \approx e^x$ (for small x). Setting $d = 2^l \log n$, we obtain

$$\Pr[\delta^{(1)} > 2^l \log n] \approx 1 - e^{-n \cdot e^{-\log n}} = 1 - e^{-n/n} = 1 - e^{-1}$$

and thus, with constant probability, the maximum head start is greater $2^l \log n$.

One the other hand, the head start δ^v is less than 2^l with probability $1 - e^{-2^l/2^l} = 1 - e^{-1}$. This occurs again with constant probability.

Since δ^v is independent from $\delta^{(1)}$ as defined above, both events above occur at the same time with constant probability. Thus, some vertex from X wins v with constant probability. The expected vertex-to-center distance is therefore at least $\Pr[\text{some } x \text{ in } X \text{ wins}] \cdot 2^l(\log n - 1) = \Omega(2^l(\log n - 1))$.

Capturing Hardness of Scales. The above example suggests that the picked scale 2^l is very bad. But what can we expect from the other scales for the notorious graph above? Let's check for each scale 2^j where $i \neq l$.

- For $i < l$, observe that the probability of a vertex $x \in X$ to have head start δ^x at least $2^l(\log n - 1)$ and thus reaches v before finishing time 0, decreases to only $\approx e^{-2^l \log n / 2^i} \approx n^{-2^{l-i}} \leq n^{-2}$. Thus, a union bound yields that the event that any vertex other than v wins v in these races is negligible. The expected vertex-to-center distance is thus going to 0 as $i \rightarrow 0$.
- For $i > l + \log \log n$, we observe a different phenomenon: we have $2^i > 2^l(\log n - 1)$ and thus, every vertex $x \in X$ is a great choice for a cluster center. For each such vertex, we have $\text{dist}_G(x, v)/2^i = O(1)$, which is exactly what we hoped for.

We see above that while at a certain scale the cluster center might be far in expectation, on average, this might not be the case.

Let us do one more example to gain all the necessary intuition. Consider the star as above, and additionally, consider attaching to each leaf x , one more vertex x' via an edge (x, x') of weight $2^i(\log n - 1)$ for some $i > l + \log \log n$. Let us denote by Y the n additional vertices.

Our question is, does this make j a bad scale? It turns out that the maximum head start obtained by n i.i.d. variables is heavily concentrated around $2^i \log(n)$ (up to additive 2^i). Thus, the gap in the largest head start between X and Y is rather small since they are of equal size. But since the distances to overcome are much smaller for vertices in X , this yields that the winner is very likely in $X \cup \{v\}$.

Generally speaking, a set of vertices at a large distance can only contain the winner with good probability if it is massively larger. Quantitatively, the set must be larger by an exponential factor in the distance gap (divided by 2^i for normalization).

Quantifying Hardness of Scales. Let us now translate our intuition into formal definitions. We use a simple ball growing process over scales l to capture hardness of scale. We define the process by a sequence of integers r_l such that balls $B(v, r_l 2^l)$ are growing monotonically.

Let $r_0 := 1$ with $|B(v, r_0 2^0)| \geq 1$ and let $r_l := r_{l-1}/2 + \Delta_l + 2$, where we choose Δ_l ($l \geq 1$) as the largest integer such that growing the radius of the ball can be paid for by an exponential growth in the number of vertices in it, namely

$$\Delta_l := \max \xi \text{ satisfying } |B(v, (r_{l-1}/2 + \xi + 2)2^l)| \geq e^{\xi/2} \cdot |B(v, r_{l-1}2^{l-1})|. \quad (10.2)$$

Above, we essentially see $r_{l-1}/2$ as the inherited part scaled by 2, ensuring that $r_l 2^l$ is monotonically growing in l . The second part Δ_l is a parameter

that only grows as long as the ball is exponentially increasing with the next level. Thus, growth can be accounted by the order of vertices that are then contained in the ball. The final term is of purely technical nature.

The remainder of this section prove the following two lemmas. The first lemma says that the distance to the center is well captured by $r_l 2^l$ on level l (in expectation). The second yields that the sum of r_l 's is small, and thus, since r_l can be thought of as the loss-per-level, we have only constant average loss over all scales. Combined, these lemmas yield Property (10.1) as a corollary.

Lemma 10.2 (Distances-to-Center). *For any $0 \leq l \leq L$, $\mathbb{E}[\text{dist}(v, \text{CENTER}_l(v))] = O(r_l \cdot 2^l)$.*

Lemma 10.3 (Sum-of-Normalized-Radii). $\sum_{l=0}^L r_l \in O(\log n)$.

Bounding the Sum of Normalized Radii (Proof of Theorem 10.3).

From the definition of Δ_l in (10.2), we have $|B(v, r_L D_L)| \geq e^{\Delta_L/2} \cdot |B(v, r_{L-1} D_{L-1})| \geq e^{\Delta_L/2} \cdot e^{\Delta_{L-1}/2} \dots |B(v, r_0 D_0)|$. But since there are at most n vertices, we have $n \geq |B(v, r_L D_L)| \geq |B(v, r_0 D_0)| \cdot \prod_{l=1}^L e^{\Delta_l/2} \geq e^{\sum_{l=1}^L \Delta_l/2}$. Thus, $\sum_{l=1}^L \Delta_l \leq 2 \ln n$. We thus get

$$\begin{aligned}
 \sum_{l=0}^L r_l &= \sum_{l=0}^L \left(r_0/2^l + \sum_{i=1}^l (\Delta_i + 2)/2^{l-i} \right) && \text{By definition of } r_l \\
 &= \sum_{l=0}^L r_0/2^l + \sum_{i=1}^L \sum_{l=i}^L (\Delta_i + 2)/2^{l-i} && \text{Reordering the summation} \\
 &\leq \sum_{l=0}^{\infty} r_0/2^l + \sum_{i=1}^L \sum_{l'=0}^{\infty} (\Delta_i + 2)/2^{l'} && \text{Using } l' = l - i \\
 &= O(\log n). && (10.3)
 \end{aligned}$$

Bounding the Vertex-to-Center Distances (Proof of Theorem 10.2).

Let us fix any level $0 < l < L$ (for $l \in \{0, L\}$ the proof is trivial). We let the variables δ^u and C^u refer to the values that these variables take in Algorithm 18 when computing $\mathcal{C}_l$.

For every $i \geq 0$, we let $B_i = B(v, (r_l + i)2^l)$ and define $I_i = B_{i+2} \setminus B_{i+1}$. We can now re-write the expectation as

$$\begin{aligned}
 \mathbb{E}[\text{dist}(v, \text{CENTER}_l(v))] &= \sum_{w \in V} \mathbb{P}[v \in C^w] \cdot \text{dist}(w, v) \\
 &\leq O(r_l \cdot 2^l) + \sum_{i \geq 1} \sum_{w \in I_i} \mathbb{P}[v \in C^w] \cdot (i+1) \cdot 2^l.
 \end{aligned}$$

It remains to bound the probabilities that a vertex in I_i wins v . Note that a vertex $w \in I_i$ cannot win v if for some $u \in B_0$, $\delta^u + i \cdot 2^l > \delta^w$, since then u finishes earlier than w in the horse race. We obtain for every $w \in I_i$

$$\begin{aligned}
\mathbb{P}[w \text{ wins } v] &\leq \mathbb{P}[\max_{u \in B_0} \delta^u < \delta^w - i \cdot 2^l] \\
&= \int_{x=i \cdot 2^l}^{\infty} \frac{1}{2^l} e^{-x/2^l} \cdot \mathbb{P}[\max_{u \in B_0} \delta^u + i \cdot 2^l < x] dx \\
&= \int_{x=i \cdot 2^l}^{\infty} \frac{1}{2^l} e^{-x/2^l} \cdot (1 - e^{-x/2^l + i})^{|B_0|} dx \\
&= \int_{x=0}^{\infty} \frac{1}{2^l} e^{-x/2^l - i} \cdot (1 - e^{-x/2^l})^{|B_0|} dx \\
&= e^{-i} \int_{x=0}^{\infty} \frac{1}{2^l} e^{-x/2^l} \cdot (1 - e^{-x/2^l})^{|B_0|} dx \\
&\leq e^{-i} \int_{x=0}^{\infty} \frac{1}{2^l} e^{-x/2^l} \cdot e^{-|B_0| \cdot e^{-x/2^l}} dx \\
&= e^{-i} \cdot \left[\frac{e^{-|B_0| \cdot e^{-x/2^l}}}{|B_0|} \right]_{x=0}^{\infty} \\
&= e^{-i} \cdot \left(\frac{1}{|B_0|} - \frac{e^{-|B_0|}}{|B_0|} \right) \\
&< \frac{e^{-i}}{|B_0|}.
\end{aligned} \tag{10.4}$$

where in the first equality, we compute the probability by integrating over all values x that δ^w can take. At each such value x , we take the density function $\frac{1}{2^l} e^{-x/2^l}$ of the exponential distribution times the probability that the maximum value δ^u over all $u \in B_0$ is smaller x . Note that we start integrating only over $x \geq i \cdot 2^l$ as for smaller values, δ^w cannot exceed $\max_u \delta^u + i \cdot 2^l$ since $\delta^u \geq 0$.

In the second inequality, we use the probability that the maximum value over all variables δ^u is at most x is given by the cumulative density function at value x to the power $|B_0|$.

We substitute the value of x in the third equality to simplify the expression, and pull out the factor e^{-i} from the integral in the forth. We then use $1 + x \leq e^x$ in the second inequality, and finally evaluate the integral.

Note that this yields exactly what we expected after investigating the bad example carefully: the size of I_i must exceed the size of B_0 by a factor exponentially scaling in the gap in distance $\Theta(i \cdot 2^i)$ to contain a winner with good probability. By the definition of r_l , we know that this is, however, not

the case: we have $|I_i| < e^{(i+2)/2}|B_0|$. Thus, we have that follows from the growth process and which yields

$$\begin{aligned}
\mathbb{E}[\text{dist}(v, \text{CENTER}_l(v))] &= O(r_l \cdot 2^l) + \sum_{i \geq 1} \sum_{w \in I_i} \mathbb{P}[v \in C^w] \cdot (i+1) \cdot 2^l \\
&\leq O(r_l \cdot 2^l) + \sum_{i \geq 1} \sum_{w \in I_i} \frac{e^{-i}}{|B_0|} \cdot (i+1) \cdot 2^l \\
&= O(r_l \cdot 2^l) + \sum_{i \geq 1} |I_i| \cdot \frac{e^{-i}}{|B_0|} \cdot (i+1) \cdot 2^l \\
&\leq O(r_l \cdot 2^l) + \sum_{i \geq 1} e^{(i+2)/2}|B_0| \cdot \frac{e^{-i}}{|B_0|} \cdot (i+1) \cdot 2^l \\
&\leq O(r_l \cdot 2^l) + e \cdot \sum_{i \geq 1} e^{-i/2} \cdot (i+1) \cdot 2^l \\
&= O(r_l \cdot 2^l)
\end{aligned} \tag{10.5}$$

by standard calculations and using that $I_i \subseteq B_{i+2}$ and that the last sum is geometric.

Chapter 11

Cut-Preserving Trees via the Multiplicative Weights Update Method

In the last lecture, we saw that we can preserve the distance metric of a graph G in expectation by sampling a single tree T . Concretely, for two vertices s and t , the distance in the tree T is at most $O(\log n) \cdot \text{dist}_G(s, t)$ in expectation.

In this lecture, we explore a similar result for the cuts of a graph $G = (V, E, c)$ where we refer to c as the capacity function. Intuitively, capacities can model the strength of connections, and small cuts indicate likely points of failure or points of narrow throughput.

We construct a *distribution* $\mathcal{T}$ over trees with the following properties:

- **Non-contraction:** For every tree $T \in \mathcal{T}$ and every cut $\emptyset \subsetneq S \subsetneq V$,

$$c_G(E_G(S, V \setminus S)) \leq c_T(E_T(S, V \setminus S)),$$

- **Expected congestion bound:** For every $\emptyset \subsetneq S \subsetneq V$,

$$\mathbb{E}[c_T(E_T(S, V \setminus S))] \leq \alpha \cdot c_G(E_G(S, V \setminus S)),$$

In a breakthrough result, Räcke showed in 2003 that such a distribution $\mathcal{T}$ can be constructed efficiently with $\alpha = O(\log n)$. Again, this congestion bound is tight, i.e., matched up to constant factors by a lower bound.

Again, the problem is well-motivated since the tree structure admits efficient algorithms to various hard questions. In the next two lectures, we study various applications.

The key focus of this lecture is on a much more general tool: the multiplicative weights update (MWU) method. This method is a powerful tool to solve various optimization problems and has applications from machine learning (the AdaBoost algorithm is inspired by it), learning theory, finance, game theory, Linear programming solvers, to graph algorithms. A comprehensive survey on MWU and its applications can be found in [AHK12].

11.1 The Multiplicative Weights Update method

In this section, we introduce the multiplicative weights update (MWU) method. Before we show how this method can be applied to solve linear programs which will yield cut-preserving trees, we study this method for a much simpler problem: the **learning from experts problem**.

In this problem, every day $t = 1, \dots, T$, our algorithm has to predict the answer to a 'Yes'/'No' question. At the end of the day, the correct answer is revealed, and we learn whether our prediction was correct.

Suppose the algorithm has access to n experts $e_1, \dots, e_n$, each of which makes a prediction for the answer to take each day.

Can we use these predictions to optimize our answers? We will measure our performance relative to the **best** expert. More precisely, we will try to minimize the regret

$$\text{reg}(T) = \# \text{mistakes made by our alg} - \# \text{mistakes made by best expert}$$

Strategy in Presence of a Perfect Expert. Consider first the case where some expert i is guaranteed to make **no** mistake. Let us use the following algorithm:

- On each day t , sample an expert i uniformly at random among the experts that have not made a mistake so far.
- Output the prediction of i as your answer.

We prove the following strong result.

Theorem 11.1. *If there is a perfect expert, then $\mathbb{E}[\text{reg}(T)] \leq \log n$.*

Proof. For the analysis, denote by E_t the set of experts that have made no mistake on days $1, 2, \dots, t-1$. Clearly, E_t is shrinking monotonically as t is increasing. The probability that our algorithm answers correctly on day t is

$$\frac{1}{|E_t|} \sum_{i \in E_t} \mathbf{1}[\text{expert } i \text{ predicts day } t \text{ correctly}] = \frac{1}{|E_t|} \cdot |E_{t+1}| = \frac{|E_{t+1}|}{|E_t|}.$$

Thus, the probability that our algorithm makes a mistake is only $1 - |E_{t+1}|/|E_t|$. Intuitively, to make this probability a small constant < 1 , a constant fraction of the experts has to make a wrong prediction. But the sets E_t can only shrink by a constant factor $O(\log n)$ times, and so, we expect a regret of $O(\log n)$.

The main trick in this analysis is to exponentiate the quantities of interest. This yields,

$$\begin{aligned}
 \mathbb{E}[\text{\#mistakes made by our alg}] &= \sum_t \mathbb{E}[\text{\#mistakes made by our alg on day } t] \\
 &= \sum_t \Pr[\text{our alg makes a mistake on day } t] \\
 &= \sum_t (1 - |E_{t+1}|/|E_t|) \\
 &\leq \sum_t \log(|E_t|/|E_{t+1}|) \\
 &= \log\left(\prod_t |E_t|/|E_{t+1}|\right) \\
 &= \log(|E_1|/|E_{T+1}|) \\
 &\leq \log n
 \end{aligned}$$

where we use linearity of expectation and then in the first inequality that $1 - \frac{1}{x} \leq \log(x)$ for $x > 0$,¹ the fact that $\log(a) + \log(b) = \log(a \cdot b)$, and finally using that E_{T+1} contains a least one expert. $\square$

The MWU Algorithm. Let us now consider a generalization of the above algorithm with some parameter $\epsilon \in (0, 1/2)$

- On each day t , sample an expert i with probability

$$p_t(i) \propto e^{\epsilon \cdot \text{\#correct predictions by expert } i \text{ up to day } t}.$$

- Output the prediction of i as the answer.

To reason a bit more about this algorithm, let us introduce some additional notation. We define the gain function

$$g_t(i) = \begin{cases} 1 & \text{if the prediction of expert } i \text{ for day } t \text{ is correct} \\ 0 & \text{otherwise} \end{cases}$$

¹This inequality can be derive from the more familiar inequality $1 + x \leq e^x$ for all x since $1 + x \leq e^x \iff x \leq e^{x-1} \iff \log(x) \leq x - 1 \iff \log(1/y) \leq 1/y - 1 \iff \log(y) \geq 1 - 1/y$ since $\log(1/y) = -\log(y)$.

and define the weight associated with expert i at time t by

$$w_t(i) = e^{\epsilon \sum_{t' < t} g_{t'}(i)} = \prod_{t' < t} e^{\epsilon \cdot g_{t'}(i)}$$

Each expert is sampled proportional to this weight $p_t(i) \propto w_t(i)$. We can now also see the reason for the name of the multiplicative weights update method: for expert i we update the weight $w_t(i)$ to obtain $w_{t+1}(i)$ as follows:

- **If expert i made a correct prediction on day t ($g_t(i) = 1$):** we obtain $w_{t+1}(i)$ by multiplying $w_t(i)$ by $e^\epsilon \approx (1 + \epsilon)$, i.e. $w_{t+1}(i) = w_t(i) \cdot e^\epsilon$.
- **If expert i made a mistake on day t ($g_t(i) = 0$):** we let $w_{t+1}(i) = w_t(i)$.

Given our new notation, let us write down the MWU algorithm more carefully:

- For each expert $1 \leq i \leq n$, initialize $w_1(i) = 1$.
- For each time $t = 1, 2, \dots, T$
 - For each expert $1 \leq i \leq n$, let the probability of being picked be $p_t(i) = w_t(i) / \Phi_t$, where $\Phi_t = \sum_i w_t(i)$.
 - Sample a single expert according to the probabilities $p_t(i)$. Output the prediction of the sampled expert i as the answer for day t .
 - For each expert $1 \leq i \leq n$, update $w_{t+1}(i) = w_t(i) \cdot e^{\epsilon \cdot g_t(i)}$.

Note that the probability that our algorithm answers correctly on day t is $\sum_i p_t(i) \cdot g_t(i) = \langle p_t, g_t \rangle$.

The key insight of MWU is exponential reinforcement. By weighting experts exponentially, small differences in the number of mistakes are amplified causing the algorithm to quickly focus on the best experts. We prove the following statement.

Theorem 11.2. *The MWU algorithm obtains expected regret*

$$\mathbb{E}[\text{reg}(T)] \leq \epsilon \cdot T + \log n / \epsilon.$$

Proof. The idea behind this proof is to show that if the probability of our algorithm answering correctly is small on day t , then the weighted sum Φ_{t+1} is not increasing much over Φ_t . Since $w_t(i) \leq \Phi_t$ for every expert i at all times t , we thus have that if Φ_t is not increasing substantially over time, then neither is $w_t(i)$ for any expert i , and thus no expert is very reliable.

Analyzing Local Change. We have

$$\begin{aligned}
\Phi_{t+1} &= \sum_i w_{t+1}(i) \\
&= \sum_i w_t(i) \cdot e^{\epsilon g_t(i)} && \text{Definition of } w_{t+1}(i) \\
&\leq \sum_i w_t(i) \cdot (1 + \epsilon g_t(i) + (\epsilon \cdot g_t(i))^2) && \text{Using } e^x \leq 1 + x + x^2 \text{ for } |x| \leq 1 \\
&\leq \sum_i w_t(i) \cdot (1 + \epsilon g_t(i) + \epsilon^2) && \text{Since } x^2 \leq 1 \text{ for } |x| \leq 1 \\
&= (1 + \epsilon^2) \sum_i w_t(i) + \epsilon \sum_i w_t(i) g_t(i) && \text{Rearranging} \\
&= (1 + \epsilon^2) \Phi_t + \epsilon \Phi_t \sum_i p_t(i) g_t(i) && \text{Since } \Phi_t = \sum_i w_t(i) \text{ and } p_t(i) = w_t(i)/\Phi_t \\
&= (1 + \epsilon^2) \Phi_t + \epsilon \Phi_t \langle p_t, g_t \rangle && \text{Using vector notation} \\
&= (1 + \epsilon^2 + \epsilon \langle p_t, g_t \rangle) \Phi_t && \text{Rearranging} \\
&\leq \Phi_t \cdot e^{\epsilon^2 + \epsilon \langle p_t, g_t \rangle} && \text{Since } 1 + x \leq e^x
\end{aligned}$$

Regret for any Expert. Thus, for any expert i ,

$$\begin{aligned}
\epsilon \cdot \sum_{t=1}^T g_t(i) &= \log(w_{T+1}(i)) \\
&\leq \log(\Phi_{T+1}) \\
&\leq \log(\Phi_0 \cdot e^{\sum_t (\epsilon^2 + \epsilon \langle p_t, g_t \rangle)}) \\
&= \log(\Phi_0) + \log(e^{\sum_t \epsilon^2}) + \log(e^{\sum_t \epsilon \langle p_t, g_t \rangle}) \\
&\leq \log n + \sum_t \epsilon^2 + \sum_t \epsilon \langle p_t, g_t \rangle \\
&= \log n + \epsilon^2 \cdot T + \epsilon \cdot \mathbb{E}[\text{\#correct answers of alg}]
\end{aligned}$$

Regret for Best-Performing Expert. In particular, for the best-performing expert i , by rescaling by $1/\epsilon$, we have

$$\begin{aligned}
\mathbb{E}[\text{reg}(T)] &= \mathbb{E}[\text{\#mistakes made by alg}] - \text{\#mistakes made by best expert } i \\
&= \text{\#correct predictions by best expert } i - \mathbb{E}[\text{\#correct answers by alg}] \\
&\leq \log n / \epsilon + \epsilon \cdot T
\end{aligned}$$

as desired. □

Generalized Learning from Experts. Finally, consider that instead of each expert i giving a 'Yes'/'No' prediction, it can output the probability that the event occurs. In our current notation, this means that $g_t(i)$ is no longer a value in $\{0, 1\}$ but instead taken from $[0, 1]$. Implementation of the MWU algorithm remains unaffected. Further, the proof of [Theorem 11.2](#) already holds for this case. In fact, it even holds for any $g_t(i) \in [-1, 1]$.

By scaling, we can generalize to $g_t(i) \in [-\rho, \rho]$ for any integer ρ . We call ρ the *width*, and refer to $g_t(i)$ as *gain*. We get the following general theorem as a corollary.

Theorem 11.3. *For any sequence of T days with n experts and gain vectors $g_t(i) \in [-\rho, \rho]$ for some width ρ , the MWU algorithm obtains regret*

$$\text{reg}(T) \leq \epsilon \rho T + \rho \log n / \epsilon.$$

Corollary 11.4. *For some $T = \Theta(\rho^2 \log n / \epsilon^2)$, the MWU algorithm with appropriately chosen ϵ' obtains regret $\text{reg}(T) \leq \epsilon \cdot T$.*

Proof. Taking $\epsilon' = \epsilon / 2\rho$ yields by [Theorem 11.3](#) a regret of at most

$$\epsilon' \rho T + \rho \log n / \epsilon' \leq \frac{\epsilon}{2} T + 2\rho^2 \log n / \epsilon$$

and thus for $T \geq 4\rho^2 \log n / \epsilon^2$, we have that the regret is at most $\epsilon \cdot T$. $\square$

11.2 Solving Linear Programs via the MWU method

Next, we discuss how to approximately solve a large class of Linear Programs (LPs) called Packing LPs via the MWU method. Let us state the canonical Packing LP

$$\begin{aligned} & \text{maximize} && c^\top x \\ & \text{subject to} && Ax \leq b \\ & && x \geq 0 \end{aligned}$$

with $A \in \mathbb{R}_{\geq 0}^{n \times m}$, $c \in \mathbb{R}_{\geq 0}^m$, $b \in \mathbb{R}_{\geq 0}^n$.

We make two assumptions in this section: 1) we assume $b = \mathbf{1}$, and 2) that we know $K = c^\top x^*$. 1) can be assumed w.l.o.g. by rescaling A . 2) can be assumed w.l.o.g. because we can binary search over values of K .

Solving a Packing LP with Constraints Satisfied on Average. But how should we solve an LP using the MWU method? It seems like quite a jump!

The main idea is the following. Suppose you know how to find a 'solution' x to the above LP where constraints are enforced only *on average*. That is, suppose you can find a solution x such that

1. $c^\top x = K$, and
2. $\frac{1}{n} \mathbf{1}^\top Ax \leq \frac{1}{n} \mathbf{1}^\top \mathbf{1} \iff \frac{1}{n} \mathbf{1}^\top Ax \leq 1$.

Finding such a solution x is much, much simpler than straight-up solving the entire LP, as $\frac{1}{n} \mathbf{1}^\top Ax \leq 1$ is now a single constraint (instead of n constraints)! We know that a feasible solution exists since the optimal solution to the Packing LP is feasible. In fact, one can obtain an optimal solution simply by finding the index $i \in \arg \max_i \frac{\mathbf{1}^\top A \mathbf{1}_i}{c^\top \mathbf{1}_i}$ and setting $x = c^\top \mathbf{1}_i / K$.

From Average to Worst-Case via MWU. But how can we go from satisfying constraints *on average* to satisfying *every* constraint? We will take an iterative approach where we find for each time $t = 1, 2, \dots, T$ a solution $x^{(t)}$ enforcing the constraints on average as above, and then output their average $\bar{x} = \frac{1}{T} \sum_t x^{(t)}$. Now, if the first iterate $x^{(1)}$ is very off on the i -th constraint, then we can scale up the i -th row in future iterations, and so $x^{(2)}, \dots, x^{(T)}$ are more likely to enforce this constraint, which results in averaging out any constraint violations.

This is exactly what the MWU method can do for us: we think of each constraint i as being an expert, whose gain $g_t(i)$ is equal to how much the i^{th} constraint was violated by, i.e. $g_t(i) = \mathbf{1}_i^\top Ax^{(t)} - 1$.

Recall that the weight of expert i is $w_t(i) = e^{\epsilon \sum_{t' < t} g_{t'}(i)}$. We ask the t^{th} solution $x^{(t)}$ to satisfy

1. $c^\top x^{(t)} = K$, and
2. $w_t^\top Ax^{(t)} \leq w_t^\top \mathbf{1} \iff p_t^\top Ax^{(t)} \leq p_t^\top \mathbf{1} \iff p_t^\top Ax^{(t)} \leq 1$.

Since rescaling by $\Phi_t = w_t^\top \mathbf{1}$ yields a probability vector p_t , most people think of the above as finding a solution to a convex combination of the constraint set. We follow this convention.

A Formal Proof. Finally, we prove that a sufficient number of iterations T yield a good solution $\bar{x}$.

Theorem 11.5. *For width $\rho = \max\{1, \max_i \max_{x \text{ s.t. } c^\top x = K} |\mathbf{1}_i^\top (Ax - b)|\}$, if the above algorithm is run for $T = O(\rho^2 \log n / \epsilon^2)$ iterations, then $\bar{x}$ satisfies*

1. $c^\top \bar{x} = K$, and

2. $A\bar{x} \leq (1 + \epsilon)\mathbf{1}$.

Proof. The first property holds clearly since $\bar{x}$ is a convex combination of $x^1, x^2, \dots, x^T$, and each x^i satisfies this property.

For the second property, we have for each t that $p_t^\top Ax^{(t)} \leq 1$, and equivalently, $\langle p_t, g_t \rangle = \langle p_t, Ax^t - \mathbf{1} \rangle \leq 0$. Thus, $\sum_t \langle p_t, g_t \rangle \leq 0$.

On the other hand, for the i -th constraint, we have by our bound on the regret from [Theorem 11.4](#) that

$$\begin{aligned}
 & \sum_t g_t(i) - \sum_t \langle p_t, g_t \rangle \leq \epsilon \cdot T \\
 \implies & \sum_t g_t(i) \leq \epsilon \cdot T \\
 \iff & \frac{1}{T} \sum_t g_t(i) \leq \epsilon \\
 \iff & \frac{1}{T} \sum_t (\mathbf{1}_i^\top Ax^{(t)} - 1) \leq \epsilon \\
 \iff & \frac{1}{T} \sum_t \mathbf{1}_i^\top Ax^{(t)} \leq 1 + \epsilon \\
 \iff & \mathbf{1}_i^\top A\bar{x} \leq 1 + \epsilon
 \end{aligned}$$

as desired. □

Remark We note that setting the width $\rho = \max\{1, \max_i \max_{x \text{ s.t. } c^\top x = K} |\mathbf{1}_i^\top (Ax - b)|\}$ is quite a crude upper bound. Often, in applications, we only consider steps $x^{(t)}$ from some set X of possible solutions and achieve width $\rho = \max\{1, \max_i \max_{x \in X \text{ s.t. } c^\top x = K} |\mathbf{1}_i^\top (Ax - b)|\}$. This works as long as the set X contains solutions with respect to any probability distribution.

11.3 Application: Cut-Preserving Trees

We learned in the last section that we can solve packing LPs via the MWU method efficiently if we can solve it to satisfy constraints on average. We now show how to compute a distribution over cut-preserving trees by utilizing this approach.

Formulating a Packing LP. Let $G = (V, E, c : E \mapsto [1, C])$ be the undirected input graph. Recall that an α -approximate tree embedding is a tree $T = (V', E')$ such that for all vertices $u, v \in V$, $\text{dist}_G(u, v) \leq \text{dist}_T(u, v)$ and $\mathbb{E}[\text{dist}_T(u, v)] \leq \alpha \cdot \text{dist}_G(u, v)$. For simplicity, we assume in this section that we can sample such tree embeddings with the additional property that each tree is a subgraph of G , i.e. $T \subseteq G$.²

We associate with any tree embedding T_i , a capacity function c_i . To define this function, let us call for any edge $e' \in T_i$ the cut formed by the two connected components in $T_i \setminus e'$ the *fundamental cut* of e' , denoted by $\delta_i(e')$. Then, we define $c_i(e') = \sum_{e \in \delta_i(e')} c(e)$. We define the congestion of T_i for any edge $e \in E$ to be the sum of capacities of the edges $e' = (u, v)$ whose uv -path in T_i contains e , i.e.

$$\text{cong}_{T_i}(e) = \frac{1}{c(e)} \cdot \sum_{\substack{e'=(u,v) \in E \\ e \in T_i[u,v]}} c(e').$$

With these two important definitions in place, we can set up the matrix that we use in our packing LP.³ We define $\mathbf{A} \in \mathbb{R}_{\geq 0}^{E \times \mathcal{T}}$ as

$$\mathbf{A} := \begin{bmatrix} \text{cong}_{T_1}(e_1) & \text{cong}_{T_2}(e_1) & \dots & \text{cong}_{T_k}(e_1) \\ \text{cong}_{T_1}(e_2) & \text{cong}_{T_2}(e_2) & \dots & \text{cong}_{T_k}(e_2) \\ \dots & \dots & \dots & \dots \\ \text{cong}_{T_1}(e_m) & \text{cong}_{T_2}(e_m) & \dots & \text{cong}_{T_k}(e_m) \end{bmatrix}$$

This allows us to model our problem as the packing LP

$$\begin{aligned} & \text{maximize} && \mathbf{1}^\top x \\ & \text{subject to} && Ax \leq \alpha \cdot \mathbf{1} \\ & && x \geq 0 \end{aligned} \tag{11.1}$$

We show that for some $\alpha = O(\log n)$, we can obtain an objective of at least 1. We will assume in fact, that the objective is exactly 1 (possibly by slightly scaling it down again). This allows us to understand x as a probability distribution where the i -th tree T_i is chosen with probability $x(i)$. The constraint for edge e enforces that for distribution over $\mathcal{T}$ induced by x , we have $\mathbb{E}_{T_i \sim \mathcal{T}}[\text{cong}_{T_i}(e)] \leq \alpha$. It is not hard to verify that $\mathcal{T}$ preserves all cuts up to factor α .

²We have seen tight tree embeddings that achieve $\alpha = O(\log n)$. If we require $T \subseteq G$, the best constructions achieve $\alpha = O(\log n \log \log n)$. Techniques in this section seamlessly extend to tree embeddings, however, and do not require $T \subseteq G$. We solely assume $T \subseteq G$ to avoid clutter.

³We can restrict to tree embeddings consisting of $O(n \log n)$ nodes, which ensures that $\mathcal{T}$ is finite. While the number of columns of this matrix is still exponential, we will not need to work with this matrix explicitly.

Lemma 11.6. *A solution x to (11.1) with $\mathbf{1}^\top x = 1$ induces a distribution over trees that approximates all cuts in G up to a factor of α .*

Proof. The Lemma follows from the following two properties:

- **Non-contraction:** For any cut $(S, V \setminus S)$, any edge $e = (x, y) \in E_G(S, V \setminus S)$ (say $x \in S, y \in V \setminus S$), we have in any tree T_i , that at least one edge $e' = (u, v) \in T_i[x, y]$ ($T_i[x, y]$ being the x -to- y path in T_i) has $u \in S$ and $v \in V \setminus S$. Since $e = (x, y)$ contributes $c(e)$ to the capacity of every edge on $T_i[x, y]$, it also contributes $c(e)$ to the capacity of edge e' and thus to the capacity of the cut $(S, V \setminus S)$ in T_i . Thus, $c_i(E_{T_i}(S, V \setminus S)) \geq c(E_G(S, V \setminus S))$.

Since this holds for any tree T_i , it also holds for any convex combination of trees from $\mathcal{T}$.

- **Expected congestion bound:** Consider a cut $\emptyset \subsetneq S \subsetneq V$. Let T_i be the tree sampled from $\mathcal{T}$. A similar argument as above yields that each edge e' in T_i that crosses the cut $(S, V \setminus S)$, contributes $c_i(e')/c(e)$ to the congestion of some edge $e \in E_G(S, V \setminus S)$.

Using that the expected congestion of each edge $e \in E$ is α , this yields an upper bound on the capacity of edges in T_i crossing the cut of

$$\begin{aligned} \mathbb{E}_{T_i \sim \mathcal{T}}[c_i(E_{T_i}(S, V \setminus S))] &\leq \mathbb{E}_{T_i \sim \mathcal{T}} \left[\sum_{e \in E_G(S, V \setminus S)} \text{cong}_{T_i}(e) \cdot c(e) \right] \\ &\leq \sum_{e \in E_G(S, V \setminus S)} \mathbb{E}_{T_i \sim \mathcal{T}}[\text{cong}_{T_i}(e)] \cdot c(e) \\ &\leq \sum_{e \in E_G(S, V \setminus S)} \alpha \cdot c(e) \\ &= \alpha \cdot c(E_G(S, V \setminus S)) \end{aligned}$$

using linearity of expectation, our bound on the expected congestion and finally the definition of the capacity of a cut.

□

Implementing the MWU. Recall that the MWU method asks to solve the problem

$$p^\top \mathbf{A}x \leq \alpha, x^\top \mathbf{1} = 1$$

at each step for p being some probability vector, i.e. $p^\top \mathbf{1} = 1$. This boils down to finding a tree embedding T that minimizes the congestion on the

edges with importance weighted by the probability vector, i.e. to minimize $\sum_{e \in E} p(e) \cdot \text{cong}_T(e)$. To this end, we exploit a duality between cuts and distances represented by trees.

Lemma 11.7 (Cut / Distance Duality). *Let $G = (V, E)$ be a graph with capacities $c : E \rightarrow \mathbb{R}_{>0}$, and let p be a probability vector over edges. Define a weight function on G by $w(e) = p(e)/c(e)$. Let $T \subseteq G$ be a tree embedding of G , and let c_T be the capacity function on T defined via fundamental cuts:*

$$c_T(e') = \sum_{e \in \delta_T(e')} c(e) \quad \text{for each } e' \in T.$$

Then

$$\sum_{e \in E} p(e) \cdot \text{cong}_T(e) \leq \sum_{e=(u,v) \in E} p(e) \cdot \frac{\text{dist}_T(u, v)}{\text{dist}_G(u, v)},$$

where $\text{dist}_G(\cdot, \cdot)$ and $\text{dist}_T(\cdot, \cdot)$ are distances measured with edge-lengths $w(\cdot)$.

Proof. By definition of congestion,

$$\begin{aligned} \sum_{e \in E} p(e) \text{cong}_T(e) &= \sum_{e \in E} \frac{p(e)}{c(e)} \sum_{\substack{e'=(u,v) \in E \\ e \in T[u,v]}} c(e') \\ &= \sum_{e \in E} w(e) \sum_{\substack{e'=(u,v) \in E \\ e \in T[u,v]}} c(e') \\ &= \sum_{e'=(u,v) \in E} c(e') \sum_{e \in T[u,v]} w(e) \\ &= \sum_{e'=(u,v) \in E} c(e') \text{dist}_T(u, v). \end{aligned}$$

Now we use the key inequality: for each original edge $f \in E$,

$$\text{dist}_G(f) \leq w(f) = \frac{p(f)}{c(f)} \implies c(f) \leq \frac{p(f)}{\text{dist}_G(f)}.$$

Plugging this into the previous sum:

$$\sum_{e \in E} p(e) \text{cong}_T(e) \leq \sum_{e'=(u,v) \in E} p(e') \frac{\text{dist}_T(u, v)}{\text{dist}_G(u, v)},$$

which proves the lemma. $\square$

Given [Theorem 11.7](#), we define weights $w(e) = p(e)/c(e)$ and find a β -approximate distance-preserving tree embedding T of G . Such a tree embedding ensures that for each $e = (u, v) \in E$,

$$\mathbb{E}[dist_T(u, v)] \leq \beta \cdot dist_G(u, v) \iff \mathbb{E}\left[\frac{dist_T(u, v)}{dist_G(u, v)}\right] \leq \beta.$$

which implies

$$\mathbb{E}\left[\sum_{e \in E} p(e) \cdot cong_T(e)\right] \leq \sum_{e \in E} p(e) \cdot \beta = \beta \cdot \sum_{e \in E} p(e) = \beta.$$

Recall that in [Chapter 10](#), we gave an algorithm to obtain such tree embeddings with $\beta = O(\log n)$. Since we expect a solution of value at most β , we have with probability $1/2$, that a random distance-preserving tree embedding has yields a weighted congestion sum of at most 2β . We can set $\alpha = 2\beta$ and then sample repeatedly until a tree satisfies the constraint for our choice of α . The expected value of samples that we require is at most 2. Thus, each iteration of the MWU method can be implemented in $\tilde{O}(m)$ time.

Bounding the Number of Iterations. It remains to analyze the *width* ρ . By [Section 11.2](#), it suffices to give an upper bound on $\left|\frac{cong_T(e)}{\alpha} - 1\right|$ for all trees T that we consider in steps and all edges e (since we scale such that $b = \alpha \cdot \mathbf{1}$).

Since we have

$$\begin{aligned} \mathbb{E}[cong_T(e)] &= \frac{1}{c_G(e)} \sum_{e'=\{u,v\} \in E_G} c_G(e') \cdot \mathbb{P}(\text{the } (u, v)\text{-path in } T \text{ uses } e) \\ &= \frac{1}{p(e)} \sum_{e'=\{u,v\} \in E_G} p(e') \cdot \frac{len(e)}{len(e')} \cdot \mathbb{P}(\text{the } (u, v)\text{-path in } T \text{ uses } e) \\ &\leq \frac{1}{p(e)} \sum_{e'=\{u,v\} \in E_G} p(e') \cdot \frac{\mathbb{E}[dist_T(u, v)]}{len(e')} \\ &\leq \frac{\alpha}{p(e)}, \end{aligned}$$

we can apply Markov's inequality, and reject sampled trees T' if $cong_{T'}(e) > 4|E| \cdot \mathbb{E}[cong_T(e)]$ for any edge e . While we cannot lower bound $p(e)$, we can instead work with the probability vector $p'(e) := \frac{1}{2}p(e) + \frac{1}{2|E|}\mathbf{1}$. Note that if $(p')^\top Ax \leq \alpha$, then $p^\top Ax \leq 2\alpha$. Thus, we can obtain width $\rho = O(m)$ while retaining $\alpha = O(\log n)$.

Chapter 12

Graph Sparsification: Preserving Distances and Cuts

In this lecture, we ask how much we can compress or sparsify a graph while retaining important properties. More formally, we study algorithms that take any n -vertex graph $G = (V, E, w)$ and then identify a subgraph $H = (V, E', w')$ such that $|E'| \lll |E|$, while a property of interest is approximately preserved (note that we allow the weight functions w and w' to differ on the same edge).

In line with current topics, we again study such algorithms for the two main properties considered in the course, distances and cuts:

1. We first study the problem of preserving distances, i.e. we want that for every $u, v \in V$, $d_G(u, v) \approx d_H(u, v)$,
2. Then, we study the problem of preserving cuts, i.e. for every $S \subseteq V$, we want that $|E_G(S, V \setminus S)| \approx |E_H(S, V \setminus S)|$.

The motivation for graph sparsification is rather obvious: if we want to compute a set of (approximate) distances, it is faster to apply the algorithm to H than to G . Thus, if we can efficiently find H given G , this can yield huge speed-ups.

In modern graph algorithms, graph sparsification is an extremely important tool. While we focus on sparsifying the edge set, i.e. finding $E' \subseteq E$ that is significantly smaller, another heavily studied notion of sparsification is vertex sparsification where the goal is to find $F = (T, E'')$ where $T \subseteq V$ is a given terminal set and then properties (like distances or cuts) between terminals in G are approximated in F . Note that in the past chapters, we have already studied the extreme case where H is taken to only be a tree, i.e. the smallest possible graph that could reasonably preserve these graph properties.

12.1 Distance-Preserving Sparsifiers

We first give a family of algorithms that compute sparsifiers with an asymptotically tight trade-off between sparsity of the sparsifier graph and approximation in the distances. Sparsifiers that preserve distances are typically referred to as *spanners*.

Theorem 12.1. *Given any n -vertex graph $G = (V, E)$ and integer $k \geq 1$, there is a polynomial time algorithm that computes a graph $H \subseteq G$ with $O(n^{1+1/k})$ edges such that for every $u, v \in V$,*

$$d_G(u, v) \leq d_H(u, v) \leq (2k - 1)d_G(u, v).$$

This trade-off is asymptotically optimal under Erdos girth conjecture. The *girth* of a graph G is defined to be minimum number of edges on any cycle in G .

Conjecture 12.2. *[Erd64] For a fixed $k \geq 1$, there exists a family of graphs on n vertices with girth at least $2k + 1$ and $\Omega(n^{1+1/k})$ edges.*

The conjecture is confirmed for $k \in \{1, 2, 3, 5\}$ [Wen91, Woo06].

The lower bound is implied since for the above graphs, taking away just a single edge means that the endpoints of the deleted edge are at distance at least $2k$ and thus no edge can be removed.

The greedy algorithm. For the rest of this part of the lecture, we focus on obtaining Theorem 12.1. We give the classic greedy algorithm from [ADD⁺93] to compute the spanner H below.

Algorithm 19 COMPUTESPANNER(G, k)

```

 $H \leftarrow (V, \emptyset)$ .
for each edge  $e = (u, v) \in E(G)$  in order monotonically increasing in
weights do
    if  $d_H(u, v) > (2k - 1)w(e)$  then
        Add  $e$  to  $H$ .
    end if
end for
return  $H$ 

```

Approximation. Let us first analyze the approximation. Note that for all $x, y \in V$, $d_H(x, y) \geq d_G(x, y)$ since $H \subseteq G$, and thus, every path (between

x and y in particular) from H , also appears in G . So certainly also the minimizer appears in G .

Note that for every edge $e = (u, v) \in E(G)$, we leave the edge only out if u and v are already at distance at most $(2k-1)w(e)$. Since we are thereafter only further adding edges to H , distance only further decrease and so this remains true.

From this we can prove the approximation factor as follows: for any $x, y \in V$, let $\pi_{x,y} = \langle x = v_1, v_2, \dots, v_k = y \rangle$ be the shortest path in G , then

$$\begin{aligned} d_H(x, y) &\leq \sum_{1 \leq i < k} d_H(v_i, v_{i+1}) \\ &\leq (2k-1) \sum_{1 \leq i < k} d_G(v_i, v_{i+1}) \\ &= (2k-1)d_G(x, y) \end{aligned}$$

where we use the triangle inequality in the first inequality, and then the above fact about preserving distances between endpoints of edges.

Sparsity. We establish the claimed sparsity in two steps: we first establish that H has large girth, and then secondly, that large girth implies sparsity.

Lemma 12.3. *Spanner $H \subseteq G$ as returned by [Algorithm 19](#) has girth larger $2k$.*

Proof. Assume H has a cycle C of length at most $2k$. Let $e = (u, v)$ be the last edge added to H , i.e. the edge on C of the highest weight. We have

$$d_{C \setminus \{e\}}(u, v) = \sum_{e' \in C \setminus \{e\}} w(e') \leq (|C| - 1) \cdot \max_{e' \in C} w(e') \leq (2k-1) \cdot w(e).$$

But clearly, $C \setminus \{e\}$ is contained in H at the point that e is considered. Thus, the distance between u and v at this point is at most $(2k-1)w(e)$ in H which violates the condition of the if-statement and thus e could not have been added to C , a contradiction. $\square$

Lemma 12.4. *Any graph $H = (V, E')$ with girth larger $2k$ consist of $O(n^{1+1/k})$ edges.*

Proof. Suppose, for a contradiction, that $|E'| > 2n^{1+1/k}$.

Let $H'' = (V'', E'')$ be the (unweighted) graph obtained by iteratively removing vertices with degree $\leq n^{1/k}$ from H . The graph H'' has the following properties:

1. $\forall v \in V'', \deg_{H''}(v) \geq n^{1/k}$: by construction, otherwise such vertex would have been removed.
2. $|E''| > n^{1+1/k}$: each vertex removed, removes at most $n^{1/k}$ edges, thus we have $|E''| \geq |E'| - n \cdot n^{1/k} \geq n^{1+1/k}$.
3. $\text{girth}(H'') \geq \text{girth}(H)$: Since H'' is a subgraph of H , we have that the girth of H'' is at least the girth of H (every cycle in H'' exists in H). Thus, the girth of H'' is larger $2k$.

Now, fix any vertex $v \in V''$. Let $H^i(v)$ be the (edge-induced) graph consisting of all edges in H'' that are on a path starting in v consisting of at most i edges. We let $N^i(v)$ be the i -neighborhood of v defined to be the vertex set of $H^i(v)$ (or alternatively, all vertices at distance at most i from v in H'').

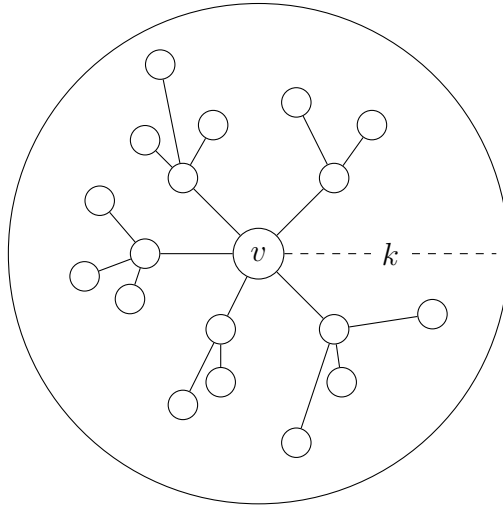
We now make the key insight! To emphasize it, we state it as a claim.

Claim 12.5. $H^k(v)$ forms a tree.

Proof. Let T be a BFS tree starting in v in $H^k(v)$. Clearly, every vertex is at distance at most k from v in this tree.

If $H^k(v)$ was not a tree, then there is an edge e that is not in T . Let π be the vx -path of length at most k containing e and ending in x that certifies that e has to be in $H^k(v)$. Since T is a tree, it also contains x at a distance at most k from v . Let $T[v, x]$ be the vx -path in T .

Since π and $T[v, x]$ are not equal since $e \in \pi$ but $e \notin T$ but consist of two vx -paths, there is a cycle in the graph $T[v, x] \cup \pi$. But this graph contains at most $|T[v, x]| + |\pi| \leq k + k = 2k$ edges, and thus there is a cycle (which also exists in $H^k(v)$) consisting of at most $2k$ edges – a contradiction. $\square$



We can then show that the i -neighborhoods are rapidly increasing in size as i increases.

Claim 12.6. $|N^{i+1}(v)| > n^{1/k} \cdot |N^i(v)|$ for all $0 \leq i < k$.

Proof. Note that every edge incident to a vertex in $N^i(v)$ for i is on a path of $\leq i + 1$ edges. Thus, $\forall u \in N^i(v)$, $\deg_{H^{i+1}(v)}(u) = \deg_{H''}(u) \geq n^{1/k}$.

Hence $|E(H^{i+1}(v))| \geq n^{1/k} \cdot |N^i(v)|$. But since $i + 1 \leq k$, we have that $H^{i+1}(v)$ forms a tree. Thus, it consists of exactly $|E(H^{i+1}(v))| + 1$ vertices. Together, we have $|N^{i+1}(v)| = |E(H^{i+1}(v))| + 1 > n^{1/k} \cdot |N^i(v)|$. $\square$

But by simple induction this implies $|N^k(v)| > (n^{1/k})^k = n$ and thus that the k -hop neighborhood of v is larger than the set of vertices in H'' , a contradiction! Hence, $|E'| \leq 2n^{1+1/k}$. $\square$

12.2 Cut-Preserving Sparsifiers

In the previous chapter, we looked at preserving distances via spanners. In this chapter, we look at preserving cut sizes. The main result that we want to attain is summarized below.

Theorem 12.7 (Cut Sparsifier). *Given an undirected graph $G = (V, E, w)$ and a parameter $0 < \epsilon < 1$, there is an algorithm that outputs a graph $H = (V, E', w')$ with $E' \subseteq E$ with $\tilde{O}(n/\epsilon^2)$ edges such that every cut $S \subseteq V$ is preserved up to a $(1 + \epsilon)$ factor, i.e.*

$$(1 - \epsilon)w(E_G(S, V \setminus S)) \leq w'(E_H(S, V \setminus S)) \leq (1 + \epsilon)w(E_G(S, V \setminus S)).$$

The algorithm succeeds w.h.p. and runs in polynomial time.

While this is one of the most useful results in modern graph algorithms, it is also quite surprising: while spanners preserve the $\binom{n}{2}$ distances in a graph, cut sparsifiers preserve each of the *exponentially* many cuts in the graph!

We achieve this result by following the following plan:

- first, we consider the rather trivial case where G is equal to the (un-weighted) complete graph K_n where computing a cut sparsifier H can be achieved by uniformly down-sampling G . On such a graph, it turns out, we can indeed get away with a union bound over all cuts.
- Then we introduce the notion of an *expander* graph, which is a graph family with essentially the same cut structure as K_n (K_n is an excellent expander). We conclude that we can down-sample by the minimum-degree of any expander, again using a union bound over all cuts.

- We then show each graph G can be partitioned edge-wise into expander graphs $G_1, G_2, \dots, G_k$ such that each vertex appears in at most $O(\log n)$ such graphs and each graph has average-degree close to minimum-degree (which is essential to ensure that each H_i has edges proportional to $|V(H_i)|$ and not to $|E(H_i)|$).
- And, finally, we show that for cut sparsifiers $H_1, H_2, \dots, H_k$ for any set of graphs $G_1, G_2, \dots, G_k$, we have that $H = \cup_i H_i$ is a cut sparsifier of $G = \cup_i G_i$.

For the rest of this lecture, we assume that G is a **simple, unweighted** graph to keep the presentation uncluttered, all techniques and algorithms extend seamlessly to weighted graphs. Note that H cannot be unweighted (otherwise it could not be sparse).

12.3 Warm up: Computing a cut sparsifier for $G = K_n$

Let G be the complete n -vertex graph. Consider the following procedure to construct H .

Algorithm 20 COMPUTECUTSPARSIFIERFORCOMPLETEGRAPH($G = K_n, \epsilon$)

$p \leftarrow 30 \cdot \frac{\log n}{\epsilon^2 n}$.

$H \leftarrow (V, \emptyset, w')$

for each edge $e = (u, v) \in E(G)$ **do**

Sample edge e i.i.d. into H with probability p with weight $1/p$.

end for

return H

The proof that this yields a cut sparsifier now follows from simply bounding the failure probabilities of all cuts.

Theorem 12.8. *Given any $\epsilon > 0$, **Algorithm 20** returns a graph H with expected number of edges $p \cdot \binom{n}{2} = \tilde{O}(n/\epsilon^2)$ that preserves all cuts in $G = K_n$ to a $(1 \pm \epsilon)$ -factor w.h.p.*

Proof. Consider any cut $S \subseteq V$ where $|S| \leq n/2$ and let $k = |S|$ (if $(S, V \setminus S)$ is preserved, then $(V \setminus S, S)$ is preserved, so this covers all cuts). By choice, we have $|E_G(S, V \setminus S)| \geq k \cdot n/2$.

Let us bound the probability of failure, i.e. cut $(S, V \setminus S)$ is not preserved. We let X_e be the indicator variable that edge $e \in$ is sampled into H , $X = \sum_{e \in E_G(S, V \setminus S)} X_e$. Since each edge is sampled with probability p that $\mathbb{E}[X] = p \cdot |E_G(S, V \setminus S)|$. Since edge in H has weight $1/p$, this yields an unbiased estimator for each cut when scaled by weight: if $X = p \cdot |E_G(S, V \setminus S)|$ then $w'(E_H(S, V \setminus S)) = |E_G(S, V \setminus S)|$. And thus $X \in (1 \pm \epsilon)p \cdot |E_G(S, V \setminus S)| = (1 \pm \epsilon)\mathbb{E}[X]$ iff the cut is preserved.

Using a simple Chernoff bound (and using that $|E_G(S, V \setminus S)| \geq k \cdot n/2$), we have

$$\begin{aligned} \mathbb{P}[|X - \mathbb{E}[X]| \geq \epsilon \cdot \mathbb{E}[X]] &\leq 2 \cdot \exp\left(\frac{-\epsilon^2 \mathbb{E}[X]}{3}\right) \\ &\leq 2 \cdot \exp\left(\frac{-\epsilon^2 p \cdot k \cdot n/2}{3}\right) \\ &= 2 \cdot \exp\left(\frac{-30 \cdot \log n \cdot k}{6}\right) \\ &= O(n^{-5k}). \end{aligned}$$

The crucial insight now is that as cuts increase in size (of the smaller side), the probability of failure goes down exponentially. The theorem then follows from a simple union bound since the number of cuts of size k is $\binom{n}{k} \leq n^k$, and therefore, we have

$$\begin{aligned} &\mathbb{P}[\cup_{S \subseteq V, |S| \leq n/2} w'(E_H(S, V \setminus S)) \notin (1 \pm \epsilon)|E_G(S, V \setminus S)|] \\ &\leq \sum_{S \subseteq V, |S| \leq n/2} \mathbb{P}[w'(E_H(S, V \setminus S)) \notin (1 \pm \epsilon)|E_G(S, V \setminus S)|] \\ &= \sum_{k=1}^{n/2} \sum_{S \subseteq V, |S|=k} \mathbb{P}[w'(E_H(S, V \setminus S)) \notin (1 \pm \epsilon)|E_G(S, V \setminus S)|] \\ &= O\left(\sum_{k=1}^{n/2} \sum_{S \subseteq V, |S|=k} n^{-5k}\right) \\ &= O\left(\sum_{k=1}^{n/2} \binom{n}{k} \cdot n^{-5k}\right) \\ &= O\left(\sum_{k=1}^{n/2} n^{-4k}\right) \\ &= O(n^{-4}). \end{aligned}$$

□

12.4 Cut Sparsifiers for Expander Graphs

Let us start by defining what an expander is. Intuitively, an expander is a graph that is well-connected. Let us first take a look at the formal definition. We let $\deg_G(v)$ denote the degree of v , and define $\deg(S) = \sum_{v \in S} \deg_G(v)$.

Definition 12.9 (Expander). *For any $0 < \phi \leq 1$ and m -edge graph $G = (V, E)$, we say a cut $(S, V \setminus S)$ is ϕ -sparse if*

$$|E_G(S, V \setminus S)| < \phi \cdot \min\{\deg_G(S), \deg_G(V \setminus S)\}.$$

G is a ϕ -expander iff it contains no ϕ -sparse cut.

Put differently, a ϕ -expander is a graph where for every set S on the smaller side, at least a ϕ -fraction of the edges incident to S are in the cut $(S, V \setminus S)$.

Note that the complete graph K_n is an excellent expander: the sets $S \subseteq V, |S| \leq n/2$, are also exactly the sets with $\deg_G(S) \leq m$, and each vertex in S sends at least $n/2$ edges over the cut $(S, V \setminus S)$ while being incident to $n - 1$ edges and thus, $|E_G(S, V \setminus S)| \geq k \cdot n/2 > \frac{1}{2} \cdot \deg_G(S)$. Thus, K_n is a $\frac{1}{2}$ -expander. Since ϕ is a ratio, this is almost the best one can achieve.

We can now generalize our algorithm to ϕ -expanders (where we will think of ϕ being $\tilde{\Omega}(1)$). However, note that when down-sampling K_n uniformly, we exploit that K_n is an r -regular graph. For general expanders, we can thus only down-sample proportional to the min-degree of the graph for the proofs to extend.

Algorithm 21 COMPUTECUTSPARSIFIERFOREXPANDER(G, ϕ, ϵ)

$\Delta \leftarrow \min_{v \in V} \deg_G(v).$

$p \leftarrow 30 \cdot \frac{\log n}{\epsilon^2 \Delta}.$

$H \leftarrow (V, \emptyset, w')$

for each edge $e = (u, v) \in E(G)$ **do**

Sample edge e independently into H with probability p and if sampled set $w'(e) = 1/p$.

end for

return H

Note that since we only down-sample proportional to the minimum degree Δ of G , the number of expected edges in H now scales in m/Δ and is therefore only close to n if the average degree in G is close to Δ .

Let us give the new proof. In red, we mark the changes necessary to the previous proof.

Theorem 12.10. *Given a ϕ -expander G and $\epsilon > 0$, **Algorithm 21** returns a graph H with expected number of edges $p \cdot m = \tilde{O}(m/(\Delta\epsilon^2))$ that preserves all cuts in G to a $(1 \pm \epsilon)$ -factor w.h.p.*

Proof. Consider any cut $S \subseteq V$ where $k = |S| \leq n/2$ and let $k = |S|$ (if $(S, V \setminus S)$ is preserved, then $(V \setminus S, S)$ is preserved, so this covers all cuts). Since G is ϕ -expander, we have $|E_G(S, V \setminus S)| \geq \phi \cdot k \cdot \Delta$.

Again, we define X_e and X as previously and recall that $X \in (1 \pm \epsilon)p \cdot |E_G(S, V \setminus S)| = (1 \pm \epsilon)\mathbb{E}[X]$ iff the cut is preserved. Using a Chernoff bound (and using that $|E_G(S, V \setminus S)| \geq \phi \cdot k \cdot \Delta$), we have

$$\begin{aligned} \mathbb{P}[|X - \mathbb{E}[X]| \geq \epsilon \cdot \mathbb{E}[X]] &\leq 2 \cdot \exp\left(\frac{-\epsilon^2 \mathbb{E}[X]}{3}\right) \\ &\leq 2 \cdot \exp\left(\frac{-\epsilon^2 p \cdot \phi \cdot k \cdot \Delta}{6}\right) \\ &= 2 \cdot \exp\left(\frac{-30 \cdot \log n \cdot k}{6}\right) \\ &= O(n^{-5k}). \end{aligned}$$

Via a union bound over all cuts, we can then again conclude that the probability that some cut is not preserved is $O(n^{-4})$. $\square$

12.5 Decomposing a graph into expanders

We now show how to decompose the graph G into $\tilde{\Omega}(1)$ -expanders.

Let us start by introducing the notion of an expander decomposition, one of the most essential tool in modern graph algorithms!

Definition 12.11. *Given a graph $G = (V, E)$, we say that a partition of V into $V_1, V_2, \dots, V_\ell$ forms a ϕ -expander decomposition if:*

1. *for every $1 \leq i \leq \ell$, the cluster $G[V_i]$ is a ϕ -expander, and*
2. *the number of edges not contained in any cluster, i.e. $E_{\text{rest}} = \cup_{i \neq j} E_G(V_i, V_j)$, is at most $\tilde{O}(\phi \cdot m)$.*

We want to prove the following theorem.

Theorem 12.12. *Given $G = (V, E)$ and some parameter $0 < \phi < 1$, there is a polynomial time algorithm that computes a ϕ -expander decomposition $V_1, V_2, \dots, V_k$ of G .*

We first prove existentially that such an expander decomposition exists for every graph. The algorithm to achieve this goal is rather straightforward: as long as there is a ϕ -sparse cut in a cluster of our partition, we decompose the cluster along such a cut.

Algorithm 22 COMPUTEEXPANDERDECOMPOSITION(G, ϕ)

```

 $\mathcal{V} = \{V\}$ .
while there is some  $V_i \in \mathcal{V}$  such that  $G[V_i]$  is not  $\phi$ -expander do
    Let  $(S, V_i \setminus S)$  be a  $\phi$ -sparse cut in  $G[V_i]$ .
    Replace cluster  $V_i$  by clusters  $S, V_i \setminus S$  in  $\mathcal{V}$ .
end while
return  $\mathcal{V}$ 

```

Clearly, each cluster $V_i \in \mathcal{V}$ as returned is a ϕ -expander as otherwise the while-loop would not have terminated. It remains to prove that the number of edge crossing clusters is bounded.

Lemma 12.13. *For any $G = (V, E)$ and parameter $0 < \phi < 1$, the algorithm [Algorithm 22](#) returns an expander decomposition $\mathcal{V} = \{V_1, V_2, \dots, V_k\}$ such that $E_{rest} = \cup_{i \neq j} E_G(V_i, V_j)$, is at most $\tilde{O}(\phi \cdot m)$.*

Proof. Whenever a new ϕ -sparse cut $(S, V_i \setminus S)$ is computed by the algorithm where w.l.o.g. $\deg_G(S) \leq \deg_G(V_i \setminus S)$, we add at most $\phi \cdot \deg_{G[V_i]}(S) \leq \phi \cdot \deg_G(S)$ edges to E_{rest} . Thus, we can charge each vertex $v \in S$ with $\phi \cdot \deg_G(v)$ to pay for the contribution to E_{rest} .

But note that every vertex in S is thereafter in a cluster S that has sum of degrees at most half as previously in V_i . Thus, each vertex can occur at most $\log_2(2m)$ times on the smaller side of such a cut along which we decompose.

Thus, E_{rest} is of size at most $\log_2(2m) \cdot \phi \cdot 2m = \tilde{O}(\phi \cdot m)$, as desired. $\square$

The proof of [Theorem 12.12](#), now almost follows directly from the fact that the algorithm has at most n while-loop iterations as each iteration refines the n -element partition $\mathcal{V}$.

But, unfortunately, computing a ϕ -sparse cut turns out to be NP-hard! But not to worry, it turns out that we can find a $O(\log n)$ -approximate sparse cut efficiently using cut-preserving tree embeddings and our proofs above extend seamlessly to also work with this approximate algorithm.

Lemma 12.14. *Given a graph $G = (V, E)$ and a parameter ϕ , there is a polynomial time algorithm such that if G is not ϕ -expander, it outputs an $O(\phi \log n)$ -sparse cut.*

Proof. Since G is not a ϕ -expander, there exists a nontrivial cut $(S, V \setminus S)$ with

$$|E_G(S, V \setminus S)| < \phi \cdot \min\{\deg_G(S), \deg_G(V \setminus S)\}.$$

We compute a cut-preserving tree distribution $\mathcal{T}$ (see [Chapter 11](#)) with congestion parameter $\alpha = O(\log n)$ in polynomial time. From the probabilistic method, we have that at least one tree $T \in \mathcal{T}$ satisfies

$$c_T(E_T(S, V \setminus S)) \leq \alpha \cdot |E_G(S, V \setminus S)|. \quad (12.1)$$

Fix such a tree T . We next prove that the fundamental cut induced by some edge $f \in E_T(S, V \setminus S)$, i.e. the cut between connected components in $T \setminus f$, is $O(\alpha\phi)$ -sparse in G . This yields an efficient algorithm, as we can verify in polynomial time per edge whether this holds true, and there are only a polynomial number of trees, each with $n - 1$ edges.

Let us now prove this claim that some fundamental cut is indeed an approximate sparse cut. Consider the connected components of $T \setminus E_T(S, V \setminus S)$. Note that these connected components partition the sets S and $V \setminus S$ since every edge in $E_T(S, V \setminus S)$ is removed. Let us focus on the connected components $C_1, C_2, \dots, C_k$ that partition S . We have that

$$\sum_{i=1}^k c_T(E_T(C_i, V \setminus C_i)) = c_T(E_T(S, V \setminus S)) \leq \alpha \cdot |E_G(S, V \setminus S)|$$

because each edge e in $E_T(S, V \setminus S)$ is incident to exactly one connected component C_i , the one that contains the endpoint that is also in S .

Next, we define $r \in V$ to be a centroid of T , i.e. such that each connected component in $T \setminus r$ is incident to at most half the degree-mass. For each edge $f \in E_T(S, V \setminus S)$, let C_f be the connected component in $T \setminus f$ that does not contain r . It remains to do a simple case distinction:

- **If any connected component $C_i \subseteq S$ contains the centroid r :** then, we have that the connected components C_f for $f \in E_T(C_i, V \setminus C_i)$ partition $V \setminus C_i$ and are each of size at most half the degree-mass. We

thus have

$$\begin{aligned}
\min_{f \in E_T(C_i, V \setminus C_i)} \frac{c_T(f)}{\deg_G(C_f)} &\leq \frac{\sum_{f \in E_T(C_i, V \setminus C_i)} c_T(f)}{\sum_{f \in E_T(C_i, V \setminus C_i)} \deg_G(C_f)} \\
&= \frac{c_T(E_T(C_i, V \setminus C_i))}{\deg_G(V \setminus S)} \\
&\leq \frac{c_T(E_T(S, V \setminus S))}{\deg_G(S)} \\
&\leq \alpha \cdot \frac{|E_G(S, V \setminus S)|}{\deg_G(S)} \\
&< \alpha \cdot \phi
\end{aligned}$$

using $\min_i a_i/b_i \leq (\sum_i a_i)/(\sum_i b_i)$ in the first inequality, the fact that the components C_f partition $V \setminus S$ in the second, [Equation \(12.1\)](#) in the third inequality along with the fact that $\deg_G(S) \leq \deg_G(V \setminus S)$ by definition, and finally that the cut S is ϕ -sparse.

It remains to use that $|E_G(C_f, V \setminus C_f)| \leq c_T(f)$ by the non-contractivity of the tree embedding $\mathcal{T}$ which yields that the fundamental cut of f is $\alpha \cdot \phi$ -sparse in G , as desired.

- **Otherwise:** since no C_i contains the centroid, we can find for each such component, a component C_f that contains C_i , i.e. $C_i \subseteq C_f$ with $f \in E_T(C_i, V \setminus C_i) \subseteq E_T(S, V \setminus S)$. Thus, $\sum_{f \in E_T(S, V \setminus S)} \deg_G(C_f) \geq \deg_G(S)$. We therefore obtain

$$\begin{aligned}
\min_{f \in E_T(S, V \setminus S)} \frac{c_T(f)}{\deg_G(C_f)} &\leq \frac{c_T(E_T(S, V \setminus S))}{\deg_G(S)} \\
&\leq \alpha \cdot \frac{|E_G(S, V \setminus S)|}{\deg_G(S)} \\
&< \alpha \cdot \phi
\end{aligned}$$

where we again use $\min_i a_i/b_i \leq (\sum_i a_i)/(\sum_i b_i)$ in the first inequality along with our insight that the clusters C_f cover S , and then appeal to [Equation \(12.1\)](#) and the sparsity of the cut $(S, V \setminus S)$, as in the previous case. Again, non-contractivity of the tree embedding yields the desired claim.

□

12.6 Decomposing the graph into almost vertex-disjoint expanders with average-degree close to minimum-degree

We can now prove our result for partitioning the graph G .

Theorem 12.15. *Given an m -edge graph $G = (V, E)$, there is a polynomial time algorithm that computes a set of $\tilde{\Omega}(1)$ -expanders $G_1, G_2, \dots, G_k$ such that w.h.p.:*

1. *the graphs G_i form an edge-wise partition of G , and*
2. *each expander G_i has minimum degree $\tilde{\Omega}(|E(G_i)|/|V(G_i)|)$, and*
3. *every vertex in V is contained in at most $O(\log n)$ graphs G_i .*

Building upon the notion of expander decompositions, we require only two small additional ideas:

- We apply the expander decomposition to G' which is the graph G where each vertex additionally has $\Delta = m/n$ self-loops. Note that G' for the resulting ϕ -expander decomposition $V_1, V_2, \dots, V_\ell$, we have that the number of edges crossing partition sets is again $\tilde{O}(\phi m)$ since G' is a $2m$ -edge graph. And the number of crossing edges is also equal to the number of crossing edges in G w.r.t. this partitioning as self-loops cannot contribute.

Further, we have that for every $v \in V_i$,

$$\begin{aligned} \deg_{G[V_i]}(v) &\geq |E_G(\{v\}, V_i \setminus \{v\})| = |E_{G'}(\{v\}, V_i \setminus \{v\})| \\ &\geq \phi \cdot \deg_{G'[V_i]}(v) \\ &\geq \phi \cdot \Delta. \end{aligned}$$

We further have that for every cluster $G[V_i]$, we have for every cut $(S, V_i \setminus S)$ that

$$\begin{aligned} |E_{G[V_i]}(S, V_i \setminus S)| &= |E_{G'}(S, V_i \setminus S)| \\ &\geq \phi \cdot \min\{\deg_{G'}(S), \deg_{G'}(V[V_i] \setminus S)\} \\ &\geq \phi \cdot \min\{\deg_G(S), \deg_G(V \setminus S)\} \end{aligned}$$

since self-loops cannot contribute to cuts, but they do contribute to degrees. Thus, each cluster $G[V_i]$ is a ϕ -expander.

Thus, $V_1, V_2, \dots, V_\ell$ forms a ϕ -expander decomposition where each vertex has minimum-degree $\tilde{\Omega}(m/n)$ in its cluster (unless the cluster is trivial).

- we choose ϕ to be such that the worst-case number of edges in E_{rest} becomes at most $m/2$. Note that for this goal, we can still find some $\phi = \tilde{\Omega}(1)$.

Then, we simply output the clusters $G[V_1], G[V_2], \dots, G[V_\ell]$ from the ϕ -expander decomposition as graphs $G_1, G_2, \dots, G_\ell$. Clearly, these graphs are vertex-disjoint.

But there is a set of edges E_{rest} of size at most $m/2$ that appear in no cluster. Therefore, we simply apply the algorithm recursively on the edge-induced graph $G[E_{rest}]$. Since across the recursive calls the number of edges in the input graphs decreases by factor 2 each time, the algorithm terminates after $O(\log n)$ recursive calls.

12.7 Putting it all together

It remains to combine our tools. We first decompose the graph G into $\tilde{\Omega}(1)$ -expanders $G_1, G_2, \dots, G_k$ via [Theorem 12.15](#). Then, we find a cut sparsifier H_i for each graph G_i using the algorithm in [Theorem 12.10](#). And finally, we output $H = \cup_i H_i$ as the cut sparsifier of G .

The expected number of edges in H is

$$\begin{aligned} \sum_i \frac{30 \log n}{\epsilon^2 \Delta(G_i)} \cdot |E(G_i)| &= \tilde{O} \left(\sum_i \frac{|V(G_i)|}{\epsilon^2 |E(G_i)|} \cdot |E(G_i)| \right) && \text{By Theorem 12.10} \\ &= \tilde{O} \left(\sum_i \frac{|V(G_i)|}{\epsilon^2} \right) && \text{Simplifying} \\ &= \tilde{O}(n/\epsilon^2) \end{aligned}$$

where the last inequality follows since by [Theorem 12.15](#), each vertex $v \in V$ appears in at most $O(\log n)$ graphs G_i .

Towards [Theorem 12.7](#), it only remains to prove that H is indeed a cut sparsifier of G .

Lemma 12.16. *For any graph G and edge-wise partition into $G_1, G_2, \dots, G_k$, we have that for $H = \cup_i H_i$ where each H_i is cut sparsifier of G_i that preserves cut to factor $(1 \pm \epsilon)$, then H preserves all cuts in G to factor $(1 \pm \epsilon)$.*

Proof. For any cut $(S, V \setminus S)$, we have

$$\begin{aligned}
 |E_G(S, V \setminus S)| &= \sum_i |E_{G_i}(S, V \setminus S)| && \text{Edge-wise Partitioning} \\
 &= (1 \pm \epsilon) \sum_i w'(E_{H_i}(S, V \setminus S)) && \text{Piece-wise cut sparsifiers} \\
 &= (1 \pm \epsilon) w'(E_H(S, V \setminus S)) && \text{Since } G \text{ is simple}
 \end{aligned}$$

□

Chapter 13

Maximum & Transshipment Flows

A central topic in graph algorithms is to compute flows that route commodities from their origin to their destination while minimizing some objective function.

In this chapter, we focus on two particular problems:

- **Transshipment Problem:** in this problem, we minimize the ℓ_1 -cost, also referred to as the length of the flow. For routing a single unit of commodity from one place to another, this is the shortest path between these points. More generally, it is the flow that sends the commodities while traveling the shortest total distance.
- **Maximum Flow Problem:** in this problem, we try to minimize the ℓ_∞ -cost of the problem, which is equivalent to determining the maximum amount of demand that can be routed while no edge carries more flow than its capacity.

The former problem typically arises in logistics, while the latter problem models throughput. Both, but especially the maxflow problem, are extremely important subroutines to a myriad of computational tasks.

In this lecture, we discuss the underlying frameworks to solve both of these problems approximately in $\tilde{O}(m)$ time. We focus on the focal ideas in these frameworks and give MWU algorithms that require $\tilde{O}(1)$ iterations. Each of these iterations can be implemented in $\tilde{O}(m)$ time, however, to keep the lecture simple enough, we only show how to implement each iteration in $\text{poly}(m)$ time.

13.1 A Brief Intro to Flows

Let us start by formalizing the flow problems. We work with an undirected graph $G = (V, E)$, however, since flows require a direction, i.e. we think of pushing flow from one endpoint towards the other, we assign an *orientation* to each edge e and write $e = (u, v) \in E$ to mean the edge is oriented from u to v (and $(v, u) \notin E$).

A *flow* $\mathbf{f} \in \mathbb{R}^E$ is a vector over the edges that indicates that for edge $e = (u, v)$, we send $|\mathbf{f}(e)|$ units of a commodity along the edge e , either from u to v if $\mathbf{f}(e) > 0$, or from v to u otherwise.

We denote by $\mathbf{d} \in \mathbb{R}^V$ the *demand* that we want to route. Here, any vertex v with $\mathbf{d}(v) > 0$ asks to receive $\mathbf{d}(v)$ units of the commodity; if $\mathbf{d}(v) < 0$, then it is the origin of $|\mathbf{d}(v)|$ units of the commodity and asks for it to be distributed to destinations. Naturally, a demand is required to sum to 0, e.g. $\mathbf{d}^\top \mathbf{1} = 0$, since we can only deliver as many units of commodities as are produced.

Given a demand $\mathbf{d}$, we then want the flow $\mathbf{f}$ to route the demand, meaning for each vertex $v \in V$, the amount of commodity received minus the amount of commodity sent away should sum to $\mathbf{d}(v)$. This property of the flow $\mathbf{f}$ is called *flow conservation*. Letting $\mathbf{B} \in \mathbb{R}^{V \times E}$ be the edge incidence matrix where for $e = (u, v) \in E$, $\mathbf{B}_e = \mathbf{1}_v - \mathbf{1}_u$, we can write the flow conservation constraint succinctly as $\mathbf{B}\mathbf{f} = \mathbf{d}$.

Using the definitions above, we can express the transshipment problem where the graph is endowed with a length function $\mathbf{l}$ and matrix $\mathbf{L} = \text{diag}(\mathbf{l})$ as the LP

$$\begin{aligned} & \text{minimize} && \|\mathbf{L}\mathbf{f}\|_1 \\ & \text{subject to} && \mathbf{B}\mathbf{f} = \mathbf{d} \end{aligned} \tag{13.1}$$

Note that the transshipment problem is a generalization of the single-source shortest paths problem for demand $\mathbf{d} = \mathbf{1} - n \cdot \mathbf{1}_s$, i.e. the problem where we need to send one unit of the commodity from source vertex s to each other vertex in the network.

The maximum flow problem for the graph endowed with a capacity function $\mathbf{c}$ and matrix $\mathbf{C} = \text{diag}(\mathbf{c})$ can be expressed as the LP

$$\begin{aligned} & \text{minimize} && \|\mathbf{C}^{-1}\mathbf{f}\|_\infty \\ & \text{subject to} && \mathbf{B}\mathbf{f} = \mathbf{d} \end{aligned} \tag{13.2}$$

In particular, if (13.2) can be solved with objective value at most 1, then we learn that the demand $\mathbf{d}$ can be routed without exceeding any of the capacities. For objective value β , we learn that we can route $1/\beta$ times the demand. The optimal value achieved by (13.2) is often referred to as the *congestion*.

The maximum amount of flow is essentially the largest number of times that the demand can be routed with congestion 1, i.e. $1/\beta$. Throughout, we assume that lengths and capacities are polynomially bounded.

13.2 Maximum Flows

Let us start by giving a $(1 + \epsilon)$ -approximate maximum flow algorithm.¹ We prove the following theorem.

Theorem 13.1. *Given a flow instance, accuracy $\epsilon > 0$ and target value K , we give an MWU algorithm that returns either*

- *a flow $\mathbf{f}$ that routes the demand with congestion at most $(1 + O(\epsilon))K$, or*
- *a violating cut S such that the demand on the cut $\mathbf{d}(S)$ almost exceeds the capacity of the cut, $\mathbf{c}(E(S, \bar{S}))$, i.e. $|\mathbf{d}(S)| > K \cdot \mathbf{c}(E(S, \bar{S}))$.*

It is not hard to see that a violating cut is a certificate that the maximum flow problem has congestion larger K . Note that by scaling $\mathbf{C}^{-1}$, we can henceforth assume that $K = 1$.

Review of the MWU framework. To obtain the algorithm above, we will apply the multiplicative weights update (MWU) framework that we saw in the last lecture. Here, we use a version of the MWU algorithm that can solve the feasibility LP, $\exists \mathbf{x} \in \mathbb{R}^n, \|\mathbf{Ax}\|_\infty + \langle \mathbf{b}, \mathbf{x} \rangle \leq K$ where $\mathbf{A}$ and $\mathbf{b}$ are allowed to have negative entries. A proof can be found in the last exercise sheet or in [Zuz23].

Theorem 13.2. *For any matrix $\mathbf{A} \in \mathbb{R}^{n \times m}$ and vector $\mathbf{b} \in \mathbb{R}^n$, for $\epsilon > 0$, value $\gamma \in \mathbb{R}$ and $\rho > 0$, given an oracle $\mathcal{O}_\epsilon$ that for any vector $\mathbf{p} \in \mathbb{R}^m$ with $\|\mathbf{p}\|_1 \leq 1$ returns a solution $\mathbf{x} \in \mathbb{R}^n$ such that*

$$\langle \mathbf{p}, \mathbf{Ax} \rangle + \langle \mathbf{b}, \mathbf{x} \rangle \leq \gamma$$

and $\|\mathbf{Ax}\|_\infty \leq \rho$. Then, there is an MWU algorithm that returns as solution $\mathbf{x} \in \mathbb{R}^n$ which is a convex combination of solutions from the oracle with $\|\mathbf{Ax}\|_\infty + \langle \mathbf{b}, \mathbf{x} \rangle \leq \gamma + \epsilon$ after making $O(\epsilon^{-2} \rho^2 \log n)$ calls to the oracle. The algorithm takes additional time $O(\epsilon^{-2} \rho^2 \log n \cdot (\text{nnz}(\mathbf{A}) + \text{nnz}(\mathbf{b})))$.

¹The presentation of this part of the lecture is partially taken from Thatchaphol Saranurak's course on [Expanders and Fast Graph Algorithms](#), which is based on the result in [She13].

A First Approach. Let us first see what happens when we apply the MWU method directly to the LP (13.2). We assume that $K = 1$ henceforth which is w.l.o.g. since we can scale capacities $\mathbf{c}$ accordingly. Since the constraint $\|\mathbf{C}^{-1}\mathbf{f}\|_\infty \leq 1$ is relatively easy to enforce, we will later implement an oracle $\mathcal{O}$ that only returns flow solutions $\mathbf{f}$ with $\|\mathbf{C}^{-1}\mathbf{f}\|_\infty \leq 1$.

It remains to enforce the inequality $\|\mathbf{d} - \mathbf{B}\mathbf{f}\|_\infty \leq 0$ which ensures that the demand is routed. This matches the format of Theorem 13.2 which (upon a good oracle implementation) returns a flow $\mathbf{f}$ with congestion at most 1 and

$$\|\mathbf{B}\mathbf{f} - \mathbf{d}\|_\infty \leq \epsilon.$$

Note that here we want to pick ϵ as close as possible to 1 since the MWU requires $\Omega(\epsilon^{-2})$ calls to our oracle.

However, observe that $\|\mathbf{B}\mathbf{f} - \mathbf{d}\|_\infty \leq \epsilon$ is not really a useful guarantee to have. Even for demand $\mathbf{d} = \mathbf{1}_s - \mathbf{1}_t$ sending one unit of commodity from a vertex s to t , means that the flow $\mathbf{f}$ that would be returned by the MWU algorithm from Theorem 13.2 could just place ϵ units of commodity at each vertex and leave it there. Thus, if s and t each have $\geq \lceil 1/\epsilon \rceil$ neighbors, $\mathbf{f}$ could simply spread the demand equally over the neighbors of s and t . This means, we wouldn't even learn from $\mathbf{f}$ whether s and t are connected in G !

Towards a Stronger Notion of Error. To strengthen our notion of error, we use a well-known fact that can be derived by taking the dual program, which yields the famous maxflow-mincut theorem. The fact then says that a demand can be routed iff there is sufficient capacity in each cut. Note that $\|\mathbf{C}\mathbf{B}^\top \mathbf{1}_S\|_1 = \mathbf{c}(E_G(S, \bar{S}))$.

Fact 13.3. *The optimal solution $\mathbf{f}^*$ to (13.2) achieves $\|\mathbf{C}^{-1}\mathbf{f}^*\|_\infty \leq 1$ if and only if $\|\mathbf{C}\mathbf{B}^\top \mathbf{1}_S\|_1 \geq |\mathbf{1}_S^\top \mathbf{d}|$ on all cuts $\emptyset \subsetneq S \subsetneq V$.*

Thus, if we really want an approximate solution to (13.2), we want to have some solution $\mathbf{f}$ that is close to optimal on **all** cuts! Previously, we only enforced this property on the singleton cuts given by the vertices.

Motivated by this insight, let us evaluate our constraints on each cut separately: $|\mathbf{1}_S^\top (\mathbf{d} - \mathbf{B}\mathbf{f})| \leq 0$ for all $\emptyset \subsetneq S \subsetneq V$. The constraint on each cut is now that the (net) amount of flow going over the cut matches the demand requirement.

When thinking about introducing some slack again for each constraint, note that Theorem 13.3 suggests that we should enforce on each cut S the constraint such that after routing the flow $\mathbf{f}$, at most $\epsilon \cdot \|\mathbf{C}\mathbf{B}^\top \mathbf{1}_S\|_1$ (positive or negative) demand remains on S . If this is true for every cut, then

Theorem 13.3 says that we can route the remaining demand with congestion ϵ , so we made a lot of progress.

Thus, we only need to enforce the constraint for each $\emptyset \subsetneq S \subsetneq V$

$$|\mathbf{1}_S^\top(\mathbf{d} - \mathbf{B}\mathbf{f})| \leq \epsilon \cdot \|\mathbf{C}\mathbf{B}^\top \mathbf{1}_S\|_1 \iff \left| \frac{\mathbf{1}_S^\top(\mathbf{d} - \mathbf{B}\mathbf{f})}{\|\mathbf{C}\mathbf{B}^\top \mathbf{1}_S\|_1} \right| \leq \epsilon \quad (13.3)$$

We are now ready to define the notion of an approximate flow.

Definition 13.4. *We say that $\mathbf{f}$ is a ϵ -approximate maxflow solution if all relaxed set constraints (13.3) are enforced, or equivalently if the residual demand $\mathbf{d} - \mathbf{B}\mathbf{f}$ can be routed with congestion at most ϵ times the congestion required to route $\mathbf{d}$.*

In the exercises, we ask you to prove the following result which says that by iteratively routing the remaining demand, we can obtain a flow that routes the demand $\mathbf{d}$ exactly and achieves close to optimal congestion.

Lemma 13.5. *Given a solver that inputted any demand $\mathbf{d}$ for which an optimal solution to (13.2) achieves objective value at most 1, returns a ϵ -approximate maxflow $\mathbf{f}$ as described in Theorem 13.4 with $\|\mathbf{C}^{-1}\mathbf{f}\|_\infty \leq 1$. Then, there is an algorithm that computes a flow $\mathbf{f}'$ that routes $\mathbf{d}$ with $\|\mathbf{C}^{-1}\mathbf{f}'\|_1 \leq 1 + O(\epsilon)$ that applies the solver only $O(\log n)$ times on the original input graph.*

Thus, if we get a good solver to the above problem, then we can output efficiently a flow that satisfies the demand and has only $(1 + O(\epsilon))$ times the congestion of the optimal flow.

Implementing the MWU oracle. Let us now give the implementation of the oracle $\mathcal{O}$ for the MWU algorithm from Theorem 13.2. Therefore, consider any probability vector $\mathbf{p}$ that weights the constraints, we denote by $\mathbf{p}(S)$ the coordinate that enforces the constraint for set $\emptyset \subsetneq S \subsetneq V$. We next give an oracle implementation that finds a flow $\mathbf{f} \in \mathbb{R}^m$ such that

$$\sum_{\emptyset \subsetneq S \subsetneq V} \frac{\mathbf{p}(S)}{\|\mathbf{C}\mathbf{B}^\top \mathbf{1}_S\|_1} \cdot \mathbf{1}_S^\top(\mathbf{d} - \mathbf{B}\mathbf{f}) \leq 0.$$

To simplify notation, let us introduce the vector $\phi = \sum_{\emptyset \subsetneq S \subsetneq V} \frac{\mathbf{p}(S)}{\|\mathbf{C}\mathbf{B}^\top \mathbf{1}_S\|_1} \cdot \mathbf{1}_S$ which lives in $\mathbb{R}^n$. Then the above constraint can be written as

$$\phi^\top(\mathbf{d} - \mathbf{B}\mathbf{f}) \leq 0 \iff \phi^\top \mathbf{d} \leq \phi^\top \mathbf{B}\mathbf{f}.$$

This looks much easier to work with. Since $\phi^\top \mathbf{d}$ does not depend on $\mathbf{f}$, we can view the above as a maximization problem

$$\max_{\mathbf{f} \in \mathbb{R}^m} \phi^\top \mathbf{B} \mathbf{f}$$

with the side constraint $\|\mathbf{C}^{-1} \mathbf{f}\|_\infty \leq 1$. Maximizing w.r.t. $\mathbf{f}$ using this vector ϕ turns out much easier since we now have

$$\phi^\top \mathbf{B} \mathbf{f} = \sum_{e=(u,v)} (\phi(u) - \phi(v)) \cdot \mathbf{f}(e)$$

which suggests to choose $\mathbf{f}(e) = \mathbf{c}(e) \cdot \text{sgn}(\phi(u) - \phi(v))$ which enforces $\|\mathbf{C}^{-1} \mathbf{f}\|_\infty = 1$ and yields objective value $\phi^\top \mathbf{B} \mathbf{f} = \sum_{e=(u,v)} |\phi(u) - \phi(v)| \cdot \mathbf{c}(e)$. It is not hard to see that $\mathbf{f}$ as chosen above maximizes $\phi^\top \mathbf{B} \mathbf{f}$ w.r.t. to our side constraint.

It looks like we are on track to prove our result. But what if even after all these calculations, we arrive at $\phi^\top \mathbf{B} \mathbf{f} < \phi^\top \mathbf{d}$? In this case, we can find a violating cut!

Lemma 13.6. *If we find $\phi^\top \mathbf{B} \mathbf{f} < \phi^\top \mathbf{d}$ in any call to the oracle, then we can find a cut $\emptyset \subsetneq S \subsetneq V$ such that $\|\mathbf{C} \mathbf{B}^\top \mathbf{1}_S\|_1 < \mathbf{1}_S^\top \mathbf{d}$.*

Proof. Let $\bar{\phi}$ be the vector obtained from ϕ by translating and scaling such that $0 = \bar{\phi}(1) \leq \bar{\phi}(2) \leq \dots \leq \bar{\phi}(n) = 1$. Note that $\bar{\phi}^\top \mathbf{B} \mathbf{f} < \bar{\phi}^\top \mathbf{d}$ since $\mathbf{1}$ is orthogonal to $\mathbf{B} \mathbf{f}$ and $\mathbf{d}$ thus translation does not effect of the inequality; and certainly neither does scaling.

Consider sampling a random threshold τ uniformly from $[0, 1]$ and consider the cut $S_{\leq \tau} = \{v \in V | \bar{\phi}(v) \leq \tau\}$. We have that the expected capacity of the cut is

$$\begin{aligned} \mathbb{E}[\|\mathbf{C} \mathbf{B}^\top \mathbf{1}_{S_{\leq \tau}}\|_1] &= \sum_{e=(u,v) \in E} \mathbf{c}(e) \cdot \Pr[\tau \in [\bar{\phi}(u), \bar{\phi}(v)]] \\ &= \sum_{e=(u,v) \in E} \mathbf{c}(e) \cdot |\bar{\phi}(u) - \bar{\phi}(v)| \\ &= \bar{\phi}^\top \mathbf{B} \mathbf{f}. \end{aligned}$$

On the other hand, we have

$$\mathbb{E}[\mathbf{1}_{S_{\leq \tau}}^\top \mathbf{d}] = \sum_{v \in V} \bar{\phi}(v) \mathbf{d}(v) = \bar{\phi}^\top \mathbf{d}.$$

But since both expectations are derived over τ and $\mathbb{E}[\|\mathbf{C} \mathbf{B}^\top \mathbf{1}_{S_{\leq \tau}}\|_1] < \mathbb{E}[\mathbf{1}_{S_{\leq \tau}}^\top \mathbf{d}]$ by assumption on $\bar{\phi}$, we have that there is a cut S s.t. $\|\mathbf{C} \mathbf{B}^\top \mathbf{1}_S\|_1 < \mathbf{1}_S^\top \mathbf{d}$, as desired. $\square$

Efficiency of the MWU Algorithm. Let us bound the width of this MWU. Note that for each solution $\mathbf{f}$ that the oracle computes, we have $\|\mathbf{C}^{-1}\mathbf{f}\| \leq 1$. Thus, $|\mathbf{1}_S^\top \mathbf{B}\mathbf{f}| \leq \|\mathbf{1}_S^\top \mathbf{B}\mathbf{C}\|_1$. Since the LP is feasible, we further again have $|\mathbf{1}_S^\top \mathbf{d}| \leq \|\mathbf{1}_S^\top \mathbf{B}\mathbf{C}\|_1$ from [Theorem 13.3](#).

This implies, that for every solution $\mathbf{f}$ returned by the oracle, we have for every $\emptyset \subsetneq S \subsetneq V$ that

$$\left| \frac{\mathbf{p}(S)}{\|\mathbf{C}\mathbf{B}^\top \mathbf{1}_S\|_1} \cdot \mathbf{1}_S^\top (\mathbf{d} - \mathbf{B}\mathbf{f}) \right| \leq \left| \frac{\mathbf{p}(S)}{\|\mathbf{C}\mathbf{B}^\top \mathbf{1}_S\|_1} \cdot 2 \cdot \|\mathbf{1}_S^\top \mathbf{B}\mathbf{C}\|_1 \right| \leq 2 \cdot \mathbf{p}(S).$$

Since $\|\mathbf{p}\|_1 \leq 1$ in each iteration, we thus have that the width $\rho \leq 2$. Thus, the MWU terminates with a $(1 + \epsilon)$ -approximate maxflow $\mathbf{f}$ after only $O(\log n / \epsilon^2)$ rounds!

Efficient Implementation via Cut-Preserving Tree Embeddings. But while the number of iterations is as low as we could possibly hope for, we face a final major obstacle: the number of constraints (and variables) is exponential in n . And indeed, it almost seems like we are stuck with a very bad runtime.

But let us re-investigate what the last lecture said: there is a collection of $\text{poly}(m)$ trees $T_1, T_2, \dots, T_k$ such that every cut S is α -approximated on average by these trees. Let us denote by $\mathcal{C}$ the collection of all fundamental tree cuts in any of the trees $T_1, T_2, \dots, T_k$.² We next show that it suffices to only enforce the constraints for the cuts in $\mathcal{C}$ to get demand-routing flow with congestion $(1 + O(\alpha \cdot \epsilon))$.

Lemma 13.7. *Consider a collection of fundamental tree cuts $\mathcal{C}$ as defined above from the support of a distribution of α -approximate cut-preserving trees. Then, if the constraints from (13.3) are enforced only on the cuts in $\mathcal{C}$, we still have them enforced up to an α -factor on all cuts.*

Proof Sketch. Let $\mathbf{f}$ be the flow returned by the MWU as implemented above that enforces (13.3) on the cuts in $\mathcal{C}$.

Fix any cut $\emptyset \subsetneq S \subsetneq V$. From the probabilistic method, we know that there is at least one tree $T = (V, E_T, \mathbf{c}_T)$ in the collection that α -approximates the cut S w.r.t. G . Let E' be the edges in T in the cut $(S, \bar{S})$. For each $e' \in E'$, let $S_{e'}$ be fundamental cut induced by e' . Let us assume for now that the sets $S_{e'}$ for $e' \in E'$ perfectly partition S and that $S_{e'} \subseteq S$.

We have $\sum_{e' \in E'} \|\mathbf{C}_T \mathbf{B}_T^\top \mathbf{1}_{S_{e'}}\|_1 \leq \alpha \cdot \|\mathbf{C}\mathbf{B}^\top \mathbf{1}_S\|_1$. Each cut $S_{e'}$ is in $\mathcal{C}$ by definition.

²Recall that a fundamental tree cut w.r.t. tree T is any cut $(S, \bar{S})$ such that for some edge $e \in E$, S and $\bar{S}$ are exactly to the two connected components of the graph $T \setminus \{e\}$.

Let $\mathbf{d}_{res} = \mathbf{d} - \mathbf{B}\mathbf{f}$. Then, we have by the triangle inequality that

$$|\mathbf{d}_{res}^\top \mathbf{1}_S| \leq \sum_{e' \in E'} |\mathbf{d}_{res}^\top \mathbf{1}_{S_{e'}}| \leq \sum_{e' \in E'} \epsilon \cdot \|\mathbf{C}_T \mathbf{B}_T^\top \mathbf{1}_{S_{e'}}\|_1 \leq \epsilon \cdot \alpha \cdot \|\mathbf{C} \mathbf{B}^\top \mathbf{1}_S\|_1.$$

This yields the desired property. Removing the assumption that the sets $S_{e'}$ properly partition S requires a refined argument that we omit here (as it gets fairly technical). $\square$

To recover a $(1 + \epsilon)$ -approximate solution, we have to set accuracy ϵ' in the algorithm to $O(\epsilon/\alpha)$. This yields $\tilde{O}((\alpha/\epsilon)^2)$ iterations of the MWU each implementable in polynomial time to obtain an approximate maxflow.³

13.3 Transshipment Flows

Next, let us discuss a $(1 + \epsilon)$ -approximate MWU algorithm to compute transshipment flows. Recall that the problem is given by the LP

$$\begin{aligned} & \text{minimize} && \|\mathbf{L}\mathbf{f}\|_1 \\ & \text{subject to} && \mathbf{B}\mathbf{f} = \mathbf{d} \end{aligned} \tag{13.4}$$

In this section, it will be more convenient to work with the dual, however. We get

$$\begin{aligned} & \text{maximize} && \mathbf{d}^\top \phi \\ & \text{subject to} && \|\mathbf{L}^{-1} \mathbf{B}^\top \phi\|_\infty \leq 1 \end{aligned} \tag{13.5}$$

Here, the task is to find a potential $\phi \in \mathbb{R}^n$ with large inner product with the demand vector.

In this section, we show the following main result.

Theorem 13.8. *Given a flow instance, accuracy $\epsilon > 0$ and target value K , we give an MWU algorithm that returns either*

- *a feasible solution $\mathbf{f}$ to the primal LP with $\|\mathbf{L}\mathbf{f}\|_1 \leq (1 + O(\epsilon))K$, or*
- *a feasible solution ϕ to the dual LP with $\mathbf{d}^\top \phi \geq K$.*

Before we delve into the MWU algorithm, let us gain some intuition and develop some first tools. Again, by scaling $\mathbf{L}$, we can assume $K = 1$ throughout the section.

³In fact, there are algorithms that compute a collection $\mathcal{C}$ of cuts with the property above for $\alpha = \tilde{O}(1)$ such that each vertex $v \in V$ occurs in $O(\log n)$ cuts on the smaller side. Analyzing our algorithm carefully, this yields an $\tilde{O}(m/\epsilon^2)$ time algorithm for approximate maxflow.

Warm-up #1: Solving the Primal via Tree Embeddings. Let us first discuss how to find a crude approximate solution for the transshipment problem given in (13.4) on G . First observe that if the demand $\mathbf{d}$ has only a single source $s \in V$ and a single sink $t \in V$ and all other coordinates in $\mathbf{d}$ are 0, i.e. $\mathbf{d} = F \cdot (\mathbf{1}_t - \mathbf{1}_s)$ for some $F \in \mathbb{R}_{>0}$, then the cheapest way to route the demand is to route F units of flow along the shortest path from s to t . Routing the demand costs $F \cdot \text{dist}_G(s, t)$.

Now consider routing the demand $\mathbf{d}$ using a distance-preserving tree embedding. Recall that a distance-preserving probabilistic tree embedding T sampled from a distribution $\mathcal{T}$ such that for any $u, v \in V$,

$$\text{dist}_G(u, v) \leq \text{dist}_T(u, v) \text{ and } \mathbb{E}[\text{dist}_T(u, v)] \leq \alpha \cdot \text{dist}_G(u, v)$$

for $\alpha = O(\log n)$. Thus, for a demand $F \cdot (\mathbf{1}_t - \mathbf{1}_s)$, routing F units along the shortest path from s to t in T , we expect that we can route at expected cost $\alpha \cdot F \cdot \text{dist}_G(s, t)$ and thus, we are α -competitive!

In fact, we claim that this strategy works also for general demands $\mathbf{d}$. To see this, let us formalize some of the concepts that we discussed above. As in the previous chapter, we assume that each tree embedding T is a subgraph of G .⁴ We claim that routing the demand $\mathbf{d}$ in the tree $T = (V, E')$, for which there is a unique flow $\mathbf{f} \in \mathbb{R}^{E'}$, achieves an α -approximate solution to (13.1) in expectation.

Let us construct the flow $\mathbf{f}$ formally: Fix an arbitrary vertex r in $T = (V, E')$. Next, construct the matrix $\mathbf{R}_T \in \mathbb{R}^{|V| \times |E'|}$ where the entry $\mathbf{R}_T(v, e')$ is 0 if e' is not on the v -to- r root path, and otherwise -1 if its orientation aligns with the path direction, and $+1$ if its orientation is reversed to the path direction. The unique flow routing demand $\mathbf{d}$ is then given by $\mathbf{f} = \mathbf{R}_T \mathbf{d}$. Let us now prove our claim.

Claim 13.9. *The flow $\mathbf{f}$ as constructed above routes demand $\mathbf{d}$ and is α -competitive in expectation to the optimal flow for the LP (13.1).*

Proof. Consider an optimal flow $\mathbf{f}^*$. For any edge $e = (u, v) \in E$, consider routing the demand $\mathbf{d}_e = \mathbf{f}^*(e)(\mathbf{1}_v - \mathbf{1}_u)$ in the tree T , i.e. routing it with flow $\mathbf{f}_e = \mathbf{R}_T \mathbf{d}_e$. We have already seen that this can be done with expected cost at most $\alpha \cdot |\mathbf{f}^*(e)| \cdot l(e)$.

By linearity of expectation, we thus have that the total cost of routing all demands $\mathbf{d}_e$ simultaneously via flow $\mathbf{f}_E = \sum_{e \in E} \mathbf{f}_e$ has expected cost

$$\|\mathbf{L} \mathbf{f}_E\|_1 \leq \sum_{e \in E} \alpha \cdot |\mathbf{f}^*(e)| \cdot l(e) = \alpha \cdot \|\mathbf{L} \mathbf{f}^*\|_1.$$

⁴And again, this requirement is not necessary but helps us to avoid clutter in the notation.

Finally, we use that $\mathbf{R}_T$ is a matrix (and thus a linear operator) which yields that $\mathbf{f}_E = \sum_{e \in E} \mathbf{f}_e = \sum_{e \in E} \mathbf{R}_T \mathbf{d}_e = \mathbf{R}_T \cdot (\sum_{e \in E} \mathbf{d}_e) = \mathbf{R}_T \mathbf{d}$ and thus $\mathbf{f}_E$ is a unique solution and $\mathbf{f}_E = \mathbf{f}$.⁵ $\square$

Warm-Up #2: Solving the Dual via Tree Embeddings. To gain some intuition for the dual, let us first consider the transshipment problem on a tree $T = (V, E)$ with lengths $\mathbf{l}$ (recall $\mathbf{L} = \text{diag}(\mathbf{l})$). For a demand $\mathbf{d} \in \mathbb{R}^n$, there is a unique flow $\mathbf{f}$ routing the demand on the tree T . W.l.o.g. let us assume that edges are oriented so that every coordinate of $\mathbf{f}$ is non-negative. Let us now construct a potential ϕ_T that is equivalent to the flow $\mathbf{f}$ in that $\mathbf{d}^\top \phi_T = \|\mathbf{L}\mathbf{f}\|_1$: we first fix an arbitrary vertex $r \in V$ as the root. We set $\phi_T(r) = 0$ and then iteratively define for each vertex $u \in V \setminus \{r\}$ with edge (u, v) to parent v the potential $\phi_T(u) = \phi_T(v) - \text{sgn}(\mathbf{f}(e)) \cdot \mathbf{l}(e)$. This means in particular that any st -tree path $T[s, t]$ with all edges carrying positive flow in the direction of the path, we have $\phi_T(t) - \phi_T(s) = \mathbf{l}(T[s, t])$. A more concise way to define the potentials is to have $\mathbf{R}_T$ be the matrix as above for tree T , then it is given by $\phi_T = \mathbf{R}_T^\top \text{sgn}(\mathbf{R}_T \mathbf{d})$.

Let us briefly verify that the flow and potential are cost-equivalent.

Lemma 13.10. *For the potential ϕ_T as defined above on a tree graph T , we have $\mathbf{d}^\top \phi_T = \|\mathbf{L}\mathbf{f}\|_1$.*

Proof. It is not hard to see that we can decompose the flow $\mathbf{f}$ into flows $\mathbf{f}_1, \mathbf{f}_2, \dots, \mathbf{f}_k$ such that each flow $\mathbf{f}_i$ routes one unit of demand from some source vertex s_i to sink vertex t_i along the tree path $T[s_i, t_i]$ and $\mathbf{f} = \sum_i \mathbf{f}_i$ and each $\mathbf{f}_i$ is non-negative in every coordinate. Thus, $\|\mathbf{L}\mathbf{f}\|_1 = \sum_i \|\mathbf{L}\mathbf{f}_i\|_1$.

On the other hand, for flow $\mathbf{f}_i$ routing from s_i to t_i , we have that it routes demand $\mathbf{d}_i = \mathbf{1}_{t_i} - \mathbf{1}_{s_i}$. So $\mathbf{d}_i^\top \phi_T = \phi_T(t_i) - \phi_T(s_i)$. But since the flow along the $T[s_i, t_i]$ path is positive in the s_i to t_i direction, we have $\phi_T(t_i) - \phi_T(s_i) = \mathbf{l}(T[s_i, t_i]) = \|\mathbf{L}\mathbf{f}_i\|_1$. Summing over all demands $\mathbf{d}_i$ and flows $\mathbf{f}_i$ yields the desired lemma. $\square$

It is now easy to get a crude solution to the dual problem: we can sample an α -approximate tree embedding T and with probability $1/2$, we have for $\phi_T = \mathbf{R}_T^\top \text{sgn}(\mathbf{R}_T \mathbf{d})$ that $\mathbf{d}^\top \phi_T = \|\mathbf{L}\mathbf{R}_T \mathbf{d}\|_1 \leq \alpha \cdot \text{OPT}$ in expectation.

It remains to scale down ϕ_T such that the constraint $\|\mathbf{L}^{-1} \mathbf{B} \phi_T\|_\infty \leq 1$ becomes feasible. But now we are hitting a major road block. The constraint $\|\mathbf{L}^{-1} \mathbf{B} \phi_T\|_\infty \leq 1$ asks for any edge $e = (u, v)$ to have $|\phi_T(u) - \phi_T(v)| \leq \ell(e)$. But the best we could hope for with a tree is to use Markov's inequality

⁵The matrix $\mathbf{R}_T$ has vectors orthogonal to $\mathbf{1}$ in its nullspace. However, all demands that we formulate are orthogonal to $\mathbf{1}$, and we therefore do not have to be overly careful.

and conclude that with probability $\geq 1/2$, at least half the constraints are enforced after scaling down by 2α . But enforcing half the constraints is not good enough, the infinity norm will punish us by the most violated constraint!

Luckily, there is a clever way around this: we simply go over the entire support of the tree embeddings. For tree embeddings sampled from the collection $\mathcal{T}$, define the linear operator

$$\mathbf{R} = \sum_{T \in \mathcal{T}} p_T \cdot \mathbf{R}_T,$$

where p_T is the probability with which T is sampled. Let us take $\mathbf{f} = \mathbf{R}\mathbf{d}$ and $\phi = \mathbf{R}^\top \text{sgn}(\mathbf{R}\mathbf{d})$.

We have that

$$\mathbf{d}^\top \phi = \mathbb{E}_T[\mathbf{d}^\top \phi_T] = \mathbb{E}_T[\|\mathbf{L}\mathbf{R}_T\mathbf{d}\|_1] \geq OPT,$$

since the flow $\mathbf{f}_T = \mathbf{R}_T\mathbf{d}$ is a feasible primal solution.

To argue dual feasibility, observe that for every tree T and every edge $e = (u, v)$, routing $\mathbf{d}$ on T induces a potential $\phi_T := \mathbf{R}_T^\top \mathbf{d}$ that satisfies

$$|\phi_T(u) - \phi_T(v)| \leq \text{dist}_T(u, v)$$

because on a tree the potential difference between two vertices equals exactly the length of the unique path between them. Taking expectations and using linearity and Jensen's inequality, we obtain

$$\begin{aligned} |\phi(u) - \phi(v)| &= |\mathbb{E}_T[\phi_T(u)] - \mathbb{E}_T[\phi_T(v)]| \\ &\leq \mathbb{E}_T[|\phi_T(u) - \phi_T(v)|] \\ &\leq \mathbb{E}_T[\text{dist}_T(u, v)] \\ &\leq \alpha \cdot \mathbf{l}(e), \end{aligned}$$

where the last inequality uses that the tree distribution has expected stretch at most α .

Thus, $\bar{\phi} = \frac{1}{\alpha}\phi$ is feasible for the dual and is indeed an α -approximate dual solution since it is of value at least OPT/α .⁶

An MWU algorithm for Transshipment Flows. We now implement a $(1 + \epsilon)$ -approximate MWU algorithm. Recall our blackbox statement for the MWU framework.

⁶Linear operators with properties like $\mathbf{R}$ are known that are computable in $\tilde{O}(m)$ time and where for each vector $\mathbf{x}$, evaluating $\mathbf{R}\mathbf{x}$ or $\mathbf{R}^\top \mathbf{x}$ takes time $\tilde{O}(m)$ (see [KGR25]). This yields an overall $\tilde{O}(m)$ algorithm for the transshipment problem.

Theorem 13.11. *For any matrix $\mathbf{A} \in \mathbb{R}^{n \times m}$ and vector $\mathbf{b} \in \mathbb{R}^n$, for $\epsilon > 0$, value $\gamma \in \mathbb{R}$ and $\rho > 0$, given an oracle $\mathcal{O}_\epsilon$ that for any vector $\mathbf{p} \in \mathbb{R}^m$ with $\|\mathbf{p}\|_1 \leq 1$ returns a solution $\mathbf{x} \in \mathbb{R}^n$ such that*

$$\mathbf{p}^\top \mathbf{A}\mathbf{x} + \langle \mathbf{b}, \mathbf{x} \rangle \leq \gamma - \epsilon$$

and $\|\mathbf{A}\mathbf{x}\|_\infty \leq \rho$. Then, there is an MWU algorithm that returns as solution $\mathbf{x} \in \mathbb{R}^n$ which is a convex combination of solutions from the oracle with $\|\mathbf{A}\mathbf{x}\|_\infty + \langle \mathbf{b}, \mathbf{x} \rangle \leq \gamma$ after making $O(\epsilon^{-2}\rho^2 \log n)$ calls to the oracle. The algorithm takes additional time $O(\epsilon^{-2}\rho^2 \log n \cdot (\text{nnz}(\mathbf{A}) + \text{nnz}(\mathbf{b})))$.

Note that **Theorem 13.11** is slightly different from **Theorem 13.2** in that we shifted γ by ϵ . This will be more natural for the transshipment problem.

Now, we want to solve the problem

$$\|\mathbf{L}^{-1}\mathbf{B}^\top\phi\|_\infty - \langle \mathbf{d}, \phi \rangle \leq 0.$$

This requires an oracle $\mathcal{O}$ that, for any vector $\mathbf{p} \in \mathbb{R}^m$, finds $\phi \in \mathbb{R}^{|V|}$ such that

$$\mathbf{p}^\top \mathbf{L}^{-1}\mathbf{B}^\top\phi - \langle \mathbf{d}, \phi \rangle \leq -\epsilon.$$

We interpret the scaled probability vector as a flow, defining

$$\mathbf{f} = \mathbf{L}^{-1}\mathbf{p} \in \mathbb{R}^{|E|},$$

and the residual demand as

$$\mathbf{d}_{\text{res}} = \mathbf{d} - \mathbf{B}\mathbf{f} \in \mathbb{R}^{|V|}.$$

The oracle's task is now to find a potential ϕ such that

$$\langle \mathbf{d}_{\text{res}}, \phi \rangle \geq \epsilon,$$

while satisfying $\|\mathbf{L}^{-1}\mathbf{B}^\top\phi\|_\infty \leq 1$.

Now, consider taking the potential $\bar{\phi} = \frac{1}{\alpha}\mathbf{R}^\top \text{sgn}(\mathbf{R}\mathbf{d})$ that we discussed before. It is indeed feasible, i.e. $\|\mathbf{L}^{-1}\mathbf{B}^\top\bar{\phi}\|_\infty \leq 1$.

Further, if $\langle \mathbf{d}_{\text{res}}, \bar{\phi} \rangle \geq \epsilon$, then we correctly implemented the oracle. It remains to consider the case that $\langle \mathbf{d}_{\text{res}}, \bar{\phi} \rangle < \epsilon$. But we have that $\|\mathbf{L}\mathbf{f}\|_1 = \|\mathbf{L}\mathbf{L}^{-1}\mathbf{p}\|_1 = \|\mathbf{p}\|_1 \leq 1$ (from the MWU, we have $\|\mathbf{p}\|_1 \leq 1$). Further since $\bar{\phi}$ is α -approximate for the dual, by duality, we now have that there is a flow $\mathbf{f}'$ routing $\mathbf{d}_{\text{res}}$ with $\|\mathbf{L}\mathbf{f}'\| < \epsilon\alpha$ and thus the objective value is at most $1 + \epsilon\alpha$. Using our solver iteratively on $\mathbf{d}_{\text{res}}$ then indeed yields a flow that routes the demand and has cost at most $1 + O(\epsilon\alpha)$ (much like **Theorem 13.5**). It remains to scale ϵ down by α to obtain the main result in **Theorem 13.8** while losing only a factor α^2 in the runtime.

Bibliography

- [ADD⁺93] Ingo Althöfer, Gautam Das, David Dobkin, Deborah Joseph, and José Soares. On sparse spanners of weighted graphs. *Discrete & Computational Geometry*, 9(1):81–100, 1993.
- [AHK12] Sanjeev Arora, Elad Hazan, and Satyen Kale. The multiplicative weights update method: a meta-algorithm and applications. *Theory of Computing*, 8(1):121–164, 2012.
- [AMS96] Noga Alon, Yossi Matias, and Mario Szegedy. The space complexity of approximating the frequency moments. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 20–29. ACM, 1996.
- [AV06a] David Arthur and Sergei Vassilvitskii. How slow is the k-means method? In *Proceedings of the Twenty-Second Annual Symposium on Computational Geometry*, SCG '06, page 144–153, New York, NY, USA, 2006. Association for Computing Machinery.
- [AV06b] David Arthur and Sergei Vassilvitskii. k-means++: The advantages of careful seeding. Technical report, Stanford, 2006.
- [BMV⁺12] Bahman Bahmani, Benjamin Moseley, Andrea Vattani, Ravi Kumar, and Sergei Vassilvitskii. Scalable k-means++. In *Proceedings of the VLDB Endowment*, volume 5, pages 622–633, 2012.
- [CJZ14] Lin Chen, Klaus Jansen, and Guochuan Zhang. On the optimality of approximation schemes for the classical scheduling problem. In *Proceedings of the twenty-fifth annual ACM-SIAM symposium on Discrete algorithms*, pages 657–668. SIAM, 2014.
- [Das08] Sanjoy Dasgupta. The hardness of k-means clustering. 2008.
- [Das13] Sanjoy Dasgupta. Lecture 3: Algorithms for k -means clustering. Course notes, 2013. Available at <https://cseweb.ucsd.edu/~dasgupta/291-geom/kmeans.pdf>.

- [DS14] Irit Dinur and David Steurer. Analytical approach to parallel repetition. In *Proceedings of the forty-sixth annual ACM symposium on Theory of computing*, pages 624–633. ACM, 2014.
- [Erd64] P. Erdős. Extremal problems in graph theory. In “*Theory of graphs and its applications*,” *Proc. Symposium Smolenice*, pages 29–36, 1964.
- [Fei98] Uriel Feige. A threshold of $\ln n$ for approximating set cover. *Journal of the ACM (JACM)*, 45(4):634–652, 1998.
- [FM85] Philippe Flajolet and G Nigel Martin. Probabilistic counting algorithms for data base applications. *Journal of computer and system sciences*, 31(2):182–209, 1985.
- [FRT03] Jittat Fakcharoenphol, Satish Rao, and Kunal Talwar. A tight bound on approximating arbitrary metrics by tree metrics. In *Proceedings of the thirty-fifth annual ACM symposium on Theory of computing*, pages 448–455. ACM, 2003.
- [Joh74] David S Johnson. Approximation algorithms for combinatorial problems. *Journal of computer and system sciences*, 9(3):256–278, 1974.
- [Kar01] David R. Karger. A randomized fully polynomial time approximation scheme for the all-terminal network reliability problem. *SIAM Rev.*, 43(3):499–522, March 2001.
- [KGR25] Rasmus Kyng, Maximilian Probst Gutenberg, and Tim Rieder. Random-shift revisited: Tight approximations for tree embeddings and ℓ_1 -oblivious routings. 2025.
- [Llo82] Stuart Lloyd. Least squares quantization in pcm. *IEEE transactions on information theory*, 28(2):129–137, 1982.
- [LY94] Carsten Lund and Mihalis Yannakakis. On the hardness of approximating minimization problems. *Journal of the ACM (JACM)*, 41(5):960–981, 1994.
- [RT87] Prabhakar Raghavan and Clark D Tompson. Randomized rounding: a technique for provably good algorithms and algorithmic proofs. *Combinatorica*, 7(4):365–374, 1987.

- [She13] Jonah Sherman. Nearly maximum flows in nearly linear time. In *2013 IEEE 54th Annual Symposium on Foundations of Computer Science*, pages 263–269. IEEE, 2013.
- [ST85] Daniel D Sleator and Robert E Tarjan. Amortized efficiency of list update and paging rules. *Communications of the ACM*, 28(2):202–208, 1985.
- [Tho15] Mikkel Thorup. Linear probing with 5-independent hashing. *arXiv preprint arXiv:1509.04549*, 2015.
- [Vat09] Andrea Vattani. k-means requires exponentially many iterations even in the plane. In *Proceedings of the Twenty-Fifth Annual Symposium on Computational Geometry*, SCG ’09, page 324–332, New York, NY, USA, 2009. Association for Computing Machinery.
- [Vaz13] Vijay V Vazirani. *Approximation algorithms*. Springer Science & Business Media, 2013.
- [Wen91] Rephael Wenger. Extremal graphs with no c_4 ’s, c_6 ’s, or c_{10} ’s. *Journal of Combinatorial Theory, Series B*, 52(1):113–116, 1991.
- [Woo06] David P Woodruff. Lower bounds for additive spanners, emulators, and more. In *Foundations of Computer Science, 2006. FOCS’06. 47th Annual IEEE Symposium on*, pages 389–398. IEEE, 2006.
- [WS11] David P Williamson and David B Shmoys. *The design of approximation algorithms*. Cambridge university press, 2011.
- [Yao77] Andrew Chi-Chin Yao. Probabilistic computations: Toward a unified measure of complexity. In *Foundations of Computer Science, 1977., 18th Annual Symposium on*, pages 222–227. IEEE, 1977.
- [Zuz23] Goran Zuzic. A simple boosting framework for transshipment. In *31st Annual European Symposium on Algorithms (ESA 2023)*, pages 104–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2023.